408 考纲知识点手册

数据结构

计算机学科专业基础综合　分册

按 2027 年全国硕士研究生招生考试大纲编排

对应大纲	第一部分数据结构
本册章数	7 章
知识点条目	覆盖大纲本部分全部条目
主要素材	语雀「计算机考研 408 笔记」知识库
	王道 2027 系列教材与课件
	工作区历年真题与强化笔记

本分册为依据 2027 年考试大纲整理的「数据结构」知识点汇编，供复习检索使用。

使用说明

本手册按《2027 年全国硕士研究生招生考试计算机学科专业基础考试大纲》的条目顺序编排，覆盖大纲「考查内容」中数据结构、计算机组成原理、操作系统、计算机网络四科的全部条目。

本册范围：本册是《408 考纲知识点手册》的「数据结构」分册，对应 2027 年考试大纲「数据结构」部分的全部考查内容，共 7 章。其余三科另见对应分册；四科合订本见《408 考纲知识点手册》。

一、编排方式

全书分四部分，共 24 章。四部分与大纲的四科一一对应，章与该科下的一级条目一一对应，章内小节按大纲的二级、三级条目顺序展开。每一章的开头用引用块照抄本章对应的大纲原文条目，便于对照检查覆盖情况；每一章末尾固定给出「本章小结表」与「易错清单」两张速查内容。

二、阅读约定

本手册用三种提示框把不同性质的内容区分开：

- 核心结论——必须记住的定义、公式、判定条件，考前反复看的就是这些。
- 易错点——统考中反复出现的陷阱，以及容易与相邻概念混淆的地方。
- 真题考法——该知识点在统考中以什么形式出现；能确认年份与题号的，如实标注。无法确认的不标，不编造。

正文中的公式一律用数学排版；算法与代码用等宽框给出；对比性内容尽量做成表格。凡涉及计算的知识点，都给出公式、符号含义、适用范围，并配一组可以自己动手复核的数字算例。

三、素材来源

本手册的素材来自三处：

- 语雀知识库「计算机考研 408 笔记」（月暗）导出的 182 篇笔记 Markdown 全文；
- 工作区中的王道 2027 系列教材、课件、强化笔记与历年真题汇编；
- 工作区 408 计算机考研目录下的各类复习笔记与答题本。

素材在整理过程中做了三件事：一是按大纲重新组织顺序；二是把原笔记中的口语化叙述、错别字与图片占位改写成规范的知识点条目，公式按上下文重新推导书写；三是逐条核对，凡素材之间口径冲突的，以大纲用词与统考真题为准，并在正文中说明差异。

四、使用建议

第一轮复习时，按章通读，重点看「核心结论」与例题，把「易错点」留到做题后再回看；第二轮复习时只看每章的「本章小结表」，遇到想不起来的再翻回正文；考前冲刺时只看「核心结论」与「易错清单」。

说明：本手册为复习资料汇编，用于知识点检索与自查，不替代教材与真题训练。

目录

使用说明 **i**

第一章　基本概念 **1**

本章考纲要求 1

一、 数据结构的基本概念 1

（一） 数据、数据元素、数据项、数据对象、数据结构 1

（二） 逻辑结构与存储结构 2

（三） 抽象数据类型 ADT 4

二、 算法的基本概念 4

（一） 算法的定义 4

（二） 算法的五个特性 5

（三） 好算法的目标 5

（四） 时间复杂度 6

本章小结表 19

易错清单 20

第二章　线性表 **22**

本章考纲要求 22

一、 线性表的基本概念 22

（一） 线性表的定义与逻辑特征 22

（二） 线性表的基本操作 23

二、 线性表的实现 24

（一） 顺序存储 24

（二） 链式存储 28

（三） 顺序表与链表的对比 34

三、 线性表的应用（算法设计题） 35

（一） （○）算法设计题的通用解题框架 36

（二） 合并两个有序表（两种存储都考） 36

（三） 就地逆置 37

（四） 快慢指针：找中间结点 40

（五） 双指针：倒数第 k 个结点（2009 年真题） 40

（六） 判断链表是否有环，并求环的入口 41

（七） 判断两个链表是否相交（2012 年真题） 42

（八） 删除重复元素 43

（九） 其他常考算法 . . . 44
（十） 补充：无序表先排序再处理的思路 . . . 46
本章小结表 . . . 46
易错清单 . . . 47
算法设计题复杂度速查表 . . . 48

第三章 栈、队列和数组 **50**
本章考纲要求 . . . 50
一、 栈和队列的基本概念 . . . 50
（一） 栈的定义与特点 . . . 50
（二） 队列的定义与特点 . . . 50
（三） 栈与队列的对比 . . . 50
（四） 上溢与下溢 . . . 51
（五） 共享栈 . . . 51
二、 栈和队列的顺序存储结构 . . . 51
（一） 顺序栈与 top 指针的两种约定 . . . 51
（二） 顺序队列的 front / rear 约定 . . . 53
（三） 循环队列：三种判满方案 . . . 53
（四） 双端队列（受限双端队列） . . . 56
三、 栈和队列的链式存储结构 . . . 56
（一） 链栈 . . . 56
（二） 链队列（带头结点） . . . 57
（三） 链队列（不带头结点） . . . 58
四、 多维数组的存储 . . . 59
（一） 数组的定义与一维地址公式 . . . 59
（二） 二维数组的行优先与列优先 . . . 59
（三） 下标从 0 与从 1 开始的差异 . . . 60
五、 特殊矩阵的压缩存储 . . . 61
（一） 压缩存储的基本思路 . . . 61
（二） 对称矩阵 . . . 61
（三） 三角矩阵 . . . 63
（四） 三对角矩阵（带状矩阵） . . . 64
（五） 稀疏矩阵 . . . 65
六、 栈、队列和数组的应用 . . . 66
（一） 栈在括号匹配中的应用 . . . 66
（二） 栈在表达式求值中的应用 . . . 66
（三） 栈在递归中的应用 . . . 69
（四） 栈在进制转换中的应用 . . . 70

（五） 队列在层次遍历中的应用 . . . 70
（六） 队列在缓冲区与 CPU 调度中的应用 . . . 71
（七） 数组的典型应用：矩阵运算与顺序表 . . . 71
（八） 栈与队列的合法出栈序列 . . . 72
（九） 用栈实现队列、用队列实现栈 . . . 74
本章小结表 . . . 75
易错清单 . . . 76

第四章 树与二叉树 **78**
本章考纲要求 . . . 78
一、 树的基本概念 . . . 78
（一） 树的定义与术语 . . . 78
（二） 树的性质 . . . 80
二、 二叉树 . . . 82
（一） 二叉树的定义及其主要特性 . . . 82
（二） 二叉树的顺序存储结构和链式存储结构 . . . 84
（三） 二叉树的遍历 . . . 87
（四） 线索二叉树 . . . 92
三、 树、森林 . . . 97
（一） 树的存储结构 . . . 97
（二） 森林与二叉树的转换 . . . 99
（三） 树和森林的遍历 . . . 101
四、 树与二叉树的应用 . . . 102
（一） 哈夫曼树与哈夫曼编码 . . . 102
（二） 并查集 . . . 107
（三） 堆 . . . 111
本章小结表 . . . 114
（四） 表 1 二叉树五个性质一览 . . . 114
（五） 表 2 一般树 / 森林的性质 . . . 114
（六） 表 3 遍历序列的对应与唯一性 . . . 115
（七） 表 4 三种线索二叉树的能力 . . . 115
（八） 表 5 并查集复杂度 . . . 115
（九） 表 6 哈夫曼树 / 堆 / 线索二叉树的关键数量 . . . 116
易错清单 . . . 116

第五章 图 **118**
本章考纲要求 . . . 118
一、 图的基本概念 . . . 118

（一） 图、有向图与无向图 . . . 118
（二） 完全图、稠密图与稀疏图 . . . 119
（三） 度、入度、出度与握手定理 . . . 119
（四） 路径、回路、简单路径与距离 . . . 120
（五） 连通性：连通、强连通、连通分量 . . . 120
（六） 子图、生成子图与生成树 . . . 121
（七） 图的基本概念小结 . . . 121
二、 图的存储及基本操作 . . . 122
（一） 邻接矩阵 . . . 122
（二） 邻接表 . . . 123
（三） 十字链表（有向图） . . . 124
（四） 邻接多重表（无向图） . . . 125
（五） 四种存储结构的对比 . . . 126
（六） 图的基本操作 . . . 126
三、 图的遍历 . . . 128
（一） 深度优先搜索（DFS） . . . 128
（二） 广度优先搜索（BFS） . . . 129
（三） BFS 的「层次」与 Dijkstra 的异同 . . . 130
（四） 遍历手工模拟 . . . 131
（五） 遍历与图的连通性 . . . 131
（六） 遍历序列的唯一性 . . . 132
（七） DFS 与 BFS 的对比 . . . 132
四、 图的基本应用 . . . 133
（一） 最小（代价）生成树 . . . 133
（二） 最短路径 . . . 137
（三） 拓扑排序 . . . 141
（四） 关键路径 . . . 144
本章小结表 . . . 148
易错清单 . . . 149

第六章 查找 **151**
一、 查找的基本概念 . . . 151
（一） 查找表、关键字、查找长度 . . . 151
（二） 用判定树统一理解 ASL . . . 152
（三） 一个最小的算例 . . . 152
二、 顺序查找法 . . . 153
（一） 一般线性表的顺序查找 . . . 153
（二） 哨兵（监视哨）的作用 . . . 153

（三） 无序表的 ASL 推导 . . . 153
（四） 有序表顺序查找的 ASL 推导 . . . 154
（五） 概率不等时：把常查的元素放前面 . . . 155
三、 分块查找法 . . . 155
1. 索引顺序表的结构 . . . 155
（一） 查找过程：两步 . . . 155
3. 用折半查找查索引表时的落点 . . . 156
（二） ASL 推导与最优块长 . . . 156
（三） 优缺点与动态版本 . . . 157
四、 折半查找法 . . . 157
（一） 适用条件 . . . 157
（二） 基本实现（左闭右闭） . . . 157
（三） 判定树的构造 . . . 158
（四） 判定树是平衡二叉树 . . . 158
（五） 树高、查找成功与失败的比较次数上限 . . . 158
（六） $ASL_{成功}$ 的推导 . . . 159
（七） $ASL_{失败}$ 的推导 . . . 160
（八） 时间复杂度与优缺点 . . . 161
五、 树形查找 . . . 161
（一） 二叉搜索树（BST） . . . 161
（二） 平衡二叉树（AVL） . . . 165
（三） 红黑树（RBT） . . . 170
六、 B 树及其基本操作、B+ 树的基本概念 . . . 173
（一） m 阶 B 树的定义 . . . 173
（二） B 树的查找 . . . 176
（三） B 树的插入 . . . 177
（四） B 树的删除 . . . 178
（五） B 树的判定 . . . 178
（六） B+ 树的基本概念 . . . 179
（七） B 树与 B+ 树的对比 . . . 179
七、 散列（hash）表 . . . 180
（一） 基本概念 . . . 180
（二） 散列函数的构造方法 . . . 180
（三） 处理冲突的方法一：拉链法 . . . 181
（四） 处理冲突的方法二：开放定址法 . . . 182
（五） 线性探测的堆积（聚集）现象 . . . 183
（六） 平方探测法与表长 $4k+3$ . . . 183

（七） 散列表 ASL 的计算（三个完整算例） . . . 184
（八） 一个经典小题 . . . 187
八、 字符串模式匹配 . . . 187
（一） 朴素算法（BF 算法） . . . 187
（二） KMP 算法的思想 . . . 188
（三） 部分匹配表 PMT 与 next 数组 . . . 188
（四） 手算 next 数组的流程 . . . 189
（五） nextval 数组（优化） . . . 190
（六） 匹配过程与复杂度 . . . 191
九、 查找算法的分析及应用 . . . 192
（一） 各查找方法的横向对比 . . . 192
（二） 选用原则 . . . 192
（三） 三个必须能自己推的结论 . . . 193
本章小结表 . . . 193
易错清单 . . . 194

第七章 排序 196
本章考纲要求 . . . 196
一、 排序的基本概念 . . . 196
（一） 什么是排序 . . . 196
（二） 稳定性（stability） . . . 196
（三） 内部排序与外部排序 . . . 197
（四） 比较次数与移动次数 . . . 197
（五） 基于比较的排序的下界 . . . 197
二、 直接插入排序 . . . 198
（一） 算法思想 . . . 198
（二） 手工过程（逐步模拟） . . . 198
（三） 参考代码 . . . 198
（四） 复杂度与特性 . . . 199
三、 折半插入排序 . . . 199
（一） 与直接插入排序的唯一区别 . . . 199
（二） 比较次数与移动次数的分别分析 . . . 200
（三） 折半查找的边界（决定稳定性） . . . 200
四、 起泡排序（bubble sort） . . . 201
（一） 算法思想 . . . 201
（二） 手工过程（逐步模拟） . . . 201
（三） 参考代码（带提前结束优化） . . . 202
（四） 复杂度与特性 . . . 202

五、 简单选择排序 . . . 203
（一） 算法思想 . . . 203
（二） 手工过程（逐步模拟） . . . 203
（三） 参考代码 . . . 203
（四） 比较次数与移动次数 . . . 204
（五） 不稳定的原因与反例 . . . 204
六、 希尔排序（shell sort） . . . 205
（一） 算法思想 . . . 205
（二） 为什么能改进——两条理由 . . . 205
（三） 手工过程（逐步模拟） . . . 205
（四） 参考代码 . . . 206
（五） 复杂度分析 . . . 206
（六） 不稳定的原因与反例 . . . 207
七、 快速排序 . . . 208
（一） 算法思想与分治结构 . . . 208
（二） 一趟划分（Partition）的完整手工过程 . . . 208
（三） 参考代码 . . . 209
（四） 时间复杂度的三种情况 . . . 210
（五） 递归深度与栈空间 . . . 210
（六） 稳定性 . . . 211
（七） 快速排序为什么” 快” . . . 211
（八） 不适合的存储结构 . . . 211
八、 堆排序 . . . 212
（一） 堆的定义与存储 . . . 212
（二） 筛选（下滤）操作 . . . 212
（三） 建堆过程（自下而上筛选，$O(n)$） . . . 213
（四） 堆排序的每一趟 . . . 214
（五） 复杂度与特性 . . . 214
（六） 不适用的存储结构与 Top-K 应用 . . . 215
九、 二路归并排序（merge sort） . . . 215
（一） 算法思想 . . . 215
（二） 手工过程（逐步模拟） . . . 216
（三） 参考代码 . . . 216
（四） 归并趟数与复杂度 . . . 217
（五） 链表上的归并排序 . . . 217
十、 基数排序 . . . 218
（一） 算法思想（非比较排序） . . . 218

（二） MSD 与 LSD 218
（三） 手工过程（逐步模拟） 218
（四） 复杂度与稳定性 219
十一、 外部排序 220
（一） 为什么需要外部排序 220
（二） 外部排序的两个阶段 220
（三） 外部排序的时间开销与优化方向 221
（四） 败者树（loser tree） 222
（五） 置换-选择排序（生成初始归并段） 222
（六） 最佳归并树与哈夫曼树 224
（七） 虚段（补空归并段）的数目计算 225
十二、 排序算法的分析与应用 226
（一） 完整对比表 226
（二） 什么情况下选哪个算法 227
（三） 算法性质反推（考试最常见的形式） 227
本章小结表 228
易错清单 229

第一章基本概念

本章考纲要求

第一部分数据结构［考查目标］

1. 掌握数据结构的基本概念、基本原理和基本方法。
2. 掌握数据的逻辑结构、存储结构及基本操作的实现，能够对算法进行基本的时间复杂度与空间复杂度的分析。
3. 能够运用数据结构的基本原理和方法进行问题的分析与求解，具备采用 C 或 C++ 语言设计与实现算法的能力。

一、基本概念（一）数据结构的基本概念（二）算法的基本概念

本章是第一章「基本概念」，对应考纲的两个二级条目。数据结构部分每年第一道选择题几乎都落在「时间复杂度」，是性价比最高的送分点，也是失分集中的重灾区；基本概念则以文字辨析题的形式出现，考查定义边界的精确理解。本章把两部分合并写透：概念部分要求能准确复述定义、能在一段描述中辨认出逻辑结构、存储结构与数据运算三个层面；算法部分要求能对任意一段循环或递归代码独立推出复杂度阶，而不是靠背题。

一、数据结构的基本概念

（一）数据、数据元素、数据项、数据对象、数据结构

这一组术语是层层包含的关系，考试喜欢用「下列选项中，（）是数据的最小单位」「一个数据元素可由若干（）组成」这类题来考边界。

- 数据（**Data**）：信息的载体，是描述客观事物属性的数、字符及所有能输入到计算机中并被计算机程序识别和处理的符号的集合。数据的含义很宽：整数、实数、字符串、图像、声音都是数据。
- 数据元素（**Data Element**）：数据的基本单位，在计算机程序中通常作为一个整体进行考虑和处理。又称元素、记录（**record**）、结点（**node**）、顶点（**vertex**）——这些叫法在不同数据结构中指的是同一层概念。
- 数据项（**Data Item**）：构成数据元素的不可分割的最小单位。又称字段（**field**）、域（**domain**）。
- 数据对象（**Data Object**）：具有相同性质的数据元素的集合，是数据的一个子集。例如整数数据对象是集合 $N = \{0, \pm1, \pm2, \dots\}$。
- 数据结构（**Data Structure**）：相互之间存在一种或多种特定关系的数据元素的集合。

举一个具体例子把五层贯通。一张学生成绩表：

层次	具体内容	说明
数据	整张表格的全部符号	最宽的范围
数据对象	所有「学生成绩记录」的集合	性质相同的元素集合
数据元素	一行记录张三，90，85	作为整体处理的基本单位

层次	具体内容	说明
数据项	姓名、数学、英语	不可再分割的最小单位

数据元素与数据项的边界

「数据的最小单位」是数据项，不是数据元素。「数据的基本单位」是数据元素。这两个词一字之差，考频极高。另外注意：数据项是「不可分割的最小单位」，但数据元素在有些实现里可以只是一个数据项（例如一个整数数组，每个元素就是一个数据项，此时数据元素 = 数据项）。

数据对象与数据的关系

数据对象是数据的子集，不是数据的同义词。数据是所有能被程序处理的符号的集合；数据对象要求「性质相同」。同一个数据集合可以划分出不同的数据对象。

数据结构这一术语本身包含三个层面的内容，缺一不可：

1. 逻辑结构：数据元素之间的抽象关系，与计算机无关。
2. 存储结构（物理结构）：逻辑结构在计算机中的表示（又称映像），是逻辑结构在存储器中的实现。
3. 数据的运算：施加在数据上的操作，包括运算的定义（针对逻辑结构，指出运算的功能）和运算的实现（针对存储结构，指出运算的具体操作步骤）。

数据结构 = 逻辑结构 + 存储结构 + 数据的运算

这是本章最常被直接命中的定义。三个层面互相牵连：逻辑结构决定「能做什么运算」，存储结构决定「运算怎么做、效率如何」。所以同一个逻辑结构在同一种存储结构下实现不同，效率可能差一个数量级——这正是后面各章反复出现的主线。

（二）逻辑结构与存储结构

（1）逻辑结构：线性结构与非线性结构

按数据元素之间关系的抽象形态，分为两大类。

线性结构：数据元素之间存在一对一的关系，除第一个元素外每个元素有且仅有一个直接前驱，除最后一个元素外每个元素有且仅有一个直接后继。典型例子：线性表、栈、队列、串、一维数组。

非线性结构：数据元素之间存在一对多或多对多的关系，一个元素可能有多个直接前驱或直接后继。典型例子：树（一对多）、图（多对多）、集合（元素之间无关系，是另一种独立的逻辑结构）。

如果写得再细一点，逻辑结构可分成四类：集合结构（元素间除「同属一个集合」外无其他关系）、线性结构、树形结构、图形结构（网状结构）。后者三类加上集合结构就是常见的四分法。

逻辑结构与存储结构必须分开判断

「栈和队列的逻辑结构相同」——正确，二者都是线性结构。「线性表的链式存储是非线性结构」——错误，存储结构的选择不改变逻辑结构。逻辑结构是「抽象关系」，存储结构是「内存里的摆放方式」。看到「链式」「顺序」这样的词，它们描述的是存储结构，不能用来判断逻辑结构。

特殊的逻辑结构判定

- 空表、只有一个元素的表，也是线性结构（一对一的定义是「除首尾外每个元素有且仅有一个前驱/后继」，单元素表首尾重合，仍满足）。
- 二叉树是非线性结构，虽然每个结点最多两个后继，但「一对多」已经不属于线性。
- 栈、队列、线性表三者逻辑结构相同（都是线性结构），只是运算定义不同（限制插入删除的位置）。这是 408 常考的文字辨析点。

（2）存储结构：四种基本方式

存储结构要在存储器中表示两件事：数据元素本身的值 + 元素之间的逻辑关系。按关系的表示方式分为四种。

存储方式	关系如何表示	核心特点	典型例子
顺序存储	用物理位置上的相邻（连续单元 + 相对位置）表示逻辑相邻	逻辑相邻 ⇔ 物理相邻；支持随机存取；插入删除需移动元素	顺序表、顺序栈、一维数组实现堆
链式存储	用附加的指针表示逻辑相邻，结点本身可以不相邻	只能顺序存取；插入删除不需移动，改指针即可；有指针域额外开销	单链表、二叉链表、邻接表
索引存储	建立索引表，每项为「关键字 + 地址」	检索速度快；索引表本身占额外空间；插入删除要维护索引表	索引顺序表（分块查找）、B 树索引、数据库索引
散列存储	由关键字通过散列函数直接算出存储地址	检索、插入、删除都近似 $O(1)$；可能产生冲突，需要处理冲突	散列表（Hash Table）

理解这四种的关键在于问一句「关系是怎么被表达的」：顺序靠位置，链式靠指针，索引靠索引表，散列靠函数计算。

（3）四种存储结构对同一组运算的代价差异

同一个逻辑结构换一种存储结构，运算的实现代价会发生质变。下面把「按下标取值、插入、删除、按值查找」四类运算在四种存储结构下的代价摆在一起，可以看出为什么「选存储结构」本质上是在权衡。

存储结构	按下标取值	插入 / 删除	按值查找	额外开销
顺序存储	$O(1)$，地址可算	平均 $O(n)$，需移动元素	$O(n)$；若有序可折半到 $O(\log n)$	无指针开销，但需一整块连续空间
链式存储	$O(n)$，必须从头走	$O(1)$（已知位置）/ $O(n)$（需先定位）	$O(n)$	每个结点多一个指针域
索引存储	取决于数据区组织	$O(n)$（需维护索引表）	先查索引表再到块内查找，平均 $O(\sqrt{n})$ 或更快	索引表本身占空间
散列存储	无法按下标访问	平均 $O(1)$，需处理冲突	平均 $O(1)$，最坏退化为 $O(n)$	需要预留装填余量，冲突处理结构

表中「平均 $O(\sqrt{n})$」对应分块查找：把表分成若干块，块间用索引表（有序），块内顺序查找。设共 n 个元素、分成 b 块、每块 s 个元素（$n = bs$），则平均查找长度为「查索引表 + 块内查找」，当取 $s = \sqrt{n}$ 时整体达到最优 $O(\sqrt{n})$。这个结论在查找一章还会展开，这里只需记住「索引存储的核心是用额外空间换取查找速度」。

顺序存储 ≠ 随机存取，但它们总是同时出现

「能随机存取」的充要条件是存储地址可计算，这与顺序存储天然吻合。链式存储的地址不可计算，只能从头指针顺着走，因此是顺序存取。注意区分两个词：随机存取（random access，可直接访问任意位置，$O(1)$）与顺序存取（sequential access，必须按顺序推进，$O(k)$）。也要与随机存储区分开——随机存储指的是访问时间与存储位置无关的存储器（如 RAM），是硬件概念。

另外：索引存储不是顺序存储，索引表本身通常按顺序存储，但数据区可以离散；散列存储的地址由函数决定，与逻辑关系的顺序无关。

真题考法

基本概念部分常以「下列叙述中，正确的是（）」形式出现，选项通常是：某结构的逻辑结构是否线性、某种存储方式是否支持随机存取、数据元素与数据项谁是基本/最小单位、一个数据元素是否可由多个数据项组成。这类题不需要计算，靠定义精确度得分。

（三）抽象数据类型 ADT

数据类型（**Data Type**）：一个值的集合和定义在这个值集合上的一组操作的总称。例如 C 语言的 `int`，值是某个范围内的整数集合，操作是加减乘除和取模。数据类型分两类：

- 原子类型：值不可再分解，如 `int`、`char`、`double`。
- 结构类型：值由若干成分按某种结构组成，可以分解，如数组、结构体。

抽象数据类型（**Abstract Data Type, ADT**）：一个数学模型以及定义在该模型上的一组操作。它的核心特征是抽象——只关心「能做什么」（逻辑特性），不关心「怎么实现」（存储与实现细节）。

ADT 由三部分构成：

1. 数据对象：该类型的数据是什么样的集合。
2. 数据关系：数据对象中元素之间的关系。
3. 基本操作：施加于数据对象上的一组运算及其语义。

用三元组表示就是 (D, S, P)，其中 D 是数据对象，S 是 D 上的关系集，P 是对 D 的基本操作集。

关键性质：**ADT 与具体实现无关**。同一个 ADT 可以有不同的实现（例如「栈」这个 ADT 可以顺序实现也可以链式实现），只要对外提供相同的操作语义，使用者就不需要改变程序。这样做的意义是信息隐藏和数据封装——使用 ADT 的人只依赖接口，实现者可以自由更换内部结构。

ADT 的判断标准

判断一段描述是不是 ADT，只看它是否只规定了逻辑特性和操作功能，而没有规定用什么存储结构、用什么算法实现。凡出现「用数组」「用指针」「用循环实现」这类字样的，都不是 ADT 的定义，而是实现。

以「栈」为例，它作为 ADT 的定义应当是：数据对象是 $D = \{a_1, a_2, \ldots, a_n\}$（$n \geq 0$ 的有限序列）；数据关系是 $R = \{\langle a_{i-1}, a_i \rangle \mid 2 \leq i \leq n\}$（相邻元素间有序偶关系）；基本操作是 `InitStack(S)`、`Push(S,x)`、`Pop(S,&x)`、`GetTop(S,&x)`、`StackEmpty(S)` 等。定义里完全没有出现「数组」或「链表」——这正是抽象的体现。等到讲栈的顺序存储与链式存储时，才是在为这个 ADT 补上两种不同实现，二者的操作语义完全一致（都满足后进先出），只是时空代价不同。

这个「定义与实现分离」的思想在后续各章反复使用：线性表、栈、队列、串都是 ADT，每一种都有顺序与链式两套实现，考纲也正是按「基本概念 → 顺序存储结构 → 链式存储结构」的顺序展开的。

二、算法的基本概念

（一）算法的定义

算法（**Algorithm**）：对特定问题求解步骤的一种描述，它是指令的有限序列，其中每条指令表示一个或多个操作。

需要把三个词区分清楚：算法（解决问题的方法与步骤）、程序（算法用某种程序设计语言的具体实现）、数据结构（算法操作的对象）。著名论断是「程序 = 数据结构 + 算法」，强调的是算法设计与数据结构选择互相依赖——同一问题换一种数据结构，算法往往完全不同。

算法 ≠ 程序

- 程序可以不满足有穷性。操作系统常驻内存的调度程序、事件驱动的服务器主循环，都是「无限循环」，它们是程序但不是算法。
- 算法必须满足有穷性。
- 算法可以用自然语言、流程图、伪代码描述；程序必须用某种语言可编译运行。

因此「算法一定是程序」「程序一定是算法」两个说法都错。

（二）算法的五个特性

特性	含义	反例（不满足该特性的情形）
有穷性	算法在执行有穷步之后终止，且每一步都在有穷时间内完成	无限循环、死循环
确定性	每条指令有确切含义，无二义性；相同输入只能得到相同输出	「取一个较大的数」「若 x 为正则加 1，否则加 2 或 3」
可行性	算法的每一步都能通过已实现的基本运算执行有穷次而实现	「令 x 等于最小的正整数使……」
输入	有零个或多个输入	无输入的算法合法，如输出常量
输出	有一个或多个输出	无输出的算法无意义，不合法

输入与输出的不对称

输入可以是零个，输出必须至少一个。这是五个特性里唯一「不对称」的一条，考试最爱在这里设选项（例如「算法可以没有输入，也可以没有输出」——后半句错）。另外「有穷性」是算法区别于程序的核心，「确定性」要求「相同输入必得相同输出」，注意这与后面要讲的「随机化算法」不冲突：如果算法内部使用随机数，那么它就不是确定性的，属于另一类算法。

（三）好算法的目标

可从五个维度衡量：

1. 正确性（**Correctness**）：能正确解决求解问题，对合法输入给出满足规格说明的结果。这是最基本、最重要的要求。
2. 可读性（**Readability**）：便于人们阅读、理解和交流，也便于调试修改。
3. 健壮性（**Robustness**）：对非法输入能做出恰当反应或处理，而不会产生莫名其妙的结果。
4. 高效率与低存储量需求：效率指算法执行的时间（时间复杂度），存储量指算法执行过程中所需的最大存储空间（空间复杂度）。这里就顺理成章地引出了本章后半部分的全部内容。

其中，正确性是前提，其余四项常在保证正确性的基础上权衡。时间与空间往往互相冲突：提高时间效率常需增加空间开销（如用散列表换查找时间），节省空间常需牺牲时间（如用多次计算代替查表）。在大多数场景下，时间比空间更宝贵，因此「以空间换时间」是常用策略。

（四）时间复杂度

1.（1）定义：问题规模、语句频度与渐进记号

- 问题规模（**Problem Size**）：算法求解问题输入量的多少，用整数 n 表示。规模的定义要看问题本身：对数组是元素个数，对矩阵是阶数，对字符串是串长，对图是顶点数与边数。
- 语句频度（**Frequency Count**）：该语句在算法中被重复执行的次数。
- 时间复杂度：算法中所有语句的频度之和，记为 $T(n)$，是问题规模 n 的函数。

注意 $T(n)$ 是「真实次数」，我们真正关心的是当 n 趋于无穷时它的增长趋势，于是引入渐进记号。

$$T(n) = O(f(n)) \iff \exists\, c > 0,\ n_0 > 0,\ \forall n \geq n_0 :\ 0 \leq T(n) \leq c\, f(n)$$

含义：当 n 足够大以后，$T(n)$ 被 $f(n)$ 的某个常数倍从上方控制住。所以 O 给出的是上界，我们通常说「算法的时间复杂度是 $O(f(n))$」。

时间复杂度的本质是增长趋势，不是运行时间

「时间复杂度分析」统计的不是算法跑了多少毫秒，而是算法运行时间随数据量变大时的增长趋势。因此：

- 忽略与 n 无关的操作：顺序执行的语句只影响常数项，可以丢掉。
- 省略所有系数：$T(n) = 3n^2 + 5n + 7$ 记为 $O(n^2)$。
- 乘法法则用于嵌套：多层嵌套循环，总频度是各层频度之积（仅当各层相互独立时，后面会看到反例）。

一套可执行的三步流程：① 找出基本操作（循环最内层的语句）→ ② 用 n 表示它的执行次数（解方程或求和）→ ③ 只保留最高阶项并去掉系数。

2.（2）常见复杂度类型与增长速度

按增长速度从低到高排列：

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$$

助记口诀「常对幂指阶」对应前四档：常数阶、对数阶、幂（多项式）阶、指数阶。当 n 充分大时，增长速度差异极为悬殊：$n = 1000$ 时 $\log_2 n \approx 10$，$n \log_2 n \approx 10^4$，$n^2 = 10^6$，2^n 已经是天文数字。

三个高频陷阱

1. $O(1)$ 不代表只执行 **1** 次，而是「执行次数与 n 无关」。下面这段代码的循环体执行 1000000 次，时间复杂度仍是 $O(1)$。
2. log 的底数写成常数时不影响阶。$\log_2 n$ 与 $\log_{10} n$ 只差一个常数因子 $\log_2 10$，所以统考里 $O(\log_{10} n)$ 与 $O(\log_2 n)$ 视为同一复杂度，统一写 $O(\log n)$。但如果题目问「循环体执行多少次」，则必须写清底数。
3. 对数 **vs** 线性对数 **vs** 线性：判断嵌套循环时，先分别求出两层各自的迭代次数，再决定是相加还是相乘，不能一律相乘。

```
/* 时间复杂度 O(1)：循环次数与 n 无关 */
void algorithm_C(int n) {
    int count = 0;
    for (int i = 0; i < 1000000; i++)   /* 固定 100 万次 */
        count++;
}
```

3.（3）各类循环结构的时间复杂度推导

这是统考第一题的核心，必须掌握「定义法」（解方程）而不是靠记忆答案。

① 线性阶：单层循环，变量每次加 **1**

```
int count = 0;
for (int i = 0; i < n; i++)
    count++;
```

设循环体执行 k 次，则退出时 $i = k$ 首次不小于 n，故 $k = n$，即

$$T(n) = \sum_{i=0}^{n-1} 1 = n = O(n)$$

`while` 写法同理：`while (i < n) { count++; i++; }`，i 从 0 增到 n，执行 n 次。

② 对数阶：变量每次乘 **2**

```
int count = 0;
for (int i = 1; i <= n; i *= 2)
    count++;
```

设循环体执行 k 次后退出，此时 $i = 2^k$ 首次超过 n，即 $2^{k-1} \le n < 2^k$，两边取以 2 为底的对数得

$$k - 1 \le \log_2 n < k \implies k = \lfloor \log_2 n \rfloor + 1$$

$$T(n) = \sum_{r=1}^{\lfloor \log_2 n \rfloor + 1} 1 = O(\log n)$$

关键操作：把「循环变量的变化规律」翻译成等式或不等式，解出 k 与 n 的关系。该方法可推广到一切「每次变化固定倍数」的循环。

③ **2011-01** 真题：起点非 **1** 的对数阶

```
x = 2;
while (x < n / 2)
    x = 2 * x;
```

变量 x 初值为 2，每轮翻倍。设循环体执行 k 次后退出，此时 $x = 2 \cdot 2^k = 2^{k+1}$ 首次不小于 $n/2$，即

$$2^{k+1} \ge \frac{n}{2} \implies k + 1 \ge \log_2 n - 1 \implies k \approx \log_2 n - 2$$

所以循环次数与 $\log_2 n$ 同阶，$T(n) = O(\log n)$，选 A。

2011-01 的两个坑

- 初值 2 和步长 2 合在一起使 $x = 2^{k+1}$ 而不是 2^k，但这只影响常数，不影响阶。

- 上界是 $n/2$ 而不是 n，$\log_2(n/2) = \log_2 n - 1$，同样只差常数。判断阶时这些细节都可以丢，但要能写出等式说明它们确实只差常数。
- 也有把条件误读成「$x < n$」而算出别样的次数，进而错选 B；只要抓住「每轮 $\times 2$」就必然是对数级。

④ **2014-01** 真题：内层与外层无关的嵌套

```
count = 0;
for (k = 1; k <= n; k *= 2)
    for (j = 1; j <= n; j++)
        count++;
```

外层：k 取 $1, 2, 4, \ldots, 2^r$，满足 $2^r \le n < 2^{r+1}$，执行 $r+1 = \lfloor \log_2 n \rfloor + 1$ 次。内层：j 从 1 到 n，每次固定执行 n 次。内层的次数与外层变量 k 完全无关，因此可以直接相乘：

$$T(n) = \sum_{r=0}^{\lfloor \log_2 n \rfloor} \sum_{j=1}^{n} 1 = \sum_{r=0}^{\lfloor \log_2 n \rfloor} n = n(\lfloor \log_2 n \rfloor + 1) = O(n \log n)$$

选 C。

⑤ 方根阶：条件写在循环变量上

```
int count = 0;
for (int i = 0; i * i < n; i++)
    count++;
```

设循环体执行 k 次后退出，$i = k$ 首次满足 $k^2 \ge n$，故 $k = \lceil \sqrt{n} \rceil$，

$$T(n) = O(\sqrt{n})$$

⑥ **2019-01** 真题：条件含平方的 **while**

```
x = 0;
while (n >= (x + 1) * (x + 1))
    x = x + 1;
```

x 从 0 开始每轮只加 1，条件成立等价于 $(x+1)^2 \le n$ 即 $x \le \sqrt{n} - 1$。因此 x 能取到 $0, 1, \ldots, \lfloor \sqrt{n} \rfloor - 1$，循环体执行 $\lfloor \sqrt{n} \rfloor$ 次，

$$T(n) = O(\sqrt{n})$$

选 B。

「条件里出现了平方」不等于 $O(n^2)$

2019-01 的迷惑项就是 $O(n^2)$。判断依据是循环体真正执行了多少次，即循环变量达到退出条件所需的迭代次数，而不是条件表达式的复杂度。条件中的乘法、比较都是常数时间操作。

⑦ **2017-01** 真题：累加类 **while**（需解方程）

```
int i = 0, sum = 0;
while (sum < n)
    sum += ++i;
return i;
```

`sum += ++i` 的含义是先把 i 加 1，再把新的 i 累加到 sum。因此每轮至少执行一次（`++i` 与加法都是常数时间），且 sum 每轮增加的量依次为 $1, 2, 3, \ldots$。

设循环体执行 k 次后 sum 首次达到或超过 n，此时

$$sum = 1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2} \geq n$$

解一元二次不等式：

$$k^2 + k - 2n \geq 0 \implies k \geq \frac{-1 + \sqrt{1 + 8n}}{2}$$

去掉常数与系数得

$$T(n) = O(\sqrt{n})$$

选 B。

不要把 sum 每轮的增加量当成固定值

本题的最大误区是认为 sum 每轮加 1，于是得出循环 n 次、复杂度 $O(n)$。实际上 sum 每轮增量依次为 $1, 2, 3, \ldots, k$，增长速度越来越快，所以根本不需要 n 轮——大约 $\sqrt{2n}$ 轮就够了。凡是遇到「累加」型循环，先写出累加量的通项，再列方程。

4.（4）嵌套循环的两种情形

嵌套循环是失分重灾区，判定的第一件事是：内层循环的次数是否依赖外层变量？

情形 **A**：内外层无关 → 直接相乘

```
for (int i = 1; i <= n; i++)          /* n 次 */
    for (int j = 1; j <= n; j++)      /* n 次，与 i 无关 */
        count++;
```

$$T(n) = \sum_{i=1}^{n} \sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = n^2 = O(n^2)$$

若两层都是倍数型：`for(i=1;i<n;i*=2) for(j=1;j<n;j*=2)`，两层各 $\log n$ 次，得 $O(\log^2 n)$。若两层都是方根型：`for(i=0;i*i<n;i++) for(j=0;j*j<n;j++)`，两层各 $\sqrt{n}$ 次，得 $O(\sqrt{n} \cdot \sqrt{n}) = O(n)$。

一般形式：若外层终止条件为 $f(i) \geq n$，内层为 $g(j) \geq n$ 且 f, g 单调，则

$$T(n) = O\big(f^{-1}(n)\big) \cdot O\big(g^{-1}(n)\big)$$

情形 **B**：内层次数依赖外层 → 求和，不能乘

```
int count = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < i; j++)
        count++;
```

内层执行 i 次，于是

$$T(n)=\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}1=\sum_{i=0}^{n-1}i=0+1+2+\cdots+(n-1)=\frac{n(n-1)}{2}=O(n^2)$$

虽然结果仍是 $O(n^2)$，但系数是 $1/2$ 而不是 1，说明必须逐轮求和。
若内层是方根型且依赖外层：

```
for (int i = 1; i * i <= n; i++)
    for (int j = 1; j <= i; j++)
        count++;
```

外层 i 从 1 到 $\lfloor\sqrt{n}\rfloor$，内层 j 从 1 到 i：

$$T(n)=\sum_{i=1}^{\lfloor\sqrt{n}\rfloor}\sum_{j=1}^{i}1=\sum_{i=1}^{\lfloor\sqrt{n}\rfloor}i=\frac{\sqrt{n}(\sqrt{n}+1)}{2}=O(n)$$

⑧ **2022-01** 真题：外层倍数型 + 内层上界为外层变量

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

外层 i 依次取 $1,2,4,\ldots,2^{k-1}$，其中 $2^{k-1}<n\le 2^k$；内层执行次数等于当前的 i。各轮内层次数依次为

$$1,\ 2,\ 4,\ \ldots,\ 2^{k-1}$$

这是一个等比数列，求和：

$$T(n)=\sum_{i\in\{1,2,4,\ldots,2^{k-1}\}}i=1+2+4+\cdots+2^{k-1}=2^k-1$$

由 $2^{k-1}<n\le 2^k$ 得 $2^k<2n$，所以

$$T(n)<2n-1=O(n)$$

选 B。

2022-01 的致命误区：把「外层 $\log n$ × 内层最大 n」相乘

这样会得到 $O(n\log n)$，是本题最强的干扰项 C。错因是：内层并不是每轮都执行 n 次，只有最后几轮才接近 n 次，前面几轮分别只有 $1,2,4,\ldots$ 次。正确做法是逐轮累加，得到等比数列之和与 n 同阶。

判定口诀：内层上界若是外层变量的指数增长结果，总和由最后一项主导，量级是「最大那一项的量级」，即 $O(n)$，而不是「层数 × 最大值」。

可以用求和上限验证：$\sum_{i=1}^{m} 2^{i-1} = 2^m - 1$，其中 $m = \log_2 n$，故和为 $2n-1$。

⑨ 内层倍数型 + 外层线性（出现 $\log(n!)$）

```
int count = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j *= 2)
        count++;
```

内层 j 从 1 每次乘 2 直到超过 i，执行次数为 $\lfloor \log_2 i \rfloor + 1$，于是

$$T(n) = \sum_{i=1}^{n} \left(\lfloor \log_2 i \rfloor + 1 \right) = \log_2 1 + \log_2 2 + \cdots + \log_2 n + n = \log_2(n!) + n$$

利用斯特林公式或简单放缩可得 $\log_2(n!) = \Theta(n\log n)$，因此

$$T(n) = O(n\log n)$$

需要记住这条常见结论：$\log(n!) = \Theta(n\log n)$。它在归并排序、堆排序、比较类排序下界等分析里反复出现。

⑩ 外层倍数型、内层外层相关的倍数型

```
int count = 0;
for (int i = 1; i <= n; i *= 2)
    for (int j = 1; j <= i; j *= 2)
        count++;
```

外层第 r 轮（$r = 1, 2, \ldots, \lfloor \log_2 n \rfloor + 1$）时 i 约为 2^{r-1}，内层次数约为 $\log_2 i + 1 = r$。故

$$T(n) = \sum_{r=1}^{\lfloor \log_2 n \rfloor + 1} r = \frac{m(m+1)}{2}, \quad m = \lfloor \log_2 n \rfloor + 1$$

$$T(n) = O(\log^2 n)$$

这类循环是「非线性的嵌套非线性变化」，两次都按倍数收敛，所以是 $O(\log^2 n)$。

⑪ 三层线性嵌套：求和公式

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= i; j++)
        for (int k = 1; k <= j; k++)
            count++;
```

$$\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=1}^{j}1=\sum_{i=1}^{n}\sum_{j=1}^{i}j=\sum_{i=1}^{n}\frac{i(i+1)}{2}=\frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6}+\frac{n(n+1)}{2}\right)=\frac{n(n+1)(n+2)}{6}=O(n^3)$$

推导中用到两个基本求和公式，建议直接记住并会代数字验证：

$$\sum_{i=1}^{n}i=\frac{n(n+1)}{2},\qquad\sum_{i=1}^{n}i^2=\frac{n(n+1)(2n+1)}{6}$$

再补充一个「方根套方根」的例子：

```
for (int i = 0; i * i < n; i++)
    for (int j = 0; j * j < i; j++)
        count++;
```

内层对每个 i 执行约 $\sqrt{i}$ 次，外层 i 到 $\sqrt{n}$：

$$T(n)=\sum_{i=1}^{\sqrt{n}}\sqrt{i}\approx\int_{0}^{\sqrt{n}}\sqrt{x}\,dx=\frac{2}{3}(\sqrt{n})^{3/2}=\frac{2}{3}n^{3/4}=O(n^{3/4})$$

虽然 $O(n^{3/4})$ 在统考中极少出现，但这个例子很好地说明：只要内外层不独立，就必须老老实实求和。

5.（*5*）递归式的时间复杂度：展开法、递归树与主定理

递归算法的时间复杂度要用递归式（递推式）描述，再用下面四种方法之一求解。统考主要考展开法（也叫迭代法），递归树与主定理属于扩展但能极大提速。

方法一：展开法（迭代展开）

思路：反复把右边的 $T(\cdot)$ 用它自己的定义替换，直到出现边界条件 $T(1)$，然后把层数与每层代价相乘。

以 2012-01 真题（求 n 的阶乘）为例：

```
int fact(int n) {
    if (n <= 1) return 1;
    return n * fact(n - 1);
}
```

递归式为 $T(n)=T(n-1)+O(1)$——注意每层只做一次乘法、一次比较，都是常数时间，所以是 $O(1)$ 而不是 $O(n)$。逐层展开：

$$T(n)=T(n-1)+O(1)=T(n-2)+2O(1)=\cdots=T(1)+n\cdot O(1)=O(n)$$

选 B，辅助空间复杂度为 $O(n)$（递归栈深度为 n）。

同理：

- $T(n)=T(n-1)+n$：展开得 $n+(n-1)+\cdots+1=O(n^2)$。

- $T(n)=T(n/2)+1$（二分查找、对数阶递归）：每次规模减半，共 $\log_2 n+1$ 层，$T(n)=O(\log n)$。
- $T(n)=T(n-1)+T(n-2)$（朴素斐波那契）：呈指数增长，$T(n)=O(2^n)$；加记忆化后为 $O(n)$。

方法二：递归树法

思路：把递归过程画成一棵树。每个结点代表一次子调用的代价，每层所有结点的代价之和是这一层的代价；把各层代价相加即得总代价。只要每层的代价和构成等比数列，求和工作就非常简单。

例：$T(n)=2T(n/2)+n$（二路归并排序的时间递推式）

层次	结点数	每个结点代价	该层代价之和
第 0 层（根）	1	n	n
第 1 层	2	$n/2$	$2\cdot n/2=n$
第 2 层	4	$n/4$	$4\cdot n/4=n$
…	…	…	…
第 j 层	2^j	$n/2^j$	n

规模每次减半，到叶子规模为 1 时停止，共 $\log_2 n+1$ 层。每层代价之和恒为 n，因此

$$T(n)=\underbrace{n+n+\cdots+n}_{\log_2 n+1\ \text{个}}=n(\log_2 n+1)=O(n\log n)$$

例：$T(n)=3T(n/4)+\Theta(n^2)$（主定理的入门例题，答案 $O(n^2)$）

第 j 层有 3^j 个结点，每个结点的分解代价为 $(n/4^j)^2=n^2/16^j$，该层代价之和为

$$3^j\cdot\frac{n^2}{16^j}=n^2\left(\frac{3}{16}\right)^j$$

这是一个公比为 $3/16<1$ 的递减等比数列，故

$$T(n)=n^2\sum_{j\ge 0}\left(\frac{3}{16}\right)^j=n^2\cdot\frac{1}{1-3/16}=\frac{16}{13}n^2=O(n^2)$$

几何级数递减 $\Rightarrow$ 总代价由根结点主导，这正是主定理第三种情况的直观来源。

方法三：代入验证法（猜答案再证明）

先猜出 $T(n)=O(f(n))$，再把归纳假设代回递归式验证 $T(n)\le c\,f(n)$ 对某常数 c 成立。统考一般不要求写完整归纳证明，但用代入法快速验证选项非常有效：把四个选项分别代入递归式看哪个能配平，往往比正推更快。

方法四：主定理（**Master Theorem**）

适用于形如

$$T(n)=a\,T\!\left(\frac{n}{b}\right)+f(n),\qquad a\ge 1,\ b>1,\ f(n)\ \text{渐进正函数}$$

的递归式。注意约束：子问题规模必须等比缩小（n/b 而不是 $n-b$），且各子问题规模相同。方法：设临界指数 $n^{\log_b a}$，把它与 $f(n)$ 比较，分三种情况。

情况	条件	解
情况一	$f(n)=O\big(n^{\log_b a-\varepsilon}\big)$，某 $\varepsilon>0$	$T(n)=\Theta\big(n^{\log_b a}\big)$
情况二	$f(n)=\Theta\big(n^{\log_b a}\big)$	$T(n)=\Theta\big(n^{\log_b a}\log n\big)$
情况三	$f(n)=\Omega\big(n^{\log_b a+\varepsilon}\big)$，某 $\varepsilon>0$，且 $a\,f(n/b)\le c\,f(n)$ 对某 $c<1$ 与足够大的 n 成立	$T(n)=\Theta\big(f(n)\big)$

三种情况的直观含义：情况一是分下去更重要（子问题数量多、叶子层代价最大），情况二是上下一样重（每层代价均衡），情况三是合起来更重要（根结点代价最大）。

情况三里的附加条件 $a\,f(n/b)\le c\,f(n)$ 称为正则性条件，它是技术性要求（保证 f 不会增长得太野），在统考里一般不用自己验证。

再理解一个细节：递归树的叶子数为 $a^{\log_b n}=n^{\log_b a}$，这就是临界指数 $n^{\log_b a}$ 的由来——它正是「分下去」这一侧的总体量级。

主定理用法示例：

- $T(n)=2T(n/2)+n$：$a=2,b=2$，$n^{\log_2 2}=n^1=n$，与 $f(n)=n$ 同阶 $\Rightarrow$ 情况二，$T(n)=\Theta(n\log n)$。
- $T(n)=T(n/2)+1$：$a=1,b=2$，$n^{\log_2 1}=n^0=1$，与 $f(n)=1$ 同阶 $\Rightarrow$ 情况二，$T(n)=\Theta(\log n)$。
- $T(n)=3T(n/4)+n^2$：$a=3,b=4$，$n^{\log_4 3}\approx n^{0.792}$，$f(n)=n^2$ 渐进更大 $\Rightarrow$ 情况三，$T(n)=\Theta(n^2)$（与选项 A 一致）。
- $T(n)=2T(n/2)+n^2$：$a=2,b=2$，$n^{\log_2 2}=n$，$f(n)=n^2$ 渐进更大 $\Rightarrow$ 情况三，$T(n)=\Theta(n^2)$。

完整案例：归并排序为什么是 $O(n\log n)$

以二路归并排序为例走一遍全过程。算法把长度为 n 的数组从中间一分为二，分别递归排序，再把两个有序子数组合并成一个有序数组。合并两个长度约为 $n/2$ 的有序段，需要沿着两个子数组各走一遍，代价是 $\Theta(n)$；子问题有两个，规模各为 $n/2$。因此

$$T(n)=2T\left(\frac{n}{2}\right)+n,\qquad T(1)=\Theta(1)$$

用递归树看：第 0 层合并代价 n，第 1 层两个结点各自合并 $n/2$、合计 n，第 2 层四个结点各自 $n/4$、合计 n……每一层的「分」与「合」代价之和恒为 n，共 $\log_2 n+1$ 层，故

$$T(n)=n(\log_2 n+1)=O(n\log n)$$

这条结论值得单独记住：当每一层的代价之和相同时，总复杂度 = 层数 × 单层代价。这也是「分治法复杂度 = 递归树高度 × 每层代价」这一通式的来源。此时若改用主定理，$a=b=2$，$n^{\log_b a}=n$ 与 $f(n)=n$ 同阶，落入情况二，结果同样是 $O(n\log n)$，两条路径互相印证。

从空间角度看，归并排序需要长度为 n 的辅助数组，故辅助空间 $O(n)$；递归栈深度为 $\log_2 n$，栈帧空间 $O(\log n)$，总空间复杂度取较大者 $O(n)$。这也说明「时间复杂度最优」并不意味着「空间复杂度也最优」——归并排序时间稳定为 $O(n\log n)$，代价是 $O(n)$ 的额外空间；而堆排序时间同为 $O(n\log n)$，额外空间只有 $O(1)$。

主定理的盲区：情况二与情况三之间存在空隙

并非所有形如 $T(n)=aT(n/b)+f(n)$ 的递归式都能套主定理。经典反例是

$$T(n) = 2T\left(\frac{n}{2}\right) + n\log n$$

这里 $n^{\log_2 2} = n$，而 $f(n) = n\log n$ 既不是 $O(n^{1-\varepsilon})$（不满足情况一），也不与 n 同阶（不满足情况二），又不满足 $f(n) = \Omega(n^{1+\varepsilon})$（不满足情况三，因为它只比 n 多一个 log 因子，不是多项式级别的更大）。这个递归式落在情况二与情况三之间的空隙里，主定理失效。

此时改用递归树：每层代价之和都是 $n\log$ 级别的递减量，共 $\log n$ 层，结果为 $T(n) = O(n\log^2 n)$。

结论：主定理是提速工具，不是万能公式。考试若不确定，用递归树或展开法从定义出发最稳。

主定理速用四步

1. 认出形式 $T(n) = aT(n/b) + f(n)$，读出 a、b、$f(n)$。
2. 算出临界指数 $n^{\log_b a}$。
3. 比较 $n^{\log_b a}$ 与 $f(n)$：谁大取谁；同阶则乘一个 $\log n$。
4. 若两者之间只差对数因子（如 n 与 $n\log n$、n 与 $n/\log n$），主定理失效，回退到递归树。

6.（6）均摊分析的一个经典例子：堆化建堆为什么是 $O(n)$

这是「估算不能只看乘起来」的最好例子，也是 2022 年真题（求最小的 10 个数）背后必须理解的分析。

把 n 个元素原地调整成一个堆（从最后一个非叶结点起，逐个向下调整），若粗略估算：需要调整约 $n/2$ 个结点，每个结点最多下沉树高 $O(\log n)$ 层，相乘得到 $O(n\log n)$。但这个估算偏大，因为它忽略了「底层结点远多于顶层结点」这一事实。

设完美二叉树高度为 h（根的高度为 h，叶结点高度为 0）。一个结点下沉的最大层数恰好等于它的高度。第 i 层有 2^{h-i} 个结点、每个结点高度为 i，故总下沉次数为

$$T(h) = \sum_{i=0}^{h} 2^{h-i}\cdot i = 2^0\cdot h + 2^1(h-1) + 2^2(h-2) + \cdots + 2^{h-1}\cdot 1$$

两边乘 2 后与原式错位相减：

$$2T(h) = 2^1 h + 2^2(h-1) + \cdots + 2^h\cdot 1$$

$$2T(h) - T(h) = T(h) = 2^h + 2^{h-1} + \cdots + 2^1 - h = 2^{h+1} - h - 2 = O(2^h)$$

又完美二叉树结点数满足 $n = 2^{h+1} - 1$，即 $2^h = \Theta(n)$，于是

$$T(h) = O(n)$$

所以输入 n 个元素并建堆的时间复杂度是 $O(n)$，而不是 $O(n\log n)$。这个例子告诉我们两条经验：其一，结点数量按层呈指数分布时，低价操作数量巨大，加权求和后由浅层主导；其二，遇到乘积估算与真实求和矛盾时，永远相信求和。

同理，堆的常用操作复杂度为：访问堆顶 $O(1)$、元素入堆 $O(\log n)$、堆顶出堆 $O(\log n)$；用堆求解「求前 k 小」问题的时间复杂度为 $O(n\log k)$、空间复杂度为 $O(k)$（维护一个大小为 k 的大顶堆，遍历数组一次）。

7.（7）空间复杂度

定义：算法运行过程中所需辅助空间随问题规模增长的趋势，同样用渐进记号表示。

算法运行中使用的内存空间分三部分：

1. 输入空间：存储输入数据。
2. 暂存空间：存储运行中的变量、对象、函数上下文等。可再细分为暂存数据（常量、变量、对象）、栈帧空间（每次函数调用在栈顶创建的上下文，函数返回即释放）、指令空间（编译后的程序指令，通常忽略不计）。
3. 输出空间：存储输出数据。

统考口径：空间复杂度一般只统计辅助空间，即「输入空间之外」的那部分，通常记为「暂存数据 + 输出数据 + 栈帧空间」，输入本身不计入。多数教材与真题按「除输入和程序本身之外所需的额外空间」理解，遇到题目明确给了口径时以题面为准。

常见类型从低到高：

$$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$$

- $O(1)$：只用了若干常量、变量或固定长度的数组。循环中每次新建的变量在进入下一轮后释放，不累积，仍算 $O(1)$。这类只需 $O(1)$ 辅助空间的算法称为就地算法（**in-place algorithm**）。
- $O(\log n)$：递归深度为对数级，如二分查找递归版、归并排序的栈帧空间。
- $O(n)$：长度为 n 的辅助数组、链表、散列表；或递归深度为 n 的递归（如阶乘递归、单链表递归逆置）。
- $O(n^2)$：二维数组；或「每层递归各开一个长度递减的数组」这类嵌套占用 $n+(n-1)+\cdots+1$ 的情形。
- $O(2^n)$：如建立高度为 n 的满二叉树，结点数 2^n-1。

空间复杂度的两个高频误区

1. 只关注最差空间复杂度。原因是内存是硬性要求，必须保证所有输入下都有足够空间预留。「最差」有两层含义：最差输入分布（如 $n>10$ 才开数组，则最差是 $O(n)$）与运行过程中的峰值内存（某一步同时占用的最大空间）。
2. 递归与循环的栈帧差别：循环里调用 n 次 `func()`，每次都返回并释放栈帧，空间仍是 $O(1)$；而递归 `recur(n-1)` 会同时存在 n 个未返回的调用，占据 $O(n)$ 栈帧空间。看到递归一定要算栈深度。

8.（8）最好、最坏与平均时间复杂度

上一节讨论的都是「执行次数随 n 增长」这一层面的问题，而当算法的执行次数还取决于输入的具体内容（而不只是规模）时，就必须分三种情况讨论。这也是统考主观题的常见小问：「说明你所设计算法的时间复杂度」——如果算法的代价与数据初始状态有关，只回答一个阶是不完整的。

- 最坏时间复杂度：在最坏输入下（执行次数最多）的复杂度，给出运行时间的上界，是最常用的分析指标，保证程序不会更慢。
- 最好时间复杂度：在最好输入下的复杂度，给出下界，参考价值最小。
- 平均时间复杂度：在所有可能输入等概率出现的前提下，算法执行次数的加权平均值，即期望复杂度。

先看一个「逐语句分析」的完整例子，体会如何从代码走到 $T(n)$：

```
void fun(int a[], int n, int s) {
    int i, j, t;
    for (i = s; i < n - 1; i++) {          /* 语句 1: 外层循环 */
        for (j = s; j < n - i - 1; j++) { /* 语句 2: 内层循环 */
            if (a[j] > a[j + 1]) {         /* 语句 3: 比较 */
                t = a[j];                  /* 语句 4: 交换，3 次赋值 */
                a[j] = a[j + 1];
                a[j + 1] = t;
            }
        }
    }
}
```

这是一段从下标 s 开始、对后缀做起泡排序的代码。设参与排序的元素个数为 $m = n - s$，则语句 1 执行 $m - 1$ 次；语句 2 在每轮外层中执行 $m - i - 1$ 次（$i = 1, 2, \ldots, m - 1$），合计

$$\sum_{i=1}^{m-1}(m - i) = \frac{m(m-1)}{2}$$

次；语句 3 与语句 2 同频度；语句 4 只在发生交换时执行。最坏情况（初始为升序，每对都要交换）语句 4 执行 $\frac{m(m-1)}{2}$ 次，最好情况（已降序）为 0 次。于是最坏时间复杂度为

$$T(n) = O\left(\frac{m(m-1)}{2}\right) = O(m^2) = O\big((n-s)^2\big)$$

这段分析给出两点经验。其一，先找准「规模」到底是哪个量：这里真正决定代价的是参与排序的元素个数 $m = n - s$，若 $s = n/2$，参与排序的只剩一半，代价降为 $O(n^2/4)$——阶不变、常数减半，所以不能因为 s 变了就换一个阶。其二，「次数最少」不等于「阶更低」：起泡排序即使输入已经有序，比较次数仍是 $\frac{m(m-1)}{2}$ 次，只是交换为 0 次，最好复杂度依然是 $O(m^2)$——这正是起泡排序必须加「本趟无交换则提前结束」标志位才能把最好情况压到 $O(n)$ 的原因。

由此总结出四条分析惯例：

1. 先写出 $T(n)$ 的表达式（求和式或递推式），再化简出阶。主观题要保留这一步。
2. 最坏情况是默认口径。题目只说「时间复杂度」而未加限定，通常按最坏情况回答；若材料同时给出最好与平均，则三者都要交代。
3. 平均复杂度必须声明概率假设。统考默认「所有可能输入等概率」，例如顺序查找默认每个位置被查到的概率都是 $1/n$。
4. 三者的阶可能相同也可能不同。起泡排序三者都是 $O(n^2)$；快速排序平均 $O(n\log n)$、最坏 $O(n^2)$，差异的关键在枢轴（中轴）的选取与初始序列是否有序。

以顺序查找为例把三种情况算全：在长度为 n 的顺序表中查找给定元素（假定查找成功且等概率）。

- 最好情况：目标在第一个位置，比较 1 次，$O(1)$。
- 最坏情况：目标在最后一个位置或不存在，比较 n 次，$O(n)$。
- 平均情况：比较次数为

$$ASL = \sum_{i=1}^{n} p_i \cdot c_i = \frac{1}{n}(1 + 2 + \cdots + n) = \frac{n+1}{2} = O(n)$$

其中 $p_i = \frac{1}{n}$ 是等概率假设，$c_i = i$ 是命中第 i 个元素所需的比较次数。如果按 2023-01 的思路考察顺序表的其他操作：按位获取第 i 个元素只需一次地址计算，平均 $O(1)$；而插入、删除平均要移动约一半元素，平均 $O(n)$；有序表的折半查找平均 $O(\log n)$。

三者的关系与考法

- 最好 $\le$ 平均 $\le$ 最坏，且一般情况下三者不同阶就是信息（例如快速排序：平均 $O(n\log n)$，最坏 $O(n^2)$，最好 $O(n\log n)$）。
- 统考问「平均时间复杂度」时，如果没说概率分布，默认等概率。
- 如果题目给出的是「有序表」「顺序存储」这两个条件，要立刻想到折半查找 $O(\log n)$ 和随机存取 $O(1)$ 这两个可能答案。

9.（9）常见算法与操作复杂度对照表

这张表建议背下来，它是各章复杂度题的答案来源，也是主观题「说明你所设计算法的时间复杂度」的评分依据。

算法 / 操作	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
顺序查找（无序/有序顺序表）	$O(n)$	$O(n)$	$O(1)$	有序时失败查找可提前终止
折半查找（有序顺序表）	$O(\log n)$	$O(\log n)$	$O(1)$（迭代）/ $O(\log n)$（递归栈）	只适用于顺序存储且有序
分块查找（索引顺序查找）	$O(\sqrt{n})$	$O(\sqrt{n})$	$O(n)$（索引表）	块内顺序查找，块间用索引
直接插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	基本有序时最好可到 $O(n)$
折半插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	比较次数降至 $O(n\log n)$，移动次数不变
起泡排序（bubble sort）	$O(n^2)$	$O(n^2)$	$O(1)$	带标志位优化后最好 $O(n)$
简单选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	比较次数固定 $\frac{n(n-1)}{2}$
希尔排序（shell sort）	约 $O(n^{1.3})$	$O(n^2)$	$O(1)$	复杂度依赖增量序列，不作精确要求
快速排序	$O(n\log n)$	$O(n^2)$	$O(\log n)$（递归栈）	最坏出现在基本有序且取首元素为枢轴
堆排序	$O(n\log n)$	$O(n\log n)$	$O(1)$	建堆 $O(n)$，每次调整 $O(\log n)$
二路归并排序（merge sort）	$O(n\log n)$	$O(n\log n)$	$O(n)$	需要辅助数组
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(r)$	d 为位数，r 为基数，稳定
顺序表按下标取值	$O(1)$	$O(1)$	$O(1)$	随机存取特性
顺序表插入（平均）	$O(n)$	$O(n)$	$O(1)$	平均移动 $n/2$ 个元素
顺序表删除（平均）	$O(n)$	$O(n)$	$O(1)$	平均移动 $(n-1)/2$ 个元素
单链表按序号查找	$O(n)$	$O(n)$	$O(1)$	顺序存取特性
单链表在已知结点后插入	$O(1)$	$O(1)$	$O(1)$	不需移动元素
栈的入栈 / 出栈	$O(1)$	$O(1)$	$O(1)$	顺序栈与链栈相同
循环队列的入队 / 出队	$O(1)$	$O(1)$	$O(1)$	只改 `front`/`rear`

算法 / 操作	平均时间复杂度	最坏时间复杂度	空间复杂度	备注
二叉树 / 树 / 图的遍历	$O(n)$	$O(n)$	$O(n)$（栈或队列）	图按邻接表为 $O(V+E)$，邻接矩阵为 $O(V^2)$
求前 k 小元素（大顶堆法）	$O(n\log k)$	$O(n\log k)$	$O(k)$	2022 年真题的最优解法
输入 n 个元素并建堆	$O(n)$	$O(n)$	$O(1)$	自底向上堆化，不是 $O(n\log n)$
堆的插入 / 删除堆顶	$O(\log n)$	$O(\log n)$	$O(1)$	向上/向下调整一层为 $O(1)$

历年真题分布（数据结构第一章）

- **2011-01**：`while (x < n/2) x = 2*x;` 求复杂度，答案 $O(\log n)$。
- **2012-01**：阶乘递归 `fact(n)`，答案 $O(n)$（辅助空间 $O(n)$）。
- **2013-01**：两个升序链表（长 m、n）合并为降序链表的最坏时间复杂度，答案 $O(\max(m,n))$；因为每个结点最多被访问移动一次，总代价 $O(m+n)$，而 $m+n\le 2\max(m,n)$，故与 $\max(m,n)$ 同阶，不能选 $O(\min(m,n))$。
- **2014-01**：外层 `k *= 2`、内层 `j <= n`，答案 $O(n\log n)$。
- **2017-01**：`while (sum < n) sum += ++i;`，答案 $O(n^{1/2})$。
- **2019-01**：`while (n >= (x+1)*(x+1)) x++;`，答案 $O(n^{1/2})$。
- **2022-01**：外层 `i *= 2`、内层 `j < i`，答案 $O(n)$。
- **2023-01**：顺序存储有序表中平均时间复杂度为 $O(1)$ 的操作，答案「获取第 i 个元素」。

规律很明显：第一章第一题几乎固定是复杂度计算，且集中在「循环变量非线性变化」与「内外层相关」两类。把上面十个推导例子练熟，这类题就是稳拿分。

本章小结表

考纲条目	核心内容	必须记住的结论
（一）数据结构的基本概念	数据、数据元素（基本单位）、数据项（最小单位）、数据对象（性质相同元素的集合）、数据结构	数据结构 = 逻辑结构 + 存储结构 + 数据的运算
逻辑结构	线性（一对一）与非线性（一对多、多对多）；四分法为集合、线性、树形、图形	栈、队列、线性表逻辑结构相同，都是线性结构
存储结构	顺序（位置表关系）、链式（指针表关系）、索引（索引表）、散列（函数算地址）	顺序存储支持随机存取；链式存储只能顺序存取
抽象数据类型	数学模型 + 定义在其上的一组操作；由数据对象、数据关系、基本操作构成，记为 (D,S,P)	ADT 与实现无关，只看「能做什么」，不看「怎么实现」
（二）算法的基本概念	算法是对特定问题求解步骤的描述，是指令的有限序列	算法 $\neq$ 程序；程序可以不满足有穷性
算法的五个特性	有穷性、确定性、可行性、输入、输出	输入可以是零个，输出必须至少一个
好算法的目标	正确性、可读性、健壮性、高效率与低存储量	正确性最重要；时间与空间常互相冲突
时间复杂度	$T(n)=O(f(n))$ 表示增长趋势的上界	只保留最高阶项、去掉系数；log 底数不影响阶

考纲条目	核心内容	必须记住的结论
循环复杂度推导	定义法：设循环变量满足退出条件，解出迭代次数	独立的嵌套相乘，相关的嵌套求和
递归复杂度	展开法、递归树法、代入法、主定理	$T(n)=2T(n/2)+n=O(n\log n)$；主定理在 $f(n)$ 与 $n^{\log_b a}$ 只差对数因子时失效
空间复杂度	辅助空间随规模的增长趋势；只关注最差	递归要算栈深度：n 层递归为 $O(n)$
最好 / 最坏 / 平均	最坏给上界，平均按等概率加权	顺序查找平均比较次数 $\frac{n+1}{2}=O(n)$

易错清单

1. 数据的最小单位是数据项，基本单位是数据元素。两者互换即为错项。
2. 数据对象是数据的子集，不能说「数据对象就是数据」。
3. 逻辑结构与存储结构不能混判。「链式存储是线性结构/非线性结构」的说法本身跑偏：链式说的是存储方式，线性/非线性说的是逻辑关系。
4. 顺序存储 $\Leftrightarrow$ 随机存取，链式存储 $\Leftrightarrow$ 顺序存取。不要写「链式存储可以随机存取」。
5. 索引存储不是顺序存储，它的数据区可以离散；散列存储的地址由散列函数算出，不是按逻辑顺序排布。
6. 算法与程序不等价：程序可以无限循环，算法必须有穷；算法可以没有输入但不能没有输出。
7. 五特性中没有「高效性」「可读性」。可读性、健壮性属于「好算法的目标」，不属于算法的特性，这是常见的混淆选项。
8. $O(1)$ 不是「执行一次」，而是「次数与 n 无关」；执行一千万次的固定循环仍是 $O(1)$。
9. $\log_2 n$ 与 $\log_{10} n$ 是同一复杂度阶，选项里同时出现 $O(\log_2 n)$ 与 $O(\log_{10} n)$ 时不能据此区分；但问「执行多少次」时必须写底数。
10. 判断循环次数要解方程，不能看条件里有什么运算。2019-01 的条件含平方，答案是 $O(\sqrt{n})$ 而不是 $O(n^2)$。
11. 累加型循环的增量不是常数。`sum += ++i` 的增量是 $1,2,3,\ldots$，方程为 $\frac{k(k+1)}{2}\geq n$，得 $O(\sqrt{n})$。
12. 内外层独立才能相乘。只要内层上界依赖外层变量，就必须逐轮求和，否则会把 2022-01 误判成 $O(n\log n)$。
13. 指数增长的内层，总和由最后一项主导。$1+2+4+\cdots+2^{k-1}=2^k-1$，与 n 同阶，不是「层数 $\times$ 最大值」。
14. $\log(n!)=\Theta(n\log n)$，见到「内层为 $\log i$、外层遍历 i 到 n」的求和要立刻反应过来。
15. 主定理不是万能的。$f(n)$ 与 $n^{\log_b a}$ 只差一个对数因子时（如 $T(n)=2T(n/2)+n\log n$）落在情况二与三的空隙中，必须改用递归树，答案为 $O(n\log^2 n)$。
16. 主定理要求子问题规模等比缩小。像 $T(n)=T(n-1)+O(1)$ 这种线性递减的递归式不满足条件，只能用展开法。
17. 空间复杂度要算递归栈深度。循环调用 n 次函数是 $O(1)$，递归 n 层是 $O(n)$。
18. 建堆是 $O(n)$，不是 $O(n\log n)$；只有「依次插入建堆」才是 $O(n\log n)$。两者要分清。
19. 平均复杂度默认等概率，且注意区分「查找成功」与「查找成功与失败都计入」两种口径——顺序查找失败时的比较次数是 n（或 $n+1$，取决于哨兵设置），会改变平均值。
20. 写主观题时要写规则：题目问「说明时间复杂度」时，必须给出 $T(n)$ 的表达式或求和式再给

出阶，只写一个 $O(n \log n)$ 容易被扣过程分。

第二章线性表

本章考纲要求

二、线性表（一）线性表的基本概念（二）线性表的实现

1. 顺序存储
2. 链式存储

（三）线性表的应用

[考查目标]

1. 掌握数据结构的基本概念、基本原理和基本方法。
2. 掌握数据的逻辑结构、存储结构及基本操作的实现，能够对算法进行基本的时间复杂度与空间复杂度的分析。
3. 能够运用数据结构的基本原理和方法进行问题的分析与求解，具备采用 C 或 C++ 语言设计与实现算法的能力。

本章是数据结构部分的第一道「大题」来源。统考的 41 题（算法设计题，15 分左右）在 2009、2010、2011、2012、2013、2015、2018、2019、2020 这些年份都出在线性表上，选择题也几乎年年涉及。因此本章的写法是「概念讲透 + 代码模板背熟」，而不是只记结论。

一、线性表的基本概念

（一）线性表的定义与逻辑特征

线性表（**Linear List**）是具有相同数据类型的 n（$n \geq 0$）个数据元素的有限序列，通常记作

$$L = (a_1, a_2, \cdots, a_i, \cdots, a_n)$$

其中 a_1 称为表头元素，a_n 称为表尾元素；i 是元素 a_i 的位序（从 1 开始数），n 是表长。当 $n = 0$ 时称为空表。

一句话把握它的逻辑特征：

线性表的四条逻辑特征

1. 有穷性：元素个数 n 是有限的。
2. 同类型：所有元素属于同一数据类型（每个元素占用的存储空间相同，这一点是顺序存储能随机存取的前提）。
3. 有序性：元素之间有顺序关系。除第一个元素外，每个元素有且仅有一个直接前驱；除最后一个元素外，每个元素有且仅有一个直接后继。
4. 一对一：a_1 无前驱，a_n 无后继，其余元素恰好一前一后。

线性表是一种线性结构，它的逻辑结构只有「一个前驱一个后继」这一种形态，与它采用什么存储方式无关。同一个线性表既可以顺序存储（顺序表），也可以链式存储（链表），二者的逻辑结构相同、存储结构不同，因而基本操作的实现方式与效率不同。这句话是本章所有对比题的题眼。

从「按逻辑顺序排列」这一点再往下推，可以得到两个必须记牢的推论。第一，线性表中的每个元素都有一个确定的位置编号，这个编号就是位序，位序是由逻辑关系决定的，不随存储方式改变：同一个线性表无论是用数组还是用链表存，a_3 永远是 a_4 的直接前驱。第二，任意两个元素之间的前后关系由它们在序列中的相对位置决定，因此「插入一个元素」在逻辑上的含义是「使原来的第 i 个元素及其后所有元素的位置编号都加一」，在「删除一个元素」时则是减一。这个逻辑语义，正是顺序存储必须搬移元素、而链式存储只需改指针的根本原因——顺序存储把「位置编号」直接编码成了「物理地址的偏移量」，一旦编号变化，物理位置就必须跟着变；链式存储把「前后关系」编码成了「指针」，编号变化时只要改指针，物理位置完全不用动。

再补充两个常考的相邻概念，避免混淆。

- 线性表与数组：数组是存储结构层面的概念（一段连续的、可随机访问的存储空间），线性表是逻辑结构层面的概念。数组可以存线性表，也可以存矩阵、堆、完全二叉树；线性表可以用数组存，也可以用链表存。因此「数组就是线性表」「线性表就是数组」都不成立。
- 线性表与集合：集合中的元素只要求「互异」，元素之间没有顺序关系；线性表要求元素之间有严格的前后次序。所以线性表不是集合，也不能说「线性表是一种特殊的集合」。
- 线性表与串：串是内容受限的线性表（限定数据元素为字符），本质上是线性表的特例，它的存储和操作都可以沿用本章的方法，只是多了模式匹配这类与「内容」有关的操作。本手册把串的内容放在第三部分展开。

「线性表」不等于「顺序表」

常见错误是把「线性表」直接当成数组。线性表是逻辑结构（一对一的有序序列），顺序表和链表是它的两种存储结构。判断题里出现「线性表只能用顺序存储实现」「线性表的元素必须连续存放」一律是错的。

（二）线性表的基本操作

统考与教材通用的操作名（务必认清函数名与语义，选择题常在「返回位序还是返回值」上设陷阱）：

操作名	语义	返回值约定
`InitList(&L)`	初始化，构造一个空表	成功/失败
`DestroyList(&L)`	销毁表，释放全部空间	无
`ListInsert(&L, i, e)`	在位序 i 处插入元素 e，原第 i 个元素及其后元素依次后移	成功/失败
`ListDelete(&L, i, &e)`	删除位序 i 的元素，用 e 带回它的值	成功/失败
`LocateElem(L, e)`	按值查找，找第一个值为 e 的元素	返回其位序；失败返回 0 或 -1
`GetElem(L, i, &e)`	按位查找，取位序 i 的元素，用 e 带回	成功/失败
`ListLength(L)`	求表长	元素个数
`PrintList(L)`	按前后次序输出所有元素	无
`Empty(L)`	判空	真/假

位序与下标必须分清

位序从 **1** 开始，数组下标从 **0** 开始。位序 i 对应的数组下标是 $i-1$。插入的合法位序范围是 $1 \le i \le n+1$（可以有 $n+1$ 个插入位置，因为可以插到表尾之后）；删除的合法位序范围是 $1 \le i \le n$。这两个范围的

差别是最容易被扣分的细节。

这一部分怎么考

- 选择题：给一段操作语句序列判断其效果（如 2024-01 让判断 `q = p->next; p->next = q->next; q->next = h->next; h->next = q;` 的作用）。
- 选择题：给顺序表 / 单链表判断某操作的平均时间复杂度（如 2023-01 问哪个操作平均时间复杂度是 $O(1)$，答案是「获取第 i 个元素」）。
- 选择题：判断涉及移动元素的操作（如 2026-01 问表头插入、表头删除、表尾插入、表尾删除哪些必然产生元素移动，答案是表头插入与表头删除）。
- 算法设计题：见本章第三节。

在九种操作中，真正需要「设计思路」的只有四类：插入、删除、按位查找、按值查找。其余操作（初始化、销毁、求表长、判空、遍历）都是顺序扫描或一次性赋值，两种存储结构下的写法都很直接。因此本章后续所有分析都围绕这四类操作展开，考纲「（二）线性表的实现」实际上就是在讨论这四类操作在两种存储结构下的不同实现代价。

这里还要提醒一个语义细节：教材中 `ListInsert(&L, i, e)` 里的 i 是位序，而 `LocateElem(L, e)` 返回的也是位序，`GetElem(L, i, &e)` 接收的是位序、带回的是元素值。这三者对「位序 / 值」的取用方向相反，是选择题的常见设错点——把 `GetElem` 说成「返回位序」、把 `LocateElem` 说成「返回值」都是错的。

二、线性表的实现

（一）顺序存储

*1.*定义与两种分配方式

顺序表用一组地址连续的存储单元依次存放线性表的元素，逻辑上相邻的元素物理上也相邻。

静态分配：数组大小在编译时确定，一经定义不能改变。

```
#define MaxSize 100         // 顺序表的最大容量
typedef int ElemType;       // 元素类型

typedef struct {
    ElemType data[MaxSize]; // 静态数组存放元素
    int length;             // 当前表长（元素个数）
} SqList;

void InitList(SqList *L) {
    L->length = 0;          // 空表长度为 0，无须清空数组内容
}
```

动态分配：运行时用 `malloc` 申请空间，空间不够时再申请一块更大的并搬移数据。

```
#define InitSize 100

typedef struct {
    ElemType *data;  // 指向动态分配数组的指针
    int MaxSize;     // 数组的最大容量
    int length;      // 当前表长
} SeqList;
```

```
// 初始化：申请初始容量
void InitList(SeqList *L) {
    L->data = (ElemType *)malloc(sizeof(ElemType) * InitSize);
    L->MaxSize = InitSize;
    L->length = 0;
}

// 扩容：三步骤——申请新空间、搬移旧数据、释放旧空间
void IncreaseSize(SeqList *L, int len) {
    ElemType *p = L->data;
    L->data = (ElemType *)malloc(sizeof(ElemType) * (L->MaxSize + len));
    for (int i = 0; i < L->length; i++) {
        L->data[i] = p[i];          // 逐个搬移，不能整体赋值
    }
    L->MaxSize = L->MaxSize + len;
    free(p);                        // 释放原来那片空间
}
```

静态与动态分配的关键差别

静态分配一旦数组满，无法扩容，只能插入失败；动态分配可以在满时扩容，但扩容要搬移全部 n 个元素，时间代价 $O(n)$，而且必须「先申请、再搬移、最后释放旧空间」，顺序不能反。注意：动态分配虽然可以扩容，但它仍然是顺序存储，仍然支持随机访问。

关于扩容还有一个容易忽略的量化结论。若每次扩容只增加固定长度 `len`（即线性增长），插入 n 个元素的累计搬移代价是 $O(n^2)$；若每次扩容把容量加倍（倍增策略），第 i 次扩容搬移 $1+2+4+\cdots+2^{i-1}\approx 2^i$ 个元素，而此后至少还能再插入 2^i 个元素，把搬移代价均摊到每次插入上是常数，因此 n 次插入的总时间仍是 $O(n)$，单次插入的均摊时间是 $O(1)$。统考不要求推导均摊分析，但选择题里出现「动态分配的顺序表插入元素一定是 $O(n)$」这种说法时可以据此判断：均摊意义下是 $O(1)$，最坏情况（恰逢扩容）才是 $O(n)$。

再强调一个概念区分：「动态分配」和「动态存储结构」不是一回事。动态分配讲的是一块顺序空间如何申请和扩充，它仍然是顺序存储；链式存储才是真正的动态存储结构。把「动态数组」理解成链表是典型错误。

最后一点实现细节：用 `malloc` 申请到的空间里是未初始化的随机数据，但顺序表只靠 `length` 判断哪些位置有效，所以初始化时不必把整个数组清零，只把 `length` 置 0 即可（本手册给出的 `InitList` 就是这么写的）。反之，若题目明确要求「初始化时清空数组内容」，则要循环赋值。

*2.*随机访问特性与地址计算

设第一个元素的存储地址为 $LOC(a_1)$，每个元素占 d 个存储单元，则位序为 i 的元素地址为

$$LOC(a_i) = LOC(a_1) + (i-1)\times d$$

只要给出位序 i，一次乘法加法就能算出地址，访问时间与表长 n 无关，这就是随机访问（随机存取）。因此 `GetElem(L, i)` 的时间复杂度是 $O(1)$。

一句话记住

顺序表 ⇒ 随机访问 ⇒ 按位查找 $O(1)$；链表 ⇒ 顺序访问 ⇒ 按位查找 $O(n)$。

「随机访问」这个特性还带来一个常被忽略的考点：顺序表可以用折半查找。前提是表内元素有序（按关键字递增或递减排列），此时按值查找的时间可以从线性查找的 $O(n)$ 降到 $O(\log_2 n)$。这是链

表做不到的——链表的按位访问是 $O(n)$，无法在 $O(1)$ 时间内取到中间元素，所以折半查找只适用于顺序存储的有序表。这一点经常与「查找」那一章联动出题：题干说「有序表」「折半查找」，那它必然是顺序表。

反过来也要注意：顺序表不一定有序。顺序表只保证「逻辑相邻 = 物理相邻」，不保证值的大小关系。随机访问是顺序存储天然具备的，折半查找则是「顺序存储 + 有序」两个条件同时具备才有的能力，二者不能混为一谈。

*3.*插入与删除：移动次数推导

插入：在位序 i 插入，需要把原来的第 $i, i+1, \cdots, n$ 个元素整体后移一位，共移动 $n-i+1$ 个元素；再把 e 放到位序 i 处，表长加 1。

```
// 在位序 i 处插入元素 e，位序从 1 开始
bool ListInsert(SqList *L, int i, ElemType e) {
    if (i < 1 || i > L->length + 1) return false; // 位序越界
    if (L->length >= MaxSize)        return false; // 表满，不能插入
    for (int j = L->length; j >= i; j--) {         // 从后往前依次后移
        L->data[j] = L->data[j - 1];
    }
    L->data[i - 1] = e;   // 位序 i 对应下标 i-1
    L->length++;
    return true;
}
```

设 P_i 是在第 i 个位置插入的概率（$1 \le i \le n+1$），并假设在每个合法位置插入的概率相等，即 $P_i = \dfrac{1}{n+1}$，则插入一个元素所需移动元素的平均次数为

$$E_{\text{ins}} = \sum_{i=1}^{n+1} P_i (n-i+1) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{1}{n+1} \cdot \frac{n(n+1)}{2} = \frac{n}{2}$$

所以插入操作的平均时间复杂度为 $O(n)$，其中最好情况（插到表尾，$i = n+1$）移动 0 次，最坏情况（插到表头，$i = 1$）移动 n 次。

删除：删除位序 i 的元素，需要把第 $i+1, i+2, \cdots, n$ 个元素整体前移一位，共移动 $n-i$ 个元素，表长减 1。

```
// 删除位序 i 的元素，并用 e 带回其值；位序从 1 开始
bool ListDelete(SqList *L, int i, ElemType *e) {
    if (i < 1 || i > L->length) return false;   // 位序越界
    *e = L->data[i - 1];                        // 先保存被删元素
    for (int j = i; j < L->length; j++) {       // 从前往后依次前移
        L->data[j - 1] = L->data[j];
    }
    L->length--;
    return true;
}
```

同样假设 $P_i = \dfrac{1}{n}$（删除时只有 n 个合法位置），平均移动次数为

$$E_{\text{del}} = \sum_{i=1}^{n} P_i (n-i) = \frac{1}{n} \sum_{i=1}^{n} (n-i) = \frac{1}{n} \cdot \frac{n(n-1)}{2} = \frac{n-1}{2}$$

所以删除操作的平均时间复杂度也是 $O(n)$，最好情况（删表尾）移动 0 次，最坏情况（删表头）移动 $n-1$ 次。

顺序表三种操作的平均次数（必背）

操作	平均移动 / 比较次数	时间复杂度
按位查找 `GetElem`	0 次（直接算地址）	$O(1)$
插入 `ListInsert`	$\frac{n}{2}$ 次	$O(n)$
删除 `ListDelete`	$\frac{n-1}{2}$ 次	$O(n)$
按值查找 `LocateElem`	$\frac{n+1}{2}$ 次（查找成功）	$O(n)$

代一组数验证：$n=5$ 时，插入平均移动 $5/2=2.5$ 次。手工核一遍：6 个插入位置分别移动 $5,4,3,2,1,0$ 次，和是 15，除以 6 得 2.5，一致。删除平均移动 $(5-1)/2=2$ 次；5 个删除位置分别移动 $4,3,2,1,0$ 次，和是 10，除以 5 得 2，一致。

关于这两个公式，有三个必须澄清的点。

第一，公式里的「平均」是概率意义上的平均。推导时用的假设是「插入到每个合法位置的概率相等」且「删除每个位置的概率相等」。如果题目另外给了概率分布（例如「插到表尾的概率是插到其他位置的 2 倍」），就必须按题目给的 P_i 重新代入公式计算，不能直接套 $\frac{n}{2}$ 和 $\frac{n-1}{2}$。

第二，「移动次数」和「时间复杂度」是两个层次的东西。移动次数是精确的操作计数，时间复杂度是增长速度的量级。插入平均 $\frac{n}{2}$ 次、删除平均 $\frac{n-1}{2}$ 次，都是 $O(n)$。考试可能只问次数（填空或选择），也可能问复杂度，看清题干的问法。

第三，最好情况与最坏情况要记牢。插入：插在表尾之后（位序 $n+1$）移动 0 次，是最好；插在表头（位序 1）移动 n 次，是最坏。删除：删表尾（位序 n）移动 0 次，是最好；删表头（位序 1）移动 $n-1$ 次，是最坏。一种便于自查的记忆法是：「移动元素个数 = 需要被跨过的元素个数」——插入时跨过的是原来第 i 到第 n 个，共 $n-i+1$ 个；删除时跨过的是第 $i+1$ 到第 n 个，共 $n-i$ 个。

第四，循环里的移动方向不能反。插入时循环变量必须从大到小（`for (j = length; j >= i; j--)`），因为元素要向后挪，必须先把最后面的挪走，否则会覆盖还没搬的数据；删除时循环变量必须从小到大（`for (j = i; j < length; j++)`），因为元素要向前挪，必须先把最前面的挪走，否则同样会覆盖。这是写代码时最实际的一个坑。

第五，顺序表的插入删除有「移动区间的选择」技巧。如果题目要求「删除所有值为 x 的元素」，用逐个删除的办法每次都要搬移其后所有元素，最坏会退化到 $O(n^2)$；正确做法是用双指针一趟扫描（见第三节），把保留的元素一次填好，最后统一改表长，时间 $O(n)$。「批量删除要用双指针，不要循环调用单次删除」是统考算法题的重要得分意识。

按值查找的比较次数推导：设要查找的元素在位序 i 的概率为 $P_i=\frac{1}{n}$，找到它需要比较 i 次，则

$$ASL_{\text{成功}}=\sum_{i=1}^{n}P_i\cdot i=\frac{1}{n}\cdot\frac{n(n+1)}{2}=\frac{n+1}{2}$$

若要把查找失败也算进去（线性表的顺序查找，失败时比较 n 次后还要再比一次才能判失败），教材常给出 $ASL_{\text{失败}}=n+1$ 与 $ASL_{\text{总}}=\frac{n+1}{2}+n+1$ 之类的写法。统考考「平均查找长度」时若未说明，按查找成功计算，即 $(n+1)/2$；遇到明确要求算失败的题目，按题目给出的概率模型老老实实列式子。

*4.*顺序表的优缺点

- 优点
  1. 随机访问：按位查找 $O(1)$，这是顺序表最大的优势。
  2. 存储密度高：只存数据本身，不需要额外的指针开销（存储密度 $=1$）。
  3. 存储空间是一整块连续空间，对 CPU 缓存友好，实际运行速度快。
- 缺点
  1. 插入、删除平均要移动约一半的元素，平均时间复杂度 $O(n)$。
  2. 需要预先分配连续空间：静态分配可能溢出，动态分配扩容要整体搬移。
  3. 存储空间利用率受「连续」约束：内存里有很多碎片但拼不出一块够大的连续空间时，仍然申请失败。

关于「存储密度」

存储密度 $=\dfrac{\text{数据本身占用的存储量}}{\text{结点占用的存储总量}}$。顺序表为 1；单链表每个结点除数据域外还要存一个指针域，所以存储密度小于 1（严格小于 1，除非结点结构里数据域是空的）。存储密度高不等于效率高，它只反映「空间换结构信息」的比例。

「随机访问」和「存储密度」是顺序表的两张底牌，而它们都来自「一整块连续空间」这一个事实。正因为空间连续，地址才能用乘法直接算出来（随机访问）；也正因为空间连续且按顺序紧密排列，才没有任何空隙需要额外的结构信息去填充（存储密度为 1）。反过来，空间连续也带来了顺序表最大的软肋——「连续」是一个很硬的约束。插入删除要维持连续，就得搬移；预留空间要维持连续，就得整块申请，一旦内存中最大的空闲连续区小于所需容量，即使总的空闲空间足够大也申请不到。这三利三弊是同一个原因的不同侧面，理解了因果关系就不需要死记。

真题考法

2013 年应用题考过顺序存储结构与链式存储结构的 ASL 对比，2023-01 考过哪类操作平均 $O(1)$，2026-01 考过「哪些操作必然导致元素移动」。这类题的共同点是先判断题目描述的是顺序表还是链表——题干里的「存储空间有足够的空闲空间」「保持元素相对顺序」这类措辞通常指向顺序表。

2026-01 那类题怎么想

题目问「在保持表内元素顺序相对不变的情况下，哪些操作会必然导致移动次数」。分析路径是：先确定讨论对象是顺序表（因为「充足的连续空闲空间」和「元素移动」是顺序表的话题）；再检查四种操作是否需要给新元素腾位置或填补空隙。表头插入要给首位置腾空间，$a_1 \sim a_n$ 全部后移，必然移动；表头删除会让 $a_2 \sim a_n$ 全部前移填补，必然移动；表尾插入只需在 `data[length]` 处赋值，不移动任何已有元素；表尾删除只需把表长减 1，也不移动。答案是「表头插入、表头删除」。关键是分清「元素移动」与「指针修改」——链表做这些操作一个元素都不用挪，但题目限制的是顺序表。

（二）链式存储

*1.*单链表的定义与头结点

单链表用一组任意的存储单元存放元素，每个结点含数据域 `data` 和指针域 `next`，`next` 指向直接后继。逻辑相邻的元素物理上不必相邻，靠指针串起来。

```
typedef struct LNode {
    ElemType data;          // 数据域
    struct LNode *next;     // 指针域，指向直接后继
} LNode, *LinkList;
```

头结点是附加在第一个数据结点之前的、不存放数据的结点，头指针指向它。为什么要引入头结点：

头结点的三个作用（常考）

1. 统一了空表和非空表的处理：空表时 head->next == NULL，非空表时 head->next 指向第一个数据结点，判空变成一条统一的语句。
2. 统一了第一个位置和其他位置的插入删除：对第一个数据结点做前插 / 删除时，其前驱是头结点而不是空，不需要单独写「如果插在表头」的分支。
3. 头指针永远非空（指向头结点），指针操作不会因为「表空导致头指针为 NULL」而崩溃。

```
// 带头结点单链表的初始化
bool InitList(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));  // 创建头结点
    if (*L == NULL) return false;         // 内存分配失败
    (*L)->next = NULL;                    // 空表
    return true;
}

// 判空：带头结点判断头结点的 next 是否为空
bool Empty(LinkList L) { return L->next == NULL; }
```

无头结点时的补救办法

如果题目给的链表不带头结点（如 2026-02 的 head = cu 这类题），可以自己申请一个 dummy 结点指向第一个数据结点，把它当头结点用，之后所有操作就与带头结点的链表完全一致。统考的应用题基本都是带头结点的链表，但选择题里会出现不带头的写法，读题时先确认「头指针指向的是头结点还是第一个数据结点」。

*2.*求表长、按位查找、按值查找

```
// 求表长（不计算头结点）
int ListLength(LinkList L) {
    int len = 0;
    LNode *p = L->next;          // 若有头结点，从头结点的后继开始
    while (p != NULL) { len++; p = p->next; }
    return len;
}

// 按位查找：返回位序为 i 的结点指针；i = 0 时返回头结点
LNode *GetElem(LinkList L, int i) {
    if (i < 0) return NULL;
    LNode *p = L;                // p 从头结点出发（头结点是位序 0）
    int j = 0;
    while (p != NULL && j < i) { p = p->next; j++; }
    return p;                    // i 超长时返回 NULL
}

// 按值查找：返回第一个数据域等于 e 的结点
LNode *LocateElem(LinkList L, ElemType e) {
    LNode *p = L->next;
    while (p != NULL && p->data != e) p = p->next;
    return p;
}
```

单链表不能随机访问：要找第 i 个结点，只能从头指针开始顺链逐个走，比较 / 移动次数与 i 成正

比。设 i 等概率取 $1 \sim n$，则按位查找的平均比较次数为 $\frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2}$，按值查找查找成功时也是 $\frac{n+1}{2}$，二者时间复杂度都是 $O(n)$。

这里要澄清一个常见的表述混淆。平均比较次数相同，不代表两种查找「一样快」——按位查找在顺序表里是 $O(1)$，在链表里是 $O(n)$；只有当两者都在「从头逐个比较」这一情形下（顺序表的按值查找、链表的按位/按值查找）才有相同量级的平均比较次数。做题时先问自己：题目问的是哪种存储结构、哪种查找。

另外，链表的按位查找与按值查找虽然量级相同，代码结构有一个重要差别：按位查找是对「位置计数器」做判断（`j < i`），走到第 i 个就停，不存在找不到的情况（除非 i 越界，此时返回 `NULL`）；按值查找是对「数据域」做判断（`p->data != e`），可能一路走到表尾都找不到，必须用 **`p != NULL`** 作循环条件并在结束后返回 **`NULL`**。写代码时把这两个循环条件写反，就会分别导致「多走 / 少走一个结点」和「解引用空指针」。

还要注意单链表的判空条件与头结点有关。带头结点时判空看 `L->next == NULL`；不带头结点时判空看 `L == NULL`。两者的差别不只是少一个空判断：带头结点时头指针永远指向一个真实的结点，因此「对第一个数据结点做前插或删除」不需要特殊分支；不带头的写法则必须专门处理「表头为空」的情形。这也是统考应用题一律给出带头结点链表的原因——它能让算法描述更简洁、更不容易在边界上出错。

*3.*插入与删除的指针操作

后插（在 p 之后插入 s）：口诀是「先连后，再连前」。

```
s->next = p->next;   // 第一步：新结点先接住 p 原来的后继
p->next = s;         // 第二步：再让 p 指向新结点
```

这两句顺序绝对不能颠倒。如果先写 `p->next = s`，那么原来 p 的后继地址就丢了，`s->next` 再也接不上原链表，链表从 s 处断成两截。

前插（在 p 之前插入）：如果只给 `p` 而没有给它的前驱，正常做法要先找到前驱，$O(n)$。但有一个 $O(1)$ 的技巧——「偷梁换柱」：

```
// 在 p 之前插入值为 x 的结点，O(1)
void InsertBefore(LNode *p, ElemType x) {
    LNode *s = (LNode *)malloc(sizeof(LNode));
    s->data = p->data;   // 先把 p 的数据复制给新结点
    s->next = p->next;   // 新结点接在 p 后面
    p->next = s;
    p->data = x;         // 再把 x 写回 p，逻辑上等于 x 插到了 p 之前
}
```

删除：

- 已知待删结点的位序 i：调用 `GetElem(L, i-1)` 找到前驱 p，再执行

```
q = p->next;            // q 指向待删结点
p->next = q->next;      // 前驱跳过待删结点
free(q);                // 释放空间
```

查找前驱需要 $O(n)$，单看删除动作本身是 $O(1)$。

- 已知待删结点的指针 p：不能直接 `free(p)`，因为无法把 p 的前驱接到 p 的后继上。用同样的「偷梁换柱」：

```
// 删除 p 所指结点，前提是 p 不是尾结点
void DeleteNode(LNode *p) {
    if (p == NULL || p->next == NULL) return;   // 尾结点无法用此法删除
    LNode *q = p->next;
    p->data = q->data;       // 用后继的数据覆盖自己
    p->next = q->next;       // 再把自己变成「后继的后继」
    free(q);                 // 删除的是原来的后继结点
}
```

删除自身的两个致命陷阱

1. 尾结点不能用这个技巧。p 是尾结点时 $p \to next$ 为 `NULL`，`q->data` 会解引用空指针。所以必须先判断 `p->next != NULL`。
2. 指针变量不能改。这个技巧只能改 p 所指结点的内容，无法让调用者手里的 p 变成别的结点。如果题目说「删除 p 所指结点并让 p 指向其后继」，必须用前驱法或额外返回新的指针。

2024-01 的典型考法

给定带头结点链表，p 指向非首非尾结点，执行 `q = p->next; p->next = q->next; q->next = h->next; h->next = q;`。逐句读：前两句把 q 从 p 后面摘下来（p 跳过 q），第三句让 q 指向原来的第一个数据结点，第四句让头结点指向 q。最终效果是把 q 所指结点移动到链表的头结点之后。这类题只要耐心逐句画图，不要去背结论。

把插入删除的指针操作归纳成一句总原则：「先保存，后修改；先连新，后断旧」。之所以必须这样，是因为链表的一切连接关系都只通过指针保存，而指针赋值是破坏性的——一旦 `p->next` 被改成别的值，原来指向的那个结点就失去了唯一的入口，再也找不回来。所以任何一段链表操作代码，都可以用这条原则检查一遍：

- 后插时先写 `s->next = p->next`（保存旧后继），再写 `p->next = s`（修改）。
- 删除时先用 `q` 记住待删结点，再改 `p->next`，最后 `free(q)`（必须先记住再改，否则 `free` 的是错的空间）。
- 双链表插入时，凡是要读「邻居的原指针」的语句都排在修改它之前。
- 遍历时若要回头用 `p->next`，先用临时变量 `nxt = p->next` 存下来。这个习惯在反转链表、K 个一组翻转、交叉合并里反复用到，是链表题不出错的最重要保障。

同时还要舍得用变量。链表题的代码量不大，但指针关系容易绕晕，多定义两个临时指针（如 **pre**、**nxt**、**tail**、**dummy**）比省变量划算得多。统考判分看的是算法正确性和复杂度，不会因为「多用一个指针变量」扣分。相反，把代码压缩成一行套一行，一旦边界错了就是全错。

关于删除还有一点要补充：被删除的结点必须 **free** 释放。不释放只是「逻辑上删掉了」，链表的实际占用空间不会减少，若题目要求「空间复杂度为 $O(1)$」或者「释放结点空间」，漏掉 `free` 会直接失分。另外，不要 **free** 之后还继续访问该结点（悬空指针），正确顺序总是「先接好前后关系，再 `free`，且 `free` 后不再使用 `q`」。

*4.*头插法与尾插法建立单链表

头插法：每读入一个元素就插到头结点之后。输入顺序与链表中的顺序相反，因此头插法天然可以用来就地逆置链表。

```
// 头插法建立单链表，输入 999 结束
LinkList ListHeadInsert(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    (*L)->next = NULL;
    ElemType x;
    scanf("%d", &x);
    while (x != 999) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = (*L)->next;    // 新结点接住头结点原来的后继
        (*L)->next = s;          // 头结点指向新结点
        scanf("%d", &x);
    }
    return *L;
}
```

尾插法：额外用一个尾指针 `r` 指向表尾，每次把新结点接在 `r` 之后并让 `r` 后移。输入顺序与链表顺序一致。如果不用尾指针、每次都从头找表尾，总时间会退化到 $O(n^2)$。

```
// 尾插法建立单链表，输入 999 结束
LinkList ListTailInsert(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    (*L)->next = NULL;
    LNode *r = *L;               // r 始终指向表尾结点
    ElemType x;
    scanf("%d", &x);
    while (x != 999) {
        LNode *s = (LNode *)malloc(sizeof(LNode));
        s->data = x;
        s->next = NULL;
        r->next = s;             // 挂到表尾之后
        r = s;                   // 尾指针后移
        scanf("%d", &x);
    }
    return *L;
}
```

头插法 vs 尾插法的用途

- 头插法：顺序反转，用于链表就地逆置、把升序链表合并成降序链表。
- 尾插法：保持相对顺序，用于合并两个升序链表为一个升序链表、拆分链表后保持原有次序。

两种建立方式的时间复杂度都是 $O(n)$（每次插入本身 $O(1)$，共 n 次）。

*5.*双链表

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;    // 前驱、后继
} DNode, *DLinkList;
```

在 p 之后插入 s（注意四条语句的顺序，可以这样记：先处理「s 的两根线」，再处理「邻居指向 s 的两根线」）：

```
s->next = p->next;          // 1. s 的后继指向 p 的后继
if (p->next != NULL)
    p->next->prior = s;     // 2. p 的原后继的前驱改为 s
s->prior = p;               // 3. s 的前驱指向 p
p->next = s;                // 4. p 的后继改为 s
```

删除 p 的后继结点 q：

```
q = p->next;
p->next = q->next;
if (q->next != NULL)
    q->next->prior = p;
free(q);
```

双链表语句顺序为什么不能乱

关键点：凡是「读取 **p->next**」的语句，必须写在「修改 **p->next**」的语句之前。上面的插入写法里第 2 句用到了 p->next，第 4 句才改 p->next，顺序是对的。2023-02 就是这样考的：题干先给了 s->next = p->next; p->next = s;，此时 p->next 已经变成 s，原来的后继 r 只能通过 s->next 找到，所以后续必须写 s->prev = s->next->prev; s->next->prev = s;——不能再写 **p->next->prev = s**，那会把 s 的前驱指向 s 自己。

*6.*循环链表

循环单链表：表尾结点的 next 不指向 NULL，而是指回头结点，整条链形成一个环。

```
// 初始化空循环单链表
bool InitList(LinkList *L) {
    *L = (LNode *)malloc(sizeof(LNode));
    if (*L == NULL) return false;
    (*L)->next = *L;          // 头结点的 next 指向自己，代表空表
    return true;
}
// 判空
bool Empty(LinkList L) { return L->next == L; }
// 判断 p 是否为表尾结点（循环单链表特有）
bool isTail(LinkList L, LNode *p) { return p->next == L; }
```

循环双链表：头结点的 prior 指向表尾，表尾的 next 指向头结点。

```
bool InitDList(DLinkList *L) {
    *L = (DNode *)malloc(sizeof(DNode));
    if (*L == NULL) return false;
    (*L)->prior = *L;       // 空循环双链表：两根指针都指自己
    (*L)->next  = *L;
    return true;
}
bool EmptyDList(DLinkList L) { return L->next == L; }  // L->prior == L 等价
```

循环链表带来的能力

1. 从任一结点出发都能遍历整条链表；普通单链表从某结点出发只能走到后继方向。
2. 若设置尾指针 `tail`，则访问表头 `tail->next` 和表尾 `tail` 都是 $O(1)$；普通单链表找表尾是 $O(n)$。
3. 插入删除本身与普通链表相同，但在表头或表尾操作时要额外维护「环」的性质：删表尾后要改尾指针，环上有结点时要保证「最后一个结点的 next 指回头结点」。

2021-01 的经典陷阱

带头结点的非空单循环链表，h 是头指针，p 是尾指针，删除第一个元素。正确语句是

```
q = h->next; h->next = q->next; if (p == q) p = h; free(q);
```

要点有两个：先保存 `q` 再改 `h->next`（否则 q 会指向新的首结点，最终释放错结点）；当链表只有一个数据结点时 `p == q`，删除后表空，必须把尾指针 p 重新指向头结点，否则 p 会指向已释放的空间。这类「边界情形」是循环链表题的固定得分点。

7.静态链表

静态链表借助数组来描述链式存储结构：结点仍有数据域和指针域，但指针域里存放的是数组下标（称为游标），而不是真正的地址。它适用于不支持指针的高级语言。

```
#define MaxSize 50
typedef struct {
    ElemType data;
    int next;          // 下一个元素的数组下标，-1 表示结束
} SLinkList[MaxSize];
```

数组的 0 号分量通常充当头结点（`SLinkList[0].next` 指向第一个数据结点）。插入删除只需修改 `next` 的值，不需要移动元素，这一点和动态链表一致；但静态链表不能随机存取，只能从头结点顺着下标找，而且容量固定、不能动态扩容。

静态链表的两个特征值

- `next == -1`：本结点是链表最后一个结点（表示「指向 NULL」）。
- `next == -2`（或题目约定的其他特殊值）：本结点空闲未使用。

构造静态链表时的通用方法：从链接地址为 **NULL** 的那个结点开始反向追溯，就能还原出完整的逻辑顺序。2016-01 就是先给出一张「地址 / 元素 / 链接地址」表，让你还原链表顺序（c → a → e → b → d）后再做插入。

2016-01 的解题流程

已知表头元素为 c，按 c(1008H) → a(1000H) → e(1010H) → b(1004H) → d(100CH) → NULL 还原出逻辑顺序；把 f 放在 1014H 并插到 a 和 e 之间后变成 c → a → f → e → b → d。于是：

- a 的直接后继变成 f，a 的链接地址为 1014H；
- e 的直接后继仍是 b，e 的链接地址仍为 1004H；
- f 的直接后继是 e，f 的链接地址为 1010H。

按题目要求的 **a**、**e**、**f** 的顺序输出，得 1014H，1004H，1010H。注意题干问的顺序（a, e, f）与链表顺序（a, f, e）不一致，审题时按题干给的顺序作答，否则会误选。

（三）顺序表与链表的对比

比较项	顺序表	链表
存储方式	连续空间，逻辑相邻即物理相邻	任意空间，靠指针建立逻辑关系
存取方式	随机存取，按位查找 $O(1)$	顺序存取，按位查找 $O(n)$
按值查找	$O(n)$，平均比较 $\frac{n+1}{2}$ 次	$O(n)$，平均比较 $\frac{n+1}{2}$ 次
插入 / 删除	平均移动 $\frac{n}{2}$ / $\frac{n-1}{2}$ 个元素，$O(n)$	修改指针，本身 $O(1)$；但找位置要 $O(n)$
空间分配	静态分配需预估容量，可能溢出；动态分配扩容要整体搬移	按需申请，不需要连续空间，不会溢出
存储密度	$=1$（无额外结构开销）	<1（每个结点多一个或两个指针域）
逻辑结构	均为线性表（一对一），逻辑结构相同	同左
适用场合	表长可预估、频繁按位访问、很少插入删除	表长变化大、频繁插入删除、难以预估规模

选择的判断口诀

「常查少改」用顺序表，「常改少查」用链表；要按位随机访问用顺序表，要频繁插删用链表。若既要求高效按位访问又要求高效插删，那是更复杂结构（如树、跳表）的用武之地，不在本章范围。

把这张表再压缩成三句可判断的话，选择题里可以直接用：

1. 问「能不能 $O(1)$ 取到第 i 个元素」：只有顺序表（等价说法：随机存取、随机访问、支持下标直接访问）。
2. 问「插入删除是否要移动元素」：顺序表要，链表不要（链表只改指针）。但要注意，若题目还问「找到位置需要几步」，答案都要加上 $O(n)$ 的定位代价。
3. 问「空间是否可能浪费 / 溢出」：顺序表有预分配问题（静态可能溢出、动态会搬移、连续空间可能申请不到），链表不会溢出，但每个结点都要付指针域的空间代价。

三个常见的对比误区

1. 不能说「链表的插入删除一定是 $O(1)$」。只有「已经拿到待操作结点的指针」时才是 $O(1)$；若题目给的是位序 i，找位置本身要 $O(n)$，整体仍是 $O(n)$。
2. 不能说「链表一定比顺序表省空间」。链表省去了「预留连续大空间」的浪费，但每个结点都要付指针域的税，存储密度更低。
3. 循环链表和双向链表不改变时间复杂度的数量级，它们带来的是「能反向走」「从任一结点出发可遍历全程」「插入删除更方便维护」这类常数级或功能性的好处。

三、线性表的应用（算法设计题）

统考 41 题的固定答题格式是三步，务必按顺序写：

1. 给出算法的基本设计思想（用文字说清楚用了什么技巧、扫描几遍、指针怎么走）；
2. 用 **C** 或 **C++** 描述算法，关键之处给出注释（函数要完整，边界要判断）；
3. 说明时间复杂度（必要时说明空间复杂度）。

答题的常见失分点

- 只写代码不写设计思想，或只写思想不写代码——评分是按三小问分点给的。
- 忘记处理空表、只有一个结点、表长小于 k、结点为尾结点等边界情形。

- 写了 sort 等库函数（408 不允许直接用库函数排序，需要自己写快排 / 归并）。
- 空间复杂度超标：题目要求 $O(1)$ 时用了数组或递归（递归栈也是空间）。

（一）（〇）算法设计题的通用解题框架

线性表的算法题看似花样多，实际上绝大多数都能归入下面五种手法之一。拿到题目先判断它属于哪一类，再套相应模板，比从零推导快得多，也稳得多。

手法	适用特征	典型题目
双指针（同向）	需要「原地筛选 / 删除 / 合并」，保持相对顺序	删重复、删值为 x 的元素、合并有序表
双指针（前后 / 距离固定）	需要「倒数第 k 个」这类定位	2009 倒数第 k 个结点、2012 求共同后缀
快慢指针	需要「中间位置」「是否有环」「环的入口」	2019 重排链表、判断环、找中点
就地逆置	需要「反转次序」或「借助反转改变顺序」	2010 循环左移、链表逆置、升序合并成降序
辅助数组 / 计数数组	题目给了值域范围（如 $data$ 的绝对值不超过 n、$0 \le a_i < n$）	2015 删绝对值重复、2013 主元素、2018 最小正整数

判断的关键在题干的两个信号：一是「空间复杂度」的要求（要求 $O(1)$ 就不能用辅助数组，只能原地改指针或引用计数）；二是题目给出的取值范围或有序性条件（给了值域范围往往暗示可以用数组下标做标记；给了有序性往往暗示可以用双指针一趟扫描）。换句话说，题目里出现的每一个条件几乎都是有用信息，不会白给。

写代码时的两条自保策略：一是先把边界情形在纸上列全（空表、单结点、全是要删的元素、一个元素都不删、表长恰好等于 k 等），写代码时逐个处理；二是写完后拿一组 **3～4** 个元素的小数据手工模拟一遍，确认指针没有走空、没有成环、没有漏结点。这两步花不了两分钟，但能避免绝大多数失分。

（二）合并两个有序表（两种存储都考）

顺序表版本：把两个升序顺序表合并为一个升序顺序表，用三个下标 i, j, k 从前往后比较，谁小谁先写。若要求原地合并且目标数组有足够尾部空间，则改用从后往前的双指针，避免覆盖还没比较的数据。

```
// 将升序顺序表 A、B 合并为升序顺序表 C
void MergeSqList(SqList A, SqList B, SqList *C) {
    int i = 0, j = 0, k = 0;
    while (i < A.length && j < B.length) {
        if (A.data[i] <= B.data[j]) C->data[k++] = A.data[i++];
        else                        C->data[k++] = B.data[j++];
    }
    while (i < A.length) C->data[k++] = A.data[i++];  // A 有剩余
    while (j < B.length) C->data[k++] = B.data[j++];  // B 有剩余
    C->length = k;
}
```

时间复杂度 $O(n+m)$，空间复杂度 $O(1)$（不计结果表本身的空间）。

链表版本：

```
// 将两个带头结点的升序单链表合并为一个升序单链表，用尾插法
LinkList MergeList(LinkList A, LinkList B) {
    LinkList C = (LinkList)malloc(sizeof(LNode));
    C->next = NULL;
    LNode *pa = A->next, *pb = B->next, *r = C;  // r 为新表尾指针
    while (pa != NULL && pb != NULL) {
        if (pa->data <= pb->data) { r->next = pa; r = pa; pa = pa->next; }
        else                      { r->next = pb; r = pb; pb = pb->next; }
    }
    r->next = (pa != NULL) ? pa : pb;    // 剩余部分整段接上，不必逐个插入
    free(A); free(B);                    // 释放两个原头结点
    return C;
}
```

升序合并成降序 = 头插法

若题目要求「两个升序链表合并为一条降序链表」（2013 年选择题的考点），必须改成头插法：每次比较取出较小的结点后，用 `s->next = C->next; C->next = s;` 插到结果表头。因为「当前较小」不等于「全体最大」，不能用尾插法。全部结点处理完后，每个结点最多被访问一次，最坏时间复杂度是 $O(n+m)=O(\max(m,n))$（m,n 同阶时二者等价，这正是 2013-01 选 D 的理由）。

（三）就地逆置

顺序表逆置：首尾双指针交换，走到中间停止，时间 $O(n)$、空间 $O(1)$。

```
void ReverseSqList(SqList *L) {
    ElemType tmp;
    for (int i = 0, j = L->length - 1; i < j; i++, j--) {
        tmp = L->data[i]; L->data[i] = L->data[j]; L->data[j] = tmp;
    }
}
```

链表逆置（迭代法，必须背下来）：把链表看成「已逆置部分」和「未逆置部分」，逐个把未逆置部分的第一个结点接到已逆置部分的最前面。

```
// 带头结点单链表的就地逆置
void ReverseList(LinkList L) {
    LNode *pre = NULL, *cur = L->next;
    while (cur != NULL) {
        LNode *next = cur->next;  // 先保存后继，因为下一步要改 cur->next
        cur->next = pre;          // 反转指针方向
        pre = cur;                // pre 后移
        cur = next;               // cur 后移
    }
    L->next = pre;                // 头结点接上新的第一个结点
}
```

迭代法反转后的一个重要性质

反转某一段结束后：**pre** 指向这一段的最后一个结点，**cur** 指向这一段之后的第一个结点（都是在原链表视角下）。这个性质在「反转部分链表」「重排链表」「K 个一组翻转」里反复使用，务必记住。

为什么迭代法比递归法和「入栈法」都好？把三种做法摆在一起看就清楚了：

方法	时间	空间	说明
迭代法（双指针）	$O(n)$	$O(1)$	原地改指针，最优解，必背
头插法	$O(n)$	$O(1)$	依靠待反转段的前驱结点；若无前驱需先造一个 dummy
递归法	$O(n)$	$O(n)$	递归栈深度为 n，且处理不当容易成环
入栈法	$O(n)$	$O(n)$	借助栈的先进后出实现逆序，空间不达标

因此只要题目要求空间 $O(1)$，答案就只能用迭代法或头插法。递归写法虽然短，但空间复杂度不达标，在「空间复杂度为 $O(1)$」的题目里会被判失分。2019 年重排链表题明确要求 $O(1)$ 空间，就是这一点的直接体现。

用头插法逆置也可以，效果相同，本质就是「把结点依次摘下再头插回头结点之后」。为什么迭代法比递归法和「入栈法」都好？把几种做法摆在一起看就清楚了：

方法	时间	空间	说明
迭代法（双指针）	$O(n)$	$O(1)$	原地改指针，最优解，必背
头插法	$O(n)$	$O(1)$	依靠待反转段的前驱结点；若无前驱需先造一个 dummy
递归法	$O(n)$	$O(n)$	递归栈深度为 n，处理不当容易成环
入栈法	$O(n)$	$O(n)$	借助栈的先进后出实现逆序，空间不达标

只要题目要求空间 $O(1)$，答案就只能用迭代法或头插法。递归写法虽然短，但空间复杂度不达标，在「空间复杂度为 $O(1)$」的题目里会被判失分。2019 年重排链表题明确要求 $O(1)$ 空间，就是这一点的直接体现。

头插法逆置的一个常见用法是把「就地逆置」写成与「头插法建表」同构的代码：

```
// 头插法就地逆置带头结点的单链表
void ReverseByHeadInsert(LinkList L) {
    LNode *p = L->next, *r;
    L->next = NULL;              // 先断开，头结点单独作为新表的头
    while (p != NULL) {
        r = p->next;             // 先保存后继
        p->next = L->next;       // 头插：新结点接住当前的第一个结点
        L->next = p;             // 头结点指向新结点
        p = r;                   // 继续处理原来的后继
    }
}
```

对照着看会发现，「头插法建表」和「头插法逆置」其实是同一段程序——这正是「输入顺序与链表顺序相反」这一特性的直接应用，也是 2013 年「升序链表合并为降序链表」能用头插法解决的原因。它的一个常见用法是把「就地逆置」和「建立链表」合并成一段代码：

```
// 头插法就地逆置带头结点的单链表
void ReverseByHeadInsert(LinkList L) {
    LNode *p = L->next, *r;
    L->next = NULL;                 // 先断开，头结点单独作为新表的头
    while (p != NULL) {
        r = p->next;                // 先保存后继
        p->next = L->next;          // 头插：新结点接住当前的第一个结点
        L->next = p;                // 头结点指向新结点
        p = r;                      // 继续处理原来的后继
    }
}
```

两段代码对照着看，会发现「头插法建表」和「头插法逆置」其实是同一段程序——这正是「输入顺序与链表顺序相反」这一特性的直接应用，也是 2013 年「升序链表合并为降序链表」能用头插法解决的原因。

「数组循环左移 p 位」（**2010** 年真题）的经典做法是三次逆置：先把前 p 个元素逆置，再把后 $n-p$ 个元素逆置，最后把整个数组逆置。

```
// 将 R[0..n-1] 循环左移 p 位，O(n) 时间、O(1) 空间
void LeftShift(int R[], int n, int p) {
    p = p % n;                  // 先取模，避免 p > n 时的无谓搬移
    Reverse(R, 0, p - 1);       // 前 p 个逆置
    Reverse(R, p, n - 1);       // 后 n-p 个逆置
    Reverse(R, 0, n - 1);       // 整体逆置
}
```

自检一下：$R=\langle 1,2,3,4,5\rangle$，$p=2$。前 2 个逆置得 $\langle 2,1,3,4,5\rangle$，后 3 个逆置得 $\langle 2,1,5,4,3\rangle$，整体逆置得 $\langle 3,4,5,1,2\rangle$，正是左移 2 位的结果。

为什么三次逆置能得到循环左移？设原序列为 XY，其中 X 是前 p 个元素、Y 是后 $n-p$ 个元素，目标是 YX。记 X' 表示 X 的逆序，则

$$(X'Y')'=YX$$

即「先分别把两段各自逆置，再把整体逆置」，就能把 XY 换成 YX。这是本题的核心设计思想，答题时写出这一步变换，比只贴代码更容易得满分。同理，循环右移 p 位只需把三段逆置的顺序调整为「整体逆置 $\to$ 前 p 个逆置 $\to$ 后 $n-p$ 个逆置」即可。

循环左移题的常见失分

一是忘记处理 $p\geq n$ 的情形（先取模 `p = p % n`）；二是把三次逆置的边界写错（`Reverse` 通常接收下标区间，注意是左闭右闭还是左闭右开）；三是空间超标——额外开一个长度 n 的辅助数组搬移虽然结果正确，但空间是 $O(n)$，不满足题目「时空两方面都尽可能高效」的要求。三次逆置法时间 $O(n)$（每个元素被访问常数次），空间 $O(1)$，是最优解。

为什么三次逆置能得到循环左移？设原序列为 XY，其中 X 是前 p 个元素、Y 是后 $n-p$ 个元素，目标是 YX。记 X' 表示 X 的逆序，则：

$$(X'Y')'=YX$$

即「先分别把两段各自逆置，再把整体逆置」，就能把 XY 换成 YX。这是本题的核心设计思想，答题时写出这一步变换，比只贴代码更容易得满分。同理，循环右移 p 位只需把三段逆置的顺序调整为「整体逆置 → 前 p 个逆置 → 后 $n-p$ 个逆置」即可。

循环左移题的常见失分

一是忘记处理 $p \geq n$ 的情形（先取模 $p = p \bmod n$）；二是把三次逆置的边界写错（Reverse 函数通常接收下标区间，注意是左闭右闭还是左闭右开）；三是空间超标——如果额外开一个长度 n 的辅助数组搬移，虽然结果正确但空间是 $O(n)$，不满足题目「时空两方面都尽可能高效」的要求。三次逆置法的时间是 $O(n)$（每个元素被访问常数次），空间 $O(1)$，是最优解。

（四）快慢指针：找中间结点

快指针一次走两步，慢指针一次走一步；快指针到表尾时，慢指针正好在中间。

```
// 带头结点单链表求中间结点
// 若结点个数为偶数，本模板返回「下中位」（靠后的那个中间结点）
LNode *MiddleNode(LinkList L) {
    LNode *slow = L, *fast = L;      // 都从头结点出发，便于偶数时返回下中位
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

中间结点的定义要看清题目

- 快慢指针都从头结点出发，n 为偶数时返回的是下中位（第 $n/2+1$ 个数据结点）。
- 快慢指针都从第一个数据结点出发，n 为偶数时返回的是上中位（第 $n/2$ 个数据结点）。

2019 年重排链表题明确要求分界点是下中位数 $\lceil n/2 \rceil$ 个结点，所以要采用第一种起点。题目怎么写，代码就怎么起步，不要凭印象。

（五）双指针：倒数第 k 个结点（**2009** 年真题）

一次遍历就能完成：设前后两个指针 p（前）和 q（后），先让 p 从头结点出发走 k 步，然后 p、q 同时后移，当 p 到达表尾（NULL）时，q 所指就是倒数第 k 个结点。

```
// 查找带头结点单链表中倒数第 k 个结点，找到则输出其数据并返回 1，否则返回 0
int SearchKthFromEnd(LinkList L, int k) {
    LNode *p = L->next, *q = L->next;
    int i = 0;
    while (p != NULL && i < k) { p = p->next; i++; }  // p 先走 k 步
    if (p == NULL && i < k) return 0;                 // 表长不足 k，查找失败
    while (p != NULL) { p = p->next; q = q->next; }   // 同步后移
    printf("%d\n", q->data);
    return 1;
}
```

时间复杂度 $O(n)$（只扫一遍），空间复杂度 $O(1)$。

边界：表长不足 k

第一个 while 结束时若 i < k，说明表长小于 k，必须返回 0。很多答案漏掉这个判断，在 p 已经为 NULL

时还接着做同步后移，逻辑上就错了。另一个写法是让 p 从头结点出发走 k 步、q 从头结点出发，最后 q->next 是倒数第 k 个——两种写法等价，但要保证前后指针的起始位置与步数匹配。

（六）判断链表是否有环，并求环的入口

判环（快慢指针相遇法）：快指针每次走两步、慢指针每次走一步。若链表无环，快指针会先到 NULL 退出；若有环，快指针必然追上慢指针。

```
// 判断带头结点单链表中是否有环
bool HasCycle(LinkList L) {
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;   // 相遇即有环
    }
    return false;                        // fast 走到 NULL，无环
}
```

为什么快慢指针一定会相遇？两个指针都进环后，每移动一轮，快指针与慢指针的距离就减少 **1**（快走 2、慢走 1）。这相当于操场上的追及问题，只要一直走，快指针一定追上慢指针。

为什么慢指针在环内走不完一圈就被追上？考虑最坏情况：慢指针刚进环时，快指针正好在它前面一个位置，此时两者相距「环长 -1」，因为每轮距离减少 1，所以最多经过「环长 -1」轮就相遇；此时慢指针在环内走了「环长 -1」步，不到一圈。其他情况比这更快。

求环的入口（第一次相遇后再用双指针）：设头结点到环入口的距离为 a，环入口到第一次相遇点的距离为 b，环长为 L。相遇时慢指针走了 $a+b$，快指针走了 $a+b+kL$，而快指针路程是慢指针的两倍，于是

$$2(a+b) = a+b+kL \implies a+b = kL \implies a = kL-b$$

即「从头结点到入口的距离 a」等于「从相遇点继续沿环走到入口的距离 $kL-b$」。所以：相遇后，让一个指针从头结点出发、另一个从相遇点出发，二者同速前进，再次相遇的位置就是环的入口。

```
// 返回环的入口结点；无环返回 NULL
LNode *DetectCycle(LinkList L) {
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {              // 第一次相遇
            LNode *p = L;
            while (p != slow) {          // 同速前进，再次相遇即入口
                p = p->next;
                slow = slow->next;
            }
            return slow;
        }
    }
    return NULL;
}
```

求环长：从第一次相遇点出发，沿环走一圈回到该点，走过的步数就是环长，时间 $O(n)$、空间 $O(1)$。

（七）判断两个链表是否相交（2012 年真题）

判定依据（必背）：两个链表如果相交，从交点之后它们共享同一段存储，因此两个链表的尾结点必然相同。注意比较的是结点地址（指针），不是数据域的值！数据域相等只是「值相同」，不代表同一个结点。

相交题的经典陷阱

看到 `p->data == q->data` 就断言相交是错的。两个链表可以有相同的值而完全不共享结点。判断相交一定要比 `p == q`（指针相等）。另外要注意：若链表带环，情况会更复杂，统考题干一般会明确「整个链式结构中不存在环」。

方法一（长度差法）：分别求两个链表长度 len_1, len_2，让较长的那个指针先走 $|len_1 - len_2|$ 步，使两个指针到表尾的距离相等，然后同步后移，第一次指针相同的位置就是交点。

```
// 带头结点，求两个单链表的第一个公共结点
LNode *FindFirstCommon(LinkList A, LinkList B) {
    int lenA = 0, lenB = 0;
    LNode *p = A->next, *q = B->next;
    while (p != NULL) { lenA++; p = p->next; }
    while (q != NULL) { lenB++; q = q->next; }
    p = A->next; q = B->next;
    int d = lenA - lenB;
    while (d > 0) { p = p->next; d--; }     // A 长，p 先走
    while (d < 0) { q = q->next; d++; }     // B 长，q 先走
    while (p != q && p != NULL && q != NULL) { p = p->next; q = q->next; }
    return p;                               // 相交返回交点，不相交返回 NULL
}
```

方法二（路程补偿，代码更短）：两个指针同时同速前进，走到 NULL 时切换到对方的起点继续走。设公共段长度为 c，则两指针总路程都是 $a+c+b+c$ 与 $b+c+a+c$，相等，因此必然在交点相遇；若不相交，则会在 NULL 处相遇。

```
LNode *GetIntersectionNode(LinkList A, LinkList B) {
    LNode *p = A->next, *q = B->next;
    while (p != q) {
        p = (p == NULL) ? B->next : p->next;   // 走到头就换到对方起点
        q = (q == NULL) ? A->next : q->next;
    }
    return p;   // 不相交时 p、q 会同时为 NULL，返回 NULL
}
```

两种方法都是时间 $O(m+n)$、空间 $O(1)$。方法二是 **2012** 年真题的主流解法，但方法一更符合「先说清设计思想」的答题习惯，建议两种都掌握。

2012 年真题的题干是「两个单词共享相同后缀」，本质就是「两个链表从某个结点起共享同一段存储」，因此求「共同后缀的起始位置」与求「第一个公共结点」完全等价。答题时的设计思想可以这样写：先分别求出两个链表的长度，算出长度差 d；让较长链表的指针先走 d 步，使两个指针到表尾的距离相等；然后两个指针同步后移，第一次指向同一结点时即交点。这段文字就是第一小问的完整答案，务必能默写。

长度差法的边界

如果两个链表根本不相交，那么同步后移时两个指针会同时变成 NULL，此时 `p == q` 成立，`while` 循环退出并返回 NULL——这恰好是正确的返回值，所以不需要额外判断。但循环条件必须写成 `p != q &&`

`p != NULL && q != NULL` 或者只写 `p != q`（依赖「同时为 NULL 时相等」这一性质）。若写成 `while (p != NULL && q != NULL && p != q)`，退出后还要再判断一次 `p == q ? p : NULL`，更啰嗦。理解「同时为 **NULL** 也算相遇」这一点，是本题最容易出彩的地方。

（八）删除重复元素

有序顺序表删重复（双指针）：因为有序，重复元素一定相邻。用快指针 `fast` 扫描，慢指针 `slow` 指向「下一个不同元素应写入的位置」。

```
// 从有序顺序表中删除重复元素，返回处理后的表长
int DeleteDuplicates(SqList *L) {
    if (L->length == 0) return 0;
    int slow = 1;                              // data[0] 必然保留
    for (int fast = 1; fast < L->length; fast++) {
        if (L->data[fast] != L->data[fast - 1]) {  // 与前一个不同，是新值
            L->data[slow] = L->data[fast];
            slow++;
        }
    }
    L->length = slow;                          // 不要忘记更新表长
    return slow;
}
```

无序表删除所有值为 x 的元素（双指针）：同样用快慢指针，把不等于 x 的元素往前填，时间 $O(n)$、空间 $O(1)$。如果题目要求「删除值在 s 到 t 之间的所有元素」，只需把判断条件改成 `if (L->data[i] < s || L->data[i] > t)`；若 $s > t$ 或表空则直接返回（判空和参数合法性是常被漏掉的得分点）。

有序表删区间 $[s,t]$（两端定位法）：先找到第一个 $\geq s$ 的元素位置 i 和最后一个 $\leq t$ 的元素位置 j，若 $i \leq j$ 则把 j 之后的所有元素整体前移 $j-i+1$ 位，表长减小 $j-i+1$。因为有序，$[s,t]$ 内的元素必然连续，这个方法比逐个比较更快（仍然是 $O(n)$，但只扫一遍找到边界就整体搬移）。

链表删除重复结点：对有序链表，只需比较相邻结点：

```
// 带头结点的有序单链表删除重复元素，每个值只保留一个
void DeleteSame(LinkList L) {
    LNode *p = L->next, *q;
    while (p != NULL && p->next != NULL) {
        q = p->next;
        if (q->data == p->data) {   // 与后继重复
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

对无序链表删重复，若不允许额外空间，则用二重循环（$O(n^2)$）；若允许空间，2015 年真题的做法是借助一个标记数组——因为 `|data| ≤ n`，可以开一个长度 $n+1$ 的数组 `flag[]`，扫描时把 `|data|` 作为下标打标记，遇到已标记的结点就删除，这样时间 $O(m)$、空间 $O(n)$，正是题目「时间复杂度尽可能高效」的意图。这类题一定要结合题目给的取值范围来设计，题干里的 `|data| ≤ n` 就是给你的提示。

（九）其他常考算法

删除单链表中值最小的结点（假设存在且唯一）：用一个指针 `minPre` 记住最小值结点的直接前驱，一趟扫描后直接删除：

```
void DeleteMin(LinkList L) {
    if (L->next == NULL) return;
    LNode *minPre = L, *p = L->next;
    while (p->next != NULL) {
        if (p->next->data < minPre->next->data) minPre = p;
        p = p->next;
    }
    LNode *q = minPre->next;
    minPre->next = q->next;
    free(q);
}
```

若要「按递增次序输出各结点的值并释放结点空间」，只需在外面套一层 `while (L->next != NULL)`，每次找出最小值输出并删除。两层扫描，时间 $O(n^2)$、空间 $O(1)$。（若允许 $O(n)$ 辅助空间，可以先存入数组排序，时间 $O(n\log n)$——考试时要看题目对时空的要求择优。）

拆分链表：把一个线性表 $C=(a_1,b_1,a_2,b_2,\cdots)$ 拆成 $A=(a_1,a_2,\cdots)$、$B=(b_n,\cdots,b_2,b_1)$ 两个表。做法是：奇数位置结点用尾插法接到 A（保持相对次序），偶数位置结点用头插法接到 B（实现逆序）。这正是「头插法逆序、尾插法保序」这一特性的直接应用。

重排链表（**2019** 年真题）：把 $L=(a_1,a_2,\cdots,a_n)$ 重排为 $(a_1,a_n,a_2,a_{n-1},\cdots)$，要求空间 $O(1)$，是三个基本功的串联：

1. 用快慢指针找到中间结点（本题取下中位 $\lceil n/2\rceil$），并把链表从中间断开；
2. 把后半段就地逆置；
3. 把前半段与逆置后的后半段交叉合并。

```
// 带头结点单链表重排为先前后尾交替
void ReorderList(LinkList L) {
    if (L->next == NULL) return;
    // 第一步：找下中位结点
    LNode *slow = L, *fast = L;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    LNode *head2 = slow->next;   // 后半段的第一个结点
    slow->next = NULL;           // 从中间断开
    // 第二步：把后半段就地逆置
    LNode *pre = NULL, *cur = head2;
    while (cur != NULL) {
        LNode *nxt = cur->next;
        cur->next = pre;
        pre = cur;
        cur = nxt;
    }
    head2 = pre;
    // 第三步：交叉合并
    LNode *p1 = L->next, *p2 = head2;
    while (p1 != NULL && p2 != NULL) {
```

```
        LNode *n1 = p1->next, *n2 = p2->next;
        p1->next = p2;
        p2->next = n1;
        p1 = n1;
        p2 = n2;
    }
}
```

重排链表的复杂度

三步各自都是 $O(n)$，总时间复杂度 $O(n)$，空间复杂度 $O(1)$（只用了常数个指针），完全满足 2019 年真题「空间复杂度为 $O(1)$ 且时间上尽可能高效」的要求。暴力解法（把结点存入数组再按下标重建）虽然也是 $O(n)$ 时间，但空间是 $O(n)$，不满足题目要求，会被扣分。

其他常见小题型（知道思路即可）：

- 数组主元素（**2013** 年真题）：摩尔投票法，$O(n)$、$O(1)$。先假设当前元素为候选、计票数为 1，遇见相同加一、不同减一，计票归零则换候选；最后再扫一遍验证候选出现次数是否真的 $> n/2$。验证这一步不能省。
- 未出现的最小正整数（**2018** 年真题）：利用「n 个元素，未出现的最小正整数必然在 $1 \sim n+1$ 之间」这一结论，用原地交换（或辅助标记数组）把值域内的元素放到对应下标处，再从头扫描找第一个不满足 `A[i] == i+1` 的位置；时间 $O(n)$、空间 $O(1)$。
- 两个等长升序序列的中位数（**2011** 年真题）：用类二分的思路，每次同时比较两个序列的中位数，取半区间，逐步把规模缩小一半，时间 $O(\log n)$、空间 $O(1)$。
- 三元组最小距离（**2020** 年真题）：三个有序数组各设一个指针，每次计算当前三元组距离并更新最小值，然后把三个指针中指向最小元素的那个前移一位，循环直到某个数组走到末尾；时间 $O(n_1+n_2+n_3)$。

关于 2011 年与 2020 年这两道题，值得多写几句设计思想，因为它们的「套路感」很强。

2011 年求两等长升序序列的中位数，设计思想可以这样表述：设两个序列为 A、B，长度均为 L。分别取 A 的中位数 a 和 B 的中位数 b（第 $\lceil L/2 \rceil$ 个元素）。若 $a=b$，则 a 就是所求；若 $a<b$，则所求中位数一定不在 A 的前半段（可以舍弃 A 的前 $\lfloor L/2 \rfloor$ 个）和 B 的后半段（舍弃 B 的后 $\lfloor L/2 \rfloor$ 个）；若 $a>b$ 则对称地舍弃 A 的后半段与 B 的前半段。每次把规模缩小一半，且两个序列始终等长，直到两个序列各只剩一个元素，较小者即为所求。因为规模每次减半，时间 $O(\log_2 n)$，只用了常数个变量，空间 $O(1)$。这类「取中位数、比大小、抛掉不可能的一半」的写法，本质与折半查找同源。

2020 年求三元组最小距离，设计思想是：$D=|a-b|+|b-c|+|c-a|$ 这个式子看起来有三个绝对值，但若设 $a \le b \le c$，则 $D=2(c-a)$——距离只由最大者和最小者的差决定。因此要让 D 变小，唯一的办法是把「当前最小的那个元素」换成它所在集合中的下一个更大的元素，从而抬高最小值、缩小极差。于是算法是：三个指针分别指向三个有序数组的第一个元素，循环计算当前 D 并更新答案，然后把三个指针中指向最小值的那个指针向后移一位（若并列最小，移动任意一个即可，但通常约定移动其中一个并继续），直到某个指针越界。每个指针最多走完自己那个数组，总时间 $O(n_1+n_2+n_3)$，空间 $O(1)$。注意题目里的排序条件是「按升序分别存储」，正是它保证了「往后移只会让最小值变大」这一单调性，去掉有序条件这个方法就不成立了。

这两个例子说明同一件事：统考算法题的最优解往往来自对题目给定条件的充分利用（有序、等长、值域范围），而不是来自高级技巧。审题时把每个条件都圈出来，是最有效的解题起点。

（十）补充：无序表先排序再处理的思路

线性表的算法题里有一类「看起来没有下手处」的题目：给一个无序数组，要求找某种满足大小关系的结果。这时一条通用的破题思路是——先用 $O(n\log n)$ 把它变成有序表，再对有序表用双指针等线性手法，总时间仍然是 $O(n\log n)$，通常就是这类题的标准答案。

典型例子是「找出数组中未出现的最小正整数」（2018 年真题）。如果直接扫描，很难判断「哪个正整数没出现」；但一旦想到「n 个元素里未出现的最小正整数一定不超过 $n+1$」，就可以用值域做下标打标记，一趟扫描解决问题。再比如「求两个数组的交集」「找出重复出现的元素」「求数组中两数之差的最小绝对值」，排序后相邻比较或双指针一遍即得。

注意事项有两条。第一，**408** 考试不允许直接调用 `sort` 等库函数，需要自己写出排序。建议把快速排序模板背下来备用：

```
// 快速排序：对 q[l..r] 就地升序排序，平均 O(nlogn)，空间 O(logn)
void QuickSort(int q[], int l, int r) {
    if (l >= r) return;                  // 递归终止：区间内元素不超过 1 个
    int i = l - 1, j = r + 1, x = q[l + r >> 1];  // x 取中间元素作分界点
    while (i < j) {
        do i++; while (q[i] < x);        // i 之前的元素都 <= x
        do j--; while (q[j] > x);        // j 之后的元素都 >= x
        if (i < j) { int t = q[i]; q[i] = q[j]; q[j] = t; }
    }
    QuickSort(q, l, j);
    QuickSort(q, j + 1, r);
}
```

第二，排序会改变元素的相对位置。如果题目要求「保持原有次序」，就不能先排序，只能改用双指针一趟扫描或其他不改变次序的方法。审题时务必看清这一点。

```
// 折半查找：在有序数组 A[low..high] 中查找 x，返回下标，失败返回 -1
int BinarySearch(int A[], int low, int high, int x) {
    while (low <= high) {                    // 左闭右闭区间
        int mid = low + (high - low) / 2;  // 防溢出写法，等价于 (low+high)/2
        if (A[mid] > x)      high = mid - 1;
        else if (A[mid] < x) low = mid + 1;
        else                 return mid;
    }
    return -1;
}
```

折半查找每次把查找区间缩小一半，最多比较 $\lceil\log_2(n+1)\rceil$ 次，时间 $O(\log_2 n)$、空间 $O(1)$。但再次强调：它要求「有序」且「顺序存储」——链表无法在 $O(1)$ 时间内取到中间元素，因此不能在链表上做折半查找。这个结论是「顺序表与链表对比」在查找问题上的延伸，常与本章一起出题。

> **「先排序再处理」的两个前提**
>
> 一是题目没有要求保持原有次序（否则排序就破坏了要求）；二是排序的代价可以接受（若题目要求 $O(n)$ 时间，那排序的 $O(n\log n)$ 就超标了，此时要另想办法，例如用哈希 / 标记数组把时间压到 $O(n)$）。这两点判断错，方向就全错了。

本章小结表

知识点	核心结论
线性表定义	同类型、有限、有序；除首尾外一前一后
位序 vs 下标	位序从 1 开始，下标从 0 开始；插入范围 $1 \sim n+1$，删除范围 $1 \sim n$
顺序表地址	$LOC(a_i) = LOC(a_1) + (i-1) \times d$，随机访问
插入平均移动	$\frac{n}{2}$ 次，时间复杂度 $O(n)$
删除平均移动	$\frac{n-1}{2}$ 次，时间复杂度 $O(n)$
按值查找（顺序）	平均比较 $\frac{n+1}{2}$ 次，$O(n)$
链表按位查找	平均比较 $\frac{n+1}{2}$ 次，$O(n)$，顺序存取
头结点作用	统一空表/非空表、统一首位置插删、头指针恒非空
头插法	顺序反转，用于逆置、升序合并成降序
尾插法	保持相对顺序，需尾指针，$O(n)$
后插口诀	先连后（`s->next=p->next`）再连前（`p->next=s`）
双链表后插	四句顺序：`s->next`、`p->next->prior`、`s->prior`、`p->next`
循环单链表判空	`head->next == head`
循环双链表判空	`head->next == head`（且 `head->prior == head`）
静态链表	数组 + 游标；`next==-1` 表结束；不能随机存取、容量固定
顺序表 vs 链表	顺序表随机存取、密度高；链表插删灵活、无需连续空间
找倒数第 k 个	前后双指针，前指针先走 k 步，一次遍历 $O(n)$
找中间结点	快慢指针，快走 2 慢走 1；起点不同结果不同（上/下中位）
判断环	快慢指针相遇即有环，$O(n)$、$O(1)$
求环入口	相遇后一个从头、一个从相遇点同速走，再次相遇即入口
判断相交	尾结点相同（比地址不比数据）；$O(m+n)$、$O(1)$
就地逆置	迭代法三句：存后继、反转指针、双指针后移
2010 循环左移	三次逆置，$O(n)$、$O(1)$
2019 重排链表	找中点 + 逆置后半 + 交叉合并，$O(n)$、$O(1)$

易错清单

本章最容易被扣分的地方

1. 把「线性表」与「顺序表」混为一谈。线性表是逻辑结构，顺序表和链表是存储结构。判断题说「线性表只能顺序存储」是错的。
2. 位序与下标混用。`L.data[i-1]` 对应位序 i；插入合法范围是 $1 \le i \le n+1$，删除是 $1 \le i \le n$。写代码时越界判断的两个条件是高频扣分点。
3. 插入移动次数记成 $\frac{n-1}{2}$、删除记成 $\frac{n}{2}$。务必用记忆锚点：插入有 $n+1$ 个位置（含表尾后），删除只有 n 个位置，所以插入的分母大、平均移动多（$n/2$），删除的平均移动少（$(n-1)/2$）。
4. 把「链表插入删除是 $O(1)$」绝对化。只有「已拿到结点指针」时才是 $O(1)$；给位序就要 $O(n)$ 找位置。
5. 后插两句顺序颠倒。先写 `p->next = s` 会丢失原来的后继地址，链表断开。
6. 双链表插入时语句顺序错乱。核心规则：读 `p->next` 的语句必须写在改 `p->next` 之前。
7. 用「删自身」技巧时忘了判尾结点。尾结点的 `p->next == NULL`，会解引用空指针；必须 `if (p->next != NULL)`。
8. 循环链表删除时忘记维护环和尾指针。链表只剩一个数据结点时删掉它，尾指针必须重新指向头结点

（2021-01、2016-02 反复考这一点）。

9. 判断相交时比较了 `data` 而不是指针。数据相等不等于共享结点。
10. 中间结点的上中位 / 下中位搞反。快慢指针起点决定返回哪一个，必须按题目要求选起点。
11. 快慢指针循环条件写错。判环、找中点时必须写 `while (fast != NULL && fast->next != NULL)`，只判 `fast` 会在 `fast->next->next` 处解引用空指针。
12. 静态链表误以为可以随机存取。它虽然用数组，但逻辑顺序由游标决定，只能从头顺着游标找，是顺序存取。
13. 算法题漏写边界与复杂度。空表、单结点、表长不足 k、参数 $s > t$ 等情形要显式判断；三小问（设计思想、代码、复杂度）要分别作答，代码里要有注释。
14. 题目要求 $O(1)$ 空间却用了数组或递归。递归调用栈算空间（递归法反转链表是 $O(n)$），辅助数组也是空间；不满足时空要求即使结果正确也会失分。

本章历年统考考点分布（按知识点）

- 顺序表：2010 循环左移（算法题）、2011 两序列中位数（算法题）、2013 主元素（算法题）、2018 未出现的最小正整数（算法题）、2020 三元组最小距离（算法题）、2023-01 顺序表操作的平均时间复杂度（选择）、2026-01 哪些操作必然产生元素移动（选择）。
- 链表基本操作与静态链表：2016-01 静态链表插入后的链接地址（选择）、2016-02 双向循环链表删除结点语句（选择）、2021-01 单循环链表删除第一个元素（选择）、2023-02 非空双向链表插入结点（选择）、2024-01 语句序列的作用（选择）、2026-02 双向链表指针调整（选择）。
- 链表算法题：2009 查找倒数第 k 个结点、2012 求两个链表的共同后缀起始位置（相交）、2015 删除绝对值相等的重复结点、2019 重排链表。
- 综合对比：2013 年应用题考过顺序存储结构与链式存储结构的 ASL 比较。

算法设计题复杂度速查表

把本章所有可能被考的算法及其时空代价集中列在这里，考前扫一眼即可。

算法	时间复杂度	空间复杂度	核心手法
顺序表按位查找	$O(1)$	$O(1)$	地址计算
顺序表按值查找	$O(n)$	$O(1)$	顺序扫描
顺序表插入 / 删除	$O(n)$	$O(1)$	搬移元素
顺序表折半查找（有序）	$O(\log_2 n)$	$O(1)$	区间减半
单链表按位 / 按值查找	$O(n)$	$O(1)$	顺链扫描
单链表求表长	$O(n)$	$O(1)$	顺链扫描
头插法 / 尾插法建表	$O(n)$	$O(1)$（不计结点空间）	头插 / 尾指针
合并两个有序表（顺序或链式）	$O(n+m)$	$O(1)$	双指针归并
升序合并成降序链表	$O(n+m)$	$O(1)$	双指针 + 头插
就地逆置（数组 / 链表）	$O(n)$	$O(1)$	首尾交换 / 双指针反转
数组循环左移 p 位	$O(n)$	$O(1)$	三次逆置
找中间结点（快慢指针）	$O(n)$	$O(1)$	快走 2 慢走 1
找倒数第 k 个结点	$O(n)$	$O(1)$	前后双指针
判断链表是否有环	$O(n)$	$O(1)$	快慢指针相遇
求环的入口 / 环长	$O(n)$	$O(1)$	相遇后同速走 / 绕环一圈

算法	时间复杂度	空间复杂度	核心手法
判断两链表是否相交（长度差法）	$O(m+n)$	$O(1)$	对齐起点后同步走
求第一个公共结点（路程补偿）	$O(m+n)$	$O(1)$	走到头切换到对方起点
有序表删除重复元素（双指针）	$O(n)$	$O(1)$	快慢指针原地填充
无序表删除值为 x 的元素	$O(n)$	$O(1)$	快慢指针原地填充
无序链表删重复（借助标记数组）	$O(n)$	$O(n)$	值域作下标打标记
升序输出并释放链表结点	$O(n^2)$	$O(1)$	每趟找最小值
重排链表（2019）	$O(n)$	$O(1)$	中点 + 逆置 + 交叉合并
数组循环左移（2010）	$O(n)$	$O(1)$	三次逆置
求两等长升序序列中位数（2011）	$O(\log_2 n)$	$O(1)$	类二分减半
求数组主元素（2013）	$O(n)$	$O(1)$	摩尔投票 + 验证
找未出现的最小正整数（2018）	$O(n)$	$O(1)$	值域作下标原地交换
三元组最小距离（2020）	$O(n_1+n_2+n_3)$	$O(1)$	三指针移动最小值

记忆要点

线性表算法题的复杂度几乎是「非 $O(n)$ 即 $O(\log n)$」，空间几乎都是 $O(1)$。出现 $O(n^2)$ 通常意味着暴力解（如「找出最小值并删除」反复执行），只有在题目没有更好要求或明确允许时才用；出现 $O(n)$ 空间通常是因为借助了辅助数组或递归，只有题目不限制空间时才用。考前把表中「$O(1)$ 空间」的那几栏重点记住即可。

第三章栈、队列和数组

本章考纲要求

> 三、栈、队列和数组（一）栈和队列的基本概念（二）栈和队列的顺序存储结构（三）栈和队列的链式存储结构（四）多维数组的存储（五）特殊矩阵的压缩存储（六）栈、队列和数组的应用

本章是数据结构的「计算题富矿」：栈与队列的出入操作序列、循环队列的判空判满、矩阵压缩存储的下标换算，三类题目几乎每年必考，且都能用固定套路在 1 分钟内算出答案。复习时要把每一条公式都自己代入一组数验证一遍，不要只背结论。

一、栈和队列的基本概念

（一）栈的定义与特点

栈（**Stack**）是只允许在一端进行插入和删除操作的线性表。允许插入和删除的一端称为栈顶（**top**），另一端称为栈底（**bottom**）。不含任何元素的栈称为空栈。

栈的插入操作称为入栈（**push**），删除操作称为出栈（**pop**）。由于插入和删除都被限制在栈顶，最后入栈的元素必定最先出栈，因此栈是后进先出（**Last In First Out**，**LIFO**）结构。

需要特别注意：栈是操作受限的线性表，这里的「受限」指「只允许在一端操作」，而不是「两端都能操作」。栈的逻辑结构与普通线性表完全相同，仍然是「一对一」的线性关系，改变的只是允许操作的端点。

易错点

「栈允许在其两端进行操作」是错误说法。栈只在栈顶一端插入和删除；支持两端插入删除的是双端队列（**deque**）。栈底是固定不动的一端，不能直接对栈底元素操作，想访问栈底元素必须先把上面所有元素弹出。

（二）队列的定义与特点

队列（**Queue**）是只允许在一端插入、在另一端删除的线性表。允许插入的一端称为队尾（**rear**），允许删除的一端称为队头（**front**）。不含任何元素的队列称为空队列。

队列的插入操作称为入队（**enqueue**），删除操作称为出队（**dequeue**）。先入队的元素必定先出队，因此队列是先进先出（**First In First Out**，**FIFO**）结构。

（三）栈与队列的对比

比较项	栈 **Stack**	队列 **Queue**
操作端点	只在一端（栈顶）插入和删除	一端（队尾）插入，另一端（队头）删除
存取特性	后进先出 LIFO	先进先出 FIFO
逻辑结构	线性表	线性表
存储实现	顺序栈、链栈	顺序队列（含循环队列）、链队列

比较项	栈 **Stack**	队列 **Queue**
典型应用	括号匹配、表达式求值、递归、进制转换、DFS	层次遍历、缓冲区、CPU 调度、BFS

（四）上溢与下溢

栈和队列在采用顺序存储时容量固定，因此存在两种异常状态：

- 上溢（**overflow**）：栈满（或队满）时仍执行入栈（入队）操作。对栈来说，满了还往里写就是上溢。
- 下溢（**underflow**）：栈空（或队空）时仍执行出栈（出队）操作。空了还往外读就是下溢。

顺序栈的上溢是真上溢，只要数组空间确实不够用就必然发生。而顺序队列的上溢可能是假上溢：由于出队只把 `front` 后移而不移动元素，数组前面的空位无法再利用，此时虽然数组内还有空位，`rear` 却已到达末尾，再入队就报「满」。循环队列正是为解决假上溢而提出的。

链式存储的栈和队列不存在上溢问题（只要内存还够就能申请新结点），但出队或出栈时仍要判空，否则会产生下溢。

易错点

「上溢」「下溢」这两个词只用于顺序存储的讨论语境。链栈、链队列不需要判断「满」，因为结点是动态申请的，所以链式存储不存在上溢。

（五）共享栈

利用栈底位置相对不变的特性，可以让两个顺序栈共享同一个一维数组空间：两个栈的栈底分别设在数组两端，两个栈顶向中间延伸。

设共享数组为 `data[0..MaxSize-1]`，用 `top0` 表示 0 号栈的栈顶，`top1` 表示 1 号栈的栈顶。采用「栈顶指针指向当前栈顶元素」的约定时：

- 初始状态：`top0 = -1`，`top1 = MaxSize`（两个栈都为空）；
- 0 号栈入栈：`data[++top0] = x`；1 号栈入栈：`data[--top1] = x`；
- 判满条件：`top0 + 1 == top1`。

共享栈的核心结论

两个栈的栈顶指针相邻时（`top0 + 1 == top1`），整个数组被占满，此时才会发生上溢。

共享栈的意义是只有在整个存储空间被占满时才上溢，让两个栈的空间相互调节。存取数据的时间复杂度仍为 $O(1)$，对存取效率没有影响。

注意区分：共享栈判满用 `top0 + 1 == top1`（全部空间可用）；循环队列牺牲一个单元判满用 `(rear+1)\%M == front`（只存 **M-1** 个）。两者形式相似，含义完全相反，考场上极易混淆。

二、栈和队列的顺序存储结构

（一）顺序栈与 top 指针的两种约定

顺序栈用一组地址连续的存储单元依次存放栈底到栈顶的元素。`top` 指针的含义有两种约定，两种都要掌握，题目给出哪种就用哪种。

比较项	约定 **A**：**top** 指向栈顶元素	约定 **B**：**top** 指向栈顶元素的下一个位置
初始化	`S.top = -1`	`S.top = 0`
判空	`S.top == -1`	`S.top == 0`
判满	`S.top == MaxSize - 1`	`S.top == MaxSize`
入栈	`S.data[++S.top] = x`（先加后存）	`S.data[S.top++] = x`（先存后加）
出栈	`x = S.data[S.top--]`	`x = S.data[--S.top]`
取栈顶	`S.data[S.top]`	`S.data[S.top - 1]`
栈中元素个数	`S.top + 1`	`S.top`

代入一组数验证约定 B（`MaxSize = 3`）：初始 `top = 0` 为空；依次入栈 A、B、C 后，写入位置为 0、1、2，`top = 3`，此时 `top == MaxSize` 判满，元素个数为 `top = 3`，正确。再出栈一次，`--top` 得 2，栈顶元素是 `data[2] = C`，正确。

易错点

考场上最容易错的不是公式本身，而是入栈时先自增还是先存值。判断方法：看 `top` 的含义。若 `top` 指向栈顶元素，则新元素必须放在 `top+1`，所以先自增再存值；若 `top` 指向空位（栈顶下一位置），则新元素直接放在 `top`，所以先存值再自增。题目里若出现 `top == 0` 表示空栈，说明采用的是约定 B。

顺序栈的完整实现（采用约定 A，`top` 指向栈顶元素）：

```
#define MaxSize 50
typedef struct {
    int data[MaxSize];   // 存放栈中元素
    int top;             // 栈顶指针，指向栈顶元素
} SqStack;

void InitStack(SqStack &S) { S.top = -1; }              // 初始化：空栈

bool StackEmpty(SqStack S) { return S.top == -1; }      // 判空

bool StackFull(SqStack S) { return S.top == MaxSize - 1; }   // 判满

bool Push(SqStack &S, int x) {                          // 入栈
    if (S.top == MaxSize - 1) return false;             // 栈满，上溢
    S.data[++S.top] = x;                                // 先加后存
    return true;
}

bool Pop(SqStack &S, int &x) {                          // 出栈
    if (S.top == -1) return false;                      // 栈空，下溢
    x = S.data[S.top--];                                // 先取后减
    return true;
}

bool GetTop(SqStack S, int &x) {                        // 读栈顶，不修改栈
    if (S.top == -1) return false;
    x = S.data[S.top];
    return true;
}
```

（二）顺序队列的 front / rear 约定

顺序队列用数组 data[0..MaxSize-1] 存放元素，用 front 和 rear 两个指针指示队头与队尾。采用「**front** 指向队头元素、**rear** 指向队尾元素的下一个位置」的约定时：

- 初始化：Q.front = Q.rear = 0;
- 判空：Q.front == Q.rear;
- 入队：Q.data[Q.rear] = x; Q.rear++;（**front** 不动，**rear** 变化）;
- 出队：x = Q.data[Q.front]; Q.front++;（**rear** 不动，**front** 变化）;
- 元素所在区间：[front, rear - 1]。

一句话记住谁动

入队时 **front** 指针不会变，变化的是 **rear** 指针；出队时 **rear** 指针不会变，变化的是 **front** 指针。选择题里经常用这句话直接排除错误选项。

这种朴素顺序队列的问题在于：不断入队出队后 front 和 rear 都单调右移，rear 到达数组末尾就无法再入队，即便数组前面还有大量空位——这就是假上溢。解决办法是把数组在逻辑上首尾相接，即循环队列。

（三）循环队列：三种判满方案

循环队列中所有指针移动都改为 (x + 1) % MaxSize。此时的困难在于：队空时 **front == rear**，队满时 **front** 也可能等于 **rear**，两种情况无法区分，必须额外增加信息。

*1.*方案一：牺牲一个存储单元

规定队列最多存放 MaxSize - 1 个元素，即数组中永远保留一个空位，用它隔开「满」和「空」两种状态。

约定：front 指向队头元素，rear 指向队尾元素的后一个位置。

- 判空：front == rear;
- 判满：(rear + 1) % MaxSize == front;
- 元素个数：(rear - front + MaxSize) % MaxSize。

推导：队满时，若再入队一个元素，新的 rear 会变成 (rear+1)%MaxSize，此时它与 front 重合，队列就再也分不清是空还是满了。因此我们在「下一个 rear 即将等于 front」时就判定为满，把那个位置空出来不用。这就是「牺牲一个单元」的由来。

代入验证(MaxSize = 5): 初始 front = rear = 0，判空成立。依次入队 10、20、30，则 data[0..2] = {10,20,30}，rear = 3。元素个数 $= (3-0+5)\%5 = 3$，与实际相符。再入队 40，rear = 4，(4+1)\%5 = 0 == front，判满成立，此时队列中确实是 4 个元素（= MaxSize - 1）。

再验证「绕回」的情形：在上面的状态下出队两次，front = 2；再入队 50、60，data[4]=50 后 rear=0，data[0]=60 后 rear=1。此时元素个数 $= (1-2+5)\%5 = 4$，队列中实际元素为 data[2],data[3],data[4],data[0] 共 4 个，正确。注意这里 **rear < front**，正是选择题最爱的陷阱情形。

*2.*方案二：增设 *size* 计数器

在结构体中增设一个 `int size`（有的教材叫 `count`、`queSize`）记录当前元素个数，所有 `MaxSize` 个单元都可以使用。

- 入队：`size++`；出队：`size--`；
- 判空：`size == 0`；
- 判满：`size == MaxSize`；
- 两个指针的判据 `front == rear` 此时已经不能用来区分空和满了（两种情况都成立）。

代入验证（`MaxSize = 5`）：连续入队 5 个元素，`size = 5` 判满，此时 `front == rear == 0`（`rear` 绕回一圈），说明仅凭指针无法判断，必须依赖 `size`。

*3.*方案三：增设 *tag* 标记

增设一个 `bool tag`，专门记录最近一次操作是入队还是出队（也有教材记「最近一次删除的是否为插入」）。

- 初始化：`front = rear = 0`，`tag = false`；
- 入队成功后：`tag = true`；
- 出队成功后：`tag = false`；
- 判空：`front == rear && tag == false`；
- 判满：`front == rear && tag == true`。

为什么有效：两个指针相等时，若上一次操作是入队，说明是「刚填满一圈追上 `front`」→ 队满；若上一次操作是出队，说明是「刚把最后一个元素取走」→ 队空。如果入队次数与出队次数恰好相等而互相抵消，则最后一次操作必为出队，`tag` 为 `false`，判空成立，正好符合实际。

三种方案的对比与代码要点：

方案	判空	判满	最大容量	需要额外变量	指针相等时的含义
牺牲一个单元	`front == rear`	`(rear+1)%M == front`	`M - 1`	无	一定是队空
设 size	`size == 0`	`size == M`	`M`	`int size`	不确定，看 `size`
设 tag	`front==rear && !tag`	`front==rear && tag`	`M`	`bool tag`	由 tag 决定

```
// 循环队列：牺牲一个存储单元的写法（rear 指向队尾元素的下一个位置）
#define MaxSize 50
typedef struct {
    int data[MaxSize];
    int front, rear;
} SqQueue;

void InitQueue(SqQueue &Q) { Q.front = Q.rear = 0; }

bool QueueEmpty(SqQueue Q) { return Q.front == Q.rear; }

bool QueueFull(SqQueue Q) { return (Q.rear + 1) % MaxSize == Q.front; }
```

```
bool EnQueue(SqQueue &Q, int x) {
    if ((Q.rear + 1) % MaxSize == Q.front) return false;   // 队满
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;                       // rear 循环后移
    return true;
}

bool DeQueue(SqQueue &Q, int &x) {
    if (Q.front == Q.rear) return false;                   // 队空
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;                     // front 循环后移
    return true;
}

int QueueLength(SqQueue Q) {
    return (Q.rear - Q.front + MaxSize) % MaxSize;
}
```

增设 tag 的入队、出队要点：

```
// 设 tag 的写法：front == rear 时由 tag 区分空满
bool EnQueue(SqQueue &Q, int x) {
    if (Q.front == Q.rear && Q.tag == true) return false;  // 队满
    Q.data[Q.rear] = x;
    Q.rear = (Q.rear + 1) % MaxSize;
    Q.tag = true;                                          // 记「刚做过入队」
    return true;
}

bool DeQueue(SqQueue &Q, int &x) {
    if (Q.front == Q.rear && Q.tag == false) return false; // 队空
    x = Q.data[Q.front];
    Q.front = (Q.front + 1) % MaxSize;
    Q.tag = false;                                         // 记「刚做过出队」
    return true;
}
```

易错点

（1）循环队列的容量不等于数组长度。用「牺牲一个单元」的方案时，MaxSize = 6 的数组最多只能存 5 个元素。

（2）取模的模数永远是数组长度 **MaxSize**，很多错误选项会写成% (MaxSize - 1)、% (M-1)，直接排除。

（3）元素个数公式必须加 **MaxSize** 再取模。写成 (rear - front) % MaxSize 在 rear < front（已绕回）时结果错误。举特例时优先举 rear < front 的例子（如 rear = 1、front = 5），最容易看出哪个含 mod 的式子正确。

（4）要看清题目给的是哪种指针约定。若题目说「front 指向队头元素的前一个位置，rear 指向队尾元素」，则判空是 (rear+1)%M == front、判满是 (rear+2)%M == front，与常见写法完全反过来。做法：先按题目约定画出初始空队列，数出两个指针的初始位置，再判断「空」和「满」时指针相差多少格。

真题考法

循环队列是本章最高频的选择题点。典型问法有三类：

- 给出一维数组 A[0..n-1] 与指针指向约定，问初始时 **front**、**rear** 的值（如 2011 年考「第一个元素要存到 A[0]，求初始指针」，答案为 front = 0，rear = n-1）；

- 给出 end1、end2 的指向与「最多容纳 M-1 个元素」，问判空判满条件（2014 年，答案为「队空 `end1 == end2`；队满 `end1 == (end2+1) mod M`」）；
- 给出队头和队尾指针的当前值，问队列中元素个数或再入队/出队后的状态。

2019 年还考过一道 10 分的应用题：要求队列「入队时允许增加空间、出队后空间可重复使用、整个队列占用空间只增不减、入队出队均为 $O(1)$」，答案是采用不设容量上限的循环单链表，队首指针指向第一个含关键字结点、队尾指针指向队尾元素的下一个空闲结点，判空 `front == rear`、判满 `rear->next == front`，出队时不释放结点只移动 `front`。

（四）双端队列（受限双端队列）

双端队列（**deque**）是允许在两端插入和删除的线性表。统考常考它的三种受限形态：

形态	允许入队	允许出队	别称
输入受限双端队列	仅一端	两端	—
输出受限双端队列	两端	仅一端	—
标准双端队列	两端	两端	—

判断某输出序列是否可由受限双端队列得到，最稳妥的办法是从输出序列的第一项倒推，或用「分段」技巧：

- 一端仅入队、另一端可入可出（输出受限）：最终输出序列必可分成前后两段，前一段相对于入队序列是逆序的（这些元素从「可出端」入队，被后来的元素压在后面），后一段是正序的（从另一端入队）。
- 判断时先尝试找一个分界点，使前面逆序、后面正序；找不到分界点则该序列不可能。

三、栈和队列的链式存储结构

（一）链栈

采用链式存储的栈称为链栈。链栈通常用不带头结点的单链表实现，并规定所有操作都在单链表的表头进行：表头结点即栈顶，表尾结点即栈底。

链栈的优点是便于多个栈共享存储空间、提高效率，且不存在栈满上溢的情况。缺点是每个元素都要额外保存指针域，且在「栈空」时需要特别处理空指针。

操作	带头结点	不带头结点
初始化	申请头结点，`S->next = NULL`	`S = NULL`（或 `top = NULL`）
判空	`S->next == NULL`	`S == NULL`
入栈	头插法：`p->next = S->next; S->next = p;`	头插法：`p->next = top; top = p;`
出栈	摘下 `S->next` 并 `free`	摘下 `top` 并 `free`

```
// 链栈：不带头结点，top 直接指向栈顶结点
typedef struct StackNode {
    int data;
    struct StackNode *next;
} StackNode, *LinkStack;
```

```
void InitStack(LinkStack &S) { S = NULL; }                    // 初始化
bool StackEmpty(LinkStack S) { return S == NULL; }            // 判空

bool Push(LinkStack &S, int x) {                              // 入栈 = 头插法
    StackNode *p = (StackNode *)malloc(sizeof(StackNode));
    if (p == NULL) return false;                              // 内存不足
    p->data = x;
    p->next = S;                                              // 新结点指向原栈顶
    S = p;                                                    // 更新栈顶指针
    return true;
}

bool Pop(LinkStack &S, int &x) {                              // 出栈 = 头删法
    if (S == NULL) return false;                              // 栈空，下溢
    StackNode *p = S;
    x = p->data;
    S = p->next;                                              // 栈顶指针下移
    free(p);
    return true;
}

bool GetTop(LinkStack S, int &x) {
    if (S == NULL) return false;
    x = S->data;
    return true;
}
```

易错点

（1）链栈的入栈是「头插法」，出栈是「头删法」，不要在链尾操作——链尾操作需要 $O(n)$ 时间找到尾结点。

（2）链栈出栈时必须先保存待删结点的地址再移动指针，否则 `free` 后无法再接上链表。

（3）链栈的存储空间是动态申请的，因此不需要判满；但出栈前必须判空，否则会对 NULL 解引用。

（二）链队列（带头结点）

链队列是同时带有队头指针和队尾指针的单链表。队头指针指向队头结点（带头结点时指向头结点），队尾指针指向队尾结点。

带头结点的链队列判空与出队都比较统一：

- 判空：`Q.front == Q.rear`（队头、队尾指针指向同一个头结点）；
- 入队：新结点接在 `Q.rear` 之后，再让 `Q.rear` 指向新结点；
- 出队：摘下 `Q.front->next`，若摘下的结点正是队尾结点，还必须把 **`Q.rear` 拉回 `Q.front`**，否则 `rear` 会变成野指针。

```
// 链队列：带头结点
typedef struct QNode {
    int data;
    struct QNode *next;
} QNode;

typedef struct {
    QNode *front, *rear;      // 队头指针、队尾指针
} LinkQueue;
```

```
void InitQueue(LinkQueue &Q) {                          // 初始化
    Q.front = Q.rear = (QNode *)malloc(sizeof(QNode));  // 建立头结点
    Q.front->next = NULL;
}

bool QueueEmpty(LinkQueue Q) { return Q.front == Q.rear; }   // 判空

void EnQueue(LinkQueue &Q, int x) {                     // 入队：只动 rear
    QNode *s = (QNode *)malloc(sizeof(QNode));
    s->data = x;
    s->next = NULL;
    Q.rear->next = s;                                   // 新结点接到队尾之后
    Q.rear = s;                                         // 更新队尾指针
}

bool DeQueue(LinkQueue &Q, int &x) {                    // 出队：只动 front
    if (Q.front == Q.rear) return false;                // 队空
    QNode *p = Q.front->next;                           // p 指向真正的队头结点
    x = p->data;
    Q.front->next = p->next;                            // 摘下队头结点
    if (Q.rear == p) Q.rear = Q.front;                  // 若摘的是最后一个结点，rear 归位
    free(p);
    return true;
}
```

（三）链队列（不带头结点）

不带头结点时，队空表现为 `Q.front == NULL && Q.rear == NULL`。出队和入队的边界情况都更多：

- 判空：`Q.front == NULL`（等价于 `Q.rear == NULL`）；
- 入队：新建结点后置 `next = NULL`；若原队列为空，则 **front** 和 **rear** 同时指向新结点，否则接到 `Q.rear` 之后并只更新 `rear`；
- 出队：若出队的是最后一个结点，出队后必须把 **front** 和 **rear** 同时置为 **NULL**，否则 `rear` 悬空。

```
// 链队列：不带头结点
void InitQueue(LinkQueue &Q) { Q.front = Q.rear = NULL; }

bool QueueEmpty(LinkQueue Q) { return Q.front == NULL; }

void EnQueue(LinkQueue &Q, int x) {
    QNode *s = (QNode *)malloc(sizeof(QNode));
    s->data = x;
    s->next = NULL;
    if (Q.front == NULL) {          // 队列原本为空
        Q.front = Q.rear = s;       // 首尾指针同时指向新结点
    } else {
        Q.rear->next = s;           // 否则只更新队尾
        Q.rear = s;
    }
}

bool DeQueue(LinkQueue &Q, int &x) {
    if (Q.front == NULL) return false;   // 队空
```

```
    QNode *p = Q.front;
    x = p->data;
    Q.front = p->next;                    // 队头指针后移
    if (Q.front == NULL) Q.rear = NULL;  // * 出队后为空，rear 也要置空
    free(p);
    return true;
}
```

带头结点与不带头结点的取舍

判空判满与边界处理的差别，本质上就是「有没有那个不存数据的头结点」。带头结点时，`front == rear` 唯一对应队空，出队永远摘 `front->next`，不用讨论「摘到最后一个」这种特殊情况；不带头结点时，队空的表示、第一个元素入队、最后一个元素出队都要单独处理。

统考选择题里最常考的两句话：

- 「最适合用作链式队列的是带有队首指针和队尾指针的非循环单链表」；
- 「只带队首指针的非循环单链表，入队需要遍历到表尾，时间复杂度 $O(n)$，因此不适合」。

另外，带尾指针的循环单链表求队头和队尾都在 $O(1)$ 内（`rear->next` 就是队头结点），也可以用作链队列；带首尾指针的循环单链表虽然可行，但每次修改指针还要维护循环特性，属于画蛇添足。

链队列同样不存在队满上溢，出队前判空是唯一必需的边界检查。与顺序队列相比，链队列的缺点是每个结点都有指针开销，且不能随机访问第 k 个元素。

四、多维数组的存储

（一）数组的定义与一维地址公式

数组是由 n ($n \geq 1$) 个相同类型的数据元素构成的有限序列。每个元素在 n 个线性关系中的序号称为该元素的下标，下标的取值范围称为数组的维界。

一维数组 `a[0..n-1]` 中，每个元素占 L 个存储单元，则：

$$LOC(a_i) = LOC(a_0) + i \times L$$

其中 $LOC(a_i)$ 表示元素 a_i 的存储地址（首地址），$LOC(a_0)$ 是数组起始地址，i 是下标，L 是单个元素占用的字节数。

若下标从 **1** 开始（即 `a[1..n]`），则 a_i 前面有 $i-1$ 个元素，公式变为：

$$LOC(a_i) = LOC(a_1) + (i-1) \times L$$

（二）二维数组的行优先与列优先

二维数组可以看成一维数组的推广。按「先存哪一维」的不同，有两种映射方式：

- 行优先（行主序）：先行后列。先存行号较小的元素，行号相同时先存列号较小的元素。
- 列优先（列主序）：先列后行。先存列号较小的元素，列号相同时先存行号较小的元素。

设二维数组 A 的行下标范围是 $[0, h_1]$、列下标范围是 $[0, h_2]$，每个元素占 L 个单元，则共有 (h_1+1) 行、(h_2+1) 列。

行优先地址公式：

$$LOC(a_{ij}) = LOC(a_{00}) + [\,i \times (h_2 + 1) + j\,] \times L$$

列优先地址公式：

$$LOC(a_{ij}) = LOC(a_{00}) + [\,j \times (h_1 + 1) + i\,] \times L$$

符号说明：i 为行下标，j 为列下标，$h_2 + 1$ 是每行的元素个数（即列数 n），$h_1 + 1$ 是每列的元素个数（即行数 m），L 为每个元素占用的存储单元数。

行优先公式的推导：a_{ij} 之前有完整的 i 行（第 0 行到第 $i-1$ 行），每行 h_2+1 个元素，共 $i \times (h_2+1)$ 个；本行中列号小于 j 的元素还有 j 个。因此 a_{ij} 是数组中的第 $i(h_2+1)+j$ 个元素（从 0 开始编号），偏移量乘以 L 即为字节偏移。

列优先公式同理：a_{ij} 之前有完整的 j 列，每列 $h_1 + 1$ 个元素，共 $j(h_1+1)$ 个；本列中行号小于 i 的元素有 i 个。

代入验证：设 A 为 4×5 的二维数组（$h_1 = 3$，$h_2 = 4$），$L = 2$ 字节，$LOC(a_{00}) = 1000$。

- 行优先求 $LOC(a_{2,3})$：偏移量 $= 2 \times 5 + 3 = 13$，地址 $= 1000 + 13 \times 2 = 1026$。
- 列优先求 $LOC(a_{2,3})$：偏移量 $= 3 \times 4 + 2 = 14$，地址 $= 1000 + 14 \times 2 = 1028$。

两者不同是正确的——同一个元素在两种存储顺序下的位置本来就不一样。再看一个极端情形：行优先下 $LOC(a_{2,0})$ 的偏移量是 10，$LOC(a_{1,4})$ 的偏移量是 9，确实 $a_{1,4}$ 排在 $a_{2,0}$ 前面，符合「先行后列」。

（三）下标从 **0** 与从 **1** 开始的差异

统考题目经常混用「C 语言数组下标从 0 开始」和「矩阵元素下标从 1 开始」（如 m_{ij} 满足 $1 \le i \le n$），这是最容易丢分的地方。

设矩阵为 n 行 m 列，行优先，每元素占 L 个单元：

- 下标从 **0** 开始（$i \in [0, n-1]$，$j \in [0, m-1]$）：

$$LOC(a_{ij}) = LOC(a_{00}) + (i \times m + j) \times L$$

- 下标从 **1** 开始（$i \in [1, n]$，$j \in [1, m]$）：

$$LOC(a_{ij}) = LOC(a_{11}) + [\,(i-1) \times m + (j-1)\,] \times L$$

两者的区别仅仅是每个下标都减 **1**，把「下标从 1 起」换成从 0 起的相对编号即可。做题时建议统一换算成「以 0 为基的偏移量」再计算，不要直接背两套公式。

真题考法

2021 年考过一道典型题：二维数组 A 按行优先存储，每个元素占 1 个存储单元，$A[0][0]$ 的地址是 100，$A[3][3]$ 的地址是 220，求 $A[5][5]$ 的地址。

解法：设每行 n 个元素，则 $LOC(a_{33}) = LOC(a_{00}) + (3n+3) \times 1 = 100 + 3n + 3$，令它等于 220 解得 $3n = 117$，即 $n = 39$。于是 $LOC(a_{55}) = 100 + (5 \times 39 + 5) = 100 + 200 = 300$。

这类题的关键是先用已知条件反解出「每行多少个元素」，再套公式。

五、特殊矩阵的压缩存储

（一）压缩存储的基本思路

矩阵的压缩存储：为多个值相同的元素只分配一个存储空间，对零元素不分配空间。

普通二维数组按行优先存放 $n \times n$ 矩阵需要 n^2 个单元；若矩阵中有大量元素相同或为零，就可以只存「有用的部分」，其余部分由映射规则推算出来，从而把空间降到 $O(n)$ 或 $O(n)$ 以下的常数倍。

压缩存储的两步固定套路：

1. 确定存哪些元素、按什么顺序存，数出一维数组的总长度（一般是一个等差数列求和）；
2. 把矩阵下标 (i, j) 映射为一维数组下标 k，即数出 a_{ij} 之前共有多少个元素。

（二）对称矩阵

若 n 阶方阵 A 中任意元素都满足 $a_{ij} = a_{ji}$，则称 A 为对称矩阵。

对称矩阵关于主对角线对称，因此只需存储主对角线 + 下三角区（或主对角线 + 上三角区），共

$$S = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

个元素。一维数组最后一个元素的下标为 $\dfrac{n(n+1)}{2} - 1$（数组下标从 0 开始）。

*1.*存主对角线 + 下三角区，按行优先

元素 a_{ij} 满足 $i \geq j$（在下三角区或主对角线上）时，直接取值：

$$k = \frac{i(i+1)}{2} + j$$

下标从 0 开始。推导：a_{ij} 之前有完整的第 0 行到第 $i-1$ 行，元素个数为 $1+2+\cdots+i = \dfrac{i(i+1)}{2}$；本行中列号小于 j 的元素还有 j 个（列号 $0, 1, \cdots, j-1$）。所以 a_{ij} 是第 $\dfrac{i(i+1)}{2} + j$ 个元素，从 0 开始编号即为该式。

代入验证：$n = 4$，数组下标从 0 开始。求 $a_{2,1}$：$k = \dfrac{2 \times 3}{2} + 1 = 3 + 1 = 4$。逐个手排验证：$a_{00} \to 0$，$a_{10} \to 1$，$a_{11} \to 2$，$a_{20} \to 3$，$a_{21} \to 4$，完全吻合。再验证对角元 $a_{3,3}$：$k = \dfrac{3 \times 4}{2} + 3 = 6 + 3 = 9$，而 $\dfrac{4 \times 5}{2} - 1 = 9$，正好是最后一个元素，吻合。

*2.*存主对角线 + 下三角区，按列优先

列优先时，第 c 列只有 $a_{cc}, a_{c+1,c}, \cdots, a_{n-1,c}$ 共 $n - c$ 个元素（因为是下三角，第 c 列第一个被存储的元素是主对角线上的 a_{cc}）。

a_{ij}（$i \geq j$）所在第 j 列的前面有第 0 列到第 $j-1$ 列，元素个数为

$$\sum_{c=0}^{j-1} (n - c) = jn - \frac{j(j-1)}{2}$$

a_{ij} 在本列中位于 a_{jj} 之后的第 $i - j$ 个位置。因此

$$k = j\,n - \frac{j(j-1)}{2} + (i-j) \quad (i \geq j,\ \text{列优先})$$

代入验证：$n=4$，数组下标从 0 开始。手排第 0 列 $a_{00}, a_{10}, a_{20}, a_{30}$ 占下标 0～3；第 1 列 a_{11}, a_{21}, a_{31} 占下标 4～6；第 2 列 a_{22}, a_{32} 占下标 7、8。

求 a_{21}（$i=2, j=1$）：$k = 1\times 4 - \dfrac{1\times 0}{2} + (2-1) = 4+1 = 5$。手排中 a_{21} 是第 1 列的第 2 个元素（a_{11} 是第 1 个），下标为 $4+1=5$，吻合。

再验证一个 j 较大的情形，a_{32}（$i=3, j=2$）：$k = 2\times 4 - \dfrac{2\times 1}{2} + (3-2) = 8-1+1 = 8$。手排中第 2 列从下标 7 开始（$a_{22}\to 7$，$a_{32}\to 8$），吻合。同时 $k=8$ 正好是数组最后一个下标 $\dfrac{4\times 5}{2} - 1 = 9$ 之前的位置——注意 a_{33} 才是最后一个元素（$k = 3\times 4 - 3 + 0 = 9$），也对得上。

对称矩阵总表（下标从 0 开始）

设 n 阶对称矩阵，只压缩存储主对角线 + 下三角区，一维数组 B 共 $\dfrac{n(n+1)}{2}$ 个单元：

存储顺序	$i \geq j$ 时 k 的取值	$i < j$ 时如何取值
行优先	$k = \frac{i(i+1)}{2} + j$	利用对称性：$a_{ij} = a_{ji}$，代入 $k = \frac{j(j+1)}{2} + i$
列优先	$k = jn - \frac{j(j-1)}{2} + (i-j)$	利用对称性：$a_{ij} = a_{ji}$，代入列优先公式（此时 $j < i$ 视为 $(i', j') = (j, i)$）

记忆要点：行优先的公式 $\dfrac{i(i+1)}{2} + j$ 最好记，也最常考。题目给出「上三角」时不要另背新公式，先用对称性把元素翻到下三角再套公式，一条公式包打天下。

翻下标的验证：$n=4$，按列优先存下三角。求 a_{02}（$i=0<j=2$，在上三角，不属于直接存储区）。用对称性得 $a_{02} = a_{20}$，代入 $i'=2$、$j'=0$：$k = 0\times 4 - \dfrac{0\times(-1)}{2} + (2-0) = 2$。手排第 0 列是 $a_{00}, a_{10}, a_{20}, a_{30}$（下标 0～3），$a_{20}$ 的下标正是 2，吻合。

*3.*存主对角线 + 上三角区

若题目要求存储主对角线 + 上三角区，则按行优先时 a_{ij}（$i \leq j$）之前有完整的第 0 行到第 $i-1$ 行，第 c 行有 $n-c$ 个元素，个数为

$$\sum_{c=0}^{i-1}(n-c) = in - \frac{i(i-1)}{2}$$

再加上本行中列号小于 j 的元素 $j-i$ 个，得

$$k = in - \frac{i(i-1)}{2} + (j-i) \quad (i \leq j)$$

代入验证：$n=4$，按行优先存上三角。手排：第 0 行 $a_{00}, a_{01}, a_{02}, a_{03}$ 占下标 0～3；第 1 行 a_{11}, a_{12}, a_{13} 占下标 4～6；第 2 行 a_{22}, a_{23} 占下标 7、8。求 a_{12}：$k = 1\times 4 - \dfrac{1\times 0}{2} + (2-1) = 4+1 = 5$，手排中 a_{12} 是第 1 行第 1 个，下标 $4+1=5$，吻合。

真题考法

对称矩阵的压缩存储是本章计算题第一高频点。统考的常见包装方式是「给 $n\times n$ 对称矩阵，只存上三角（或下三角）部分，按行（或列）优先存入下标从 0 开始的一维数组 N，求某元素的下标」。三条实战建议：

1. 先判断该元素属于哪一半。若元素在下三角而题目只存上三角，用 $a_{ij}=a_{ji}$ 翻过去（如 2020 年题给 $m_{7,2}$，10×10 对称矩阵只存上三角，$7>2$ 在下三角，翻成 $m_{2,7}$）。
2. 老老实实数「它前面有几个元素」，不要临时推导新公式。例如 2018 年题：12×12 对称矩阵只存上三角、按行优先，求 $m_{6,6}$。第 1 行存 12 个、第 2 行存 11 个、……、第 5 行存 8 个，共 $12+11+10+9+8=50$ 个；$m_{6,6}$ 是第 6 行第一个元素，故下标为 50。
3. 注意题目用的是 **1** 起始下标还是 **0** 起始下标。题目说「m_{ij}，$1\le i\le j\le12$」说明矩阵下标从 1 起，而「存入下标从 0 开始的一维数组」说明数组从 0 起，两者各按各的口径数。

（三）三角矩阵

- 下三角矩阵：主对角线及下三角区以外的元素（即上三角区）全部为同一个常数 c（通常为 0）。
- 上三角矩阵：主对角线及上三角区以外的元素（即下三角区）全部为同一个常数 c。

与对称矩阵的关键区别：三角矩阵不可利用对称性互相转换，下三角矩阵的零区不能通过下标翻转来访问，必须单独存一个常数。

压缩策略（以下三角矩阵为例）：按行优先存主对角线 + 下三角区，并把那个常数存在一维数组的最后一个位置。

一维数组长度为

$$S=\frac{n(n+1)}{2}+1$$

最后一个元素的下标为 $\dfrac{n(n+1)}{2}$。

映射关系（下标从 0 开始）：

$$k=\begin{cases}\dfrac{i(i+1)}{2}+j, & i\ge j\ (\text{下三角区或主对角线})\\[2ex] \dfrac{n(n+1)}{2}, & i<j\ (\text{上三角区，取常数}c)\end{cases}$$

代入验证：$n=3$，下三角矩阵

$$A=\begin{pmatrix}a_{00} & c & c\\ a_{10} & a_{11} & c\\ a_{20} & a_{21} & a_{22}\end{pmatrix}$$

手排（行优先，存下三角 + 常数）：下标 0 存 a_{00}，1 存 a_{10}，2 存 a_{11}，3 存 a_{20}，4 存 a_{21}，5 存 a_{22}，6 存常数 c。数组长度为 $\dfrac{3\times4}{2}+1=7$，最后下标为 6。用公式求 a_{21}：$k=\dfrac{2\times3}{2}+1=3+1=4$，吻合。求 a_{02}（$i<j$）：$k=\dfrac{3\times4}{2}=6$，取得常数 c，吻合。

上三角矩阵按行优先存主对角线 + 上三角区时，a_{ij}（$i\le j$）的映射为

$$k=in-\frac{i(i-1)}{2}+(j-i)$$

其余位置（$i>j$）取一维数组最后一个单元中的常数。

易错点

（1）对称矩阵与三角矩阵的公式看起来一样，但 $i<j$（或 $i>j$）时的处理完全不同：对称矩阵靠 $a_{ij}=a_{ji}$ 翻下标访问，三角矩阵直接返回常数区的那个单元。

（2）三角矩阵的一维数组要多开 **1** 个单元存常数，长度是 $\frac{n(n+1)}{2}+1$ 而不是 $\frac{n(n+1)}{2}$。考试若问「数组至少需要多少个元素」，答 $\frac{n(n+1)}{2}+1$。

（3）用「行优先」还是「列优先」要看清题干，两者的 k 公式不同。若题目没说明，统考默认按行优先。

（四）三对角矩阵（带状矩阵）

三对角矩阵又称带状矩阵：n 阶方阵 A 中，当 $|i-j|>1$ 时 $a_{ij}=0$。也就是说非零元素只集中在以主对角线为中心的 **3** 条对角线上：主对角线、主对角线下方一条、主对角线上方一条。

每行的非零元素个数：第 1 行（下标 $i=1$）只有 2 个，第 2 行到第 $n-1$ 行各有 3 个，第 n 行只有 2 个。总数为

$$2+3(n-2)+2=3n-2$$

一维数组最后一个元素的下标为 $3n-3$（下标从 0 开始）。

按行优先压缩存储的映射公式（下标从 1 开始，即 $1\le i,j\le n$）：

$$k=2i+j-3\quad(|i-j|\le 1)$$

推导：a_{ij} 之前有完整的 $i-1$ 行。其中第 1 行有 2 个元素，第 2 行到第 $i-1$ 行共有 $i-2$ 行、每行 3 个，合计 $2+3(i-2)=3i-4$ 个。a_{ij} 在本行中是第 $j-i+2$ 个（本行元素依次是 $a_{i,i-1},a_{ii},a_{i,i+1}$，对应本行内序号 $1,2,3$；下标 j 对应序号 $j-(i-1)+1=j-i+2$）。所以 a_{ij} 是全体中的第 $(3i-4)+(j-i+2)=2i+j-2$ 个，从 0 开始编号即为 $2i+j-3$。

代入验证一（边界）：$i=1$，取 a_{11}，$k=2+1-3=0$，正是压缩数组的第 1 个元素，正确。取 a_{22}（$i=2,j=2$）：$k=4+2-3=3$。手排：第 1 行 a_{11},a_{12} 占 0、1；第 2 行 a_{21},a_{22},a_{23} 占 2、3、4，a_{22} 下标为 3，正确。

代入验证二（统考真题数据）：100 阶三对角矩阵，元素 $m_{i,j}$（$1\le i,j\le 100$）按行优先压缩存入下标从 **0** 开始的一维数组 N，求 $m_{30,30}$ 的下标。用公式：$k=2\times 30+30-3=87$。

换一种独立方法核算：第 1 行有 2 个元素；第 2 行到第 29 行共 28 行、每行 3 个，共 84 个；合计 $2+84=86$ 个元素排在第 30 行之前，故第 30 行第一个元素 $m_{29,30}$ 的下标为 86。第 30 行按 $m_{29,30},m_{30,30},m_{30,31}$ 的顺序存放，$m_{30,30}$ 再后移一位，下标为 $86+1=87$。两种方法结果一致。

若按列优先存储（下标从 1 开始），由对称的推导可得

$$k=2j+i-3\quad(|i-j|\le 1)$$

即把行优先公式中的 i、j 互换。

易错点

三对角矩阵题的三个坑：

1. 下标基数。公式 $k=2i+j-3$ 是矩阵下标从 **1** 开始的形式。若题目给出的是 C 语言风格的 0 起始下标，需要先按 $+1$ 换算，或直接用「数一数前面有几个元素」的方法避免混淆。
2. 首行和末行只有 **2** 个元素。若直接按「每行 3 个」算，会多算 1 个，答案偏大。
3. 一维数组的下标是 **0** 起还是 **1** 起。题目说「压缩存入下标从 0 开始的一维数组」时，公式结果直接用；若说「一维数组 $N[1..3n-2]$」，则结果要按 1 起始再调一次。

（五）稀疏矩阵

稀疏矩阵：矩阵中非零元素的个数 t 相对于矩阵元素总数 s 非常少，即 $s \gg t$ 的矩阵。稀疏矩阵中零元素的分布通常没有规律，因此不能靠下标映射推算，必须显式记录每个非零元素的位置。

统考要求掌握两种压缩存储结构：

*1.*三元组表（顺序存储）

为每个非零元素存一个三元组 <行下标，列下标，值 >，所有三元组按行优先（或列优先）顺序排成一个顺序表。此外还必须额外保存矩阵的总行数、总列数（通常还有非零元素个数）。

为什么必须额外保存行数和列数：三元组只记录「非零元素出现在哪里」，无法确定矩阵的整体规模。如果最后几行或最后几列全是零，它们不会出现在任何三元组中，只凭三元组无法知道这些全零行、全零列是否存在。

```
#define MaxSize 1000
typedef struct {
    int i, j;        // 行下标、列下标
    int v;           // 元素值
} Triple;

typedef struct {
    Triple data[MaxSize];   // 非零元素的三元组表
    int mu, nu, tu;         // 矩阵的行数、列数、非零元素个数
} TSMatrix;
```

三元组表适合顺序存取，但不适合随机存取：要按行号查找某个元素必须从头扫描。若需频繁按行、按列访问，应改用十字链表。

*2.*十字链表（链式存储）

十字链表的每个非零元素对应一个结点，结点内同时挂在行链表和列链表上，因此可以同时按行、按列遍历非零元素。它是稀疏矩阵的链式压缩存储方式，也是图的「有向图十字链表」的同名不同物结构（图那一章的十字链表用于存弧）。

十字链表的结构要点：

- 每个非零元素结点含 5 个域：行下标 `i`、列下标 `j`、值 `v`、指向同列下一个非零元素的 `down` 指针、指向同行下一个非零元素的 `right` 指针；
- 设立行头指针数组和列头指针数组（或把行头、列头结点也串成一个链表）；
- 整个矩阵用一个总的头结点记录行数、列数、非零元素个数。

三元组表与十字链表的对比：

比较项	三元组表	十字链表
存储方式	顺序存储	链式存储
每个非零元素的空间	3 个域	5 个域（含 2 个指针）
按行访问	顺序扫描，较慢	沿行链 O(行内非零元个数)
按列访问	需转置或重建，很慢	沿列链，与按行同效
插入/删除非零元素	需移动大量元素	修改指针即可

比较项	三元组表	十字链表
需要保存的信息	行数、列数、非零元个数	行数、列数、非零元个数（存在总头结点中）

真题考法

- 2017 年考「适用于压缩存储稀疏矩阵的两种存储结构」，答案是三元组表和十字链表。干扰项「邻接矩阵」（需要为所有行列位置分配空间，本质是完整二维数组，不属于压缩存储）和「二叉链表」（不是矩阵的存储结构）都要能排除。
- 2023 年考「用三元组表存储稀疏矩阵，除三元组外还需要保存什么」，答案是矩阵的总行数和总列数（「包含非零元素的行数/列数」可以通过扫描三元组统计出来，不是重建矩阵所必需的规模信息）。
- 三元组表还可以考转置算法：朴素转置按列扫描，时间复杂度 $O(nu \times tu)$；快速转置先统计每列非零元个数并计算各列起始位置，再一趟扫描直接放到目标位置，时间复杂度 $O(nu + tu)$。

六、栈、队列和数组的应用

（一）栈在括号匹配中的应用

算法原理：从左到右扫描表达式。遇到左括号就入栈（后遇到的左括号要先闭合，因此放在栈顶）；遇到右括号时，若栈空说明右括号多余，匹配失败；否则弹出栈顶左括号，检查类型是否相同，不同则失败。扫描结束后栈必须为空，否则说明左括号多余。

判定的三种失败情形：右括号到来时栈空（右括号多）；弹出的左括号类型不匹配（如 `[` 配 `)`）；扫描结束时栈非空（左括号多）。

```
bool BracketMatch(char str[], int n) {
    SqStack S; InitStack(S);
    if (n % 2 == 1) return false;             // 长度为奇数必不匹配
    for (int i = 0; i < n; i++) {
        if (str[i]=='(' || str[i]=='[' || str[i]=='{') {
            Push(S, str[i]);                   // 左括号入栈
        } else {
            if (StackEmpty(S)) return false;   // 右括号多余
            char top;
            Pop(S, top);                       // 弹出栈顶左括号
            if (str[i]==')' && top!='(') return false;
            if (str[i]==']' && top!='[') return false;
            if (str[i]=='}' && top!='{') return false;
        }
    }
    return StackEmpty(S);                      // 栈空才算匹配
}
```

代入验证：输入 `{[()]}`。扫描 `{`、`[`、`(` 依次入栈；遇 `)` 弹出 `(` 匹配；遇 `]` 弹出 `[` 匹配；遇 `}` 弹出 `{` 匹配；结束栈空，返回 `true`。输入 `([)]`：遇 `)` 时栈顶是 `[`，类型不同，返回 `false`，正确。

（二）栈在表达式求值中的应用

*1.*前缀、中缀、后缀表达式

- 中缀表达式：运算符在两个操作数之间，如 `a+b`。中缀表达式需要括号和优先级规则才能确定运算顺序，计算机直接处理不方便。

- 后缀表达式（逆波兰式）：运算符在两个操作数之后，如 ab+。后缀表达式不需要括号就能唯一确定运算顺序，最适合用栈求值。
- 前缀表达式（波兰式）：运算符在两个操作数之前，如 +ab。

手算转换方法（推荐）：先给中缀表达式按运算顺序加括号，再把每个括号内的运算符移到对应位置的后面（后缀）或前面（前缀），最后去掉所有括号。

例 **1**：中缀 a+b-c*d。

- 确定运算顺序并加括号：((a+b)-(c*d))；
- 后缀：把 + 移到 ab 后得 ab+，* 移到 cd 后得 cd*，- 移到最后得 ab+cd*-；
- 前缀：+ 移到 ab 前得 +ab，* 移到 cd 前得 *cd，- 移到最前得 -+ab*cd。

例 **2**：中缀 a/b+(c*d-e*f)/g。

- 加括号：((a/b)+(((c*d)-(e*f))/g))；
- 后缀：ab/cd*ef*-g/+；
- 前缀：+/ab/-*cd*efg。

例 **3**（统考真题数据）：中缀 x+y*(z-u)/v。

- 加括号：(x+((y*(z-u))/v))；
- 后缀：xyzu-*v/+。

对照 2024 年该题的四个选项，只有 xyzu-*v/+ 与之一致。

*2.*用栈实现后缀表达式的计算

使用一个操作数栈，从左到右扫描后缀表达式：

- 遇到操作数：直接入栈；
- 遇到运算符：连续弹出两个操作数，先出栈的是右操作数，后出栈的是左操作数，执行左 op 右运算，把结果压回栈；
- 扫描结束后，栈中唯一的元素即为表达式的值。

代入验证：后缀 34+5*（对应中缀 (3+4)*5）。

- 遇 3，栈 [3]；
- 遇 4，栈 [3,4]；
- 遇 +，弹出 4（右）、3（左），算 $3+4=7$，栈 [7]；
- 遇 5，栈 [7,5]；
- 遇 *，弹出 5（右）、7（左），算 $7\times 5=35$，栈 [35]。

结果为 35，正确。注意减法、除法必须区分左右操作数：后缀 53- 的值是 $5-3=2$，不是 $3-5=-2$。

*3.*中缀转后缀的算法

初始化一个栈，用于保存暂时还不能确定运算顺序的运算符。从左到右扫描中缀表达式：

- 遇到操作数：直接加入后缀表达式；
- 遇到左括号 (：直接入栈；

- 遇到右括号)：依次弹出栈内运算符并加入后缀表达式，直到弹出 (为止，(不加入后缀表达式；
- 遇到运算符：
  1. 若优先级高于栈顶运算符（或栈空、栈顶是 (），当前运算符直接入栈；
  2. 若优先级小于等于栈顶运算符，则不断弹出优先级大于等于当前运算符的栈顶元素加入后缀表达式，直到栈空或栈顶为 (，再把当前运算符入栈；
- 扫描结束后，把栈中剩余运算符依次弹出并加入后缀表达式。

代入验证：中缀 a/b+(c*d-e*f)/g，扫描到 f 时栈中的内容（栈底到栈顶）。

- 读 a → 输出 a；
- 读 / → 栈空，入栈。栈：/；
- 读 b → 输出 b；
- 读 + → 优先级 1 ≤ 栈顶 /（2），弹出 / 输出；栈空，+ 入栈。栈：+；
- 读 (→ 直接入栈。栈：+，(；
- 读 c → 输出 c；
- 读 * → 栈顶是 (，直接入栈。栈：+，(，*；
- 读 d → 输出 d；
- 读 - → 优先级 1 ≤ 栈顶 *（2），弹出 * 输出；栈顶为 (，停止；- 入栈。栈：+，(，-；
- 读 e → 输出 e；
- 读 * → 优先级 2 > 栈顶 -（1），直接入栈。栈：+，(，-，*；
- 读 f → 输出 f（此时栈底到栈顶依次是 +、(、-、*）。

这个结果与 2014 年真题的答案选项一致。

*4.*用栈直接计算中缀表达式

初始化两个栈：操作数栈 num 和运算符栈 op。

- 遇到操作数：压入 num；
- 遇到 (：压入 op；
- 遇到)：不断执行「求值」(弹出一个运算符、弹出两个操作数、运算、结果压回 num)，直到 op 栈顶为 (，然后弹出 (；
- 遇到运算符：当 op 非空且栈顶运算符优先级大于等于当前运算符时，不断求值，再把当前运算符压入 op；
- 全部扫描完后，继续求值直到 op 为空，num 栈顶即为结果。

这里的「中缀转后缀」与「后缀求值」两套逻辑被合并在一次扫描中完成，效率和正确性都更好。

```
// 中缀表达式直接求值：操作数栈 + 运算符栈
void eval(stack<int> &num, stack<char> &op) {
    int a = num.top(); num.pop();      // 先弹出的是右操作数
    int b = num.top(); num.pop();      // 后弹出的是左操作数
    char p = op.top(); op.pop();
    if (p == '+') num.push(b + a);
    else if (p == '-') num.push(b - a);   // * 注意是 b - a
    else if (p == '*') num.push(b * a);
    else num.push(b / a);                 // * 注意是 b / a
}
```

易错点

（1）减法与除法必须区分左右操作数。后缀求值时先出栈的是右操作数，写成 `a - b` 就错了。

（2）中缀转后缀时「不入后缀表达式，遇到）弹出到「并把「丢弃。

（3）中缀转后缀的「弹出」条件是优先级「小于等于」（同级也要弹出，保证左结合），而「直接入栈」的条件是优先级「大于」。选择题常在这里设置干扰。

（4）不要声称「消除递归必须使用栈」。尾递归可以直接用循环改写，迭代法不一定需要显式栈。2017 年真题正是考这个错误说法。

（5）中缀表达式本身不唯一。同一个中缀表达式在运算符优先级相同时（如 `a+b+c`）可以由不同的结合顺序得到不同的后缀形式，但结果值相同。统考默认按「左优先」原则转换。

（6）中缀转后缀时不要漏掉扫描结束后的收尾步骤。全部字符扫描完后，必须把操作符栈中剩余的运算符依次弹出并追加到后缀表达式末尾，否则会丢掉最后几个运算符。选择题常问「扫描到某个字符时栈内还剩哪些运算符」，此时收尾步骤尚未执行。

三种表达式与栈的对应关系

表达式形式	例子（中缀 `a+b-c*d`）	用栈求值的方式
前缀（波兰式）	`-+ab*cd`	从右向左扫描；遇操作数入栈，遇运算符弹两个操作数运算后压回
中缀	`a+b-c*d`	需要两个栈（操作数栈 + 运算符栈）同时处理优先级与括号
后缀（逆波兰式）	`ab+cd*-`	从左向右扫描；遇操作数入栈，遇运算符弹两个操作数运算后压回

前缀求值时先出栈的是左操作数，后缀求值时先出栈的是右操作数——这是因为两者扫描方向相反，弹栈的先后顺序自然也相反。这个对应关系记清了，两类题都不会错。

（三）栈在递归中的应用

函数调用的特点是最后调用的函数最先被执行（**LIFO**）。调用函数时，系统需要用函数调用栈保存三类信息：调用返回地址、实参、局部变量。递归调用时，这个栈称为递归工作栈。

- 每进入一层递归，就把该层的返回地址、实参、局部变量压入栈顶；
- 每退出一层递归，就从栈顶弹出这些信息，恢复到上一层。

递归的优缺点：代码简洁、可读性好；但效率较低，递归层数过多会导致栈溢出（栈空间耗尽），且可能存在大量重复计算（如朴素的斐波那契递归）。

递归与栈的关系（**2015** 年真题考法）：程序 `main()` 调用 `S(1)`，`S(1)` 调用 `S(0)`，则递归最深时，从栈底到栈顶依次是 `main()`、`S(1)`、`S(0)`。这也解释了为什么「用非递归方式重写递归程序时不一定必须使用栈」——尾递归可以改成循环，但函数调用本身必须用栈保存现场。

递归的深度与栈容量的关系：设递归函数每层占用 s 个字节的栈帧，递归深度为 d，则需要的栈空间至少为 $d \times s$ 字节。若系统栈总容量为 S，能支持的最大递归深度约为 $\lfloor S/s \rfloor$。这解释了为什么「太深的递归会栈溢出」——是栈空间不够，而不是算法逻辑错误。

用栈把递归改写成迭代的思路：把递归函数的「参数 + 局部变量 + 返回位置」打包成一个栈元素显式压栈，用循环模拟系统调用栈的行为。二叉树的中序遍历非递归版本（讲义中的 LeetCode 94）就是典型例子：

- 一个结点入栈后有两种处理方式：要么立即出栈（说明该结点没有左孩子），要么让下一个结点

入栈（有左孩子）；

- 一个结点出栈后也有两种处理方式：要么继续出栈（该结点没有右孩子），要么把下一个结点入栈（有右孩子）。

（四）栈在进制转换中的应用

把十进制整数 N 转换为 r 进制，使用除 r 取余法：反复用 N 除以 r，把每次的余数依次入栈，直到商为 0；再依次出栈，得到的序列即为转换结果。

为什么用栈：除 r 取余法得到的余数是从低位到高位产生的，而输出需要从高位到低位，正好是一次逆序，与栈的 LIFO 特性吻合。

代入验证：把十进制的 1348 转换为八进制。

步骤	除法	商	余数（入栈顺序）
1	$1348 \div 8$	168	4
2	$168 \div 8$	21	0
3	$21 \div 8$	2	5
4	$2 \div 8$	0	2

余数入栈顺序为 4、0、5、2，出栈顺序为 2、5、0、4，故 $1348 = (2504)_8$。验算：$2\times8^3+5\times8^2+0\times8+4 = 1024+320+0+4 = 1348$，正确。

时间复杂度：需要做 $\lfloor \log_r N \rfloor + 1$ 次除法，每次入栈和出栈都是 $O(1)$，故总时间复杂度为 $O(\log_r N)$；栈的最大深度也等于这个数量级。

小数部分的转换用乘 r 取整法，取整顺序与输出顺序一致，因此不需要用栈，直接按顺序输出即可。

（五）队列在层次遍历中的应用

树的层次遍历（广度优先遍历）是队列最经典的应用：从根结点开始，根结点入队；每次取出队头结点并访问，然后把它的所有孩子依次入队；重复直到队空。

```
void LevelOrder(BiTree T) {
    SqQueue Q; InitQueue(Q);
    BiTree p;
    if (T != NULL) EnQueue(Q, T);        // 根结点入队
    while (!QueueEmpty(Q)) {
        DeQueue(Q, p);                    // 队头结点出队
        visit(p);                         // 访问该结点
        if (p->lchild != NULL) EnQueue(Q, p->lchild);   // 左孩子入队
        if (p->rchild != NULL) EnQueue(Q, p->rchild);   // 右孩子入队
    }
}
```

为什么层次遍历必须用队列：队列保证「先被发现（入队）的结点先被访问（出队）」，因此访问顺序自然是从上到下、从左到右的层序。若换成栈，就变成了深度优先遍历（DFS）。

同样的逻辑用于图的广度优先搜索（**BFS**）：队列存待访问的顶点，保证按「距起始顶点的距离」由近到远访问，因此 BFS 可以求无权图的最短路径。

（六）队列在缓冲区与 **CPU** 调度中的应用

- 缓冲区管理：为了解决 CPU/主机与慢速外设（打印机、键盘等）速度不匹配的问题，设置数据缓冲区。主机依次写入、外设依次取出，逻辑上就是 FIFO 队列。2009 年真题直接考这一点。
- **CPU** 进程调度：先来先服务（FCFS）调度算法用一个就绪队列保存等待处理机的进程，每次把队首进程投入运行，运行结束或时间片用完再挂到队尾，即轮转（**Round Robin**）调度。多级反馈队列、优先级调度同样以队列作为基本组织形式。
- 其他典型场景：消息传递（异步通信）、线程池任务分发、打印任务排队、电商订单处理等所有需要「先来后到」的场合。

（七）数组的典型应用：矩阵运算与顺序表

数组在统考中除了「压缩存储的下标计算」，还会以矩阵运算和顺序表操作的形式出现。常见的两类：

1. 矩阵的转置。设原矩阵 A 为 $m\times n$，转置矩阵 B 为 $n\times m$，则 $B[j][i]=A[i][j]$。若两个矩阵都用按行优先的一维数组存储，朴素转置需要对 A 做 n 遍扫描（每遍取一列），时间复杂度为 $O(m\times n)$；而如果只按二维数组逐元素赋值，时间复杂度为 $O(m\times n)$、空间复杂度 $O(m\times n)$。对于稀疏矩阵，转置要用到三元组表，见本章第五节的说明。

2. 用数组实现「地址计算」类操作。这类题的核心就是本章第四节的 LOC 公式：给定起始地址、元素大小和行列下标，求目标元素的地址。做题步骤固定为：

- 判断是行优先还是列优先；
- 确定「每行/每列有多少个元素」；
- 数出目标元素前面有多少个元素（这是偏移量）；
- 地址 $=$ 起始地址 $+$ 偏移量 $\times L$。

代入验证：三维数组 $A[0..1][0..2][0..3]$ 按行优先存储，每个元素占 4 字节，起始地址为 0，求 $A[1][1][2]$ 的地址。先按行优先展开为等效二维数组，每行有 $3\times 4=12$ 个元素（即后两维的长度之积）。$A[1][1][2]$ 的前面有完整的第 0 层（12 个元素）、第 1 层中的第 0 行（4 个元素）、第 1 行中的前 2 个元素（2 个），共 $12+4+2=18$ 个元素。地址 $=0+18\times 4=72$。

一般地，n 维数组按行优先存储时，下标 $(i_1,i_2,\ldots,i_n)$ 对应的一维偏移量为

$$\text{offset}=i_1\cdot(d_2d_3\cdots d_n)+i_2\cdot(d_3\cdots d_n)+\cdots+i_{n-1}\cdot d_n+i_n$$

其中 d_k 是第 k 维的长度。这个「多项式求值」形式与行优先的二维公式完全一致，理解来龙去脉后无需死记。

真题考法

数组类的统考题目主要有三种包装：

1. 给起始地址和某个元素的地址，反解出每行元素个数，再求另一个元素的地址（2021 年，见本章第四节）。
2. 问「按行优先存储」「按列优先存储」时某元素的下标或地址（近年多次出现）。
3. 问数组占用的总存储空间：$m\times n$ 的二维数组共需 $m\times n\times L$ 字节；若用压缩存储（如对称矩阵），则只需 $\dfrac{n(n+1)}{2}\times L$ 字节。这类问法的关键是先判断题目问的是压缩前还是压缩后。

（八）栈与队列的合法出栈序列

这是本章综合度最高、也最常出选择题的部分。

*1.*判断一个序列是否为合法出栈序列

模拟法（万能方法）：按入栈序列顺序不断压栈，每压入一个元素后，只要栈顶与目标出栈序列的当前元素相等就弹出并推进目标序列；全部入栈完毕后若栈空且目标序列走完，则该序列合法，否则不合法。

手算技巧：不需要写完整模拟。抓住一条核心性质——

合法出栈序列的判定准则

对任意两个元素 x、y，若它们在入栈序列中 x 在 y 之前，则在出栈序列中：

- 二者顺序可以任意（先出 x 后出 y，或先出 y 后出 x 都合法）；
- 但绝不允许出现这样的模式：y 已经出栈，而比 y 先入栈的 x 尚未出栈、且 x 与 y 之间的元素也仍未出栈——本质上是「出栈序列中不可能出现跨栈的逆序交叉」。

更实用的判定表述：对于出栈序列中的任意两个元素 a、b（a 在出栈序列中排在 b 前面），若在入栈序列中 a 排在 b 后面，则所有在入栈序列中位于 a、b 之间的元素，在出栈序列中也必须排在 b 之前。

代入验证：入栈序列为 `1,2,3`，判断 `3,1,2` 是否合法。3 先出栈，说明 `1,2,3` 全部已入栈，栈内从底到顶为 `1,2`，下一个只能出 `2`，不可能出 `1`。故 `3,1,2` 不合法。

再验证 `2,3,1`：逐一模拟——`1` 入栈，`2` 入栈，弹出 `2`；`3` 入栈，弹出 `3`；弹出 `1`。得到 `2,3,1`，合法。「`3,2,1`」合法（全部入栈后依次弹出）；「`1,2,3`」合法（每入栈一个就立即弹出）。

*2.*卡特兰数与出栈序列的个数

n 个不同元素依次进栈（进栈顺序固定），其可能的出栈序列个数为卡特兰（**Catalan**）数：

$$N=\frac{1}{n+1}C_{2n}^{n}=\frac{(2n)!}{n!\,(n+1)!}$$

符号说明：n 是元素个数（$n\geq 1$），C_{2n}^{n} 是从 $2n$ 个位置中选 n 个的组合数，N 是不同出栈序列的总数。

卡特兰数的推导思路

把「入栈」看作 $+1$、「出栈」看作 -1，任何时刻已入栈的元素个数不少于已出栈的元素个数，即前缀和恒非负（栈不能为空时出栈）。长度 $2n$ 的 ±1 序列中共有 C_{2n}^{n} 个满足「n 个 $+1$、n 个 -1」，其中前缀和非负的序列数为 $\frac{1}{n+1}C_{2n}^{n}$。这正是卡特兰数。

卡特兰数在数据结构中还有一个用途：n 个结点的不同形态二叉树（二叉排序树形态数）也是卡特兰数。因为「先序序列可视为入栈次序、中序序列可视为出栈次序」，二者的对应关系把两个计数问题统一起来。

代入验证一：$n=5$ 时，

$$N=\frac{1}{6}C_{10}^{5}=\frac{1}{6}\times\frac{10\times9\times8\times7\times6}{5\times4\times3\times2\times1}=\frac{30240}{6\times120}=\frac{30240}{720}=42$$

与逐个模拟的结果一致（$n=1,2,3,4,5$ 时 N 分别为 $1,2,5,14,42$）。

代入验证二（先序序列计数）：先序序列为 `a,b,c,d` 的不同二叉树有多少棵？先序序列是入栈次序，

中序序列是出栈序列，$n=4$：

$$N=\frac{1}{5}C_8^4=\frac{1}{5}\times 70=14$$

即共有 14 棵树。

*3.*带限制条件的出栈序列计数

统考常在纯卡特兰数的基础上加条件，此时不能直接套公式，必须按条件分类讨论。常见限制有两类：

限制一：指定某个元素为第一个出栈元素。要让 d 第一个出栈（入栈序列 a,b,c,d,e），必须先把 a,b,c,d 全部压栈，再立即弹出 d。此时栈内从底到顶为 a,b,c，e 尚未入栈。接下来分两类：

- 先弹出 c，则剩余 a,b 已全在栈中、e 未入栈，e 的入栈时机有三种，得到 d,c,b,a,e、d,c,b,e,a、d,c,e,b,a；
- 在弹出 c 之前先把 e 入栈，则 e 必须先出栈，得到 d,e,c,b,a。

共 **4** 种，与 2011 年真题答案一致。

限制二：禁止连续 k 次同类操作。如 2010 年考「不允许连续三次出栈」。判定方法是逐项构造操作序列：若某序列在构造过程中不可避免地要连续弹出 3 次（即栈内积压了 3 个以上元素而没有新的入栈操作可以打断），则该序列不可能。

以入栈序列 a,b,c,d,e,f、「不允许连续三次出栈」为例，逐项验证四个候选序列：

候选序列	构造过程	是否可行
d,c,e,b,f,a	abcd 入栈，弹 d、c（连续 2 次）；e 入栈弹 e；弹 b；f 入栈弹 f；弹 a	可行
c,b,d,a,e,f	abc 入栈，弹 c、b（2 次）；d 入栈弹 d；弹 a；e 入栈弹 e；f 入栈弹 f	可行
b,c,a,e,f,d	ab 入栈弹 b；c 入栈弹 c；弹 a；de 入栈弹 e；f 入栈弹 f；弹 d	可行
a,f,e,d,c,b	a 入栈弹 a；bcdef 全部入栈（此时栈内积压 5 个元素）；弹 f 后，栈顶依次是 e,d,c,b，已无未入栈元素可以打断，只能连续弹 f,e,d 三次	不可行

表中第四项正是 2010 年真题的答案（选项 D）。这类题的通用做法是：找出序列中「连续递减且已无剩余元素可入栈」的那一段，若长度达到 k 就违规。

限制三：入栈序列与出栈序列的对应关系（2022 年考法）。给定完整的 in 和 out，总是可以用模拟法判断 out 是否合法，因此「不能判断」的说法错误。此外：若每入栈一个就立即出栈，out 可以与 in 完全相同；若全部入栈后再依次弹出，out 恰为 in 的倒序。故「in 与 out 一定不同」错误，「可能互为倒序」正确。

限制四：双端队列的输出序列计数。双端队列的两端都能操作，情况比栈复杂，但卡特兰数仍是它的下界参照：输出受限（仅一端出）时可行序列更多。判断时先用「分段逆序」技巧筛选，再用模拟法确认。

易错点

（1）卡特兰数公式只适用于「入栈顺序固定、无额外限制、元素互不相同」的情形。一旦题目加了「不允许连续三次出栈」「以某元素开头」等条件，必须重新分类计数。

（2）卡特兰数计的是「不同出栈序列的个数」，不是「不同操作序列的个数」。若问的是「有多少种不同的入栈/出栈操作序列」，答案会更大。

（3）题目若给的是「栈的入栈序列确定，求可能的出栈序列个数」，注意栈容量可能有限制。若题目还限制了栈的容量，就不能用卡特兰数。

（4）判断合法出栈序列时不要只看首尾元素。要看元素之间的「跨越逆序」关系，或者干脆老老实实按模拟法走一遍。

（5）卡特兰数的递推公式 $N_n = \sum_{k=1}^{n} N_{k-1}N_{n-k}$（$N_0 = 1$）有时也会用到。它对应「最后一个出栈的元素把式子分成左右两部分」的分治思想，与二叉树的形态计数完全一致。

（九）用栈实现队列、用队列实现栈

统考的应用题与选择题都爱考「受限结构的互相模拟」，核心是用两个栈/两个队列抵消掉各自的方向性。

*1.*用两个栈实现队列

设 `A` 为入队栈，`B` 为出队栈：

- 入队：元素直接压入 `A`；
- 出队：若 `B` 非空，直接从 `B` 弹栈顶；若 `B` 为空，则把 `A` 中所有元素依次弹出并压入 `B`，再从 `B` 弹栈顶；
- 查队头：同出队逻辑，但不弹出；
- 元素个数：`A.size() + B.size()`。

为什么正确：`A` 中元素从栈底到栈顶是「早入队 → 晚入队」，全部倒入 `B` 后顺序反转，`B` 的栈顶恰好是最早入队的元素，符合 FIFO。

代入验证：空队列执行 `push(1)`，`push(2)`，`pop`，`push(3)`，`pop`。

操作	`A`（底 → 顶）	`B`（底 → 顶）	输出
`push(1)`	1	—	—
`push(2)`	1, 2	—	—
`pop`	—	—	`B` 空，把 `A` 全部倒入 `B`：`B` = 2, 1（底 → 顶），弹出 1
`push(3)`	3	2	—
`pop`	3	2	`B` 非空，直接弹出 2

输出序列为 1、2，正确。

时间复杂度分析（重要考点）：单个 `pop` 操作最坏为 $O(n)$（需要倒栈），但每个元素至多被压入 `A` 一次、倒入 `B` 一次、从 `B` 弹出一次，因此 n 次操作的总时间复杂度为 $O(n)$，均摊（摊还）时间复杂度为 $O(1)$。

*2.*用两个队列实现栈

设 `Q1` 为主队列，`Q2` 为辅助队列：

- 入栈：元素直接入 `Q1`；

- 出栈：把 Q1 中除队尾元素外的所有元素依次出队并转入 Q2，剩下的那个队尾元素就是栈顶，出队并返回；然后交换 Q1 和 Q2 的角色（或把 Q2 的元素再转回 Q1）；
- 取栈顶：同出栈过程，但不删除最后那个元素。

时间复杂度：单次出栈为 $O(n)$，n 次出栈的总时间复杂度为 $O(n^2)$。用队列实现栈比用栈实现队列代价更高，这是两者的本质差别——栈的 LIFO 特性比队列的 FIFO 更容易「叠加」出来。

真题考法

「用两个栈实现队列」是应用题常客，典型四问是：（1）写出结构定义；（2）说明入队出队过程；（3）画出空队列执行 `push(1)`，`push(2)`，`pop`，`push(3)`，`pop` 的过程；（4）求 n 个元素任意顺序进出后队列为空的总时间复杂度。第（4）问的标准答案是 $O(n)$——每个元素两次入栈、两次出栈，共 $4n$ 次 $O(1)$ 操作。若答成 $O(n^2)$ 会被扣分，因为那是按「单次最坏」而非「总量」估算的。

另一类常考应用题是最小栈：用「数据栈 + 辅助栈」实现 $O(1)$ 取最小值。入栈时，若新元素 $\le$ 辅助栈顶，则同时压入辅助栈；出栈时，若弹出的元素等于辅助栈顶，则辅助栈也弹出。这样辅助栈顶永远保存当前最小值。注意条件是「小于等于」而不是「小于」——用「小于」时，遇到重复的最小值会导致出栈后辅助栈提前弹空。

本章小结表

知识点	核心公式 / 结论	关键条件
栈的特点	后进先出 LIFO，只在栈顶一端操作	逻辑结构仍是线性表
队列的特点	先进先出 FIFO，队尾入、队头出	入队只动 `rear`，出队只动 `front`
顺序栈（top 指向栈顶元素）	判空 `top == -1`；判满 `top == MaxSize-1`；入栈 `data[++top]=x`	先加后存
顺序栈（top 指向栈顶下一位置）	判空 `top == 0`；判满 `top == MaxSize`；入栈 `data[top++]=x`	先存后加；元素个数 = `top`
共享栈	判空 `top0 == -1 && top1 == MaxSize`；判满 `top0 + 1 == top1`	全部 `MaxSize` 个单元可用
循环队列（牺牲一个单元）	判空 `front == rear`；判满 `(rear+1)%M == front`；个数 `(rear-front+M)%M`	容量只有 $M-1$；模数永远是 M
循环队列（设 size）	判空 `size == 0`；判满 `size == M`	容量 M，指针相等时分不清空满
循环队列（设 tag）	判空 `front==rear && !tag`；判满 `front==rear && tag`	入队置 `tag=true`，出队置 `tag=false`
链队列（带头结点）	判空 `front == rear`；出队若摘掉尾结点需 `rear = front`	最适合做链队列
链队列（不带头结点）	判空 `front == NULL`；出队至空需 `front = rear = NULL`	边界情况更多
链栈	入栈头插法、出栈头删法；无上溢	通常在表头操作
二维数组行优先	$LOC(a_{ij}) = LOC(a_{00}) + [\,i \times n + j\,] \times L$	n 为列数
二维数组列优先	$LOC(a_{ij}) = LOC(a_{00}) + [\,j \times m + i\,] \times L$	m 为行数
对称矩阵（下三角，行优先）	$k = \frac{i(i+1)}{2} + j$（$i \ge j$）；$i < j$ 时用 $a_{ij} = a_{ji}$ 翻下标	数组共 $\frac{n(n+1)}{2}$ 个单元，下标从 0

知识点	核心公式 / 结论	关键条件
三角矩阵（下三角，行优先）	$i \geq j$：$k = \frac{i(i+1)}{2} + j$；$i < j$：$k = \frac{n(n+1)}{2}$（取常数）	数组共 $\frac{n(n+1)}{2} + 1$ 个单元
三对角矩阵（行优先）	$k = 2i + j - 3$；数组长度 $3n - 2$	矩阵下标从 1 起，$\lvert i - j \rvert \leq 1$；首末行各 2 个
稀疏矩阵	三元组表（顺序，3 域）、十字链表（链式，5 域）	都需额外保存矩阵的总行数、总列数
卡特兰数	$N = \frac{1}{n+1} C_{2n}^{n}$	元素互不相同、入栈顺序固定、无其他限制
中缀转后缀	运算符优先级 $\leq$ 栈顶时弹出；（入栈不输出，）弹到（为止	操作数直接输出
后缀求值	遇运算符弹两个操作数，先出栈的是右操作数	结果压回栈，最后栈顶即答案

易错清单

1. 栈「两端都能操作」是错的。栈只在栈顶一端操作，两端都能操作的是双端队列。
2. 顺序栈的入栈顺序取决于 `top` 的含义。`top == 0` 表示空栈 ⇒ `top` 指向栈顶下一位置 ⇒ 入栈是「先存后加」；`top == -1` 表示空栈 ⇒ `top` 指向栈顶元素 ⇒ 入栈是「先加后存」。
3. 共享栈的判满与循环队列的判满形式相似、含义相反。共享栈 `top0 + 1 == top1` 时是全部空间用满；循环队列 `(rear+1)%M == front` 时是只用了 **M-1** 个。
4. 循环队列的取模模数永远是数组长度 `MaxSize`，不是 `MaxSize-1`，也不是队列容量。
5. 循环队列元素个数必须写成 `(rear - front + MaxSize) % MaxSize`。不加 `MaxSize` 在 `rear < front`（指针已绕回）时出错。
6. 入队只改 `rear`，出队只改 `front`。选择题中用这一点能快速排除一半选项。
7. 链队列出队时若摘掉的是队尾结点，带头结点实现要把 `rear` 拉回 `front`，不带头结点实现要同时把 `front`、`rear` 置空，否则 `rear` 悬空。
8. 带头结点的链队列判空是 `front == rear`；不带头结点是 `front == NULL`。两者不可混用。
9. 链栈、链队列没有上溢问题，但出栈、出队前必须判空，否则下溢。
10. 二维数组的行优先偏移用「列数」乘 i，列优先偏移用「行数」乘 j，两个数不要搞反。
11. 矩阵下标从 **1** 开始时每个下标都要减 **1**。看到「m_{ij}，$1 \leq i, j \leq n$」先换算成从 0 起的相对编号再套公式最保险。
12. 对称矩阵与三角矩阵的公式形式相同，但 `i<j` 时的处理不同：对称矩阵靠 $a_{ij} = a_{ji}$ 翻下标，三角矩阵直接返回常数单元。
13. 三角矩阵的一维数组要多开 **1** 个单元存常数，长度是 $\dfrac{n(n+1)}{2} + 1$。
14. 三对角矩阵的首行、末行各只有 **2** 个元素，数「前面有多少个元素」时不能按「每行 3 个」一刀切。
15. 三元组表必须额外保存矩阵的总行数和总列数。「含非零元素的行数/列数」可以扫描统计出来，不是必需的规模信息。
16. 稀疏矩阵的两种压缩存储是三元组表和十字链表。邻接矩阵本质是完整二维数组，不属于压缩存储；二叉链表与矩阵无关。
17. 后缀表达式中减法和除法要先出栈的是右操作数。写成 `a-b`、`a/b` 会得到相反的结果。

18. 中缀转后缀时（和）都不进入后缀表达式，右括号处理完后要把左括号弹出丢弃。
19. 「消除递归必须使用栈」是错误说法。尾递归可以直接改写成循环；但函数调用本身确实需要用栈保存返回地址、实参、局部变量。
20. 卡特兰数只适用于「入栈顺序固定、元素互不相同、无额外限制」的出栈序列计数。题目一旦加条件（如禁止连续三次出栈、指定首元素），必须分类讨论。
21. 卡特兰数同时是「n 个结点的不同形态二叉树棵数」，两处计数问题通过「先序为入栈、中序为出栈」的对应关系统一。
22. 判断出栈序列合法性时，不要只看首尾元素。必须检查「跨越逆序」关系，或者直接用模拟法逐元素走一遍。

第四章树与二叉树

本章考纲要求

四、树与二叉树（一）树的基本概念（二）二叉树

1. 二叉树的定义及其主要特性
2. 二叉树的顺序存储结构和链式存储结构
3. 二叉树的遍历
4. 线索二叉树的基本概念和构造

（三）树、森林

1. 树的存储结构
2. 森林与二叉树的转换
3. 树和森林的遍历

（四）树与二叉树的应用

1. 哈夫曼（Huffman）树和哈夫曼编码
2. 并查集及其应用
3. 堆及其应用

本章是数据结构部分分值最高、题型最全的一章：选择题几乎年年必考（性质计算、遍历序列、线索二叉树、哈夫曼编码、堆的调整），算法设计题也多次落在这一章（二叉树的递归遍历改造、带权路径长度、表达式树等）。「五个性质」和「遍历序列」是本章的两根主线，其余考点都可以挂在这两根主线上。

一、树的基本概念

（一）树的定义与术语

*1.*树的定义

树（**Tree**）是 n（$n \geq 0$）个结点的有限集合。当 $n = 0$ 时称为空树；当 $n > 0$ 时满足：

1. 有且仅有一个特定的结点称为根（**Root**）；
2. 其余结点可分为 m（$m > 0$）个互不相交的有限集合 $T_1, T_2, \cdots, T_m$，其中每个集合本身又是一棵树，称为根的子树（**Subtree**）。

定义本身是递归的，这决定了树上的算法几乎都用递归实现。

从图论角度看，树是连通无环图，也是 n 个顶点的极小连通图、极大无环图。因此：

树的边数与结点数

对含 n 个结点的树，边数恒为 $n-1$，即 $e = n - 1$。这条结论是后续所有计数性质的出发点。

*2.*结点分类

- 根结点：只有后继没有前驱，非空树中有且仅有一个。
- 叶子结点 / 终端结点：没有后继，度为 **0**。
- 分支结点 / 非终端结点：有后继，度大于 **0**。除根之外的 branch 结点也称内部结点。

易错

「叶子结点」等价于「度为 0 的结点」，但不等于「最后一层的结点」。完全二叉树中最后一层也可能有非叶结点（它还有下一层）；反过来，倒数第二层的结点也可能是叶子。

*3.*基本术语（考点密集）

术语	含义	说明
结点的度	该结点的孩子个数（子树个数、分支个数）	叶子结点的度为 0
树的度	树中所有结点度的最大值	度为 m 的树 $\Leftrightarrow$ 树中至少有一个结点的度为 m
孩子 / 双亲	某结点子树的根 / 对应的上层结点	除根外每个结点恰有一个双亲
兄弟	同一双亲的孩子互称兄弟	双亲在同一层的结点互称堂兄弟
祖先 / 子孙	从根到该结点路径上的所有结点 / 以该结点为根的子树中所有结点	
层次	根为第 1 层，根的孩子为第 2 层，依次类推	
结点的深度	从根到该结点路径上的结点个数，即该结点的层次	自顶向下累加
结点的高度	以该结点为根的子树的高度（该子树中最长路径上的结点数）	自底向上累加
树的高度 / 深度	树中结点的最大层次	根的高度 = 树的高度
有序树 / 无序树	各结点的子树从左到右有 / 无次序	二叉树是有序树
森林	m（$m \geq 0$）棵互不相交的树的集合	树去掉根就变成森林
路径	两结点间所经过的结点序列	树中路径自上向下，兄弟之间无路径
路径长度	路径上边的个数	不是结点个数

深度与高度的区分与联系

- 深度从根往下数，高度从叶子往上数，同一结点两者可以不等。
- 恒有 $height(v) + depth(v) \leq height(T)$，当且仅当 v 位于某条「根到最深叶子」的链上时取等号。
- 若 p 是 v 的双亲，则 $h(p) = \max\{h(\text{所有子结点})\} + 1$，故 $h(p) \geq h(v) + 1$。
- 树的路径长度指从根到每个结点的路径长度之和，其最大值等于「树高 -1」。它与哈夫曼树的带权路径长度 **WPL** 是两个完全不同的概念，不要混用。

真题考法

直接用「深度 / 高度 / 层次」的定义设陷阱：给出某结点的层次，要求最深叶子的层次；或给出一棵树的各度结点数，反求叶子结点数（2010 年统考的经典形式）。此外「路径长度」默认按边数计，若题目按结点数计会明确说明。

（二）树的性质

*1.*结点数与度数的关系

设度为 i 的结点数为 n_i（$i=0,1,\cdots,m$），n 为结点总数。

树的第一性质（最常用）

树中结点总数 = 所有结点的度数之和 + **1**。

$$n=\sum_{i=0}^{m} n_i=\sum_{i=1}^{m} i\cdot n_i+1$$

理由：除根结点外，每个结点恰好有一条边连到它的双亲，故边数 $e=n-1$；另一方面每个度为 i 的结点贡献 i 条边，故 $e=\sum\limits_{i=1}^{m} i\cdot n_i$。两者联立即得。

算叶结点数的标准步骤：

1. 用非叶结点算出总边数 $e=\sum\limits_{i\geq 1} i\cdot n_i$；
2. 由 $n=e+1$ 得总结点数 n；
3. $n_0=n-\sum\limits_{i\geq 1} n_i$。

例：某度为 4 的树中，度为 4、3、2、1 的结点数依次为 20、10、1、10，求叶子结点数。

边数 $e=20\times 4+10\times 3+1\times 2+10\times 1=80+30+2+10=122$，故 $n=122+1=123$。非叶结点共 $20+10+1+10=41$ 个，所以 $n_0=123-41=82$。

*2.*每层最多多少个结点

度为 m 的树第 i 层最多 m^{i-1} 个结点

第 1 层只有根，$1=m^0$；第 2 层最多 m 个；第 3 层最多 m^2 个……由归纳即得第 i 层最多 m^{i-1} 个结点（$i\geq 1$）。m 叉树同理。

*3.*高度为 h 的 m 叉树最多 / 最少多少个结点

容量与下界

- 高度为 h 的 m 叉树最多有 $\dfrac{m^h-1}{m-1}$ 个结点（等比求和 $m^0+m^1+\cdots+m^{h-1}$）。当 $m=2$ 时即 2^h-1。
- 高度为 h 的 m 叉树最少有 h 个结点（每层只挂 1 个，退化成链）。
- 高度为 h、度为 m 的树最少有 $h+m-1$ 个结点（前 $h-1$ 层各 1 个，最后一层挂满 m 个孩子）。

*4.*具有 n 个结点的 m 叉树的最小高度

最小高度公式

$$h_{\min}=\left\lceil \log_m\big(n(m-1)+1\big)\right\rceil$$

推导：要让 n 个结点装进尽量矮的树，就应让每层都放满。高度为 h 时最多容纳 $\dfrac{m^h-1}{m-1}$ 个结点，故必须有

$$n\leq\frac{m^h-1}{m-1}\implies n(m-1)\leq m^h-1\implies m^h\geq n(m-1)+1$$

两边取以 m 为底的对数得 $h \geq \log_m\big(n(m-1)+1\big)$；$h$ 是整数，取下界即得上式。取 $h_0 = \lceil \log_m(n(m-1)+1) \rceil$ 后按层序填满即可达到 h_0，故下界可达。

验证：$m=3$、$n=244$ 时，$n(m-1)+1 = 244\times 2+1 = 489$，$\log_3 489 \approx 5.65$，取整得 $h_{\min}=6$。（也可直接数：高度 5 的满三叉树最多 $\frac{3^5-1}{2}=121<244$；高度 6 的满三叉树最多 $\frac{3^6-1}{2}=364\geq 244$。）

当 $m=2$、$n=1000$ 时，$n(m-1)+1=1001$，$\log_2 1001 \approx 9.97$，$h_{\min}=10$。

*5.*具有 n 个结点的度为 m 的树的最大高度

最大高度为 $n-m+1$

除一个结点的度为 m 外，其余 $n-m$ 个结点的度均为 1，这样树能拉得最长：

$$h_{\max} = (n-m)+1 = n-m+1$$

*6.*度为 m 的树与 m 叉树的区别（高频陷阱）

对比项	度为 m 的树	m 叉树
结点度的上限	不超过 m	不超过 m
是否必须有度为 m 的结点	必须至少有一个	允许所有结点度都 $<m$
能否为空	不能，至少有 $m+1$ 个结点	可以是空树

易错

- 「度为 m 的树」是非空树，因为「度为 m」本身就要求存在一个度为 m 的结点。
- 「m 叉树」可以是空树，所以 n 个结点的 m 叉树高度可以为 0（空树）——但统考一般默认非空，遇到「含 n 个结点」时 $n\geq 1$。
- m 叉树（有序）与 m 叉树形（不考虑左右次序）在计数问题中结果不同。

7.n 个结点能构成多少种不同形态的二叉树 / 树

卡特兰数

- 含 n 个结点的不同形态二叉树数目为第 n 个卡特兰数：

$$b_n = \frac{1}{n+1}C_{2n}^{n} = \frac{(2n)!}{n!(n+1)!}$$

- 含 n 个结点的不同形态树（普通有序树）数目为 $t_n = b_{n-1} = \frac{1}{n}C_{2n-2}^{n-1}$。

  推导（二叉树的递推）：n 个结点的二叉树可看成「1 个根 + i 个结点的左子树 + $n-i-1$ 个结点的右子树」，于是

$$b_0 = 1, \qquad b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1}$$

  代入得 $b_1=1$，$b_2=2$，$b_3=5$，$b_4=14$，$b_5=42$。

  推导（树的递推）：任意一棵普通树经「左孩子右兄弟」表示后必对应一棵根无右子树的二叉树；而 n 个结点的树去掉根后剩下 $n-1$ 个结点，所以 $t_n = b_{n-1}$。

真题考法

「先序序列为 a,b,c,d 的不同二叉树个数」这类题，因为固定先序后每一种树形唯一对应一种填入方式，答案就是结点数对应的卡特兰数（$n=4$ 时为 14）。注意区分问的是二叉树还是普通树，后者要用 $t_n = b_{n-1}$。

二、二叉树

（一）二叉树的定义及其主要特性

*1.*定义

二叉树（**Binary Tree**）是 n（$n \geq 0$）个结点的有限集合。它或为空树，或由一个根结点及两棵互不相交的、分别称为左子树和右子树的二叉树组成。

二叉树 $\neq$ 度为 2 的有序树

- 二叉树可以为空，度为 2 的树不能为空；
- 二叉树的每个结点至多有两棵子树，且左右次序严格区分，即使某结点只有一个孩子，这个孩子也必须明确是左孩子还是右孩子；
- 度为 2 的有序树中，若某结点只有一个孩子，就不存在「左右次序」问题。

这条差别是「先序 + 后序能否唯一确定二叉树」等问题的根源。

二叉树的 5 种基本形态：空树、只有根、根 + 左子树、根 + 右子树、根 + 左右子树。

*2.*几种特殊的二叉树

满二叉树（**Perfect / Full Binary Tree**）：高度为 h 且含 $2^h - 1$ 个结点的二叉树，即每一层都填满。按层序从 1 开始编号后，编号 i 的左右孩子为 $2i$、$2i+1$，双亲为 $\lfloor i/2 \rfloor$。

完全二叉树（**Complete Binary Tree**）：结点编号与同高度满二叉树的编号一一对应（即只允许最下面一层从右往左缺结点）。其特点：

- 只有最下面两层可能出现叶子结点；
- 最多只有一个度为 **1** 的结点，且该结点只有左孩子；
- 按层序从 1 编号后，$i \leq \lfloor n/2 \rfloor$ 的结点是分支结点，$i > \lfloor n/2 \rfloor$ 的结点是叶子结点。

满二叉树与完全二叉树的关系

满二叉树一定是完全二叉树；完全二叉树不一定是满二叉树。二者都适合顺序存储。

二叉排序树 / 二叉搜索树（**BST**）：左子树上所有结点关键字 < 根，右子树上所有结点关键字 > 根，左右子树又各是 BST。中序遍历得到递增有序序列。（详见 D06 查找章。）

平衡二叉树（**AVL**）：任一结点的左、右子树高度之差的绝对值不超过 1。（详见 D06 查找章。）

真二叉树 / 严格二叉树 / 正则二叉树：不存在度为 1 的结点，即结点的度只能是 0 或 2。此时 $n_2 = n_0 - 1$，即内部结点数比叶子结点数少 1。哈夫曼树一定是真二叉树，但不一定是完全二叉树。

*3.*二叉树的五个重要性质（全部推导）

性质 1：第 i 层最多 2^{i-1} 个结点

$i=1$ 时只有根，$1 = 2^0$ 个，成立。设第 i 层最多 2^{i-1} 个，则因每个结点至多两个孩子，第 $i+1$ 层最多 $2 \cdot 2^{i-1} = 2^i$ 个。由数学归纳法得第 i 层最多 2^{i-1} 个结点（$i \geq 1$）。

性质 2：高度为 h 的二叉树最多 2^h-1 个结点

把各层最大值相加（等比数列）：

$$1+2+4+\cdots+2^{h-1}=\frac{2^h-1}{2-1}=2^h-1$$

恰好达到这个上界的二叉树就是满二叉树。反之，高度为 h 的二叉树至少有 h 个结点（每层至少 1 个，退化成单链）。

性质 3：$n_0=n_2+1$

设度为 0、1、2 的结点数分别为 n_0,n_1,n_2，则

- 结点总数：$n=n_0+n_1+n_2$；
- 分支（边）总数：由每个度为 i 的结点引出 i 条分支，得 $e=n_1+2n_2$；
- 又因除根外每个结点恰有一条来自双亲的边，$e=n-1$。

于是 $n_1+2n_2=n_0+n_1+n_2-1$，两边消去 n_1+n_2 得

$$n_0=n_2+1$$

这条性质对任何非空二叉树都成立，是换算 n_0、n_1、n_2 的核心。

性质 4：完全二叉树的编号关系

完全二叉树按层序从 **1** 开始编号为 $1\sim n$，则对编号 i 的结点：

关系	从下标 **1** 开始	从下标 **0** 开始	存在条件
双亲	$\lfloor i/2\rfloor$	$\lfloor (i-1)/2\rfloor$	非根结点
左孩子	$2i$	$2i+1$	$<n$（1 起）/ $\le n-1$（0 起）
右孩子	$2i+1$	$2i+2$	$<n$（1 起）/ $\le n-1$（0 起）
分支结点判据	$i\le\lfloor n/2\rfloor$	$i\le\lfloor n/2\rfloor-1$	——
叶子结点判据	$i>\lfloor n/2\rfloor$	$i>\lfloor n/2\rfloor-1$	——

推导：按层序编号时，第 i 层有 2^{i-1} 个结点，其编号区间为 $[2^{i-1},2^i-1]$。编号 j 的结点之前共有 $j-1$ 个结点，其中叶子之外的第 k 层（$k\le$ 该结点所在层 -1）的结点按「每个分支结点贡献两个孩子」依次编号，逐层推算即得 $j=2i$、$2i+1$、$\lfloor j/2\rfloor=i$ 的对应。$i=\lfloor n/2\rfloor$ 是最后一个有孩子的结点，故 $i>\lfloor n/2\rfloor$ 者必为叶子。

若数组下标从 **0** 开始存储，则编号为 i 的结点：双亲 $\lfloor (i-1)/2\rfloor$，左孩子 $2i+1$，右孩子 $2i+2$。

性质 5：n 个结点的完全二叉树高度为 $\lceil\log_2(n+1)\rceil$

推导：完全二叉树前 $h-1$ 层是满的（共 $2^{h-1}-1$ 个结点），第 h 层至少有 1 个结点，故

$$n_{\min}=(2^{h-1}-1)+1=2^{h-1}$$

当第 h 层也填满时取得最大结点数 $n_{\max}=2^h-1$。因此

$$2^{h-1}\le n\le 2^h-1$$

由左边得 $2^{h-1}\le n<2^h$，取对数 $h-1\le\log_2 n<h$，因 $h-1$ 是整数，故 $h=\lfloor\log_2 n\rfloor+1$。由右边得 $2^{h-1}<n+1\le 2^h$，取对数 $h-1<\log_2(n+1)\le h$，故 $h=\lceil\log_2(n+1)\rceil$。

结论：

$$h = \lfloor \log_2 n \rfloor + 1 = \lceil \log_2 (n+1) \rceil$$

两个式子恒等，考试用哪个都可以。注意这个结论只对完全二叉树成立；一般二叉树只知道 n 时，$h_{\max} = n$（单链），$h_{\min} = \lceil \log_2 (n+1) \rceil$。

验证：$n = 768$ 时，$\lfloor \log_2 768 \rfloor + 1 = \lfloor 9.58 \rfloor + 1 = 9 + 1 = 10$，而 $2^9 - 1 = 511 < 768 \le 1023 = 2^{10} - 1$，一致。

*4.*完全二叉树的 n_0、n_1、n_2 速算

由 n 直接推出 n_0, n_1, n_2

完全二叉树中 $n_1 \in \{0, 1\}$，结合 $n_0 = n_2 + 1$ 得 $n_0 + n_2 = 2n_2 + 1$ 必为奇数：

- 若 $n = 2k$（偶数）：$n_1 = 1$，$n_0 = k$，$n_2 = k - 1$；
- 若 $n = 2k - 1$（奇数）：$n_1 = 0$，$n_0 = k$，$n_2 = k - 1$。

验证：$n = 768 = 2 \times 384$，故 $n_0 = 384$，$n_1 = 1$，$n_2 = 383$，$384 = 383 + 1$ 成立。

*5.*二叉树的空指针域个数

二叉链表有 $n + 1$ 个空指针域

含 n 个结点的二叉链表共有 $2n$ 个指针域；除根结点外，每个结点恰好被一个指针指向，即非空指针数为 $n - 1$。故空指针域个数为

$$2n - (n - 1) = n + 1$$

这条结论是线索二叉树存在的依据：用这 $n + 1$ 个空域存放前驱 / 后继线索。

真题考法

- 由完全二叉树的「第 k 层有 m 个叶结点」求结点总数（2009 年形式）：要让总数最大就把第 k 层剩下的结点都补上两个孩子，算出第 $k + 1$ 层最多结点数再相加；要让总数最小就让叶结点尽量靠右。
- 由 n 直接求 n_0（2011 年形式）：直接用上面的 $n = 2k$ / $n = 2k - 1$ 结论。
- 「非空完全二叉树所有叶结点同层且非叶结点都有 2 个孩子」（2018 年形式）：这实际是一棵满二叉树，于是 $n = 2n_0 - 1$。

（二）二叉树的顺序存储结构和链式存储结构

*1.*顺序存储结构

用一维数组按层序依次存放结点，结点间的父子关系由下标计算隐含给出。

顺序存储的下标约定

从下标 **1** 开始（统考、教材通用）：

- 结点 i 的双亲 $= \lfloor i/2 \rfloor$，左孩子 $= 2i$，右孩子 $= 2i + 1$；
- 若 $i > n$ 或该位置为空，则对应孩子不存在。

从下标 **0** 开始（C 语言数组天然形式）：

- 结点 i 的双亲 $= \lfloor (i - 1)/2 \rfloor$，左孩子 $= 2i + 1$，右孩子 $= 2i + 2$。

必须显式标记空结点才能唯一表示一棵二叉树：因为层序序列不包含空结点信息，同一个层序序列

可能对应多棵不同的二叉树。常用做法是把每个数组元素设计为带 isEmpty 标记的结点：

```
typedef struct TreeNode {
    int data;        // 结点数据元素
    bool isEmpty;    // 该位置是否为空结点
} TreeNode;

// 初始化：全部标记为空
void InitSqBiTree(TreeNode t[], int length) {
    for (int i = 0; i < length; i++)
        t[i].isEmpty = true;
}

// 判断下标为 index 的结点是否为空（下标从 1 开始）
bool isEmpty(TreeNode t[], int length, int index) {
    if (index >= length || index < 1) return true;
    return t[index].isEmpty;
}

// 取下标为 index 的结点的左孩子下标，无则返回 -1
int getLchild(TreeNode t[], int length, int index) {
    int lChild = index * 2;
    if (isEmpty(t, length, lChild)) return -1;
    return lChild;
}

// 取下标为 index 的结点的右孩子下标，无则返回 -1
int getRchild(TreeNode t[], int length, int index) {
    int rChild = index * 2 + 1;
    if (isEmpty(t, length, rChild)) return -1;
    return rChild;
}

// 取下标为 index 的结点的双亲下标，无则返回 -1
int getFather(TreeNode t[], int length, int index) {
    if (index == 1) return -1;              // 根结点无双亲
    int father = index / 2;                 // 整数除法自动向下取整
    if (isEmpty(t, length, father)) return -1;
    return father;
}

// 顺序存储二叉树的先序遍历
void PreOrderSqTree(TreeNode *t, int length, int index) {
    if (isEmpty(t, length, index)) return;
    visitNode(t[index]);
    PreOrderSqTree(t, length, getLchild(t, length, index));
    PreOrderSqTree(t, length, getRchild(t, length, index));
}
```

顺序存储的优缺点

优点：连续内存、缓存友好、不需存指针、支持随机访问（含完全二叉树的 $O(1)$ 找双亲）。

缺点：需要连续空间；插入删除代价高；稀疏时空间浪费严重。

关键结论：顺序存储只适合完全二叉树（含满二叉树）——因为空结点只出现在最底层右侧，可省略不存，n 个结点恰好占 n 个单元。一般二叉树最坏情况下，一棵高度为 h 的单支树只有 h 个结点，却要占据 2^h-1 个单元。

易错：「需要的存储单元数量至少是多少」

题目给定结点数和高度、要求用顺序存储，问最少需要多少单元时：由于树的形态未知，必须按最坏情况考虑，即最深结点恰好落在编号最大的位置上。此时数组长度由高度决定，为 2^h-1（从 1 编号）或 2^h-1 个单元（从 0 编号时需能存到下标 2^h-2，仍为 2^h-1 个单元）。

例：高度为 5、10 个结点的任意二叉树用顺序存储，最少需要 $2^5-1=31$ 个存储单元（2020 年形式）。这里的答案是 **31**，而不是 **10** 或 **15**。

易错：从数组反推二叉树是否合法

若给定数组（含 -1 表示空）问「能否表示一棵二叉树」，判据是：每个非空结点的双亲位置也必须非空。若某结点下标 i 的值非空，而 $\lfloor (i-1)/2 \rfloor$ 或 $\lfloor i/2 \rfloor$ 处为空，就出现了「悬空结点」，不合法。

例（从 0 开始编号）：`R[] = {17,20,35,-1,18,45,-1,-1,19,27}` 中下标 8 的结点 19，其双亲下标 $\lfloor (8-1)/2 \rfloor = 3$，而 `R[3] = -1` 为空，故 19 悬空，该数组不能表示二叉树（2025 年形式）。

*2.*链式存储结构

二叉链表：数据域 + 左指针 + 右指针。

```
typedef struct TreeNode {
    int data;                           // 数据域
    struct TreeNode *lchild, *rchild;   // 左、右孩子指针
} TreeNode, *BiTree;
```

三叉链表：在二叉链表上增加一个指向双亲的指针，便于找双亲。

```
typedef struct TreeNode {
    int data;                           // 数据域
    struct TreeNode *lchild, *rchild;   // 左、右孩子指针
    struct TreeNode *parent;            // 双亲指针
} TreeNode, *BiTree;
```

三种链式结构对比

结构	域	左右指针域中的空域数	找双亲	全部指针域中的空域数
二叉链表	data, lchild, rchild	$n+1$	需从根遍历	$n+1$
三叉链表	data, lchild, rchild, parent	$n+1$	$O(1)$	$2n+1$
线索链表	data, lchild, rchild, ltag, rtag	线索化后为 0	中序线索可 $O(1)$ 找前驱后继	——

三叉链表共 $3n$ 个指针域，非空 $n-1$ 个，空 $3n-(n-1)=2n+1$。注意：考试若只问「`lchild`、`rchild` 中的空指针」，答案仍是 $n+1$（与二叉链表相同），因为多出来的 `parent` 域不参与计数。

真题考法

本考点以「求空指针域个数」「求找双亲的最坏时间」「给出一段找兄弟 / 找堂兄弟的操作问复杂度」等形式出现。没有双亲指针时，找兄弟必须先从根遍历到双亲，这是常见的复杂度设陷阱点。

（三）二叉树的遍历

*1.*遍历的定义与四种方式

遍历：从根出发，按某种次序访问树中所有结点，使每个结点被访问一次且仅一次。

以 N 表示根、L 表示左子树、R 表示右子树，「前 / 中 / 后」指的是根在遍历中的位置：

遍历	次序	口诀
先序（前序）	$N \to L \to R$	根在最前
中序	$L \to N \to R$	根在中间
后序	$L \to R \to N$	根在最后
层次	自上而下、自左而右逐层访问	用队列

前三种属于深度优先遍历（**DFS**），可用递归或栈实现；层次遍历属于广度优先遍历（**BFS**），用队列实现。

四条速记结论

- 先序首个 = 后序末个 = 层次首个 = 根结点；
- 中序序列中，根左边的全部属于左子树，右边全部属于右子树；
- 先序 + 中序、后序 + 中序、层次 + 中序均可唯一确定二叉树；先序 + 后序不行；
- 一棵二叉树的中序序列唯一，但不同的二叉树可能有相同的中序序列（如先序 + 后序的组合）。

*2.*递归遍历（三种）

```
// 先序遍历：根 左 右
void PreOrder(BiTree T) {
    if (T != NULL) {
        visit(T);             // 访问根结点
        PreOrder(T->lchild); // 递归遍历左子树
        PreOrder(T->rchild); // 递归遍历右子树
    }
}

// 中序遍历：左 根 右
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}

// 后序遍历：左 右 根
void PostOrder(BiTree T) {
    if (T != NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

递归遍历的时空复杂度

- 时间：$O(n)$，每个结点恰好访问一次。
- 空间：$O(h)$，即递归工作栈的深度。最好情况（平衡 / 完全二叉树）$h=\lceil \log_2(n+1) \rceil = O(\log_2 n)$；最坏情况（单支链）$h=n=O(n)$。

注意空间复杂度考的是辅助栈而非结点本身，故应为 $O(h)$，只有最坏时才写成 $O(n)$。

*3.*非递归遍历（栈模拟）

核心思想

递归调用的本质是「把局部变量、参数、返回地址压入调用栈」。因此用显式栈完全可以模拟递归：递归隐式维护一个栈，迭代显式维护一个栈，两者等价。

- 先序：访问与处理顺序一致（都是根），代码最简——出栈即访问，然后把右孩子先入栈、左孩子后入栈（保证出栈顺序为「左、右」）。
- 中序：访问顺序（先到最左下）与处理顺序（根在中间）不一致，必须用一个指针 `cur` 负责「访问」、用栈负责「处理」。
- 后序：最难，因为从左子树返回时无法判断右子树是否已访问过，需增加 `pre` 指针记录上一个被访问的结点。

先序遍历（迭代）：

```
void PreOrderIter(BiTree T) {
    if (T == NULL) return;
    BiTree stack[MaxSize];
    int top = -1;
    stack[++top] = T;
    while (top >= 0) {
        BiTree p = stack[top--];   // 出栈即访问
        visit(p);
        if (p->rchild != NULL) stack[++top] = p->rchild; // 右孩子先入栈
        if (p->lchild != NULL) stack[++top] = p->lchild; // 左孩子后入栈
    }
}
```

中序遍历（迭代）：

```
void InOrderIter(BiTree T) {
    BiTree stack[MaxSize];
    int top = -1;
    BiTree cur = T;
    while (cur != NULL || top >= 0) {
        if (cur != NULL) {
            stack[++top] = cur;      // 一路向左，沿途压栈
            cur = cur->lchild;
        } else {
            cur = stack[top--];      // 左子树走完，弹栈处理
            visit(cur);
            cur = cur->rchild;       // 转向右子树
        }
    }
}
```

后序遍历（迭代，用 `pre` 记录历史）：

```
void PostOrderIter(BiTree T) {
    BiTree stack[MaxSize];
    int top = -1;
    BiTree cur = T, pre = NULL;
    while (cur != NULL || top >= 0) {
        while (cur != NULL) {          // 步骤 1：沿左孩子一路入栈
            stack[++top] = cur;
            cur = cur->lchild;
        }
        cur = stack[top--];            // 步骤 2：弹出栈顶，其左子树必已访问完
        // 步骤 3：无右子树，或右子树刚被访问完（pre 指向右孩子），则可访问 cur
        if (cur->rchild == NULL || pre == cur->rchild) {
            visit(cur);
            pre = cur;
            cur = NULL;                // 防止再次进入步骤 1
        } else {
            stack[++top] = cur;        // 右子树未访问，cur 重新入栈
            cur = cur->rchild;         // 先去处理右子树
        }
    }
}
```

后序遍历的取巧写法：先按「根 → 右 → 左」的顺序遍历（即把先序代码中左右孩子入栈次序互换），得到「NRL」序列，再把结果逆序，即得「LRN」。若不允许用逆序函数，可再设一个辅助栈，访问时压入辅助栈，遍历结束后依次出栈。

后序非递归遍历的重要性质

在上述后序迭代过程中，当访问到结点 p 时，栈中恰好是 p 的全部祖先结点，且从栈底到栈顶再加上 p 本身，恰好构成从根到 p 的路径。

这条性质可用于：求根到某结点的路径、求二叉树的最近公共祖先、求根到叶结点的数字之和。

*4.*层次遍历（队列）

```
void LevelOrder(BiTree T) {
    if (T == NULL) return;
    BiTree Q[MaxSize];
    int front = 0, rear = 0;
    Q[rear++] = T;                  // 根结点入队
    while (front != rear) {
        BiTree p = Q[front++];      // 队头出队
        visit(p);
        if (p->lchild != NULL) Q[rear++] = p->lchild; // 左孩子入队
        if (p->rchild != NULL) Q[rear++] = p->rchild; // 右孩子入队
    }
}
```

若要按层分行输出，可在每轮循环开始时记录 `size = rear - front`，只处理这 `size` 个结点。

完全二叉树层次遍历的队列规模

完全二叉树做层次遍历时，辅助队列的最大规模等于叶子结点个数 $= \lceil n/2 \rceil$。也就是说，队列容量只要开到叶子结点数就够用。

队列规模在整个遍历过程中是先增后减、单峰对称的：从 0 开始逐层上升到达峰值，再逐层回落到 0。峰值出现在最宽的一层（即最后一层）的结点全部入队、而它们尚未出队完毕的时刻。

5. 由遍历序列构造二叉树

唯一性总表

序列组合	能否唯一确定	原因
先序 + 中序	能	先序定根，中序分左右
后序 + 中序	能	后序定根，中序分左右
层次 + 中序	能	层次定根，中序分左右
先序 + 后序	不能	单孩子结点无法区分左右
层次 + 先序	不能	同上
层次 + 后序	不能	同上
先序 + 后序（严格二叉树）	能	无单孩子结点，歧义消失
层次 + 先序（完全二叉树）	能	单孩子必为左孩子，形状受限

（1）先序 + 中序

构造步骤（设结点值互不相同）：

1. 先序序列的第一个元素是当前子树（初始为整棵树）的根；
2. 在中序序列中定位该根，根左边的元素构成左子树的中序序列，右边的构成右子树的中序序列；
3. 由左子树结点个数 k，把先序序列切成「根 | 左子树先序（k 个）| 右子树先序」；
4. 对左右子树递归执行上述过程。

关键一步

第 3 步「用左子树的结点数去切分先序序列」是本题型的得分点。很多同学会漏掉这一步，只在中序里切分，导致先序无法继续划分。

例：先序 A B D E C F G，中序 D B E A F C G。

- 先序首元素 A 是根；中序中 A 的位置把序列分为 D B E 与 F C G；
- 左子树 3 个结点，故先序中 A 之后 3 个 B D E 属左子树，其余 C F G 属右子树；
- 递归：左子树先序 B D E、中序 D B E → 根 B，左 D 右 E；右子树先序 C F G、中序 F C G → 根 C，左 F 右 G。

（2）后序 + 中序

与上一情形完全对称，只是根取后序序列的最后一个元素：

1. 后序序列的最后一个元素是根；
2. 在中序中定位根，划分左右子树的中序序列；
3. 由左子树结点个数 k，把后序序列切成「左子树后序（k 个）| 右子树后序 | 根」；
4. 递归。

易错

「后序 + 中序」中，左子树的后序序列在下标上位于前面，右子树紧随其后。如果切分时把右子树放在前面就会全错。建议每次切分后手工验证一次后序（左子树后序 + 右子树后序 + 根是否等于原序列）。

（3）层次 + 中序

1. 层次序列的第一个元素是整棵树的根；
2. 在中序序列中定位该根，根左边是左子树全部结点，右边是右子树全部结点；
3. 从层次序列中筛选出属于左子树的结点（保持原有相对顺序），得到左子树的层次序列；同理得到右子树的层次序列；
4. 对左右子树递归。

为什么必须「重新筛选」

层次序列是整棵树的层次序，左右子树的结点在原序列中是交错出现的，必须各自挑出来组成新的层次序列，才能继续递归。每一步挑出后，新层次序列的第一个元素就是该子树的根。

例：层次 A B C D E F，中序 D B E A F C。

- 根为 A；中序分左右：左 D B E，右 F C；
- 从左子树结点集 $\{D, B, E\}$ 在原层次序列中筛选，得到左子树层次 B D E；右子树层次 C F；
- 左子树：层次首元素 B 为根，中序 D B E 中 B 居中 → 左 D 右 E；
- 右子树：层次首元素 C 为根，中序 F C 中 C 末尾 → 只有左孩子 F。

(4) 先序 + 后序为什么不能唯一确定

根本原因：单孩子子树无法区分左右

- 先序去掉根后是「左子树先序 + 右子树先序」；
- 后序去掉根后是「左子树后序 + 右子树后序」；
- 但你不知道左子树有几个结点，无法据此切分。

当某个结点同时有左右两个孩子时，才能通过「先序的第二个元素 = 左子树根」「后序的倒数第二个元素 = 右子树根」确定分界。一旦某结点只有一个孩子，该孩子在左在右都不改变先序和后序的结果，歧义就产生了。

最小反例：树 1 中 B 是 A 的左孩子且 C 是 B 的右孩子；树 2 中 B 是 A 的右孩子且 C 是 B 的左孩子。两棵树的先序都是 A B C，后序都是 C B A，但结构不同。

什么时候先序 + 后序能唯一确定

在结点值互不相同、且不附加形状限制时：当且仅当该二叉树是严格二叉树（真二叉树，每个结点度为 **0** 或 **2**），先序 + 后序才能唯一确定；同理，层次 + 先序、层次 + 后序也只有在这种情况下能唯一。

若附加形状约束，条件可以放宽：例如完全二叉树中单孩子只能是左孩子，形状约束本身就消除了歧义，所以「层次 + 先序」对完全二叉树可唯一确定。

典型应用：表达式树是一棵真二叉树，因此给出它的先序（前缀表达式）和后序（后缀表达式）就能唯一还原它。

*6.*遍历序列相等的条件（高频小题）

序列相同 ⇔ 形状受限

条件	等价的结构要求
先序 = 中序	每个非叶结点只有右子树（不存在左孩子）
后序 = 中序	每个非叶结点只有左子树（不存在右孩子）
先序 = 后序	树中只有一个结点（$n = 1$）
先序 = 层次	每个非叶结点只有右子树，且树是单链

条件	等价的结构要求
中序 = 层次	每个非叶结点只有右子树（单链）

先序 = 中序的推导：先序为 N L R，中序为 L N R，二者唯一的差别是根与左子树的先后。若某非叶结点有左子树，先序先访问该结点再访问左子树，中序先访问左子树再访问该结点，两序列必然不同。故所有非叶结点都不能有左子树，只能有右子树。

注意：「所有结点的度均为 **1**」是不够的——度为 1 还可能包含「只有左孩子」的情况，此时先序与中序不再相同。

*7.*中序序列相邻结点的结构含义

中序前驱 / 后继的结构性质

- 若 v 有左子树，则 v 的中序直接前驱是 v 左子树中最右下的结点，且该结点必无右孩子；
- 若 v 有右子树，则 v 的中序直接后继是 v 右子树中最左下的结点，且该结点必无左孩子。

推导：中序按「左—根—右」进行，访问 v 之前的最后一个结点必在左子树中，且是「最后被访问」的那个，即最右下的结点；若它还有右孩子，则中序还要先访问它的右子树，它就不可能紧邻 v。

这条性质是 **2024** 年真题「中序序列形如⋯**,p,v,q,**⋯，v 有两个孩子」的解答关键：由 v 有左子树知 p 是左子树中最右下结点，故 p 无右孩子；由 v 有右子树知 q 是右子树中最左下结点，故 q 无左孩子。

（四）线索二叉树

*1.*基本概念

在含 n 个结点的二叉链表中，有 $n+1$ 个空指针域。传统二叉链表只能体现父子关系，求某结点的前驱 / 后继必须从根重新遍历。线索二叉树就是把这些空指针利用起来，指向该结点在某种遍历次序下的前驱 / 后继。

规定：

- 若无左子树，令 `lchild` 指向其前驱；
- 若无右子树，令 `rchild` 指向其后继。

为了区分指针指向的是孩子还是线索，需增加两个标志域：

标记	值	含义
`ltag`	0	`lchild` 指向左孩子
`ltag`	1	`lchild` 指向前驱
`rtag`	0	`rchild` 指向右孩子
`rtag`	1	`rchild` 指向后继

```
typedef struct ThreadNode {
    int data;                                // 数据域
    struct ThreadNode *lchild, *rchild;  // 左、右指针
    int ltag, rtag;                          // 左、右线索标志
} ThreadNode, *ThreadTree;
```

三个必背结论

- 加上线索的二叉链表称为线索链表，对应的二叉树称为线索二叉树；
- 线索化的实质是在遍历过程中把空指针改为前驱 / 后继线索，所以线索化的过程就是遍历的过程；
- 线索二叉树类似于一个非循环的双向链表：遍历序列的第一个结点前驱指向 NULL，最后一个结点后继指向 NULL（不带头结点时）。

*2.*线索化的构造过程

核心技巧：设两个指针 pre（刚访问过的结点）与 p（正在访问的结点），则 **pre** 是 **p** 的前驱。遍历到 p 时：

- 若 p->lchild == NULL，把 p->lchild 指向 pre，p->ltag = 1；
- 若 pre->rchild == NULL，把 pre->rchild 指向 p，pre->rtag = 1。

中序线索化：

```
// pre 为全局或引用参数，始终指向刚访问过的结点
void InThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        InThread(p->lchild, pre);            // 递归线索化左子树
        if (p->lchild == NULL) {             // 左孩子为空，建立前驱线索
            p->lchild = pre;
            p->ltag = 1;
        } else {
            p->ltag = 0;
        }
        if (pre != NULL && pre->rchild == NULL) { // 前驱的右孩子为空，建立后继线索
            pre->rchild = p;
            pre->rtag = 1;
        } else if (pre != NULL) {
            pre->rtag = 0;
        }
        pre = p;                             // 更新 pre
        InThread(p->rchild, pre);            // 递归线索化右子树
    }
}

void CreateInThread(ThreadTree root) {
    ThreadNode *pre = NULL;
    if (root != NULL) {
        InThread(root, pre);
        pre->rchild = NULL;                  // 处理中序最后一个结点
        pre->rtag = 1;
    }
}
```

先序线索化：把「建线索 + 更新 pre」的部分提到递归之前，并且递归左子树前必须判断 **p->ltag == 0**：

```
void PreThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        if (p->lchild == NULL) { p->lchild = pre; p->ltag = 1; }
        else { p->ltag = 0; }
        if (pre != NULL && pre->rchild == NULL) { pre->rchild = p; pre->rtag = 1; }
        else if (pre != NULL) { pre->rtag = 0; }
        pre = p;
```

```
        if (p->ltag == 0) PreThread(p->lchild, pre);  // 必须判断，否则死循环
        if (p->rtag == 0) PreThread(p->rchild, pre);  // 必须判断，否则死循环
    }
}
```

先序线索化的死循环陷阱

先序线索化时，结点 p 的左指针可能刚被改成指向前驱 `pre`。如果此后不加判断就递归进入 `p->lchild`，程序会顺着线索走回祖先，形成死循环。所以先序线索化递归左右子树前必须先检查 `ltag` / `rtag`。中序线索化因为「建线索」发生在左递归之后，天然没有这个问题。

后序线索化：把「建线索 + 更新 pre」的部分放到左、右递归之后。递归左右子树时不需要判断标记（因为进入时线索尚未建立），但建立线索时要用 `pre`：

```
void PostThread(ThreadNode *p, ThreadNode *&pre) {
    if (p != NULL) {
        PostThread(p->lchild, pre);
        PostThread(p->rchild, pre);
        if (p->lchild == NULL) { p->lchild = pre; p->ltag = 1; }
        else { p->ltag = 0; }
        if (pre != NULL && pre->rchild == NULL) { pre->rchild = p; pre->rtag = 1; }
        else if (pre != NULL) { pre->rtag = 0; }
        pre = p;
    }
}
```

三种线索化只需改动两处：① 线索化改造代码段的位置；② 递归调用左右子树的位置。顺序为：

线索类型	代码顺序
先序	建线索 → 递归左 → 递归右
中序	递归左 → 建线索 → 递归右
后序	递归左 → 递归右 → 建线索

*3.*带头结点的线索二叉树

仿照带头结点的链表，可为线索二叉树增设一个头结点：

- 头结点的 `lchild` 指向二叉树的根结点（`ltag = 0`）；
- 头结点的 `rchild` 指向中序遍历最后一个访问的结点（`rtag = 1`）；
- 中序遍历第一个结点的 `lchild` 指向头结点；
- 中序遍历最后一个结点的 `rchild` 指向头结点。

这样整棵线索二叉树构成一个双向线索链表，可以方便地从任一点出发向两个方向遍历：从头结点的左孩子出发可正向中序遍历，从头结点出发沿 `rchild` 链可逆向遍历。

```
ThreadTree CreateInThreadWithHead(ThreadTree root) {
    ThreadNode *head = (ThreadNode *)malloc(sizeof(ThreadNode));
    head->ltag = 0;                  // 左指针指向根
    head->rtag = 1;                  // 右指针始终为线索
    head->rchild = head;             // 先初始化指向自己
```

```
    if (root != NULL) {
        head->lchild = root;          // 头结点的左指针指向根结点
        ThreadNode *pre = head;
        InThread(root, pre);          // 中序线索化
        pre->rchild = head;           // 最后一个结点的后继指向头结点
        pre->rtag = 1;
        head->rchild = pre;           // 头结点的后继指向最后一个结点
        head->rtag = 1;
    } else {
        head->lchild = head;          // 空树时左指针指向自己
    }
    return head;
}
```

易错

- 先序线索化不能用头结点的 `lchild` 作为 `pre` 的初值后再无条件递归，因为先序的第一个结点是根，前驱应为 `NULL`（或头结点）；
- 后序线索化中，若最后一个结点有右孩子，**`pre->rtag` 不能被设为 1**，否则线索化错误。处理「最后一个结点」时必须先判断其右指针是否为空。

*4.*在线索二叉树上找前驱 / 后继

中序线索二叉树：找后继

- 若 `p->rtag == 1`：后继就是 `p->rchild`；
- 若 `p->rtag == 0`：后继是其右子树中最左下的结点（右子树中第一个被中序访问的结点）。

中序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱就是 `p->lchild`；
- 若 `p->ltag == 0`：前驱是其左子树中最右下的结点（左子树中最后一个被中序访问的结点）。

```
// 以 p 为根的子树中，第一个被中序遍历访问的结点（最左下）
ThreadNode *FirstNode(ThreadNode *p) {
    if (p == NULL) return NULL;
    while (p->ltag == 0) p = p->lchild;   // 循环到最左下，不一定是叶结点
    return p;
}

// 以 p 为根的子树中，最后一个被中序遍历访问的结点（最右下）
ThreadNode *LastNode(ThreadNode *p) {
    if (p == NULL) return NULL;
    while (p->rtag == 0) p = p->rchild;
    return p;
}

// 中序线索二叉树中求 p 的中序后继
ThreadNode *NextNode(ThreadNode *p) {
    if (p->rtag == 0) return FirstNode(p->rchild); // 右子树最左下
    else return p->rchild;                          // 直接读线索
}

// 中序线索二叉树中求 p 的中序前驱
ThreadNode *PreNode(ThreadNode *p) {
    if (p->ltag == 0) return LastNode(p->lchild);   // 左子树最右下
```

```
    else return p->lchild;                          // 直接读线索
}

// 利用线索做中序遍历（非递归，空间 O(1)）
void InOrderThreadTree(ThreadNode *root) {
    for (ThreadNode *p = FirstNode(root); p != NULL; p = NextNode(p))
        visit(p);
}

// 逆向中序遍历
void RevInOrderThreadTree(ThreadNode *root) {
    for (ThreadNode *p = LastNode(root); p != NULL; p = PreNode(p))
        visit(p);
}
```

中序线索二叉树的最大优势

中序遍历不需要栈，空间复杂度为 $O(1)$（若不计线索化本身的额外标记域）。这是中序线索二叉树被广泛使用的原因。

先序线索二叉树：找后继

- 若 `p->ltag == 0`（有左孩子）：左孩子就是其后继；
- 若 `p->ltag == 1` 且 `p->rtag == 0`（无左孩子有右孩子）：右孩子就是其后继；
- 若 `p->ltag == 1` 且 `p->rtag == 1`（叶结点）：`p->rchild` 即线索，直接给出后继。

先序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱 = `p->lchild`；
- 若 `p->ltag == 0`：先序遍历的顺序是「根—左—右」，因此 `p` 的左右孩子都只能是 `p` 的后继，不可能是前驱。此时必须从头（根）重新开始遍历才能找到 `p` 的前驱。

先序线索二叉树找前驱的代价

先序线索二叉树不便于找前驱。若确实需要频繁求先序前驱，可把二叉链表改为三叉链表（增加 `parent` 域），此时分四种情况：

情况	p 的先序前驱
p 是根	无前驱
p 是双亲的左孩子	p 的双亲
p 是双亲的右孩子，且双亲无左孩子	p 的双亲
p 是双亲的右孩子，且双亲有左孩子	双亲左子树中最后一个被先序访问的结点（先下左、无左则下右，直至叶子）

后序线索二叉树：找前驱

- 若 `p->ltag == 1`：前驱 = `p->lchild`；
- 若 `p->ltag == 0`（p 必有左孩子）：
  - 若 p 有右孩子：后序前驱 = 右孩子（后序为「左、右、根」，最后访问完右子树再访问根）；
  - 若 p 无右孩子：后序前驱 = 左孩子。

后序线索二叉树：找后继

- 若 `p->rtag == 1`：后继 = `p->rchild`；
- 若 `p->rtag == 0`（p 有右孩子）：后序中左右子树中的结点都只能是 p 的前驱，不可能是后继，除非从根重新遍历，或改用三叉链表。

用三叉链表时，p 的后序后继分四种情况：

情况	p 的后序后继
p 是根	无后继
p 是双亲的右孩子	p 的双亲
p 是双亲的左孩子，且双亲无右孩子	p 的双亲
p 是双亲的左孩子，且双亲有右孩子	双亲右子树中第一个被后序访问的结点（有左走左、无左走右，直至叶子）
附：p 有右孩子（不论位置）	右子树中第一个被后序访问的结点（有左走左、无左走右）

后序线索二叉树遍历仍需栈

后序线索二叉树的遍历仍然需要栈。原因：后序遍历最后访问根结点，若从右孩子返回去访问双亲结点，由于双亲的右孩子指针不一定是空的（它可能指向真实的右孩子而非线索），无法用线索判断「是否已从右子树返回」，因而无法仅靠指针遍历整棵树。

三种线索二叉树的能力对比（必背）

能力	中序线索	先序线索	后序线索
找前驱	✓ 直接可得	× 需从头遍历 / 三叉链表	✓ 直接可得
找后继	✓ 直接可得	✓ 直接可得	× 需从头遍历 / 三叉链表
不用栈完成遍历	✓	✓	× 仍需栈

记忆口诀：「中序两全，先序缺前驱，后序缺后继」。

真题考法

- 给出二叉树的形状与遍历序列，问某结点的左右线索指向谁（2014 年形式）：先把对应的遍历序列完整写出，再看该结点在序列中的前后邻居，同时注意「有真实孩子的一侧不建线索」。
- 后序线索二叉树中叶结点 X 有左兄弟 Y，问 X 的右线索指向谁（2013 年形式）：X 是右孩子，后序遍历完 X 后两个子树都已处理完，下一个是双亲。
- 先序线索二叉树中非根叶结点 X 有左兄弟 Y，问 X 的左线索（先序前驱）指向谁：是以 Y 为根的子树中先序最后访问的结点。
- 后序线索二叉树中，X 是其双亲 p 的左孩子且 p 存在右子树，问 X 的右线索（后序后继）：p 的右子树中后序第一个访问的结点（从 p 的右孩子出发，有左走左、无左走右，直至叶结点），不是「最右下的结点」。

三、树、森林

（一）树的存储结构

树是一对多的逻辑结构，存储时既要保存数据，又要保存父子关系。三种表示法的本质区别只在于「每个结点额外记录谁」。

*1.*双亲表示法

用一维数组顺序存放全部结点，每个结点附一个 parent 域记录双亲在数组中的下标；根结点的 parent 约定为 -1。

```
#define MAX_TREE_SIZE 100

typedef struct {
    ElemType data;                  // 数据域
    int parent;                     // 双亲在数组中的下标
} PTNode;

typedef struct {
    PTNode nodes[MAX_TREE_SIZE];    // 结点数组
    int n;                          // 结点数
} PTree;
```

操作特性：

- 找双亲 $O(1)$，沿 parent 链可一路向上找到根——这正是并查集的基础；
- 找孩子 $O(n)$：需扫描整个数组看谁的双亲是自己。

删除结点的两种策略（被删结点为叶子；非叶需先递归删子树）：

策略	做法	时间复杂度	缺点
打标记	把 parent 改为 -2（区别于根的 -1）	$O(1)$	结点仍占数组空间
物理搬移	把数组尾部元素前移填补，并修正受影响结点的下标	$O(n)$	删除变慢

易错

双亲表示法靠 parent 域显式记录父子关系；二叉树的顺序存储靠下标计算隐含父子关系（结点 i 的左孩子为 $2i$）。一般树的孩子个数不固定，不能直接套用二叉树的顺序存储。

*2.*孩子表示法

把每个结点的所有孩子串成一条单链表，n 个结点就有 n 条孩子链表（叶子对应空链表）；所有链表的头指针集中放在一个数组（表头数组）中，数组每个位置同时保存结点数据。

链表结点中保存的是孩子在表头数组中的下标，而不是孩子数据本身。

```
typedef struct CCNode {             // 孩子链表的结点
    int child;                      // 孩子在表头数组中的下标
    struct CCNode *next;            // 下一个孩子
} CCNode;

typedef struct {
    ElemType data;                  // 数据域
    CCNode *firstChild;             // 孩子链表头指针
} TreeNode;

typedef struct {
    TreeNode nodes[MAX_TREE_SIZE];
```

```
    int n;
} CTree;
```

操作特性：找孩子快（沿链表走一遍），找双亲慢（需遍历所有链表，$O(n)$）。

实用的混合方案

把双亲表示法和孩子表示法拼在一起（结点同时带 parent 域和孩子链表头），找双亲、找孩子都快，代价是多花一点空间。这就是带双亲的孩子链表。

结构对比记忆：孩子表示法与图的邻接表、散列表的拉链法、基数排序的分配收集在示意图和结构体定义上高度相似，可放在一起记忆。

*3.*孩子兄弟表示法（二叉链表 / 二叉树表示法）

每个结点设两个指针：

- 左指针 $\rightarrow$ 第一个孩子（firstChild）；
- 右指针 $\rightarrow$ 下一个兄弟（nextSibling）。

想枚举某结点的全部孩子：先沿左指针找到老大，再沿老大的右指针依次往后找，直到右指针为空。

```
typedef struct CSNode {
    ElemType data;                  // 数据域
    struct CSNode *firstChild;      // 左指针：第一个孩子
    struct CSNode *nextSibling;     // 右指针：下一个兄弟
} CSNode, *CSTree;
```

操作特性：找孩子容易、找双亲难（同孩子表示法）；若需快速回溯双亲，可再加 parent 域构成三叉链表。

孩子兄弟表示法的最大价值

它的存储结构与二叉树的二叉链表在物理形式上完全一致，因此同一棵树可以毫不费力地「解读」成一棵二叉树——这正是下一节「树与二叉树相互转换」的物理基础。

*4.*三种存储结构对比

对比项	双亲表示法	孩子表示法	孩子兄弟表示法
额外记录	双亲下标	孩子链表头	第一个孩子 + 下一个兄弟
找双亲	$O(1)$，极快	需遍历，慢	需遍历，慢
找孩子	需扫描全表，慢	沿链表，快	沿左指针 + 右链，快
存储形式	顺序（数组）	顺序 + 链式	纯链式（二叉链表）
典型用途	并查集等向上回溯场景	需要频繁枚举孩子的场景	树 / 森林与二叉树的转换

（二）森林与二叉树的转换

树、森林与二叉树之间存在一一对应关系：任何一棵树（或森林）都能唯一地转成一棵二叉树，反之亦然。这意味着——凡是在树上不好做的事，都可以转成二叉树来做，做完再翻译回去。

*1.*树 → 二叉树

转换规则：左孩子右兄弟

二叉树中结点 p 的左指针指向原树中 p 的第一个孩子；右指针指向原树中 p 的下一个兄弟。

手工三步走：

1. 断开：对每个结点，只保留它与第一个孩子的连线，其余父子连线全部擦掉；
2. 连线：把同一双亲的兄弟结点从左到右用线串起来；
3. 整形：以树根为轴心顺时针旋转约 45°，兄弟连线就变成「右指针」。

重要结论

根结点没有兄弟，所以由单棵树转成的二叉树，其根结点必定没有右子树。反过来，一棵二叉树若根结点没有右子树，它才可以还原成「一棵树」。

逆过程（二叉树 → 树）：把每个结点的左孩子还原为「第一个孩子」，把沿右指针串起来的一串结点还原为同一双亲的一排兄弟。

*2.*森林 → 二叉树

森林是若干棵树的集合，转换建立在单棵树转换的基础上。推荐用虚构根结点法：

1. 加虚根：虚构一个根结点 r，把森林中每棵树都当作 r 的子树（于是各棵树的根互为兄弟）；
2. 转换：对「虚根 + 森林」整体应用「左孩子右兄弟」规则；
3. 删虚根：把虚根 r 及与它相连的边擦掉，剩下的就是森林对应的二叉树。

等效的手工画法（不画虚根）：先把森林中每棵树各自转成二叉树（它们的根都没有右子树），然后把第 2 棵树的根接在第 1 棵树的根的右指针上、第 3 棵接在第 2 棵的右指针上……依次串联。本质上就是「把各棵树的根视为兄弟」。

逆过程（二叉树 → 森林）：

- 若二叉树根结点无右子树 → 对应一棵树；
- 若二叉树根结点有右子树 → 对应一个森林：把根结点的右指针断开，右子链上每断开一处就分离出一棵新树，对每棵分离出的二叉树分别还原。

两个重要计数结论（真题直接考）

1. 转换前森林中的叶子结点数 = 转换后二叉树中「左孩子指针为空」的结点数。理由：左指针指向第一个孩子，故「有孩子 ⇔ 左指针非空」；叶子 ⇔ 无孩子 ⇔ 左指针为空。右指针反映的是「有没有下一个兄弟」，与是否为叶子无关。
2. 若森林（或树）中有 n 个分支结点，则转换后二叉树中「右孩子指针为空」的结点数恰为 $n+1$。理由：每个分支结点的最右孩子没有下一个兄弟，贡献 1 个空右指针，共 n 个；再加上最后一棵树的根（在森林中）或该树的根（在单棵树中）没有兄弟，贡献 1 个，合计 $n+1$。

例：一棵有 2011 个结点、116 个叶结点的树，转成二叉树后无右孩子的结点个数是多少?

分支结点数 $= 2011 - 116 = 1895$，故无右孩子的结点数 $= 1895 + 1 = 1896$。（2011 年形式）

例：某森林 F 转换为二叉树 T，F 中叶子的个数等于 T 中「左孩子指针为空」的结点个数。（2014 年形式）

（三）树和森林的遍历

*1.*树的遍历

树的孩子个数不固定，无法像二叉树那样把根「夹在两棵子树中间」，所以树没有中根遍历，只有两种（外加层序）：

- 先根遍历：先访问根结点，再从左到右依次先根遍历根的每一棵子树。
- 后根遍历：先从左到右依次后根遍历根的每一棵子树，最后访问根结点。
- 层次遍历：借助队列，自上而下、自左而右逐层访问（原理同二叉树层次遍历）。

例：树 `X(Y(V), Z(U,T), W)`

- 先根遍历序列：`X Y V Z U T W`
- 后根遍历序列：`V Y U T Z W X`
- 层次遍历序列：`X Y Z W V U T`

*2.*森林的遍历

森林的遍历是递归定义的（设森林非空）：

先序遍历森林：

1. 访问第一棵树的根结点；
2. 先序遍历「第一棵树的根的所有子树」组成的森林；
3. 先序遍历「去掉第一棵树后剩下的树」组成的森林。

效果 = 依次对每棵树做先根遍历。

中序遍历森林：

1. 中序遍历「第一棵树的根的所有子树」组成的森林；
2. 访问第一棵树的根结点；
3. 中序遍历「去掉第一棵树后剩下的树」组成的森林。

效果 = 依次对每棵树做后根遍历。

例：森林 F（`A(B(D), C)`、`E(F)`、`G(H)`）

- 先序遍历序列：`A B D C E F G H`
- 中序遍历序列：`D B C A F E H G`

术语口径差异

部分教材把森林的「中序遍历」称作「后序遍历」（因为对每棵树做的是后根遍历）。两种叫法指的是同一个序列。统考以题目采用的口径为准，看到「森林的后序遍历」多数是指对每棵树做后根遍历。

*3.*与二叉树遍历的对应关系（必背）

对应关系总表

树	森林	对应二叉树
先根遍历	先序遍历	先序遍历
后根遍历	中序遍历（部分教材称后序遍历）	中序遍历
层次遍历	——	（无直接对应）

记忆口诀：「先对先，后对中」——树的先根 / 森林的先序 ≡ 二叉树先序；树的后根 / 森林的中序 ≡ 二叉树中序。

用例子验证：

- 树 T_2 先根 `X Y V Z U T W` = 对应二叉树的先序序列 ✓；T_2 后根 `V Y U T Z W X` = 对应二叉树的中序序列 ✓
- 森林 F 先序 `A B D C E F G H` = 对应二叉树先序 ✓；F 中序 `D B C A F E H G` = 对应二叉树中序 ✓

为什么「后根 = 中序」：转换后用左指针连第一个孩子、右指针连后续兄弟。对二叉树做中序遍历时，先沿左指针进入第一个孩子，孩子的右兄弟链在遍历子树的过程中依次处理，所有孩子及其子树处理完成后才访问当前根——这与原树「先遍历各子树、后访问根」的后根遍历完全一致。

最小验证例：原树根 a 有孩子 b、c。原树后根遍历为 `b c a`。转换后二叉树中 a 的左孩子是 b，b 的右孩子是 c，其中序遍历同样为 `b c a`。

真题考法

- 2009：森林转二叉树后，若 u 是 v 的双亲的双亲，问原森林中 u、v 的关系（枚举 L-L、L-R、R-L、R-R 四种路径即可）。
- 2019：树 T 转二叉树 BT 后，与 T 的后根遍历序列相同的是 BT 的中序遍历。
- 2020：已知森林的先根与后根序列，求对应二叉树的后序序列——先用「先根 = 二叉树先序、后根 = 二叉树中序」还原二叉树，再求后序。
- 2021：给出二叉树 T 的先序与中序序列，问 F 中树的棵数——森林中树的棵数 = 二叉树根结点沿右指针链的结点数（即根到最右下的右链长度）。

四、树与二叉树的应用

（一）哈夫曼树与哈夫曼编码

*1.*基本概念

- 结点的权：赋予结点的、代表某种含义的数值（如字符出现频率、判断概率）。
- 路径长度：路径上的分支（边）数目。
- 结点的带权路径长度：从该结点到根的路径长度与该结点权的乘积 $w_i \times l_i$。
- 树的带权路径长度 **WPL**：

WPL 定义

$$WPL = \sum_{i=1}^{n_0} w_i\, l_i$$

其中 n_0 为叶子结点个数，w_i 为第 i 个叶子的权，l_i 为该叶子到根的路径长度。

WPL 只计算叶子结点，不计算内部结点。

*2.*哈夫曼树的定义

哈夫曼树（**Huffman Tree**）又称最优二叉树：在 n_0 个带权叶子结点构成的所有二叉树中，**WPL** 最小的那棵。

要让 WPL 小，就要让权大的叶子离根近（路径短），权小的叶子离根远（路径长）。

*3.*构造算法

构造步骤

1. 初始：构造含 n_0 棵二叉树的森林 $F=\{T_1,\dots,T_{n_0}\}$，每棵树只有一个权为 w_i 的根结点。
2. 选两个最小：从 F 中选取根权值最小的两棵树 T_x、T_y（选两次，可用小顶堆实现）。
3. 合并：新建结点 T_j，权值 $=T_x$ 根权 $+T_y$ 根权，以 T_x 为左子树、T_y 为右子树，把 T_j 加入 F。
4. 重复 2、3，直到 F 中只剩一棵树，即哈夫曼树。

例：权值 $\{7,5,2,4\}$ 构造哈夫曼树。

步骤	森林 F	操作
1	$\{7,5,2,4\}$	初始，4 棵树各一个结点
2	$\{7,5,6\}$	选最小的 2 和 4 合并，新结点权 $2+4=6$
3	$\{7,11\}$	选最小的 5 和 6 合并，新结点权 $5+6=11$
4	$\{18\}$	选 7 和 11 合并，根权 $7+11=18$

$$WPL_{\text{按叶子算}}=7\times1+5\times2+2\times3+4\times3=7+10+6+12=35$$

WPL 的第二种算法（更快）

哈夫曼树的 **WPL** = 所有非叶（内部）结点的权值之和。

$$\text{WPL}=6+11+18=35$$

原理：每次合并产生的新结点权值会被累加进所有子孙叶子的路径长度中；把每个分支结点的权值拆开分配，恰好等于所有叶子的带权路径长度之和。考试中用这个算法能省一半时间。

哈夫曼编码的代码实现（顺序存储，含选两个最小值）：

```
typedef struct HTNode {
    int weight;         // 权值
    char data;          // 待编码字符
    int parent;         // 双亲下标
    int lchild, rchild; // 左右孩子下标
} HTNode, *HuffmanTree;

typedef char **HuffmanCode;

// 在 0..n-1 中选出两个根结点的最小值下标（parent == 0 表示尚未被合并）
void Select(HuffmanTree HT, int n, int &min1, int &min2) {
    int i, min;
    for (i = 0; i < n; i++)          // 第一个最小值
        if (HT[i].parent == 0) { min = i; break; }
    for (i = min + 1; i < n; i++)
```

```
        if (HT[i].parent == 0 && HT[i].weight < HT[min].weight) min = i;
    min1 = min;
    for (i = 0; i < n; i++)          // 第二个最小值（跳过 min1）
        if (HT[i].parent == 0 && i != min1) { min = i; break; }
    for (i = min + 1; i < n; i++)
        if (HT[i].parent == 0 && i != min1 && HT[i].weight < HT[min].weight) min = i;
    min2 = min;
}

// 构造哈夫曼树：n 个叶子，总结点数 m = 2n - 1
void Huffman_Create(HuffmanTree &HT, int *w, char *data, int n) {
    int m = 2 * n - 1;
    HT = (HuffmanTree)calloc(m, sizeof(HTNode));  // calloc 自动清零 parent/lchild/rchild
    for (int i = 0; i < n; i++) {
        HT[i].weight = w[i];
        HT[i].data = data[i];
    }
    for (int i = n; i < m; i++) {
        int min1, min2;
        Select(HT, i, min1, min2);                // 只在 i 之前（已建好的）中选
        HT[i].weight = HT[min1].weight + HT[min2].weight;
        HT[min1].parent = i;
        HT[min2].parent = i;
        HT[i].lchild = min1;                      // 较小的作左孩子
        HT[i].rchild = min2;
    }
}

// 从叶子向上回溯得到编码（得到的是逆序，从后往前填）
void Huffman_Coding(HuffmanTree HT, HuffmanCode &HC, int n) {
    HC = (HuffmanCode)malloc(sizeof(char *) * n);
    char *code = (char *)malloc(sizeof(char) * n);
    code[n - 1] = '\0';
    for (int i = 0; i < n; i++) {
        int j = i, p = HT[j].parent, start = n - 1;
        while (p != 0) {
            code[--start] = (HT[p].lchild == j) ? '0' : '1';
            j = p;
            p = HT[j].parent;
        }
        HC[i] = (char *)malloc(sizeof(char) * (n - start));
        strcpy(HC[i], &code[start]);
    }
    free(code);
}
```

*4.*哈夫曼树的性质

五条性质

1. 结点总数：若哈夫曼树有 n_0 个叶结点，则总结点数为 $2n_0-1$，度为 2 的结点数为 n_0-1。推导：哈夫曼树没有度为 1 的结点，故 $n=n_0+n_2$；又 $n_0=n_2+1$，代入得 $n_2=n_0-1$，$n=2n_0-1$。
2. 严格二叉树：哈夫曼树中结点的度只能是 0 或 2，没有度为 **1** 的结点。
3. 权值单调性：从根往下，任意非叶结点的权值不小于下一层任一结点的权值（越靠近根权值越大）。
4. 两个最小权值必为兄弟：构造过程中权值最小的两个结点一定在第一次合并时成为同一新结点的两个孩子。
5. 形态不唯一，但 **WPL** 唯一：左右子树可以互换、权值相同时合并顺序不同，都会得到不同形态但同为

最优的哈夫曼树；**WPL** 始终相同且最小。高度也可能不同。

哈夫曼树不一定是完全二叉树

哈夫曼树是严格二叉树，但不是完全二叉树。当权值分布很不均匀时，较小权值会被反复合并，树形可能明显向一侧加深，不满足完全二叉树的层序连续性。（2010 年真题的考点）

*5.m 叉哈夫曼树

由 n_0 个初始（叶）结点构造 m 叉哈夫曼树：

补充虚结点的公式

1. 计算 $t=(n_0-1) \bmod (m-1)$。若 $t\neq 0$，需补充 $m-1-t$ 个权为 **0** 的虚叶结点；若 $t=0$ 则不需补充。
2. 每次从所有结点中取出 m 个权值最小的结点，作为新结点的孩子，新结点权值为各孩子权值之和，再加入结点集合。每轮结点数减少 $m-1$ 个。
3. 重复直到只剩一个结点。

为什么要补：保证最后一次合并恰好是 m 个结点，使构造出的树是真正的 m 叉哈夫曼树（根结点恰好有 m 个孩子）。虚结点的权为 0，不影响 **WPL**。

例：权值 $\{2,3,4,5,6,7\}$ 构造三叉哈夫曼树（2013 年统考形式）。

$n_0=6$，$m=3$，$t=(6-1) \bmod 2=1\neq 0$，需补充 $3-1-1=1$ 个权为 0 的虚结点，集合变为 $\{0,2,3,4,5,6,7\}$。

1. 取最小的三个 $0,2,3$ 合并 $\to 5$，集合 $\{4,5,5,6,7\}$
2. 取最小的三个 $4,5,5$ 合并 $\to 14$，集合 $\{6,7,14\}$
3. 取 $6,7,14$ 合并 $\to 27$，根权 27

树的形状：根 27 的三个孩子依次为 6、7、14；14 的三个孩子依次为 4、5、5′；5′ 的三个孩子依次为 0（虚）、2、3。

叶子	权	到根的路径长度
6	6	1
7	7	1
4	4	2
5	5	2
2	2	3
3	3	3
（虚）	0	3

$$WPL=6\times 1+7\times 1+4\times 2+5\times 2+2\times 3+3\times 3+0\times 3=6+7+8+10+6+9=46$$

用「内部结点权值之和」验证：$5+14+27=46$ ✓ 两种算法一致（虚结点权为 0，不产生贡献）。

m 叉哈夫曼树的 WPL 计算

m 叉情况下 $WPL=$ 所有非叶结点的权值之和仍然成立（每次合并产生的权值都是其所有子树中叶子带权路径长度的一份贡献，逐层累加即为 WPL）。这条算法比按叶子逐个数路径长度更可靠。$m=2$ 时两种算法必然一致，可以作为自查手段。

*6.*哈夫曼编码

编码：把数据转换成 0/1 串的过程。

前缀编码：没有任何一个编码是另一个编码的前缀。这样译码时从左到右扫描，每遇到一个完整编码即可立即确定一个字符，无需回溯，实现唯一可译。

前缀编码的判定方法

把给出的编码逐个比较：

- 若存在某个编码是另一个编码的前缀，则不是前缀编码；
- 等长编码一定是前缀编码；
- 「首位互不相同」或「某一位上已经区分开」都可安全排除前缀关系。

例：$\{01, 0000, 0001, 001, 1\}$ 是前缀编码；$\{0, 100, 110, 1110, 1100\}$ 不是，因为 `110` 是 `1100` 的前缀。

哈夫曼编码：对一棵有 n_0 个叶子的哈夫曼树，左分支赋 **0**、右分支赋 **1**（也可以反过来），从根到每个叶子的路径上的赋值序列就构成该叶子对应字符的哈夫曼编码。

哈夫曼编码的四条结论

1. 哈夫曼编码是最优前缀编码，能使文件压缩后的二进制长度最短；
2. 左右分支的 0/1 赋值无强制规定，且权值相同的结点合并顺序可以不同，因此哈夫曼树形态不唯一 $\rightarrow$ 哈夫曼编码不唯一；
3. 但 **WPL** 唯一且最小，即总编码长度唯一；
4. 出现频率高的字符编码短、频率低的编码长——这是哈夫曼编码高效的根源。

例：文件 f 由字符集 $\{U, V, W, X, Y, Z\}$ 组成，出现次数分别为 $\{8, 10, 25, 12, 15, 30\}$。

合并过程：$8+10=18$；$12+15=27$；$18+25=43$；$27+30=57$；$43+57=100$。

字符	出现次数	编码（左 **0** 右 **1**）	编码长度
U	8	000	3
V	10	001	3
W	25	01	2
X	12	100	3
Y	15	101	3
Z	30	11	2

$$WPL = 8\times3 + 10\times3 + 25\times2 + 12\times3 + 15\times3 + 30\times2 = 24+30+50+36+45+60 = 245$$

等价的快速算法：$WPL = 18+27+43+57+100 = 245$ ✓

与定长编码对比：6 个字符用定长编码需 $\lceil \log_2 6 \rceil = 3$ 位，总长 $100\times3 = 300$ 位。

$$\text{压缩率} = \frac{300-245}{300} \approx 18.3\%$$

加权平均长度

加权平均长度 L 表示平均每个符号占用的编码位数，衡量单位符号的编码效率：

$$L=\frac{\sum_{i=1}^{n} f_i l_i}{\sum_{i=1}^{n} f_i}=\frac{WPL}{F}=\sum_{i=1}^{n} p_i l_i$$

其中 f_i 为符号频次，$F=\sum f_i$ 为总频次，$p_i=f_i/F$ 为概率。**WPL** 是总量，L 是平均值，二者通过 $WPL=L\times F$ 互相换算。

例：频次 $\{3,4,5,6,8,10\}$，$F=36$。按哈夫曼算法合并：$3+4=7$；$5+6=11$；$7+8=15$；$10+11=21$；$15+21=36$。

$$WPL=7+11+15+21+36=90,\qquad L=\frac{90}{36}=2.5$$

（2023 年统考形式）

一个重要的常数关系

哈夫曼编码的最长编码长度 +1 = 哈夫曼树的最大高度（按根为第 1 层、高度等于层数的约定）。

真题考法（哈夫曼这一节年年出题）

- 判断题：哈夫曼树是否是完全二叉树（否）、是否有度为 1 的结点（无）、两个最小权值是否必为兄弟（是）、非叶结点权值是否不小于下层（是）——2010 年形式。
- 给出一组编码问哪个是前缀编码——2014 年形式。逐个检查是否有编码是另一个的前缀。
- 给定权值序列求最小 WPL——用「非叶结点权值之和」最快（2021、2023 年形式）。算例：权值 $\{10,12,16,21,30\}$，合并过程为 $10+12=22$、$22+16=38$、$21+30=51$、$38+51=89$，故

$$WPL_{\min}=22+38+51+89=200$$

注意：算出来的 89 是根结点的权值，不是 WPL。看到选项里有 89 不要直接选——那是典型干扰项（2021 年真题的正确选项为 200）。

- 两级 WPL 的对比：「从根到两个叶子的路径权值序列」，要求每个非叶结点权值 = 孩子权值之和（2015 年形式）。
- 给出哈夫曼编码串要求译码——从前向后贪心切分，因为前缀编码的任意前缀都不是完整编码（2017 年形式）。
- 哈夫曼树与定长编码树的对比：定长编码的叶结点全部位于同一层（2022 年形式）。
- 同一哈夫曼树上哪些叶子同层：按合并过程画出树，直接读深度（2026 年形式）。

（二）并查集

*1.*定义与核心思想

并查集（**Disjoint Set Union**，**DSU**）是处理不相交集合的合并与查询问题的数据结构。

操作	功能
`Initial(S)`	把集合 S 中每个元素初始化为独立的单元素集合
`Find(S,x)`	查找 x 所在子集合的根结点
`Union(S,Root1,Root2)`	把 `Root2` 并入 `Root1`（要求两者属于不同集合）

核心思想：用一棵树表示一个集合，根结点作为集合的代表。

- 判断元素属于哪个集合 → 找该元素所在树的根；

- 合并两个集合 → 把一棵树的根挂到另一棵树的根下。

*2.*存储结构：树的双亲表示法

用一个数组：

- 数组的下标代表元素（结点编号）；
- 数组元素的值代表该元素的双亲下标；
- 根结点的值为负数，其绝对值表示该集合的元素个数。

```
#define SIZE 100

typedef struct {
    int data;       // 元素数据值
    int parent;     // 双亲下标（负数表示根，绝对值表示集合大小）
} UFSType;

typedef struct {
    UFSType *elements;  // 双亲指针数组
    int n;              // 元素个数
} UnionFindSet;
```

根结点用负数的好处

根结点既标识了集合（谁是代表），又顺带记录了集合大小，一举两得。按大小合并时直接比较绝对值即可，无需额外数组。

例：数组 `S = [-4, 0, 0, 1, -4, 4, 5, 6]`（下标 0~7）对应：

- 集合一：根 0（`S[0]=-4`），1、2 是 0 的孩子，3 是 1 的孩子 → $\{0,1,2,3\}$
- 集合二：根 4（`S[4]=-4`），5 是 4 的孩子，6 是 5 的孩子，7 是 6 的孩子 → $\{4,5,6,7\}$

数组中只有 2 个负数，故共有 **2** 个集合。

*3.*基本操作

```
// 初始化：每个元素自成一个集合
void Initial(int S[], int n) {
    for (int i = 0; i < n; i++) S[i] = -1;
}

// 查找 x 所在集合的根（朴素版）
int Find(int S[], int x) {
    while (S[x] >= 0) x = S[x];   // 沿双亲链向上，遇到负数即为根
    return x;
}

// 合并：把 Root2 挂到 Root1 下（朴素版）
void Union(int S[], int Root1, int Root2) {
    if (Root1 == Root2) return;
    S[Root1] += S[Root2];         // 累加集合大小
    S[Root2] = Root1;
}
```

朴素实现的致命问题

朴素的 `Union` 不考虑两棵树的大小，反复执行 `Union(i+1, i)` 会退化成一条长度为 n 的链，此时 `Find`

的代价为 $O(n)$。把 n 个元素合并成一个集合的总代价可达 $O(n^2)$。这就是必须优化的原因。

*4.*优化一：路径压缩

在 Find 的过程中，顺便把查找路径上的所有结点直接挂到根下，压缩树高。

```
// 路径压缩 · 递归版
int Find(int S[], int x) {
    if (S[x] >= 0)
        return S[x] = Find(S, S[x]);   // 递归找根，回溯时把 x 挂到根下
    else
        return x;
}

// 路径压缩 · 迭代版（推荐，避免递归深度）
int Find(int S[], int x) {
    int root = x;
    while (S[root] >= 0) root = S[root];   // 第一步：找到根
    while (x != root) {                    // 第二步：把路径上所有结点挂到根下
        int tmp = S[x];
        S[x] = root;
        x = tmp;
    }
    return root;
}
```

路径压缩的性质

- 路径压缩是事后优化：第一次查找仍可能很慢，但之后同一路径上的结点到根的距离变为 1；
- 路径压缩不改变元素所属的集合（根不变），只是改变同一棵集合树内部的父子关系；
- 仅用路径压缩不保证树高有上界（单次最坏仍可能 $O(n)$），但均摊效果好。

*5.*优化二：按秩 / 按大小合并

合并时把较小的树挂到较大的树的根下，避免树高增长过快。

```
// 按大小合并（小树挂大树）
void Union(int S[], int Root1, int Root2) {
    if (Root1 == Root2) return;
    if (S[Root2] < S[Root1]) {      // Root2 的绝对值更大，即集合更大
        S[Root2] += S[Root1];       // 累加集合大小
        S[Root1] = Root2;           // Root1 挂到 Root2 下
    } else {
        S[Root1] += S[Root2];
        S[Root2] = Root1;           // 大小相等时把 Root2 挂到 Root1 下
    }
}
```

按秩合并的高度上界

采用按大小合并后，集合树的深度不超过 $\lfloor \log_2 n \rfloor + 1$。

证明：关键结论是秩（高度上界）为 k 的树至少有 2^k 个结点。归纳：秩首次变为 k 时，必是两棵秩为 $k-1$ 的树合并，结点数 $\geq 2^{k-1} + 2^{k-1} = 2^k$。于是 $n \geq 2^k$，得 $k \leq \lfloor \log_2 n \rfloor$。

*6.*复杂度总表（必背）

实现方式	单次 **Find**	单次 **Union**	n 个元素合并为 **1** 个集合
朴素实现	$O(n)$（最坏树高）	$O(1)$	$O(n^2)$
仅按秩合并	$O(\log_2 n)$（最坏）	$O(1)$	$O(n\log_2 n)$
仅路径压缩	单次最坏 $O(n)$，均摊 $O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
路径压缩 + 按秩合并	均摊 $O(\alpha(n))$	$O(1)$	$O(n\,\alpha(n))$

关于 $\alpha(n)$

$\alpha(n)$ 是 Ackermann 函数的反函数，增长极慢：$n \le 10^{80}$ 时 $\alpha(n) \le 4$，可视为常数。因此并查集的操作近乎 $O(1)$。

注意 $\alpha(n)$ 是均摊界，不是单次最坏界：单次 `Find` 最坏仍可能是 $O(\log_2 n)$，只是在整条操作序列上摊薄后接近常数。

易错辨析

- 「只要采用路径压缩，任意时刻树高都不超过 $\lfloor \log_2 n \rfloor + 1$」——错。树高上界是按秩合并带来的保证，路径压缩只压低被访问路径的高度。
- 「路径压缩可能改变元素所属的集合」——错。压缩前后元素的根不变。
- 并查集不支持高效地拆分集合（不能把一棵子树从集合树中分离出来）。
- n 个元素合并为一个集合时，若不优化会退化成链，`Find` 最坏 $O(n)$。

*7.*典型应用

（1）图的连通性问题：每个顶点初始为一个集合，每条边对应一次 `Union`，判断两顶点是否连通即比较 `Find(u) == Find(v)`。

（2）Kruskal 最小生成树：用并查集判断新加入的边是否会与已选边构成环。

1. 把所有边按权值从小到大排序；
2. 依次考察每条边 (u, v)：若 `Find(u) != Find(v)`（不连通，不成环），选中该边并 `Union`；否则舍弃；
3. 直到选够 $n-1$ 条边。

注意：图不连通时，Kruskal 得到的是最小生成森林，而不是最小生成树。

例：顶点集 $V = \{0,1,2,3,4,5,6\}$，边按权递增排列为 `e1=(0,1,1) e2=(1,2,2) e3=(3,4,3) e4=(0,2,4) e5=(4,5,5) e6=(2,3,6) e7=(1,5,7) e8=(5,6,8)`

边	结果	原因
`e1=(0,1,1)`	选中	0、1 不连通
`e2=(1,2,2)`	选中	1、2 不连通
`e3=(3,4,3)`	选中	3、4 不连通
`e4=(0,2,4)`	舍弃	0、2 已连通，成环
`e5=(4,5,5)`	选中	4、5 不连通
`e6=(2,3,6)`	选中	合并 $\{0,1,2\}$ 与 $\{3,4,5\}$
`e7=(1,5,7)`	舍弃	1、5 已连通，成环

边	结果	原因
e8=(5,6,8)	选中	顶点 6 接入生成树

最小生成树为 $\{e1, e2, e3, e5, e6, e8\}$，总权值 $1+2+3+5+6+8=25$。处理完 e7 后 $\{0,1,2,3,4,5\}$ 已连通、6 单独成一个连通分量，故有 **2** 个连通分量。

真题考法

- 给出并查集数组，问某个 Find 之后 S[x] 变成什么（路径压缩后，路径上所有结点都直接指向根）；
- 给出操作序列要求写最终数组（按大小合并时要注意「大小相等时第二个根挂到第一个根下」的约定）；
- 在 Kruskal 算法中并查集的作用（判环）。

（三）堆

*1.*定义

堆（**Heap**）是满足「堆序性」的完全二叉树：

- 大根堆（大顶堆）：任一结点的关键字不小于其左右孩子的关键字，即 $a_i \geq \max\{a_{2i}, a_{2i+1}\}$；
- 小根堆（小顶堆）：任一结点的关键字不大于其左右孩子的关键字，即 $a_i \leq \min\{a_{2i}, a_{2i+1}\}$。

堆的三条基本事实

1. 逻辑上是一棵完全二叉树，因此适合顺序存储：下标从 1 开始时，i 的左孩子 $2i$、右孩子 $2i+1$、双亲 $\lfloor i/2 \rfloor$；
2. 堆不是二叉排序树：堆只要求「父不小于（不大于）孩子」，不要求「左子树全小于根、右子树全大于根」，因此中序遍历不有序；
3. 大根堆的次大值一定在根的下一层（即根的左孩子或右孩子之一，或两者）；同理小根堆的次小值也必在根的孩子中。

```
#define MAXSIZE 100

typedef struct {
    int elem[MAXSIZE];   // 下标从 1 开始存储，elem[0] 不用
    int n;               // 元素个数
} Heap;
```

*2.*建堆：*Floyd* 自下而上算法 $O(n)$

关键点：一次性先把整个序列按层序摆成完全二叉树，然后从最后一个非叶结点开始向前逐个「下坠调整」，而不是一个一个插入。这是考试与代码实现的分水岭。

```
// 对以 k 为根的子树进行「下坠」调整（大根堆）
void SiftDown(Heap &H, int k) {
    H.elem[0] = H.elem[k];                          // 暂存待调整的根
    int i = k;
    for (int j = 2 * i; j <= H.n; j = 2 * i) {   // 沿关键字较大的孩子往下
        if (j < H.n && H.elem[j] < H.elem[j + 1]) j++;   // j 指向较大的孩子
        if (H.elem[0] >= H.elem[j]) break;       // 已满足大根堆，结束
        H.elem[i] = H.elem[j];                   // 孩子上移
        i = j;
```

```
    }
    H.elem[i] = H.elem[0];                          // 就位
}

// 自下而上建堆（Floyd 算法）：时间 O(n)
void BuildHeap(Heap &H) {
    for (int i = H.n / 2; i >= 1; i--)   // 从最后一个非叶结点开始
        SiftDown(H, i);
}
```

为什么建堆是 $O(n)$ 而不是 $O(n\log_2 n)$

「下坠」的代价与结点所在高度成正比。自下而上建堆时，绝大多数结点在底层，下坠距离很短：

- 高度为 h 的结点约有 $n/2^{h+1}$ 个，每个下坠至多 h 层；
- 总代价 $\le \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{n}{2^{h+1}} \cdot h < 2n = O(n)$。

对比：如果从空堆开始逐个插入建堆，每个元素平均要上升 $O(\log_2 n)$，总代价 $O(n\log_2 n)$。两种建堆方式得到的堆不一定相同。

建堆的常见错误

1. 不要从最后一个结点开始：叶子结点天然满足堆序，从 $n/2$ 开始（最后一个非叶结点）即可；
2. 下坠后要继续探测子树：把根与较大孩子交换后，被交换的孩子所在的子树可能不再满足堆序，必须继续沿该路径往下调整；
3. 不要每交换一次就重新从头扫描：正确的做法是沿着一条根到叶的路径一次下坠到底，代价 $O(\log_2 n)$。

建堆比较次数的量级

自下而上建大根堆的过程中，关键字比较次数最多约 $4n$，时间复杂度 $O(n)$。这是统考中「建堆比较次数」问题的标准答案量级。

*3.*插入元素（向上调整）

步骤：把新元素放到最后（第 $n+1$ 个位置），然后沿双亲方向向上调整，直到满足堆序。

```
// 大根堆插入：先放末尾，再向上调整
bool HeapInsert(Heap &H, int x) {
    if (H.n >= MAXSIZE - 1) return false;
    int i = ++H.n;
    H.elem[i] = x;
    while (i > 1 && H.elem[i] > H.elem[i / 2]) {  // 比双亲大就交换
        int tmp = H.elem[i];
        H.elem[i] = H.elem[i / 2];
        H.elem[i / 2] = tmp;
        i = i / 2;
    }
    return true;
}
```

插入的调整次数

插入时只需与双亲比较，不需要和兄弟比较（因为兄弟一定也满足与双亲的大小关系）。比较次数最多为树高 -1，即 $\lfloor \log_2 n \rfloor$，时间复杂度 $O(\log_2 n)$。

这一点要和「排序时的取数调整」区分开：排序时是从根往下与孩子比较（可能还要先比较两个孩子），每次下坠一层最多 2 次比较。

例：小根堆 8,15,10,21,34,16,12 中插入 3。

第 9 个位置放 3，双亲为 $\lfloor 9/2 \rfloor = 4$ 号元素 21：$3 < 21$，交换；3 到第 4 位，双亲为 2 号元素 15：$3 < 15$，交换；3 到第 2 位，双亲为 1 号元素 8：$3 < 8$，交换。最终根为 3。（2009 年形式）

例：大根堆 25,13,10,12,9 尾部插入 18，比较次数是多少?

18 放第 6 位，双亲 $6/2 = 3$ 号元素 10，比较 $18 > 10$ 交换（第 1 次）；18 到第 3 位，双亲为 1 号元素 25，比较 $18 < 25$ 停止（第 2 次）。共 **2** 次比较。（2011 年形式）

*4.*删除堆顶（向下调整）

步骤：取走堆顶（通常与最后一个元素交换后，令 n 减 1），再把新的根下坠调整。

```
// 大根堆删除堆顶：把末尾元素补到根，再下坠
bool HeapDeleteMax(Heap &H, int &x) {
    if (H.n == 0) return false;
    x = H.elem[1];
    H.elem[1] = H.elem[H.n--];   // 末尾元素补到根，堆规模减 1
    SiftDown(H, 1);
    return true;
}
```

删除的调整次数

从根下坠到叶子，最多走树高 -1 层，每层最多 2 次关键字比较（先比两个孩子、再比孩子与根），时间复杂度 $O(\log_2 n)$。

例：小根堆 8,15,10,21,34,16,12 删除 8 后的比较次数。

用末尾的 12 补到根，有效序列 12,15,10,21,34,16：

1. 比较根的两个孩子 15 与 10，选较小的 10（第 1 次）；
2. 比较 12 与 10，$12 > 10$ 交换（第 2 次）；
3. 12 下移到第 3 位，该位置只有左孩子 16：比较 12 与 16，$12 < 16$ 停止（第 3 次）。

共 **3** 次比较。（2015 年形式）

*5.*堆与堆排序

堆排序（**Heap Sort**）的两阶段：

1. 建堆：对 n 个元素的序列自下而上建成大根堆（升序排序用大根堆），时间 $O(n)$；
2. 反复取数：把堆顶（当前最大）与堆尾交换，堆规模减 1，再对新根下坠调整；重复 $n-1$ 次。每次调整 $O(\log_2 n)$，共 $O(n \log_2 n)$。

堆排序的时空特性

指标	值
建堆时间	$O(n)$
单次调整	$O(\log_2 n)$
总时间	$O(n \log_2 n)$（最好、最坏、平均均如此）
空间	$O(1)$，仅需常数辅助空间

指标	值
稳定性	不稳定（建堆和堆顶、堆尾交换可能使相等元素跨越移动）
适用存储	顺序表，不适用于单链表

每趟都能确定一个元素的最终位置——堆排序和简单选择排序、快速排序一样，属于「每趟至少定位一个元素到最终位置」的排序。

堆的两个实用特性

1. 适合在海量数据中找出前 k 大 / 前 k 小的元素（维护一个大小为 k 的堆）；
2. 堆排序的左右子树调整互不干扰，可以并行执行。

堆排序不稳定的例子

原序列 1 2 2。建大根堆后两个 2 中的一个上到根；取数时先取出的必是根上的 2，剩下两个元素中原本在前的 1 反而先被取出，最终顺序变为 1 2 2（原来的第二个 2 排到了第一个 2 前面），稳定性被破坏。

真题考法

- 「逻辑上是完全二叉树」「可以顺序存储」正确，「可以看成二叉排序树」错误（2020 年形式）；
- 依次插入建堆与自下而上建堆结果不一定相同（2021 年形式特别注意）；
- 插入 / 删除时的关键字比较次数（2011、2015、2024 年形式）——必须逐步模拟；
- 堆排序与外部排序的关系：海量数据（远大于内存）的排序应采用外部排序（多路归并），堆排序虽能取前 k 大但不适合整体外排。

本章小结表

（四）表 1 二叉树五个性质一览

编号	内容	公式	适用范围
1	第 i 层最多结点数	2^{i-1}	任意二叉树
2	高度 h 最多结点数	2^h-1	任意二叉树（取满即满二叉树）
3	叶结点与度 2 结点	$n_0=n_2+1$	任意非空二叉树
4	完全二叉树编号关系	双亲 $\lfloor i/2\rfloor$，左 $2i$，右 $2i+1$	完全二叉树（从 1 编号）
5	完全二叉树高度	$\lceil\log_2(n+1)\rceil=\lfloor\log_2 n\rfloor+1$	完全二叉树

（五）表 2 一般树 / 森林的性质

内容	公式
树的边数	$e=n-1$
结点数与度数	$n=\sum_{i\geq 1} i\cdot n_i+1$
度为 m 的树第 i 层最多	m^{i-1}
高度 h 的 m 叉树最多	$\frac{m^h-1}{m-1}$

内容	公式
高度 h 的 m 叉树最少	h
高度 h、度为 m 的树最少	$h+m-1$
n 个结点的 m 叉树最小高度	$\lceil \log_m(n(m-1)+1) \rceil$
n 个结点的度为 m 的树最大高度	$n-m+1$
n 个结点的二叉树形态数	$b_n = \frac{1}{n+1}C_{2n}^{n}$
n 个结点的树形态数	$t_n = b_{n-1} = \frac{1}{n}C_{2n-2}^{n-1}$

（六）表 3 遍历序列的对应与唯一性

序列组合	唯一性	构造要点
先序 + 中序	唯一	先序首元素定根，用中序切分左右，用左子树结点数切分先序
后序 + 中序	唯一	后序末元素定根，用中序切分左右，用左子树结点数切分后序
层次 + 中序	唯一	层次首元素定根，从中序分左右，从层次序列筛选子树的层次序列
先序 + 后序	不唯一	只对严格二叉树唯一
层次 + 先序 / 层次 + 后序	不唯一	只对严格二叉树唯一；对完全二叉树因形状受限也可唯一

遍历与树 / 森林的对应	对应二叉树的遍历
树的先根遍历 / 森林的先序遍历	先序遍历
树的后根遍历 / 森林的中序遍历	中序遍历
（无对应）	后序遍历

（七）表 4 三种线索二叉树的能力

能力	中序线索	先序线索	后序线索
找前驱	可直接得（`ltag=0` 时取左子树最右下）	需从头遍历 / 三叉链表	可直接得
找后继	可直接得（`rtag=0` 时取右子树最左下）	可直接得	需从头遍历 / 三叉链表
不用栈遍历	可以	可以	仍需栈

（八）表 5 并查集复杂度

实现	单次 **Find**	单次 **Union**	n 个元素合并
朴素	$O(n)$	$O(1)$	$O(n^2)$

实现	单次 **Find**	单次 **Union**	n 个元素合并
仅按秩合并	$O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
仅路径压缩	均摊 $O(\log_2 n)$	$O(1)$	$O(n\log_2 n)$
路径压缩 + 按秩合并	均摊 $O(\alpha(n))$	$O(1)$	$O(n\,\alpha(n))$

（九）表 **6** 哈夫曼树 / 堆 / 线索二叉树的关键数量

结构	关键数量
二叉链表（n 个结点）	空指针域 $n+1$ 个
哈夫曼树（n_0 个叶子）	总结点 $2n_0-1$，内部结点 n_0-1，度为 1 的结点 0 个
线索二叉树（n 个结点）	线索（`ltag` 或 `rtag` 为 1 的域）共 $n+1$ 条
完全二叉树（层次遍历）	队列最大规模 = 叶子结点数 = $\lceil n/2\rceil$
森林转二叉树（n 个分支结点）	无右孩子的结点 $n+1$ 个；叶子数 = 左孩子为空的结点数

易错清单

1. 「树的高度」与「结点的深度 / 高度」混用。深度从根往下数、高度从下往上数；树高 = 根的高度 = 最深层号。路径长度按边数计，不是结点数。
2. 「度为 m 的树」当成「m 叉树」。前者非空且必须有度为 m 的结点（至少 $m+1$ 个结点）；后者可以为空。
3. 顺序存储二叉树时只按结点数估算空间。若题目强调「任意一棵」二叉树或给了高度，必须按最深结点编号（最坏情况 2^h-1）开空间。
4. 用「层序序列 + 空结点」反推二叉树时忘了校验双亲。任何非空结点的双亲位置都必须非空，否则是悬空结点，该数组非法。
5. 误认为「先序 + 后序」能唯一确定二叉树。只有严格二叉树才可以；一般情况会出现「单孩子结点在左还是在右」的歧义。
6. 误认为「先序序列相同 + 结点数相同」的二叉树个数可以随便猜。答案就是卡特兰数 $\dfrac{1}{n+1}C_{2n}^{n}$；但问普通树的形态数时要换成 $t_n=b_{n-1}$。
7. 「先序 = 中序」误答成「结点的度均为 **1**」。正确条件是所有非叶结点只有右子树（不存在左孩子），必须排除「只有左孩子」的情形。
8. 中序线索二叉树找前驱 / 后继时搞反「最左下」和「最右下」。后继取右子树的最左下，前驱取左子树的最右下。
9. 先序线索化递归时不判断 `ltag` / `rtag`，会顺着刚建立的线索走回祖先，造成死循环。
10. 误认为三种线索二叉树都能不用栈遍历。后序线索二叉树仍需栈，因为无法判断是否已从右子树返回。
11. 误认为三种线索二叉树都能直接找前驱和后继。口诀：中序两全、先序缺前驱、后序缺后继；缺的那一侧必须借助三叉链表（`parent`）或从头遍历。
12. 线索二叉树不带头结点时忘了处理首尾。中序第一个结点的左线索、最后一个结点的右线索应指向 `NULL`；带头结点时则指向头结点，构成双向线索链表。
13. 森林转二叉树后「无右孩子结点数」漏掉 **+1**。n 个分支结点贡献 n 个「最右孩子」，还要加上最后一棵树的根（它没有兄弟），共 $n+1$。

14. 混淆「森林的叶子数」与二叉树中哪种空指针对应。森林叶子数 = 二叉树中左孩子为空的结点数（不是右孩子为空）。
15. 树 / 森林遍历与二叉树遍历的对应关系记混。口诀「先对先，后对中」：先根 / 先序 $\leftrightarrow$ 二叉树先序；后根 / 中序 $\leftrightarrow$ 二叉树中序。
16. 哈夫曼树当成完全二叉树。哈夫曼树是严格二叉树（无度为 1 的结点），但不一定完全。
17. 构造 m 叉哈夫曼树时忘记补虚结点。判据是 $(n_0-1) \bmod (m-1)$ 是否为 0；补 $m-1-t$ 个权为 **0** 的结点。虚结点不影响 WPL。
18. **WPL** 把内部结点也乘路径长度算进去。WPL 只累加叶子；反过来，用「所有非叶结点权值之和」也能算出同一个 WPL（哈夫曼树特有两算法）。
19. 把「编码不唯一」和「**WPL** 不唯一」混为一谈。哈夫曼树形态不唯一 $\rightarrow$ 编码不唯一；但 **WPL**（总编码长度）唯一且最小。
20. 插入堆和删除堆的调整方向搞反。插入向上调（只与双亲比），删除堆顶向下调（要先比两个孩子）。
21. 建堆时从最后一个结点开始，或每交换一次就重新扫描整个数组。正确做法是从 $i=\lfloor n/2 \rfloor$ 开始向前，每个结点沿一条路径一次下坠到底，总代价 $O(n)$。
22. 认为「逐个插入建堆」与「自下而上建堆」结果相同。二者得到的堆不一定相同，且前者时间 $O(n\log_2 n)$、后者 $O(n)$。
23. 并查集按大小合并时比较方向写反。根结点的值是负数，`S[Root1]`、`S[Root2]` 中绝对值更大的是更大的集合，应把小的挂到大的下面。
24. 误认为路径压缩能保证树高 $O(\log_2 n)$。树高上界来自按秩合并；只做路径压缩时单次最坏仍可能是 $O(n)$。
25. 误认为并查集支持高效的集合拆分。并查集只擅长「合并」与「查询」，不支持拆分。
26. 在 **Kruskal** 中把并查集的作用说成「排序」或「求最短路径」。并查集的唯一作用是判断新加入的边是否会与已选边构成环；排序是排序算法的工作。

第五章图

本章考纲要求

五、图（一）图的基本概念（二）图的存储及基本操作

1. 邻接矩阵
2. 邻接表
3. 邻接多重表、十字链表

（三）图的遍历

1. 深度优先搜索
2. 广度优先搜索

（四）图的基本应用

1. 最小（代价）生成树
2. 最短路径
3. 拓扑排序
4. 关键路径

图是数据结构部分分值最高的章节之一，特点是概念判断题多、应用算法题多、算法设计题常考。本章的复习策略是：概念部分把「度数关系」「连通性边数上下界」「存储结构的空间与效率」这三类结论背死；应用部分把 Prim、Kruskal、Dijkstra、Floyd、拓扑排序、关键路径这六个算法的手工执行流程练到能闭眼画表。考纲里的「图的基本应用」四个条目全部要求会手算，不只是会写代码。

一、图的基本概念

(一）图、有向图与无向图

图（**Graph**）由顶点集（Vertex）和边集（Edge）组成，记为 $G=(V,E)$。其中 V 是顶点的有限非空集合，E 是顶点之间关系的集合。$|V|$ 表示顶点个数，称为图的阶；$|E|$ 表示边的条数。要注意图与线性表、树的区别：线性表可以是空表，树可以是空树，但图不能为空图，V 一定是非空集（E 可以为空）。

无向图（undirected graph）：E 是无向边的有限集合。无向边是顶点的无序对，记为 (v,w) 或 (w,v)，二者表示同一条边。

有向图（directed graph，digraph）：E 是有向边的有限集合。有向边又称弧（arc），是顶点的有序对，记为 $<v,w>$，其中 v 称为弧尾（tail，起点），w 称为弧头（head，终点），读作「从 v 到 w 的弧」。显然 $<v,w>\neq<w,v>$。

邻接（adjacent）：两顶点之间存在边时称二者邻接。在有向图中要说清方向——$<v,w>$ 表示 v 邻接到 w，w 邻接自 v。

简单图与多重图：

1. 简单图：不存在重复边，也不存在顶点到自身的边（自环）。

2. 多重图：某两个顶点之间的边数多于一条，或允许顶点通过同一条边与自身关联。

数据结构课程中只讨论简单图，后文若不特别说明，「图」一律指简单图。

带权图（网）：每条边都标有表示某种含义的数值，该数值称为权值（weight）或代价（cost），这样的图称为带权图，也叫网（network）。一条路径上所有边的权值之和称为该路径的带权路径长度。

（二）完全图、稠密图与稀疏图

完全图是边数达到上限的简单图：

- 无向完全图：任意两个顶点之间都存在边，边数为 $\frac{n(n-1)}{2}$。
- 有向完全图：任意两个顶点之间都存在方向相反的两条弧，边数为 $n(n-1)$。

由此可得普通简单图的边数取值范围：n 个顶点的无向图满足 $0 \le e \le \frac{n(n-1)}{2}$；有向图满足 $0 \le e \le n(n-1)$。

边数上下界（必背）

条件	边数
n 个顶点的无向图，e 的范围	$0 \le e \le n(n-1)/2$
n 个顶点的有向图，e 的范围	$0 \le e \le n(n-1)$
n 个顶点的连通无向图，e 最少	$n-1$（即生成树）
n 个顶点的非连通无向图，e 最多	$\frac{(n-1)(n-2)}{2}$，即 C_{n-1}^2
n 个顶点的强连通有向图，e 最少	n（构成一个环）

「非连通图最多 C_{n-1}^2 条边」的含义是：把 $n-1$ 个顶点凑成一个完全图，剩下 1 个顶点孤立，此时边数最多而图仍不连通。

稠密图与稀疏图：边数很少的图称为稀疏图，边数很多的图称为稠密图。二者没有绝对的界限，一般把满足 $|E| < |V|\log|V|$ 的图视为稀疏图。这个区分直接决定了应该用邻接矩阵还是邻接表存储，以及应该用 Prim 还是 Kruskal 求最小生成树。

（三）度、入度、出度与握手定理

无向图的度：顶点 v 的度 $TD(v)$ 是指依附于该顶点的边的条数。

有向图的度：以 v 为终点的弧的数目称为入度 $ID(v)$，以 v 为起点的弧的数目称为出度 $OD(v)$，二者之和为 v 的度：

$$TD(v) = ID(v) + OD(v)$$

握手定理（度的总和）：在具有 n 个顶点、e 条边的无向图中

$$\sum_{i=1}^{n} TD(v_i) = 2e$$

即无向图全部顶点的度数之和等于边数的 **2** 倍，因为每条边给两个端点各贡献 1 度。

在有向图中，每条弧给一个起点贡献出度、给一个终点贡献入度，因此

$$\sum_{i=1}^{n} ID(v_i) = \sum_{i=1}^{n} OD(v_i) = e$$

由握手定理推出的两个高频结论

1. 无向图中度为奇数的顶点个数一定是偶数。因为总度数是偶数，其中偶数度顶点贡献偶数，所以奇数度顶点必须两两配对成偶数个。
2. 无论有向图还是无向图，全部顶点的度数之和都是 $2e$（有向图中「度」按入度加出度理解）。做题时若已知部分顶点的度数，用这条等式可以反推剩余顶点的度数。

（四）路径、回路、简单路径与距离

- 路径（path）：顶点 v_p 到 v_q 之间的一条路径是指顶点序列 $v_p, v_{i1}, v_{i2}, \cdots, v_{im}, v_q$。有向图中的路径必须沿弧的方向走。
- 回路（环，cycle）：第一个顶点和最后一个顶点相同的路径。
- 简单路径（simple path）：路径序列中顶点不重复出现的路径。
- 简单回路：除第一个顶点和最后一个顶点外，其余顶点不重复出现的回路。
- 路径长度：路径上边的数目（不是权值之和）。
- 点到点的距离：从 u 到 v 的最短路径的长度；若不存在路径，距离记为 ∞。

「回路」与「简单路径」的关系

回路不是简单路径。简单路径要求所有顶点互不重复，而回路的首尾顶点必然相同，本身就已经重复了。类似地要区分「简单回路」的定义：只允许首尾顶点重复，中间顶点不允许重复。做题时把「顶点不重复」当成唯一判据就不会错。

（五）连通性：连通、强连通、连通分量

无向图的连通性：

- 连通：无向图中若从顶点 v 到顶点 w 有路径存在，则称 v 和 w 是连通的。
- 连通图：无向图中任意两个顶点都是连通的，则称该图为连通图。
- 连通分量（connected component）：无向图中的极大连通子图。「极大」的含义是：子图必须连通，并且包含尽可能多的顶点和边；只要还能再加进一个原图中的顶点（及关联边）而不破坏连通性，就不算极大。

有向图的连通性：

- 强连通：有向图中若从 v 到 w、从 w 到 v 都有路径，则称 v 与 w 强连通。
- 强连通图：有向图中任何一对顶点都强连通。
- 强连通分量：有向图中的极大强连通子图。

连通分量的三点判别

1. 一个连通图的连通分量只有 1 个，就是它自己；非连通图至少有 2 个连通分量。
2. 极大连通子图强调「极大」，不是「最大」。一个包含 3 个顶点的连通分量和一个包含 5 个顶点的连通分量可以同时存在，二者都是极大的。
3. 孤立顶点（度数为 0 的顶点）本身就是一个连通分量，因为它无法再与任何顶点构成连通子图。

（六）子图、生成子图与生成树

子图：设有图 $G=(V,E)$ 和 $G'=(V',E')$，若 V' 是 V 的子集且 E' 是 E 的子集，则称 G' 是 G 的子图。

生成子图：若子图 G' 满足 $V(G')=V(G)$（顶点集与原图完全相同），则称 G' 是 G 的生成子图。注意「子图」不要求顶点齐全，「生成子图」要求顶点一个不少。

生成树：连通图的生成树是包含图中全部顶点的一个极小连通子图。「极小」的含义是：边尽可能少，但必须保持连通。

生成树的三条必用性质：

1. n 个顶点的连通图，其生成树恰好含有 $n-1$ 条边。
2. 生成树中任意两个顶点之间有且仅有一条通路。
3. 对生成树而言，砍去一条边就变成非连通图，加上一条边就会形成一个回路。

生成森林：非连通图中，各个连通分量的生成树构成该图的生成森林。

推论：n 个顶点的图，如果边数大于 $n-1$，则一定含有回路。反过来，边数恰为 $n-1$ 且连通，则必是一棵树。

为什么生成树恰好是 $n-1$ 条边？这可以从两个方向理解。一方面，要让 n 个顶点全部连通，至少需要 $n-1$ 条边（每加一条边最多把两个尚未连通的「块」连起来，从 n 个孤立的块合并成 1 个块需要 $n-1$ 次合并）；另一方面，一旦边数达到 n 条，由「边数大于 $n-1$ 必有回路」可知此时必然出现回路，就不再是极小连通子图。两条合起来就锁定了 $n-1$ 这个数字。生成树的三条性质——边数恰为 $n-1$、任意两点间有且仅有一条通路、删一边即不连通加一边即出回路——其实是同一件事的三种说法，考试中经常把它们打散成判断题来考。

由「极小」与「极大」这一对概念还可以串起本章最容易混的三个术语：连通分量是极大连通子图（边尽量多、点尽量多，但不破坏连通性）；生成树是极小连通子图（点一个不少，边尽量少）；最小生成树是代价最小的生成树（在生成树这个大集合里，再按权值之和挑最小的）。三者都涉及「子图」，但一个求极大、一个求极小、一个求最优，考纲把它们放在不同条目下，答题时务必对号入座。

树：不存在回路且连通的无向图。因此 n 个顶点的树必定有 $n-1$ 条边。

有向树：一个顶点入度为 0、其余顶点入度均为 1 的有向图。

（七）图的基本概念小结

概念	无向图	有向图
边的记法	(v,w)，无序对	$<v,w>$，有序对，v 弧尾、w 弧头
完全图边数	$n(n-1)/2$	$n(n-1)$
顶点的度	$TD(v)$ 为依附边数	$TD(v)=ID(v)+OD(v)$
度数总和	$\sum TD(v)=2e$	$\sum ID(v)=\sum OD(v)=e$
连通性术语	连通、连通图、连通分量	强连通、强连通图、强连通分量
连通所需最少边数	$n-1$	n
生成树	有（仅连通图有）	无（有向图讲生成树需先转成无向图）

二、图的存储及基本操作

（一）邻接矩阵

邻接矩阵（adjacency matrix）用一个 $n \times n$ 的矩阵表示图。设矩阵为 A、顶点表为 V，则 $A[i][j]=1$ 表示顶点 $V[i]$ 到 $V[j]$ 之间存在边，$A[i][j]=0$ 表示无边。

```
#define MAXV 100          // 顶点数目的最大值

typedef struct {
    int numVertices;          // 图中实际的顶点数
    int numEdges;             // 图中实际的边数
    char vertexList[MAXV];    // 顶点表
    int edges[MAXV][MAXV];    // 邻接矩阵
} MGraph;
```

核心性质（必背）：

1. 主对角线元素没有意义，因为简单图不允许自环。
2. 无向图的邻接矩阵关于主对角线对称，即 $A[i][j]=A[j][i]$。因此可以只存上三角或下三角区，把空间压缩到约 $n(n+1)/2$ 个元素，对应一维数组下标 $k=n\cdot i-\dfrac{i(i-1)}{2}+(j-i)$（行优先、含主对角线）。
3. 无向图中，第 i 个顶点的度 = 第 i 行（或第 i 列）的非零元素个数。
4. 有向图中遵循「行出列入」：第 i 个顶点的出度 = 第 i 行的非零元素个数，入度 = 第 i 列的非零元素个数。

因此在邻接矩阵上求某个顶点的度、出度、入度，都需要扫描一整行或一整列，时间复杂度为 $O(|V|)$。

```
// 计算顶点 V 的出度（行内非零元素个数）
int OutDegree(MGraph graph, int index) {
    int count = 0;
    for (int j = 0; j < graph.numVertices; ++j) {
        if (graph.edges[index][j] != 0) count++;
    }
    return count;
}

// 计算顶点 V 的入度（列内非零元素个数）
int InDegree(MGraph graph, int index) {
    int count = 0;
    for (int i = 0; i < graph.numVertices; ++i) {
        if (graph.edges[i][index] != 0) count++;
    }
    return count;
}
```

带权邻接矩阵（网）：把矩阵元素从 0/1 换成权值。$A[i][j]=k$ 表示从 $V[i]$ 到 $V[j]$ 的边存在且权值为 k；$A[i][j]=\infty$（代码中常写 `INT_MAX` 或约定的大数）表示两顶点之间无边。主对角线元素通常置 **0**（自己到自己的距离为 0），也可以置 ∞，两种约定统考都出现过，解题时按题目给定的矩阵读即可。

```
// 带权图的邻接矩阵初始化（无向网）
void initWeightedMatrix(int n, int edges[][MAXV]) {
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            edges[i][j] = (i == j) ? 0 : INT_MAX;   // 对角线 0，其余无穷表示无边
}
```

空间与效率：邻接矩阵的空间复杂度为 $O(|V|^2)$，与边数无关，因此适合存储稠密图。由于可以直接下标寻址，增删改查一条边的时间复杂度都是 $O(1)$。代价是当图很稀疏时会浪费大量空间。

什么时候该用邻接矩阵？判断依据只有一条：边数是否接近 n^2 量级。若图是稠密的（例如 e 与 $n(n-1)/2$ 同阶），邻接矩阵不浪费空间，同时还能享受 $O(1)$ 判边、$O(1)$ 改边、天然对称性、以及「用矩阵幂数路径」这些便利；若图是稀疏的（例如 e 与 n 同阶，像道路网、程序调用图），邻接表只花 $O(n+e)$ 空间，而邻接矩阵仍要老老实实开 n^2 个单元。考试中常把这一点包装成一句叙述让你判断对错，记住「邻接矩阵的空间代价只与顶点数有关，与边数无关」这一句就足以应对。

邻接矩阵的幂与路径数目

设图 G 的邻接矩阵为 A（元素为 $0/1$），则 A^n 的元素 $A^n[i][j]$ 等于由顶点 i 到顶点 j 的长度为 n 的路径的数目。这个结论对无向图和有向图都成立。

用法：题目给出邻接矩阵让你求 $\sum_i A^2[i][i]$（即 2 步回路的数目）、或者判断两点间是否存在长度为 2 的路径时，只需做一次矩阵乘法。注意 $A^2[i][j]=\sum_k A[i][k]\cdot A[k][j]$，本质上就是枚举中转顶点 k，这与 Floyd 算法的思想一脉相承。

（二）邻接表

邻接表（adjacency list）对图中每个顶点 v_i 建立一个单链表，第 i 个单链表的结点表示依附于 v_i 的边（对有向图则是以 v_i 为弧尾的弧），这个单链表称为顶点 v_i 的边表（有向图中称为出边表）。

边表的头指针和顶点数据采用顺序存储，称为顶点表。因此邻接表中存在两种结点：顶点表结点和边表结点，这一点可以与树的孩子表示法对比记忆。

```
#define MAXV 1000                 // 顶点数目的最大值

typedef struct ArcNode {          // 边表结点
    int adjvex;                   // 该弧所指向的顶点在顶点表中的下标
    struct ArcNode *next;         // 指向下一条弧的指针
    // int info;                  // 网中可加边权值
} ArcNode;

typedef struct VNode {            // 顶点表结点
    char data;                    // 顶点信息
    ArcNode *firstarc;            // 指向第一条依附于该顶点的弧
} VNode, AdjList[MAXV];

typedef struct {
    int numVertices, numEdges;    // 图中实际的顶点数和边数
    AdjList vertices;             // 顶点表
} ALGraph;
```

空间复杂度：

- 无向图：每条边 (v,w) 在 v 的表和 w 的表中各存一次，边结点总数为 $2|E|$，空间复杂度 $O(|V|+$

$2|E|$）。

- 有向图：只记录出边，边结点总数为 $|E|$，空间复杂度 $O(|V|+|E|)$。

邻接表只存实际存在的边，因此适合存储稀疏图。代价是查找一条边需要遍历链表，时间效率不如邻接矩阵。

有向图的出度与入度：

- 出度很容易求：找到该顶点，数它所在链表中的结点个数即可，时间复杂度 $O(1)$（定位）+ $O(d)$（数链）。
- 入度很麻烦，因为邻接表只记出边。两种方法：
  1. 遍历整个顶点表的所有链表，统计 `adjvex` 等于目标顶点下标的结点个数，时间复杂度 $O(|V|+|E|)$。
  2. 建立逆邻接表——每个顶点的链表中记录的是以该顶点为弧头的弧。逆邻接表求出度难、求入度容易，正好与邻接表互补。

```
// 邻接表上求某顶点入度：需要遍历所有顶点的链表
int InDegree(ALGraph graph, int index) {
    int count = 0;
    for (int j = 0; j < graph.numVertices; ++j) {
        for (ArcNode *p = graph.vertices[j].firstarc; p; p = p->next) {
            if (p->adjvex == index) count++;
        }
    }
    return count;
}
```

邻接表表示不唯一

用邻接矩阵存储图，其表示方法是唯一的；用邻接表存储图，其表示方法不唯一。因为各条边在单链表中的次序可以任意（头插、尾插、按编号排序都合法）。这一条直接导致后面「邻接表存储时 DFS/BFS 遍历序列不唯一、生成树不唯一」的结论，是选择题的常考陷阱。

（三）十字链表（有向图）

用邻接表存有向图能快速求出度但求入度难，用逆邻接表则相反。十字链表（orthogonal list）是一种同时能快速求入度和出度的有向图存储结构。

核心思想：把所有顶点存到顺序表中，同时为每个顶点配备两个链表——一个链表记录以当前顶点为弧头的弧，另一个链表记录以当前顶点为弧尾的弧。

- 顶点表结点：`data` 存顶点信息；`firstin` 指向以该顶点为弧头的第一条弧；`firstout` 指向以该顶点为弧尾的第一条弧。
- 边表结点：`tailvex` 存弧尾顶点在顶点表中的下标；`headvex` 存弧头顶点的下标；`hlink` 指向弧头相同的下一条弧；`tlink` 指向弧尾相同的下一条弧；`info` 可存权值。

```
#define MAXV 1000
typedef struct ArcNode {          // 边表结点
    int tailvex, headvex;         // 该弧的弧尾、弧头顶点在顶点表中的下标
    struct ArcNode *hlink, *tlink;// 分别指向弧头相同、弧尾相同的下一条弧
    // int info;                  // 网中可加边权值
} ArcNode;
```

```
typedef struct VNode {            // 顶点表头结点
    char data;                    // 顶点信息
    ArcNode *firstin, *firstout;  // 分别指向第一条入弧、第一条出弧
} VNode, XJList[MAXV];

typedef struct {
    int numVertices, numEdges;
    XJList vertices;
} OLGraph;                        // 十字链表
```

手工理解十字链表的走法：以 $< A, B >$ 和 $< A, C >$、$< C, A >$ 和 $< D, A >$ 为例（A, B, C, D 编号为 $0, 1, 2, 3$）：

1. 对 $< A, B >$、$< A, C >$，A 是弧尾，B、C 是弧头。顶点表 A 的 `firstout` 指向以 A 为弧尾的第一条弧（即边结点 B），边结点 B 的 `tlink` 指向弧尾相同的下一条弧（边结点 C），C 的 `tlink` 为空。
2. 对 $< C, A >$、$< D, A >$，A 是弧头。顶点表 A 的 `firstin` 指向以 A 为弧头的第一条弧（边结点 C），C 的 `hlink` 指向弧头相同的下一条弧（边结点 D），D 的 `hlink` 为空。

因此：

- 沿 **tlink** 链遍历，可找到指定顶点的所有出边；统计该链的结点数即为出度。
- 沿 **hlink** 链遍历，可找到指定顶点的所有入边；统计该链的结点数即为入度。

十字链表的空间复杂度为 $O(|V| + |E|)$，因为每条弧只对应一个边表结点。

（四）邻接多重表（无向图）

无向图中 (V_i, V_j) 与 (V_j, V_i) 是同一条边，但在邻接表中每条边都要存两份。如果需要对无向图的边做大量插入、删除操作，邻接表就得处理两次，效率低下。邻接多重表（adjacency multilist）就是为解决这个问题而设计的无向图存储结构。

核心思想：所有顶点存到顺序表中，每个顶点配备一个链表，链表的每个结点代表一条与当前顶点直接关联的边——注意是「一条边一个结点」，而不是「一条边两个结点」。

- 顶点结点：`data` 存顶点数据；`firstedge` 指向依附于该顶点的第一条边。
- 边结点：`i`、`j` 分别存边两端顶点在顶点表中的下标；`ilink` 指向下一条依附于顶点 `i` 的边；`jlink` 指向下一条依附于顶点 `j` 的边；`info` 可存权值。

```
#define MAXV 1000
typedef struct ArcNode {          // 边结点
    int i, j;                     // 边的两个端点在顶点表中的下标
    struct ArcNode *ilink, *jlink;// 分别指向依附于 i、j 的下一条边
    // int info;                  // 网中可加边权值
} ArcNode;

typedef struct VNode {            // 顶点表头结点
    char data;
    ArcNode *firstedge;           // 指向与该顶点相连的第一条边
} VNode, AdjMuList[MAXV];

typedef struct {
```

```
    int numVertices, numEdges;
    AdjMuList vertices;
} AMLGraph;                          // 邻接多重表
```

查找与某顶点关联的所有边（以 A 为例）：

1. 由 A 的 `firstedge` 找到第一条与 A 关联的边结点 (A, B)。
2. 该边结点的 `i` 域存的是 A 的下标，因此沿 `ilink` 找到下一条边结点 (A, D)。
3. (A, D) 的 `i` 域仍是 A 的下标，但其 `ilink` 为空，查找结束。

邻接多重表与邻接表最大的不同：无向图中每条边在邻接表要存两份，在邻接多重表只存一份，因此删除边、删除顶点等操作只需修改指针即可，非常方便。空间复杂度 $O(|V|+|E|)$。

删除顶点时别以为「方便」就等于 $O(1)$

在十字链表和邻接多重表中，一个边结点可能同时被两个顶点的指针链引用。删除顶点时，我们能轻易定位到该顶点关联的边，但不知道该边结点另一端可能存在的另一个前驱指针在哪，仍需遍历，最坏时间复杂度为 $O(|V|+|E|)$。只有在「已知边结点的前驱位置」时才能做到 $O(1)$。

（五）四种存储结构的对比

比较项	邻接矩阵	邻接表	十字链表	邻接多重表
适用对象	稠密图（有向、无向均可）	稀疏图（有向、无向均可）	仅有向图	仅无向图
空间复杂度	$O(\|V\|^2)$	无向：$O(\|V\|+2\|E\|)$；有向：$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
每条边存储份数	无向图存 2 份（对称）	无向图存 2 份	1 份	1 份
求某顶点出度	$O(\|V\|)$	容易，$O(1)+O(d)$	容易，沿 `tlink` 链	无向图不区分出/入度
求某顶点入度	$O(\|V\|)$	难，需遍历整个邻接表	容易，沿 `hlink` 链	无向图不区分出/入度
找相邻边	遍历对应行或列，$O(\|V\|)$	遍历该顶点的边表	很方便	很方便
删除一条边	很方便，$O(1)$	需定位前驱，$O(1)\sim O(\|V\|)$	较方便	方便
删除一个顶点	需处理行与列，$O(\|V\|)$（若要求紧凑移动则 $O(\|V\|^2)$）	无向图不方便	不方便	方便
表示是否唯一	唯一	不唯一	不唯一	不唯一

（六）图的基本操作

考纲「图的存储及基本操作」要求掌握以下接口，并能说出它们在两种主要存储结构下的时间代价：

操作	含义	邻接矩阵	邻接表
`Adjacent(G,x,y)`	判断是否存在边 (x,y) 或弧 $<x,y>$	$O(1)$	$O(1)\sim O(\|V\|)$
`Neighbors(G,x)`	列出与 x 邻接的顶点	$O(\|V\|)$	无向图 $O(1)\sim O(\|V\|)$；有向图出边易、入边难 $O(\|V\|+\|E\|)$
`InsertVertex(G,x)`	插入顶点 x	$O(1)$	$O(1)$
`DeleteVertex(G,x)`	删除顶点 x	$O(\|V\|)$，需将该行该列清零	无向图 $O(\|V\|+\|E\|)$
`AddEdge(G,x,y)`	若无边则插入	$O(1)$	$O(1)$（头插法）
`RemoveEdge(G,x,y)`	若有边则删除	$O(1)$	$O(1)\sim O(\|V\|)$
`FirstNeighbor(G,x)`	求 x 的第一个邻接点	$O(\|V\|)$	$O(1)$
`NextNeighbor(G,x,y)`	求 x 在 y 之后的下一个邻接点	$O(\|V\|)$	$O(1)$

其中 `FirstNeighbor` 与 `NextNeighbor` 是遍历算法的基础原语。它们的语义是：`FirstNeighbor(G, x)` 返回 x 的第一个邻接点编号，不存在则返回 -1；`NextNeighbor(G,x,y)` 返回在 y 之后的下一个 x 的邻接点，不存在则返回 -1。有了这两个函数，图的遍历就可以写成与存储结构无关的代码。

```
// 邻接矩阵版：求顶点 v 的第一个邻接点
int FirstNeighbor(MGraph *G, int v) {
    for (int i = 0; i < G->numVertices; ++i)
        if (G->edges[v][i]) return i;
    return -1;
}

// 邻接矩阵版：求顶点 v 在 w 之后的下一个邻接点
int NextNeighbor(MGraph *G, int v, int w) {
    for (int i = w + 1; i < G->numVertices; ++i)
        if (G->edges[v][i]) return i;
    return -1;
}
```

出题角度

邻接矩阵与邻接表的存储结构是本章考查次数最多的知识点（历年统考中出现 8 次以上）。常见问法：(1) 给出邻接矩阵，求各顶点的度或入度出度（如按「行出列入」直接数）；(2) 给出邻接表，问求某顶点入度需要多少时间、或问「用邻接表存储的无向图，边结点个数是多少」（答 $2e$）；(3) 判断关于存储结构的叙述正误，典型正确项是「邻接矩阵表示唯一、邻接表表示不唯一」，典型错误项是「存储稀疏图用邻接矩阵比邻接表更省空间」。(4) 给出邻接多重表的示意图，直接问某两个顶点的度——此时要注意每条边只算一次，但每个顶点在边结点中出现的次数就是它的度。

三、图的遍历

图的遍历是指从某个顶点出发，沿图中的边依次访问每个顶点，确保每个顶点都只访问一次。为避免重复访问，需要设置一个 visited 数组标记顶点是否已被访问。从某个顶点出发一次遍历只能访问到与它连通的所有顶点，因此对非连通图必须从多个起点出发，调用次数就是连通分量数。

（一）深度优先搜索（DFS）

深度优先搜索（Depth First Search，DFS）是一种「优先走到底、无路可走再回头」的遍历方式：从某顶点出发，访问它的一个邻接顶点，再从这个邻接顶点继续深入，直到走到尽头时回溯，再换一条没走过的分支继续深入，直到所有顶点都被访问。

递归实现：

```
bool visited[MAXV];

void visit(char v) { printf("%c ", v); }

// 从顶点 v 出发对图 G 做深度优先遍历
void DFS(ALGraph G, int v) {
    visit(G.vertices[v].data);       // 访问顶点 v
    visited[v] = true;               // 标记已访问
    for (ArcNode *p = G.vertices[v].firstarc; p; p = p->next) {
        if (!visited[p->adjvex])     // 邻接点未访问过则继续深入
            DFS(G, p->adjvex);
    }
}

// 遍历整张图（处理非连通图）
void DFSTraverse(ALGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    for (int i = 0; i < G.numVertices; ++i)
        if (!visited[i]) DFS(G, i);   // 每调用一次，就找到一个连通分量
}
```

非递归实现（借助显式栈）：递归的本质是系统栈，改成显式栈即可。

```
// 深度优先遍历的非递归实现（显式栈）
void DFS_NonRecursive(ALGraph G, int v) {
    int stack[MAXV], top = -1;
    stack[++top] = v;                // 起点入栈
    while (top >= 0) {
        int cur = stack[top--];      // 弹出一个顶点
        if (visited[cur]) continue;  // 可能被重复压栈，需判重
        visit(G.vertices[cur].data);
        visited[cur] = true;
        // 邻接点全部压栈（压栈顺序决定遍历序列）
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            if (!visited[p->adjvex])
                stack[++top] = p->adjvex;
        }
    }
}
```

要特别注意：非递归版本中一个顶点可能被多次压栈，因此弹出时必须再次检查 **visited**，否则会

重复访问。

性能分析：遍历时间 = 访问各顶点的时间 + 探索各条边的时间。

存储结构	访问顶点	探索边	总时间复杂度	空间复杂度
邻接矩阵	$O(\|V\|)$	对每个顶点扫描一整行，共 $O(\|V\|^2)$	$O(\|V\|^2)$	$O(\|V\|)$（递归深度/栈）
邻接表	$O(\|V\|)$	每条边被访问 2 次（无向图），共 $O(\|E\|)$	$O(\|V\|+\|E\|)$	$O(\|V\|)$（`visited` 数组 + 递归栈）

深度优先生成树：DFS 过程中经过的边（由「发现新顶点」的那条边构成，即 DFS 树边）和全部顶点组成深度优先生成树。对非连通图做 DFS，得到的是深度优先生成森林。

（二）广度优先搜索（BFS）

广度优先搜索（Breadth First Search，BFS）是一种「由近及远、层层扩张」的遍历方式：从某顶点出发，先访问它的所有邻接顶点（第一层），再依次访问第一层各顶点的所有未访问邻接顶点（第二层），以此类推。BFS 借助队列实现——队列「先进先出」的性质正好对应「按发现的先后次序逐层处理」。

```
void BFS(ALGraph G, int v) {
    int queue[MAXV], front = 0, rear = 0;
    visit(G.vertices[v].data);
    visited[v] = true;
    queue[rear++] = v;                          // 起点入队
    while (front < rear) {
        int cur = queue[front++];               // 队头出队
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            int w = p->adjvex;
            if (!visited[w]) {                  // 未被访问过则访问并入队
                visit(G.vertices[w].data);
                visited[w] = true;              // 入队时立即标记，防止重复入队
                queue[rear++] = w;
            }
        }
    }
}

void BFSTraverse(ALGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    for (int i = 0; i < G.numVertices; ++i)
        if (!visited[i]) BFS(G, i);             // 每调用一次找到一个连通分量
}
```

BFS 中「标记」的位置

BFS 必须在顶点入队的同时就置 `visited = true`，而不能等到出队时再标记。否则同一个顶点可能被其多个邻接点先后发现并重复入队，导致队列中出现大量冗余元素，甚至死循环。DFS 中则是在访问时立即标记。

性能分析：

- 空间复杂度：两种存储结构都需要一个辅助队列，最坏情况下队列大小为 $O(|V|)$（例如所有顶点都与起点相邻）。
- 时间复杂度：邻接矩阵为 $O(|V|)+O(|V|^2)=O(|V|^2)$；邻接表为 $O(|V|)+O(|E|)=O(|V|+|E|)$。

广度优先生成树：BFS 过程中「发现新顶点」的边与全部顶点构成广度优先生成树。非连通图的 BFS 得到广度优先生成森林。

BFS 求无权图单源最短路径：无权图可以看作所有边权都为 1 的带权图。BFS 按层扩展，第一次访问到某顶点时经过的边数就是它到源点的最短距离。

```
// 从源点 u 出发求无权图单源最短路径
void BFSShortestPath(ALGraph G, int u, int d[], int path[]) {
    for (int i = 0; i < G.numVertices; ++i) {
        d[i] = -1; path[i] = -1; visited[i] = false;   // -1 表示不可达
    }
    int queue[MAXV], front = 0, rear = 0;
    d[u] = 0; visited[u] = true; path[u] = -1;
    queue[rear++] = u;
    while (front < rear) {
        int cur = queue[front++];
        for (ArcNode *p = G.vertices[cur].firstarc; p; p = p->next) {
            int w = p->adjvex;
            if (!visited[w]) {
                visited[w] = true;
                d[w] = d[cur] + 1;        // 层数加一即为最短距离
                path[w] = cur;            // 记录前驱，用于还原路径
                queue[rear++] = w;
            }
        }
    }
}
```

注意目的：`d[]` 存距离，`path[]` 存前驱。求 u 到某顶点的具体路径时，从该顶点沿 `path` 反向回溯到 u 再倒序输出即可。

（三）BFS 的「层次」与 Dijkstra 的异同

BFS 求无权图单源最短路径的做法，本质上是把图按层切开：源点是第 0 层，与源点直接相邻的顶点是第 1 层，再往外是第 2 层……顶点 v 的层号就是它到源点的最短路径长度（以边数计）。Dijkstra 也有「逐轮确定一个顶点」的外壳，而且两者的算法骨架极其相似（都是维护一个候选距离数组、每轮选出一个最小值、确定它、再用它去松弛邻居），这正是考生最容易混淆的地方。二者的差异可以归结为三点。

第一，距离的定义与累加方式不同。BFS 中 `d[v] = d[u] + 1`，每条边的「代价」固定为 1，所以层号可以直接加一；Dijkstra 中 `dist[v] = dist[u] + w(u,v)`，必须把边权累加进去。正因为标量不同，BFS 可以只用一个整数计数器在层与层之间递增，而 Dijkstra 必须对每个顶点单独维护一个 `dist` 值。

第二，能否「一次性确定」的依据不同。BFS 能够一旦发现某顶点就认准它是最短路径，靠的是「所有边权相等」这一前提：先被发现的层号必然不大于后被发现的层号。Dijkstra 之所以能在每轮挑出 `dist` 最小的未确定顶点并宣布它已确定，靠的是「所有边权非负」这一前提：任何绕经其他未确定顶点的路径都不可能更短。一旦出现负权边，Dijkstra 的这个依据就崩掉了（见后文反例），而 BFS 的那一层保证也随之失效，因为一条「更深」的路径可能因为负权而总长更短。

第三，是否保留「层次」这个中间概念。BFS 天然产生一个层次结构，因此除了求距离，它还能直接用来判断二分图（按层交替染色）、求无权图的直径、做多源泛洪等；Dijkstra 不产生层次，它只输出一组距离值和一棵最短路径树。反过来，Dijkstra 能处理任意非负权值，而 BFS 面对「边权不相等」的图只能给出边数意义下的距离，与题目要求的最短路径无关。

一句话概括：**BFS** 是「边权全为 **1** 时的 **Dijkstra**」，而 **Dijkstra** 是 **BFS** 在「边权非负但可能不等」时的推广。

（四）遍历手工模拟

设无向图 G 有 6 个顶点 $1 \sim 6$，边为 1–2、1–3、2–4、3–5、4–5、5–6，采用邻接表存储且每个顶点的边表都按顶点编号升序排列。分别从顶点 1 出发做 DFS 与 BFS：

步骤	**DFS**（栈/递归）	**BFS**（队列）
起始	访问 1	访问 1，队 = [1]
第 1 步	1 的邻接点中最小未访问者为 2，访问 2	1 出队；邻接点 2、3 均未访问，访问 2、3，队 = [2,3]
第 2 步	2 的邻接点 1 已访问，取 4，访问 4	2 出队；邻接点 1 已访问，4 未访问，访问 4，队 = [3,4]
第 3 步	4 的邻接点 2 已访问，取 5，访问 5	3 出队；邻接点 1 已访问，5 未访问，访问 5，队 = [4,5]
第 4 步	5 的邻接点 3 未访问，访问 3	4 出队；邻接点 2、5 均已访问
第 5 步	5 的邻接点 6 未访问，访问 6	5 出队；邻接点 3、4 已访问，6 未访问，访问 6，队 = [6]
第 6 步	6 的邻接点 5 已访问，回溯至 5、4、2、1，均无未访问邻接点，结束	6 出队；邻接点 5 已访问，结束
序列	**1, 2, 4, 5, 3, 6**	**1, 2, 3, 4, 5, 6**

由此得到的两棵生成树也不同：深度优先生成树的树边为 1–2、2–4、4–5、5–3、5–6（呈现一条长链）；广度优先生成树的树边为 1–2、1–3、2–4、3–5、5–6（呈现逐层展开的形态）。两棵树都恰好有 $5 = n - 1$ 条边，也印证了「生成树边数恒为 $n - 1$」。若把邻接表中某两个邻接点的次序对调，上面两个序列都会随之改变——这正是「邻接表表示不唯一 $\Rightarrow$ 遍历序列不唯一」的直观体现。

（五）遍历与图的连通性

对无向图做遍历时：

- 调用 **BFS/DFS** 函数的次数 = 连通分量数。
- 若图连通，只需调用 1 次。

对有向图做遍历时，调用次数需要具体分析，规律是：

- 若从起始顶点出发能到达其余所有顶点，则只需调用 1 次。
- 对强连通图，从任一顶点出发都只需调用 1 次。
- 一般情况下，有向图一次遍历能到达的是「该顶点能到达的所有顶点」集合，与强连通分量并不一一对应。

求无向图连通分量个数的算法（统考算法设计题的常见题型）：

1. 初始化 `visited` 数组全为 `false`，计数器 `count = 0`。
2. 依次检查每个顶点 i：若 `visited[i]` 为 `false`，则以 i 为起点做一次 DFS（或 BFS），并把 `count` 加 1。
3. 搜索过程中跳过已访问顶点，未访问的顶点必然与 i 属于同一连通分量。
4. 遍历结束后 `count` 即为连通分量个数。

```
int count = 0;
void CountComponents(MGraph G) {
    for (int i = 0; i < G.numVertices; ++i) visited[i] = false;
    count = 0;
    for (int i = 0; i < G.numVertices; ++i) {
        if (!visited[i]) {          // 每个未访问顶点开启一个新的连通分量
            DFS_Matrix(G, i);       // 泛洪标记整个连通分量
            count++;
        }
    }
}
```

时间复杂度：无论 DFS 还是 BFS 实现，最坏情况（完全图）下每个顶点都要询问其余 $|V|-1$ 个顶点是否已访问，因此为 $O(|V|^2)$；若用邻接表实现则为 $O(|V|+|E|)$。

同一问题也可以用并查集解决：遍历邻接矩阵的上三角区，对每条边的两端点做 `Union`；遍历结束后统计父数组中值为 -1（即根结点）的元素个数，即为连通分量个数。

（六）遍历序列的唯一性

遍历序列何时唯一（高频结论）

- 邻接矩阵表示唯一 $\Rightarrow$ 图的结构与邻接点次序都固定 $\Rightarrow$ **DFS/BFS** 遍历序列唯一，生成树也唯一（前提是起点固定）。
- 邻接表表示不唯一（边在链表中的次序可任意）$\Rightarrow$ **DFS/BFS** 遍历序列不唯一，生成树也不唯一。

唯一性的充要条件：给定图 G 和起点 v，从 v 出发的 DFS（或 BFS）序列唯一，当且仅当遍历过程中每一步「当前可选的未访问邻接点」都只有一个。换句话说，只要途中某个顶点存在两个及以上未访问的邻接点，选择次序不同就会得到不同的遍历序列。

（七）**DFS** 与 **BFS** 的对比

比较项	深度优先搜索 **DFS**	广度优先搜索 **BFS**
辅助结构	栈（递归的系统栈或显式栈）	队列
扩展方式	一条路走到黑，回溯	由近及远，逐层扩散
邻接矩阵时间复杂度	$O(\|V\|^2)$	$O(\|V\|^2)$
邻接表时间复杂度	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$
空间复杂度	$O(\|V\|)$	$O(\|V\|)$
生成树名称	深度优先生成树（森林）	广度优先生成树（森林）
典型用途	判断环、求连通分量、逆拓扑序列、割点	求无权图单源最短路径、层次遍历、判二分图

四、图的基本应用

（一）最小（代价）生成树

1.（*1*）最小生成树的概念与唯一性

最小生成树（Minimum-Spanning-Tree，MST）是连通网中各边权值之和最小的那棵生成树。

必背结论：

1. 一个连通网的最小生成树可能有多个，但各边权值之和（代价）总是唯一且最小的。
2. 当图中各边的权值互不相等时，最小生成树唯一。
3. 若无向连通图的边数比顶点数少 1（$e = n - 1$），则它本身就是一棵树，最小生成树就是它自己。
4. 只有连通图才有生成树，非连通图只有生成森林。

MST 唯一的充要条件：设 G 是无向连通带权图，T 是其一棵最小生成树，则 T 是 G 的唯一最小生成树，当且仅当对于不在 T 中的每一条边 e，都有 e 的权重大于 $T + e$ 所产生的圈中其它每条边的权重。

证明思路：往 T 中加入任一条不属于 T 的边 e，必然形成唯一一个圈（因为 T 已经包含全部顶点且连通无环）。若 e 的权小于或等于圈中某条边的权，就可以用 e 替换那条边得到一棵权值和不增大的新生成树，从而 T 不唯一；反之若 e 严格大于圈中所有其它边，任何替换都会使总权值增大，故 T 唯一。

2.（*2*）*Prim* 算法（选点）

算法思想：从某个顶点开始构建生成树，每次把与当前生成树顶点集合距离最近的那个新顶点及其对应边纳入生成树，直到所有顶点都被纳入。

执行步骤：

1. 初始化：设置 `visited[]` 数组标记顶点是否已进入 MST；设置 `dist[]` 数组，`dist[v]` 表示「顶点 v 到当前 MST 顶点集合的最短边长」。任选一个起点 v_0，令 `dist[v0] = 0`、`visited[v0] = true`，其余顶点的 `dist` 初始化为 v_0 到它们的直接边长（无边则为 ∞）。
2. 选点：在所有未加入 **MST**（`visited` 为 **false**）的顶点中挑出 `dist` 值最小的顶点 u。
3. 纳入：令 `visited[u] = true`，把边 $(u,$ 产生 dist[u] 的那个顶点$)$ 加入 MST，累加权值。
4. 更新：对 u 的每个未加入 MST 的邻接顶点 k，执行

$$dist[k] = \min\{dist[k],\ w(u,k)\}$$

注意这里用的是边的权值，不是把路径长度累加。

5. 重复步骤 2 ~ 4 共 $n - 1$ 次，直到所有顶点都加入 MST。

Prim 的 dist 数组含义（与 Dijkstra 的分水岭）

- **Prim**：`dist[k]` = 顶点 k 到 **MST** 顶点集合的最短边权。更新式为 $dist[k] = \min\{dist[k],\ w(u,k)\}$。
- **Dijkstra**：`dist[k]` = 顶点 k 到源点的最短路径长度。更新式为 $dist[k] = \min\{dist[k],\ dist[u] + w(u,k)\}$。

两者的代码结构几乎相同，但一个「不累加、只看单条边」，一个「累加、看整条路径」，这是考试中最容易被混淆的地方。

```
void Prim(MGraph G, int v0) {
```

```
    int dist[MAXV];
    bool visited[MAXV];
    int sum = 0;
    for (int i = 0; i < G.numVertices; ++i) {
        dist[i] = (i == v0) ? 0 :
                  (G.edges[v0][i] != 0 && G.edges[v0][i] < INT_MAX ? G.edges[v0][i] : INT_MAX);
        visited[i] = false;
    }
    visited[v0] = true;
    for (int i = 1; i < G.numVertices; ++i) {     // 共执行 n-1 轮
        int u = -1, min = INT_MAX;
        for (int j = 0; j < G.numVertices; ++j) { // 找 dist 最小的未访问顶点
            if (!visited[j] && dist[j] < min) { min = dist[j]; u = j; }
        }
        if (u == -1) break;                       // 图不连通
        visited[u] = true;
        sum += min;
        for (int k = 0; k < G.numVertices; ++k) { // 用 u 的边更新 dist
            if (!visited[k] && G.edges[u][k] != 0 && G.edges[u][k] < INT_MAX
                && G.edges[u][k] < dist[k])
                dist[k] = G.edges[u][k];
        }
    }
    printf("MST 总权值 = %d\n", sum);
}
```

手工模拟：设有 6 个顶点 A、B、C、D、E、F，无向带权边为 AB=6、AC=1、AD=5、BC=5、BE=3、CD=5、CE=6、CF=4、DF=2、EF=6。从 A 出发运行 Prim：

轮次	当前 **dist**（未确定顶点的候选值）	选中的顶点	加入的边	累计权值
初始	B:6, C:1, D:5, E:∞, F:∞	—	—	0
第 1 轮	选中最小者 C（1）	C	A–C	1
更新	B:min(6,5)=5, D:min(5,5)=5, E:min(∞,6)=6, F:min(∞,4)=4			
第 2 轮	最小者 F（4）	F	C–F	5
更新	B:min(5,∞)=5, D:min(5,2)=2, E:min(6,6)=6			
第 3 轮	最小者 D（2）	D	F–D	7
更新	B:min(5,5)=5, E:min(6,∞)=6			
第 4 轮	最小者 B（5）	B	C–B	12
更新	E:min(6,3)=3			
第 5 轮	最小者 E（3）	E	B–E	15

所得 MST 的边为 {A–C(1), C–F(4), F–D(2), C–B(5), B–E(3)}，总代价 15，共 5 条边，等于 $n-1$。

复杂度：外层循环执行 $n-1$ 次，内层两个并列的非嵌套循环各为 $O(|V|)$，因此总时间复杂度为 $O(|V|^2)$，与边数无关。空间复杂度 $O(|V|)$。**Prim** 适合求解边稠密的图（点少边多）。

3.（*3*）*Kruskal* 算法（选边）

算法思想：每次选择一条权值最小的边，若这条边的两个端点尚未连通则加入 MST，否则跳过（因为加入会形成回路），直到所有顶点连通。Kruskal 是并查集的典型应用。

执行步骤：

1. 把所有边按权值从小到大排序。
2. 初始化并查集，每个顶点各成一个集合。
3. 依次取出排序后的每条边 (u,v)：查询 u、v 的根结点，若不同则合并两集合并把该边加入 MST；若相同则跳过该边。
4. 当加入 MST 的边数达到 $|V|-1$ 时算法结束。
5. 若所有边都处理完仍未达到 $|V|-1$ 条，说明原图不连通，不存在生成树。

```
int parent[MAXV];
void initUF(int n) { for (int i = 0; i < n; ++i) parent[i] = i; }
int find(int x) { return parent[x] == x ? x : (parent[x] = find(parent[x])); }  // 路径压缩
bool unite(int x, int y) {
    int rx = find(x), ry = find(y);
    if (rx == ry) return false;      // 已连通，该边会成环
    parent[ry] = rx;
    return true;
}
int cmpEdge(const void *a, const void *b) {
    int wa = ((Edge *)a)->weight, wb = ((Edge *)b)->weight;
    return (wa > wb) - (wa < wb);    // 避免直接相减溢出
}

void Kruskal(MGraph *G, Edge edgeList[]) {
    int cnt = 0, sum = 0;
    initUF(G->numVertices);
    qsort(edgeList, G->numEdges, sizeof(Edge), cmpEdge);   // 按权值升序
    for (int i = 0; i < G->numEdges && cnt < G->numVertices - 1; ++i) {
        if (unite(edgeList[i].u, edgeList[i].v)) {
            sum += edgeList[i].weight;
            cnt++;
        }
    }
    if (cnt == G->numVertices - 1) printf("MST 总权值 = %d\n", sum);
    else printf(" 图不连通，无生成树\n");
}
```

手工模拟：仍用上一小节的 6 顶点图，按下表依次处理排序后的边：

序号	边	权值	两端是否已连通	处理结果	已选边数
1	A–C	1	否	加入	1
2	D–F	2	否	加入	2
3	B–E	3	否	加入	3
4	C–F	4	否（{A,C} 与 {D,F} 尚未合并）	加入	4
5	A–D	5	是（A–C–F–D 已连通）	跳过	4

序号	边	权值	两端是否已连通	处理结果	已选边数
6	B–C	5	否（{B,E} 与 {A,C,D,F} 尚未合并）	加入	5
—	—	—	已达 $n-1=5$ 条	结束	5

所得 MST 的边为 {A–C(1), D–F(2), B–E(3), C–F(4), B–C(5)}，总代价 15。可以看到 Prim 与 Kruskal 得到了不同的边集但相同的总代价——这正说明该图的 MST 不唯一（因为存在权值相同的边）。

复杂度：边排序需要 $O(|E|\log|E|)$；每条边做一次并查集查询，路径压缩后近似 $O(1)$，至多 $|E|$ 次，因此总时间复杂度为 $O(|E|\log|E|)$（可写为 $O(|E|\log|V|)$）。空间上需要 $O(|V|)$ 的并查集数组与边表。**Kruskal** 适合求解边稀疏而顶点较多的图。

Prim 与 Kruskal 的选择口诀

- **Prim** 选点（P 开头，Point 也是 P 开头），时间复杂度 $O(|V|^2)$，只与顶点数有关，适合边稠密的图。
- **Kruskal** 选边（K 开头，可以联想「快排也是先排序」），时间复杂度 $O(|E|\log|E|)$，只与边数有关，适合边稀疏而顶点较多的图。

4.（*4*）两个算法为什么正确

两个算法都是贪心，它们的正确性依赖同一条性质，可以称为切割性质（cut property）：把图的顶点任意划分成两个非空集合 S 与 $V-S$，横跨这两个集合的边中权值最小的那条边（若权值并列则任取其一）一定属于某棵最小生成树。直觉解释是：设 T 是一棵 MST，若这条最小横跨边 $e=(u,v)$ 不在 T 中，把它加入 T 会形成一个圈，而这个圈必然还有另一条横跨 S 与 $V-S$ 的边 e'（因为从 $u\in S$ 出发绕圈回到 $v\in V-S$ 至少要横跨一次）。由 e 是最小横跨边可知 $w(e)\le w(e')$，于是用 e 换掉 e' 得到的新生成树权值不增大，仍是 MST，故存在一棵 MST 包含 e。

- **Prim** 的做法就是反复应用切割性质：取 S = 当前已加入 MST 的顶点集合，$V-S$ = 尚未加入的顶点集合，此时「横跨 S 与 $V-S$ 的最小边」恰好就是 `dist` 数组中的最小值，所以每轮选 `dist` 最小的顶点是安全的。
- **Kruskal** 的做法则是另一种视角：按权值从小到大逐条考察，当考察到边 $e=(u,v)$ 时，若 u、v 已连通就跳过（加入必成环，且圈中已有边的权值都不大于 $w(e)$，换掉任何一条都不会更优）；若未连通就一定要加入（否则这一时刻 u 所在连通分量与 v 所在连通分量之间的所有横跨边权值都 $\ge w(e)$，而最终必须有一条横跨边被选中，选 e 不会比选别的差）。

理解了切割性质，就能解释两个常考的现象：其一，同一张图用 Prim 和 Kruskal 可能得到不同的 MST 边集，但它们的总代价必然相同（都等于所有被切割性质「锁定」的边权之和的最小值）；其二，只要某条边的权值是全图唯一最小的，它必然出现在每一棵 MST 中；反之若有若干条同样小的边构成环，就不能把它们全部选入，因此「所有权值最小的边一定出现在所有 MST 中」是错误的。

5.（*5*）最小生成树与最短路径的区别

比较项	**Prim**（最小生成树）	**Dijkstra**（单源最短路径）
目标	使生成树所有边的权值之和最小	使源点到每个顶点各自的路径长度最小
`dist` 含义	顶点到已选顶点集合的最短边权	顶点到源点的最短路径长度

比较项	**Prim**（最小生成树）	**Dijkstra**（单源最短路径）
更新方式	$dist[k] = \min(dist[k],\ w(u,k))$	$dist[k] = \min(dist[k],\ dist[u] + w(u,k))$
结果结构	一棵包含全部顶点的树	一棵以源点为根的最短路径树
权值要求	无限制（无向图）	不能有负权边
时间复杂度	$O(\|V\|^2)$	$O(\|V\|^2)$

最关键的区别：MST 保证「整棵树的总权值最小」，但不保证树上源点到某顶点的路径是原图中的最短路径；最短路径树保证「源点到每个顶点的路径最短」，但整棵树的边权之和通常比 MST 大。二者不能互相替代。特别地，即使图的所有边权都为 1，Prim 和 Kruskal 也不能用来求单源最短路径。

出题角度

最小生成树在历年统考中出现 5 次左右，选择题与应用题都会考。典型问法：(1) 判断关于 MST 的叙述——「最小生成树的代价唯一」正确，「所有权值最小的边一定出现在所有 MST 中」错误（等权边成环时不可能全选），「Prim 从不同顶点开始得到的 MST 一定相同」错误；(2) 给出带权图，问「可能是 Kruskal 第 2 次选中但不是 Prim（从某点开始）第 2 次选中的边」，解法是分别按两种规则手算前两步再取差集；(3) 应用题要求写出 Prim 或 Kruskal 逐次选边过程并计算总代价。

（二）最短路径

1.（*1*）无权图的单源最短路径——*BFS*

无权图（或所有边权都相等）的单源最短路径可以直接用 BFS 求。算法过程：

1. 设置队列 `q`、访问标记数组 `visited`、距离数组 `d`、前驱数组 `path`。
2. 源点 s 入队，置 `visited[s] = true`、`d[s] = 0`、`path[s] = -1`。
3. 循环：弹出队头 u，遍历 u 的所有邻接点 v；若 v 未访问，则置 `visited[v] = true`、`d[v] = d[u] + 1`、`path[v] = u`，并把 v 入队。
4. 队列为空时，`d[]` 即为源点到各顶点的最短距离；若 `d[i]` 仍为无穷，说明 i 从源点不可达。

时间复杂度：邻接矩阵 $O(|V|^2)$，邻接表 $O(|V| + |E|)$。

2.（*2*）*Dijkstra* 算法（带权单源最短路，非负权）

BFS 只适用于无权图或各边权值相等的图。当边权不同时，更深的路径完全可能比更浅的路径更短，此时必须使用 Dijkstra 或 Floyd 算法。

Dijkstra 算法需要三个辅助数组：

- `dist[]`：`dist[i]` 表示当前已知的从源点到顶点 i 的最短路径长度。
- `path[]`：`path[i]` 表示源点到 i 的最短路径上 i 的前驱顶点，用于还原路径。
- `final[]`：`final[i] = true` 表示顶点 i 的最短路径已经确定，不会再被更新。

执行步骤：

1. 初始化：`final[]` 全置 `false`；`dist[i] = Edge[s][i]`（源点到 i 的直接边长，无边则 ∞）；`path[i] = (Edge[s][i] 存在) ? s : -1`。令 `final[s] = true`、`dist[s] = 0`、`path[s] = -1`。
2. 选点：在 `final` 为 `false` 的顶点中，找出 `dist` 值最小的顶点 u。

3. 确定：令 `final[u] = true`。此时 `dist[u]` 就是源点到 u 的最短距离。
4. 松弛：检查所有由 u 出发的边 $< u, v >$。若 `final[v] == false` 且

$$dist[u] + w(u, v) < dist[v]$$

则更新 `dist[v] = dist[u] + w(u,v)`，并令 `path[v] = u`。

1. 重复步骤 2 ~ 4 共 $n - 1$ 轮，直到所有顶点的最短路径都确定。

```
void Dijkstra(MGraph G, int v, int dist[], int path[]) {
    bool final[MAXV];
    int n = G.numVertices;
    for (int i = 0; i < n; ++i) {                  // 初始化
        final[i] = false;
        int w = G.edges[v][i];
        bool hasEdge = (w != 0 && w < INT_MAX);
        dist[i] = hasEdge ? w : INT_MAX;
        path[i] = hasEdge ? v : -1;
    }
    final[v] = true; dist[v] = 0; path[v] = -1;
    for (int i = 1; i < n; ++i) {                  // 共 n-1 轮
        int u = -1, min = INT_MAX;
        for (int j = 0; j < n; ++j)                // 选点：找 dist 最小的未确定顶点
            if (!final[j] && dist[j] < min) { min = dist[j]; u = j; }
        if (u == -1) break;                        // 剩余顶点均不可达
        final[u] = true;
        for (int j = 0; j < n; ++j) {              // 松弛：只用「从 u 出发」的边
            if (!final[j] && G.edges[u][j] != 0 && G.edges[u][j] < INT_MAX
                && dist[u] != INT_MAX
                && dist[u] + G.edges[u][j] < dist[j]) {
                dist[j] = dist[u] + G.edges[u][j];
                path[j] = u;
            }
        }
    }
}
```

手工模拟：设 5 个顶点 $v_0 \sim v_4$，有向边 $v_0 \to v_1 = 10$、$v_0 \to v_4 = 5$、$v_4 \to v_1 = 3$、$v_4 \to v_2 = 9$、$v_4 \to v_3 = 2$、$v_1 \to v_2 = 1$、$v_2 \to v_3 = 4$、$v_3 \to v_0 = 7$、$v_3 \to v_2 = 6$。以 v_0 为源点：

轮次	已确定集合	`dist（v0,v1,v2,v3,v4）`	本轮选中	本轮 `path` 变化
初始	{v0}	0, 10, ∞, ∞, 5	—	`path[1]=0, path[4]=0`
第 1 轮	{v0,v4}	0, **8**, **14**, **7**, 5	v4（5）	`path[1]=4, path[2]=4, path[3]=4`
第 2 轮	{v0,v4,v3}	0, 8, **13**, 7, 5	v3（7）	`path[2]=3`（7+6=13<14）
第 3 轮	{v0,v4,v3,v1}	0, 8, **9**, 7, 5	v1（8）	`path[2]=1`（8+1=9<13）
第 4 轮	全部	0, 8, 9, 7, 5	v2（9）	无

最终结果：$v_0 \to v_1 = 8$（$v_0 \to v_4 \to v_1$），$v_0 \to v_2 = 9$（$v_0 \to v_4 \to v_1 \to v_2$），$v_0 \to v_3 = 7$（$v_0 \to v_4 \to v_3$），$v_0 \to v_4 = 5$。

注意第 1 轮后 dist[1] 从 10 被更新为 8、dist[2] 从 ∞ 变为 14、dist[3] 从 ∞ 变为 7——这说明「中转一个顶点」就能让原本不可达的顶点变得可达，让原本较长的路径变短，这也正是每轮都要重新松弛的原因。

复杂度：两层 for 循环，外层执行 $n-1$ 轮，每轮选点 $O(|V|)$，松弛 $O(|V|)$，总时间复杂度 $O(|V|^2)$。空间复杂度 $O(|V|)$（三个一维数组）。若用邻接表加优先队列（堆）优化，可降到 $O(|E|\log|V|)$，但统考按 $O(|V|^2)$ 理解。

Dijkstra 为什么不能处理负权边

Dijkstra 的正确性建立在一条贪心性质上：当前 dist 最小的未确定顶点，其最短路径已经确定，此后不会再被任何路径更新。当所有边权非负时，任何绕路（经过其他未确定顶点）只会让路径更长或持平，因此这条性质成立。

一旦出现负权边，这条性质就失效了。反例：有向图 $1\to2$ 权值为 3，$1\to3$ 权值为 4，$3\to2$ 权值为 -2。

- 初始化：dist[2]=3，dist[3]=4。
- 第 1 轮：dist 最小的未确定顶点是 2（值为 3），令 final[2]=true，认定 1 到 2 的最短距离是 3。
- 第 2 轮：选中顶点 3（值为 4），尝试用 $3\to2$ 松弛，得到 $dist[2]$ 应为 $4+(-2)=2<3$——但 final[2] 已经是 true，按算法规则不会再更新它。

于是 Dijkstra 输出 3，而真实最短路径是 $1\to3\to2$，长度为 2。结论：**Dijkstra** 只适用于所有边权非负的图；求解带负权（但无负权回路）的图必须用 Floyd 或 Bellman-Ford。

3.（3）Floyd 算法（各顶点对之间的最短路径）

算法思想：动态规划。依次允许经过 $v_0, v_1, v_2, \cdots, v_{n-1}$ 中转，每允许增加一个中转顶点，就检查是否能让某些点对之间的距离变短，若可以就更新。设 $dist[i][j]$ 表示当前已知的从 i 到 j 的最短路径长度，则状态转移为

$$dist[i][j]=\min\{dist[i][j],\ dist[i][k]+dist[k][j]\}$$

执行步骤：

1. 初始化：dist[][] 直接取邻接矩阵（注意 $i=j$ 时置 **0**，无边时置 ∞）；path[][] 初始化为 -1（也可初始化为 i，表示从 i 直接到 j）。
2. 三层循环：外层固定中转顶点 k（$0\sim n-1$），中层固定起点 i，内层固定终点 j。对每个 (i,j) 判断 $dist[i][k]+dist[k][j]<dist[i][j]$，若成立则更新距离并记录 path[i][j] = k。
3. 循环结束时（若无负权回路），dist[i][j] 即 i 到 j 的最短路径长度。

```
void Floyd(MGraph G, int dist[MAXV][MAXV], int path[MAXV][MAXV]) {
    int n = G.numVertices;
    for (int i = 0; i < n; ++i) {                       // 初始化
        for (int j = 0; j < n; ++j) {
            int w = G.edges[i][j];
            if (i == j) dist[i][j] = 0;
            else dist[i][j] = (w != 0 && w < INT_MAX) ? w : INT_MAX;
            path[i][j] = (dist[i][j] != INT_MAX && i != j) ? i : -1;
        }
    }
    for (int k = 0; k < n; ++k) {                       // 外层：中转顶点 k（必须在最外层）
        for (int i = 0; i < n; ++i) {                   // 中层：起点 i
            for (int j = 0; j < n; ++j) {               // 内层：终点 j
```

```
                if (dist[i][k] < INT_MAX && dist[k][j] < INT_MAX
                    && dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                    path[i][j] = k;                    // 记录中转点
                }
            }
        }
    }
}
```

手工模拟（十字法）：设 4 个顶点 $0,1,2,3$，有向边 $0\to1=1$、$1\to2=3$、$0\to2=5$、$2\to3=1$、$0\to3=10$、$1\to3=8$。∞ 表示不可达。

初始 $dist$（k 尚未允许）：

从\到	**0**	**1**	**2**	**3**
0	0	1	5	10
1	∞	0	3	8
2	∞	∞	0	1
3	∞	∞	∞	0

- $k=0$：没有任何顶点能到达 0，矩阵无变化。
- $k=1$（允许经 1 中转）：只需检查第 1 列非 ∞ 的行与第 1 行非 ∞ 的列。
  - $dist[0][1]+dist[1][2]=1+3=4<5\Rightarrow dist[0][2]=4$，`path[0][2]=1`。
  - $dist[0][1]+dist[1][3]=1+8=9<10\Rightarrow dist[0][3]=9$，`path[0][3]=1`。
- $k=2$（允许经 2 中转）：
  - $dist[0][2]+dist[2][3]=4+1=5<9\Rightarrow dist[0][3]=5$，`path[0][3]=2`。
  - $dist[1][2]+dist[2][3]=3+1=4<8\Rightarrow dist[1][3]=4$，`path[1][3]=2`。
- $k=3$：3 没有出边，无变化。

最终 $dist[0][3]=5$，还原路径：`path[0][3]=2` $\Rightarrow$ 到 3 之前先到 2；`path[0][2]=1` $\Rightarrow$ 到 2 之前先到 1；`path[0][1]=0` $\Rightarrow$ 到 1 之前先到 0。于是路径为 $0\to1\to2\to3$，长度 $1+3+1=5$，与 $dist$ 一致。

Floyd 的三重循环顺序为什么不能换

k 必须放在最外层。因为 k 代表的是「当前允许作为中转的顶点集合」$\{0,1,\cdots,k\}$，这是动态规划的阶段；i,j 是同一阶段内的状态。只有先固定阶段再枚举状态，转移方程 $dist_k[i][j]=\min(dist_{k-1}[i][j],\ dist_{k-1}[i][k]+dist_{k-1}[k][j])$ 才成立。

如果把 k 放到里层（例如写成 i,j,k 三层），就会在同一阶段内反复使用尚未确定的中间结果，导致某些路径组合被漏算。做题时若题目要求「依次允许 $v_0,v_1,\cdots$ 中转」写出每一轮矩阵，本质上就是在验证这个顺序。

复杂度：三重循环，时间复杂度 $O(|V|^3)$；空间复杂度 $O(|V|^2)$（距离矩阵 + 路径矩阵）。

负权支持：Floyd 可以处理带负权边但不能有负权回路的图。原因在于算法会对所有顶点的所有入边做松弛，不受「已确定」限制。若图中存在负权回路，则绕该回路可以无限缩短路径，最短路径不存在（长度趋于 $-\infty$）。

4.（4）三种最短路径算法对比

比较项	**BFS**	**Dijkstra**	**Floyd**
解决问题	无权图单源最短路径	非负权图单源最短路径	各顶点对之间的最短路径
边权要求	无权或各边权相等	所有边权非负	允许负权，不允许负权回路
算法思想	按层扩展	贪心，分已确定/未确定两个集合	动态规划，逐个允许中转顶点
辅助数组	`visited`、`d`、`path`	`dist`、`path`、`final`	`dist[][]`、`path[][]`
时间复杂度	邻接表 $O(\|V\|+\|E\|)$；邻接矩阵 $O(\|V\|^2)$	$O(\|V\|^2)$	$O(\|V\|^3)$
空间复杂度	$O(\|V\|)$	$O(\|V\|)$	$O(\|V\|^2)$
适用场景	无权网络、层次关系	非负权单源问题	需要所有点对距离

出题角度

最短路径历年考查 5 次以上，是选择题与应用题的双料重点。典型问法：(1) 给出图，问「依次得到的各最短路径的目标顶点是」，需按 Dijkstra 逐轮记录被确定顶点的顺序；(2) 给出图与初始 `dist` 数组，问「求出第 2 条最短路径后 `dist` 中的内容」，需精确模拟两轮松弛；(3) 问「所有边权均为 1 的连通图中，一定能求出单源最短路径的算法」，答案是只有 **BFS**，Prim 和 Kruskal 求的是最小生成树，不保证源点到各顶点的路径最短；(4) 判断「Dijkstra 能否处理负权图」，注意题干若强调「无负权回路」，则能处理的是 Floyd 而不是 Dijkstra。

（三）拓扑排序

1.（1）AOV 网与拓扑序列

AOV 网（Activity On Vertex Network）：用顶点表示活动、用有向边表示活动之间的先后制约关系的有向图。例如课程 A 是课程 B 的先修课，就画一条 $A \to B$ 的弧。

拓扑排序（Topological Sorting）：由一个有向无环图的顶点组成的一个线性序列，当且仅当满足以下条件时称为该图的一个拓扑排序——对图中任意一条弧 $<u,v>$，u 在序列中都排在 v 之前。

等价说法：拓扑排序是对有向图的一种遍历，在这种遍历中，对一个顶点 v 的访问只发生在它所有前驱顶点都被访问之后。

必背性质：

1. 只要图为 **DAG**（有向无环图），就至少存在一种拓扑排序结果。每个 AOV 网都有一个或多个拓扑序列。
2. 有向图存在拓扑序列 $\Leftrightarrow$ 图中不存在回路 $\Leftrightarrow$ 图是 **DAG**。
3. 拓扑排序与图是否连通无关。互不连通的分量互不依赖，它们在拓扑序中的相对顺序是任意的。
4. 若某顶点有多个直接后继，拓扑排序结果通常不唯一。唯一的情形是：每个时刻可选的入度为 0 的顶点只有一个，等价于图中存在一条包含全部顶点的路径（哈密顿路径），此时拓扑序就是这条路径上顶点的顺序。
5. 若一个有向图不能构成拓扑排序，则该图一定有环，即一定含有顶点数目大于 **1** 的强连通分量。注意此时该图不一定是强连通图——有环只是存在强连通分量的充分条件，不是整个图强连通。

怎么判断拓扑序列是否唯一、怎么数拓扑序列的个数？这是选择题的高频操作，方法是模拟人脑版

的 **Kahn** 算法并做分支统计：

1. 在草稿纸上写出当前所有入度为 0 的顶点，这就是当前这一步「可选」的集合。
2. 若某个时刻可选集合中只有 1 个顶点，则这一步没有选择余地，只能选它。
3. 若某个时刻可选集合中有 $k \geq 2$ 个顶点，则这里产生了 k 个分支，需要分别往下推演。
4. 把每条分支路上「所有选择都是被迫的」这一情况记为一个完整的拓扑序列，最后统计叶子数即为拓扑序列总数。

由此可以推出两个直接可用的判据：只要全过程中每一步的可选集合都恰好只有 **1** 个顶点，拓扑序列就唯一；而只要出现过一次可选集合大于 1，序列就不唯一。更本质地看，「每一步可选集合都只有 1 个顶点」等价于「图中存在一条依次经过所有顶点的有向路径」，也就是拓扑序恰好是这条路径上的顶点次序。这也解释了为什么「每个顶点都有唯一的前驱和后继」时拓扑序唯一，而「某个顶点有多个直接后继」时通常不唯一。

2.（*2*）基于入度的拓扑排序（*Kahn* 算法）

算法思想：一个顶点的入度为 **0**，说明它依赖的所有顶点都已被处理，此时就可以输出它。先把所有入度为 0 的顶点入队，然后不断出队、输出，并把出队顶点的所有邻接顶点的入度减 1，若某个邻接顶点入度减到 0 就入队。重复直到队列为空。这是一个顺序的拓扑排序过程。

执行步骤：

1. 统计每个顶点的入度，存入数组 `indegree[]`。
2. 把所有入度为 0 的顶点入队。
3. 循环直到队列为空：
   (a) 队头顶点 u 出队，加入结果序列 `res`，计数 `count++`。
   (b) 遍历 u 的所有邻接顶点 v，令 `indegree[v]--`。
   (c) 若 `indegree[v]` 减到 0，则把 v 入队。
4. 循环结束后判环：若 **`count == n`**，则排序成功，**`res`** 即为一个拓扑序列；否则图中存在环，无拓扑序列（有环的顶点入度永远减不到 0）。

```
bool TopologicalSort(ALGraph G, int res[]) {
    int indegree[MAXV] = {0}, cnt = 0, q[MAXV], front = 0, rear = 0;
    for (int i = 0; i < G.numVertices; ++i)          // 1. 统计入度
        for (ArcNode *p = G.vertices[i].firstarc; p; p = p->next)
            indegree[p->adjvex]++;
    for (int i = 0; i < G.numVertices; ++i)          // 2. 入度为 0 者入队
        if (indegree[i] == 0) q[rear++] = i;
    while (front < rear) {                            // 3. 出队并更新邻接点入度
        int u = q[front++];
        res[cnt++] = u;
        for (ArcNode *p = G.vertices[u].firstarc; p; p = p->next) {
            int v = p->adjvex;
            if (--indegree[v] == 0) q[rear++] = v;
        }
    }
    return cnt == G.numVertices;                      // 4. 判环
}
```

手工模拟：设有向图顶点 A、B、C、D、E、F，弧为 $A \to C$、$A \to D$、$B \to D$、$B \to E$、$C \to F$、$D \to F$。

步骤	各顶点入度（**A,B,C,D,E,F**）	队列内容	输出序列
初始	0, 0, 1, 2, 1, 2	A, B	—
出 A	0, 0, 0, 1, 1, 2	B, C	A
出 B	0, 0, 0, 0, 0, 2	C, D, E	A, B
出 C	0, 0, 0, 0, 0, 1	D, E	A, B, C
出 D	0, 0, 0, 0, 0, 0	E, F	A, B, C, D
出 E	0, 0, 0, 0, 0, 0	F	A, B, C, D, E
出 F	全部为 0	空	A, B, C, D, E, F

`count = 6 = n`，拓扑排序成功，得到一个合法序列 A,B,C,D,E,F。由于 A、B 入队次序可交换、C、D、E 的处理次序也有多种选择，该图的拓扑序列不唯一（例如 B,A,E,C,D,F 同样合法）。

复杂度：统计入度需扫描所有边，$O(|V|+|E|)$；每个顶点至多入队出队一次，$O(|V|)$；每条弧在出队时被扫描一次，$O(|E|)$。因此邻接表存储时总时间复杂度为 $O(|V|+|E|)$；若用邻接矩阵存储，每次找入度为 0 的顶点都要扫描一整行，复杂度升为 $O(|V|^2)$。空间复杂度 $O(|V|)$。

3.（3）基于 DFS 的拓扑排序

DFS 也能实现拓扑排序：在 DFS 中，顶点 u 的所有后继顶点一定会在 u 之前完成（回溯）。因此只要按「完成时间」把顶点压入结果栈（或逆序记录），最后得到的序列就是拓扑序列。若直接按完成次序顺序输出，得到的则是逆拓扑序列。

逆拓扑序列的判别

把 DFS 中「输出顶点信息」的语句移到退出递归之前（即先递归处理所有后继，再输出当前顶点），遍历一个有向无环图，输出结果中每个顶点都排在其所有后继之后，因此得到的是逆拓扑有序序列。反过来，要得到正序拓扑序列，需要把完成序列反转（或用栈）。这个「改一行语句」的考法在统考中出现过。

4.（4）拓扑排序与判环

有向图判环的常用方法：

1. 拓扑排序法（最常用）：执行 Kahn 算法后，若出队顶点数 `count < n`，说明存在环。因为环上的顶点入度永远无法减到 0。
2. **DFS** 三色标记法：DFS 中用一个 `state[]` 数组区分「未搜索 (0) / 搜索中 (1) / 已完成 (2)」。若在搜索过程中遇到了一个 `state` 为 1 的顶点，说明回到了当前递归路径上的顶点，即存在环。搜索完成一个顶点后置 `state = 2`，遇到 2 的顶点直接返回（不会重复检测）。
3. **DFS** 路径栈法：用栈记录当前遍历路径上的顶点，若遇到已在栈中的顶点则有环。

无向图判环的方法：

1. **DFS/BFS** 法：从某顶点 v 出发搜索，若在搜索过程中再次遇到 v，则图有环。对每个顶点都做一次即可找出所有环上的顶点。
2. 剥洋葱法：把度为 1（或 0）的顶点不断入队并删除，同时更新邻接点的度；若最终出队顶点数 `count < n`，说明剩下的是环，图有环。
3. 并查集法：依次处理每条边，若边的两端点已在同一集合中，则加入该边会成环。
4. 边数与连通性法：对 n 个顶点、e 条边的无向图，若 $e > n-1$ 则一定有环；若 $e = n-1$ 且连通则一定是树（无环）。

拓扑排序与邻接矩阵的三角形态

DAG（AOV 网）中各顶点的地位是平等的，顶点编号是人为的。因此可以按照拓扑排序的结果对顶点重新编号，使得新编号下所有的弧都从小编号指向大编号，从而新生成的邻接矩阵是（上）三角矩阵。

反之，若一个图的邻接矩阵是三角矩阵，则该图必然存在拓扑序列，但拓扑序列不一定唯一。注意方向：三角矩阵 ⇒ 有拓扑序列（存在性）；但「有拓扑序列 ⇒ 邻接矩阵是三角矩阵」需要允许重新编号后才成立。

典型考法：给出邻接矩阵「主对角线以下元素均为零」，问拓扑序列的结论。此时按矩阵下标编号，只有小编号指向大编号的弧，沿任何有向路径顶点编号都严格增大，故不可能有环，拓扑序列一定存在；但由于没有具体图的信息，无法判断是否唯一，结论是「存在，可能不唯一」。

DAG 描述表达式：有向无环图可以让相同的子表达式共用同一个顶点，从而避免重复建立结点，这是 DAG 在编译原理中用于表达式优化的经典应用，也是统考出现过的冷门考点。它之所以能用 DAG 而不是树来描述表达式，正是因为「同一个子表达式在表达式中出现多次」这一情形可以被合并在一个顶点上，而树结构无法共享子树。

以表达式 $(x+y)\cdot((x+y)/x)$ 为例，逐步建图：

1. 操作数 x 一个顶点，操作数 y 一个顶点。
2. 子表达式 $(x+y)$ 一个顶点，接收 x、y 两条入弧。它在原表达式中出现了两次，但只需建一个顶点。
3. 子表达式 $((x+y)/x)$ 一个顶点，入弧分别来自 $(x+y)$ 顶点和 x 顶点。
4. 最外层的乘法一个顶点，入弧来自 $(x+y)$ 顶点和 $((x+y)/x)$ 顶点。
5. 输出顶点（也可省去）指向该乘法顶点。

因此最少需要的顶点数为 x、y、加法、除法、乘法共 **5** 个。若改用二叉树表示而不共享子表达式，则需要重复建出第二个加法结点，顶点数会增加到 6 个以上——这正是题目「需要的顶点个数至少是」的考点所在。解题要领是：先把表达式按树展开，再把所有结构相同的子表达式合并成一个顶点，最后数顶点。

出题角度

拓扑排序历年考查 8 次左右，是本章最高频的考点之一。典型问法：(1) 问「可以得到不同拓扑序列的个数」，解法是逐步枚举当前入度为 0 的顶点，做树形分支统计；(2) 给出邻接表存储的有向图，问拓扑排序时间复杂度，答案是 $O(n+e)$；(3) 判断某个序列是否是该图的拓扑序列，逐个检查每条弧的方向约束即可；(4) 问「仅当图中存在一条包含所有顶点的路径时，拓扑序列唯一」；(5) 结合邻接矩阵的三角形态考查拓扑排序与压缩存储。

（四）关键路径

1.（1）AOE 网与关键路径的概念

AOE 网（Activity On Edge Network）：在一个表示工程的带权有向图中，用顶点表示事件、用有向边表示活动、用边上的权值表示活动的持续时间。其中没有入边的顶点称为源点（工程的开始），没有出边的顶点称为汇点（工程的结束）。

AOE 网的两条性质：

1. 只有在某顶点所代表的事件发生后，从该顶点出发的各条有向边所代表的活动才能开始。
2. 只有在进入某顶点的各条有向边所代表的活动都已结束时，该顶点所代表的事件才能发生。另外，有些活动是可以并行进行的。

AOV 网与 **AOE** 网对比：

比较项	**AOV 网**	**AOE 网**
顶点含义	表示活动	表示事件
边含义	无权值，仅表示活动之间的先后顺序	有权值，表示活动的持续时间
典型问题	拓扑排序、判环	关键路径、工程最短工期

路径长度：路径上各个活动的持续时间之和。从源点到汇点具有最大长度的路径称为关键路径，关键路径上的活动称为关键活动。关键路径的长度就是完成整个工程所需的最短时间。

必背性质：

1. 关键路径可能不唯一，但所有关键路径的长度都相同。
2. 增大某个关键活动的持续时间，一定会使关键路径变长，从而延长工期。
3. 缩短某个关键活动的持续时间，不一定能缩短工期——因为可能存在多条关键路径，只有同时缩短所有关键路径上的关键活动才能缩短总工期。
4. 关键活动不能任意缩短：一旦缩短到一定程度，该关键活动可能变成非关键活动，关键路径会转移到其他路径上。

2.（*2*）四个关键量

量	含义	计算公式
$ve(k)$	事件 v_k 的最早发生时间，也是从源点到 v_k 的最长路径长度	$ve(\text{源点})=0$；$ve(k)=\max\{ve(j)+w(j,k)\}$，$j$ 取遍 k 的所有前驱
$vl(k)$	事件 v_k 的最迟发生时间，即在不推迟整个工程的前提下该事件最迟必须发生的时间	$vl(\text{汇点})=ve(\text{汇点})$；$vl(k)=\min\{vl(j)-w(k,j)\}$，$j$ 取遍 k 的所有后继
$e(i)$	活动 a_i 的最早开始时间，即该活动弧的起点所表示事件的最早发生时间	若边 $<v_k,v_j>$ 表示 a_i，则 $e(i)=ve(k)$
$l(i)$	活动 a_i 的最迟开始时间，即该活动弧的终点的最迟发生时间减去该活动持续时间	若边 $<v_k,v_j>$ 表示 a_i，则 $l(i)=vl(j)-w(v_k,v_j)$

时间余量：$d(i)=l(i)-e(i)$，表示在不增加整个工程总时间的情况下，活动 a_i 可以拖延的时间。

$$d(i)=0 \iff l(i)=e(i) \iff a_i \text{ 是关键活动}$$

关键活动组成的路径就是关键路径。

计算 ve 与 vl 的方向不要弄反

- ve 从源点向汇点正推，用 **max**（取最长路径，因为事件必须等所有前驱活动都完成）。
- vl 从汇点向源点倒推，用 **min**（取最紧的约束，因为不能推迟后面任何一个后继）。

一个常见的错误是算 vl 时误用 max，或算 ve 时误用 min。记住口诀：正推求最早取最大，倒推求最迟取最小。

3.（*3*）标准解法五步

1. 正推求所有事件的 $ve()$：从源点开始，按拓扑序依次计算 $ve(k)=\max\{ve(j)+w(j,k)\}$。
2. 倒推求所有事件的 $vl()$：令 $vl(\text{汇点})=ve(\text{汇点})$，按逆拓扑序依次计算 $vl(k)=\min\{vl(j)-w(k,j)\}$。
3. 求所有活动的最早开始时间 $e()$：对表示活动的边 $<v_k,v_j>$，$e=ve(k)$。
4. 求所有活动的最迟开始时间 $l()$：对表示活动的边 $<v_k,v_j>$，$l=vl(j)-w(v_k,v_j)$。
5. 求所有活动的时间余量 $d=l-e$：$d=0$ 的活动就是关键活动，由关键活动连成的源点到汇点的路径就是关键路径。

4.（*4*）完整手工模拟

设有 AOE 网，顶点 $v_1\sim v_6$，v_1 为源点、v_6 为汇点，活动（边）如下：

活动	弧	持续时间	活动	弧	持续时间
a_1	$v_1\to v_2$	3	a_5	$v_3\to v_4$	4
a_2	$v_1\to v_3$	2	a_6	$v_3\to v_6$	3
a_3	$v_2\to v_4$	2	a_7	$v_4\to v_6$	2
a_4	$v_2\to v_5$	3	a_8	$v_5\to v_6$	1

第一步：正推求 ve（从 v_1 向 v_6，取 **max**）

$$ve(1)=0$$

$$ve(2)=ve(1)+3=3$$

$$ve(3)=ve(1)+2=2$$

$$ve(4)=\max\{ve(2)+2,\ ve(3)+4\}=\max\{5,\ 6\}=6$$

$$ve(5)=ve(2)+3=6$$

$$ve(6)=\max\{ve(3)+3,\ ve(4)+2,\ ve(5)+1\}=\max\{5,\ 8,\ 7\}=8$$

第二步：倒推求 vl（从 v_6 向 v_1，取 **min**），$vl(6)=ve(6)=8$

$$vl(6)=8$$

$$vl(5)=vl(6)-1=7$$

$$vl(4)=vl(6)-2=6$$

$$vl(3)=\min\{vl(6)-3,\ vl(4)-4\}=\min\{5,\ 2\}=2$$

$$vl(2)=\min\{vl(5)-3,\ vl(4)-2\}=\min\{4,\ 4\}=4$$

$$vl(1)=\min\{vl(2)-3,\ vl(3)-2\}=\min\{1,\ 0\}=0$$

第三、四、五步：求 e、l、d（$e=ve$(弧尾)，$l=vl$(弧头) − 权值）

活动	弧	权值	$e=ve$(尾)	$l=$ vl(头) − 权值	$d=l-e$	是否关键
a_1	$v_1\to v_2$	3	0	$4-3=1$	1	否
a_2	$v_1\to v_3$	2	0	$2-2=0$	**0**	是
a_3	$v_2\to v_4$	2	3	$6-2=4$	1	否
a_4	$v_2\to v_5$	3	3	$7-3=4$	1	否
a_5	$v_3\to v_4$	4	2	$6-4=2$	**0**	是
a_6	$v_3\to v_6$	3	2	$8-3=5$	3	否
a_7	$v_4\to v_6$	2	6	$8-2=6$	**0**	是
a_8	$v_5\to v_6$	1	6	$8-1=7$	1	否

关键活动为 a_2、a_5、a_7，关键路径为 $v_1\to v_3\to v_4\to v_6$，长度 $2+4+2=8$，与 $ve(6)=8$ 一致。结果的工程解读：整个工程最短需要 8 个时间单位完成。事件 v_3 必须准时在第 2 个时间单位发生（$ve(3)=vl(3)=2$），也就是活动 a_2 一天都不能拖；事件 v_4 必须在第 6 个时间单位发生，由 a_5 保障。反观活动 a_6（$v_3\to v_6$，时长 3），它的最早开始时间是 2、最迟开始时间是 5，余量为 3，意味着只要在第 **5** 个时间单位之前开工即可，提前或推后 3 个单位都不影响总工期。活动 a_8（$v_5\to v_6$，时长 1）余量为 1，a_1、a_3、a_4 的余量也都是 1。这些余量正是工程调度中「松弛时间」的概念。

如果要把工期从 **8** 压缩到 **7**，应当做什么？必须先找出所有关键路径（本例中只有一条：$v_1\to v_3\to v_4\to v_6$），然后缩短这条路径上某个活动的持续时间。但要注意两个约束：其一，如果图中有多条关键路径，必须同时缩短每一条关键路径上的活动，只缩短其中一条路径上的活动不会改变总工期（这也是统考反复考查的结论）；其二，缩短幅度受到余量的限制——若把 a_5 从 4 缩到 3，则关键路径长度变为 $2+3+2=7$，此时路径 $v_1\to v_3\to v_4\to v_6$ 仍然是唯一最长路径；但若继续把 a_5 缩到 2，关键路径长度变为 6，而 $v_1\to v_2\to v_5\to v_6$ 的长度为 $3+3+1=7$，关键路径就转移到了另一条路上，此时 a_5 反而不是关键活动了。这就是「关键活动不能任意缩短，缩到一定程度会变成非关键活动」的具体含义。

5.（*5*）快速手算关键路径：删边法

考试时间紧张时可以用下面这个技巧：

1. 只求 $ve(k)$，从源点开始正推。
2. 观察每个顶点：如果指向 k 的多条边产生的 ve 值不相等（即 $ve(j)+w(j,k)$ 有大有小），那么产生较小值的那条边一定是非关键边，可以把它从图中删掉；产生最大值的边（可能有两条及以上）保留。

3. 删边后可以继续向源点方向回溯删除，直到遇到一个出度大于 **1** 的交叉点为止。
4. ve 全部求完后，在剩余的边上找出能连通源点和汇点的路径，这些路径就是关键路径。

以本章的模拟为例：$ve(4)=\max\{ve(2)+2,\ ve(3)+4\}$，其中 $ve(2)+2=5<6$，因此边 $v_2\to v_4$（a_3）可删；继续向源点回溯，v_2 的入度只有 1，可继续删 $v_1\to v_2$（a_1），此时遇到 v_1（出度 $2>1$），停止。同理 $ve(6)=\max\{5,8,7\}$，$v_3\to v_6$ 与 $v_5\to v_6$ 可删。剩余边中连接 v_1 与 v_6 的路径只有 $v_1\to v_3\to v_4\to v_6$，即为唯一的关键路径。

出题角度

关键路径历年考查 4 次左右。典型问法：(1) 给出 AOE 网，问「活动 d 的最早开始时间和最迟开始时间分别是」，需要完整算出 ve、vl 再套 e、l 公式；(2) 问「时间余量最大的活动」，余量最大者偏离关键路径最远；(3) 问「加快哪些活动的进度可以缩短工程工期」，答案是必须覆盖所有关键路径的公共活动集合；(4) 判断叙述正误，注意「关键路径是从源点到汇点边数最多的路径」是错的（比的是持续时间之和而不是边数），「缩短任一关键活动一定能缩短工期」也是错的（多条关键路径时未必）。

本章小结表

一、核心公式与结论速查

结论	表达式 / 内容
无向图度数之和	$\sum TD(v_i)=2e$
有向图入度、出度之和	$\sum ID(v_i)=\sum OD(v_i)=e$
无向完全图边数	$n(n-1)/2$
有向完全图边数	$n(n-1)$
连通图最少边数	$n-1$（生成树）
非连通无向图最多边数	$(n-1)(n-2)/2$
强连通有向图最少边数	n（一个环）
生成树边数	$n-1$
最短路径所含边数上限（非负权）	任意连通图两点间最短路径至多 $n-1$ 条边（最短路径必是简单路径）
度数为奇数的顶点个数	必为偶数
邻接矩阵幂	$A^k[i][j]=i$ 到 j 长度为 k 的路径数目

二、六类应用算法总表

算法	算法思想	关键数组 / 结构	时间复杂度	空间复杂度	适用
Prim	选点，贪心	`dist`、`visited`	$O(\|V\|^2)$	$O(\|V\|)$	边稠密图
Kruskal	选边，贪心 + 并查集	边表排序、`parent`	$O(\|E\|\log\|E\|)$	$O(\|V\|)$	边稀疏、点多
Dijkstra	贪心，分已确定/未确定集合	`dist`、`path`、`final`	$O(\|V\|^2)$	$O(\|V\|)$	非负权单源最短路
Floyd	动态规划，逐个允许中转	`dist[][]`、`path[][]`	$O(\|V\|^3)$	$O(\|V\|^2)$	全源最短路，允许负权无负环

算法	算法思想	关键数组 / 结构	时间复杂度	空间复杂度	适用
拓扑排序	入度为 0 者入队，出队后邻接点入度减 1	`indegree`、队列	邻接表 $O(\|V\|+\|E\|)$；邻接矩阵 $O(\|V\|^2)$	$O(\|V\|)$	DAG、判环
关键路径	求最长路径，找 $d=0$ 的活动	`ve`、`vl`、`e`、`l`	$O(\|V\|+\|E\|)$	$O(\|V\|+\|E\|)$	AOE 网工期分析

三、四种存储结构速查

结构	空间	求入度	求出度	删边	删点	适用	唯一性
邻接矩阵	$O(\|V\|^2)$	$O(\|V\|)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	稠密图	唯一
邻接表	$O(\|V\|+\|E\|)$	难，$O(\|V\|+\|E\|)$	易	需定位	不方便（无向图）	稀疏图	不唯一
十字链表	$O(\|V\|+\|E\|)$	易（`hlink`）	易（`tlink`）	较方便	不方便	有向图	不唯一
邻接多重表	$O(\|V\|+\|E\|)$	无向图不区分	无向图不区分	方便	方便	无向图	不唯一

易错清单

概念类易错

1. 回路不是简单路径。简单路径要求顶点全部不重复，而回路首尾顶点相同，至少重复了两个位置。要区分「简单路径」与「简单回路」。
2. 图的「距离」是边数，不是权值之和。带权图中两点间的最短路径长度才是权值之和，术语上不要混用。
3. 连通分量是「极大连通子图」，不是「最大连通子图」，也不是「最小连通子图」。极小连通子图指的是生成树。
4. 强连通图最少 n 条边（构成一个环），但 n 条边不一定是强连通图（可能只是几个不相交的环加上别的结构）。
5. 有向图存在拓扑序列 $\Leftrightarrow$ 无环，与是否连通无关。有环时一定含顶点数大于 1 的强连通分量，但整个图未必强连通。
6. 非连通图只有生成森林，没有生成树。生成树只对连通图定义。
7. 稀疏图与稠密图没有绝对界限，一般以 $|E|<|V|\log|V|$ 为界。不要把它当成严格定义来判定。
8. 有向树要求恰好一个顶点入度为 **0**、其余顶点入度均为 **1**，仅有这一条，出度不限制。

存储结构类易错

1. 「存储稀疏图用邻接矩阵比邻接表更省空间」是错的。邻接矩阵恒为 $O(|V|^2)$，与边数无关；邻接表为 $O(|V|+|E|)$。稀疏图中后者更省。
2. 邻接表表示不唯一，邻接矩阵表示唯一。这条决定了遍历序列与生成树是否唯一。
3. 无向图的邻接表边结点数是 $2e$，有向图是 e。无向图每条边存两份；若问「邻接多重表的边结点数」，则是 e（每边一份）。
4. 邻接表求有向图的入度需要遍历整个邻接表，$O(|V|+|E|)$，不是 $O(1)$；求入度快的是逆邻接表和十字链表。

5. 删除顶点在邻接矩阵中是 $O(|V|)$（将该行该列清零）；只有当题目要求把矩阵元素移动成紧凑形式时才需要 $O(|V|^2)$。看清题目问的是哪种。
6. 十字链表用于有向图，邻接多重表用于无向图，不要写反。
7. 邻接矩阵的带权形式中，主对角线元素是 **0** 还是 ∞ 要看题目约定，解题时按给定矩阵读，不要自己假设。

遍历类易错

1. **BFS** 必须在顶点入队时就标记 `visited`，不能等到出队时再标记，否则会重复入队。
2. **DFS** 非递归实现弹出时要再次判 `visited`，因为一个顶点可能被多次压栈。
3. 对无向图，调用 **BFS/DFS** 的次数等于连通分量数；对有向图不能这样直接下结论。
4. 邻接矩阵存储时遍历序列唯一（起点固定时），邻接表存储时不唯一。生成树同理。
5. **BFS** 求单源最短路径只适用于无权图或各边权相等的图，带不同权值时必须用 Dijkstra。
6. 无论 DFS 还是 BFS，空间复杂度都是 $O(|V|)$——BFS 是队列大小，DFS 是递归栈深度。

应用算法类易错

1. **Prim** 选点、**Kruskal** 选边。Prim 的时间复杂度只与顶点数有关，适合稠密图；Kruskal 只与边数有关，适合稀疏图。写反是最常见的失分点。
2. **MST** 唯一性：只有「各边权值互不相等」才是 MST 唯一的充分条件；等权边存在时可能有多个 MST，但代价总是唯一的。
3. 「所有权值最小的边一定出现在所有 **MST** 中」是错的。若若干条最小权边构成环，不可能全部选入。
4. **Prim** 与 **Dijkstra** 的 `dist` 含义完全不同：Prim 是「到已选集合的最短边权」（不累加），Dijkstra 是「到源点的最短路径长」（累加）。代码结构相似但更新式不同。
5. **Dijkstra** 不能处理负权边。原因是贪心性质失效：一个顶点的最短距离一旦被确定就不再更新，而负权边可能让后来确定的顶点提供更短的路径。
6. **Dijkstra** 的三层判断中，松弛只用「从 u 出发」的边，不是「与 u 相连」的边——有向图与无向图的区别就在这里。
7. **Floyd** 的三重循环 k 必须在最外层。k 是 DP 的阶段（允许中转的顶点集合），i, j 是同阶段内的状态。顺序写错结果会出错。
8. **Floyd** 允许负权边但不允许负权回路；存在负权回路时最短路径不存在。
9. 拓扑排序的出队顺序不唯一，但「每个时刻入度为 0 的顶点只有一个」时唯一，此时图中存在一条包含所有顶点的路径。
10. 关键路径比的是路径上活动持续时间之和，不是边的条数。ve 正推取 max，vl 倒推取 min，切勿弄反。
11. 缩短关键活动不一定缩短工期：存在多条关键路径时，只有同时缩短所有关键路径上的活动才行。而延长任一关键活动一定延长工期。
12. 关键活动不能任意缩短，缩短到一定程度它会变成非关键活动，关键路径转移。
13. **AOV** 网顶点表示活动，**AOE** 网顶点表示事件；AOE 网的边才带权值（活动持续时间）。

第六章查找

本章考纲要求

（一）查找的基本概念

（二）顺序查找法

（三）分块查找法

（四）折半查找法

（五）树形查找

1. 二叉搜索树
2. 平衡二叉树
3. 红黑树

（六）B 树及其基本操作、B+ 树的基本概念

（七）散列（hash）表

（八）字符串模式匹配

（九）查找算法的分析及应用

查找这一章在统考中的份量很重，选择题几乎年年必出，而且往往是「两道单选 + 一道大题」的组合。它的特点是知识点碎、公式多、每一种结构都要会手算。真正拉开分差的不是定义，而是三类基本功：**ASL** 的推导与计算、插入/删除后的调整手算、判定类问题的边界。本章按考纲条目顺序展开，所有 ASL 公式都给出推导过程。

一、查找的基本概念

（一）查找表、关键字、查找长度

查找（**search**）：在数据集合中寻找满足某种条件的数据元素的过程。

查找表（**search table**）：用于查找的数据集合，由同一类型的数据元素（或记录）组成。查找表可以是一个数组、一个链表、一棵树、一张散列表，因此「查找表」讲的是逻辑上的数据集合，不是某种特定存储结构。

关键字（**key**）：数据元素中唯一标识该元素的某个数据项的值。基于关键字的查找，结果应当是唯一的。

按对查找表的操作方式，可以把查找表分成两类：

类型	支持的操作	典型结构
静态查找表	只查找，不改变表的内容	顺序表上的顺序查找、折半查找
动态查找表	查找之外还要插入、删除	二叉搜索树、AVL 树、红黑树、B 树、散列表

这个区分很重要：折半查找不能用在动态查找表上，因为有序顺序表插入删除要移动大量元素。

查找长度（**search length**）：一次查找运算中比较关键字的次数。注意它数的是「比较关键字的次数」，不是「访问结点的次数」，也不是「循环次数」。

平均查找长度（**ASL, Average Search Length**）：所有查找过程中关键字比较次数的平均值。

$$ASL = \sum_{i=1}^{n} P_i C_i$$

其中 P_i 是查找第 i 个元素的概率，C_i 是找到第 i 个元素所需的比较次数。等概率时 $P_i = \frac{1}{n}$。

ASL 必须分「成功」和「失败」两种情况讨论，两者用的分母完全不同：

- $ASL_{\text{成功}} = \dfrac{\text{查找成功时比较关键字的总次数}}{\text{表中元素的个数}}$
- $ASL_{\text{失败}} = \dfrac{\text{查找失败时比较关键字的总次数}}{\text{查找失败的情况数（即失败位置的个数）}}$

失败 ASL 的分母最容易搞错

$ASL_{\text{失败}}$ 的分母不是元素个数 n，而是「查找失败可能落在几个位置上」。

- 顺序表的顺序查找：失败位置有 $n+1$ 个（在每个元素前后以及表尾之后）；
- 折半查找判定树：失败结点（外部结点）有 $n+1$ 个；
- 散列表的开放定址法：分母是初次散列可能落在的地址个数，即散列函数值域的大小，通常是 p，不一定是表长 m。

分母选错，整道题就全错。

（二）用判定树统一理解 ASL

所有查找算法都可以画成一棵判定树（**decision tree**），ASL 就是判定树上各结点的平均层数。这让不同算法的分析变得统一：

- 圆结点（内部结点）：查找表里真实存在的元素，对应「查找成功」；成功时的查找长度 = 该结点所在层数。
- 矩形结点（外部结点 / 失败结点）：表示一个查找失败的区间，对应「查找失败」；失败时的查找长度 = 其父结点（内部结点）所在层数，因为到达失败结点之前，比较动作都发生在内部结点上，最后那次比较完才知道「不存在」。

判定树的三条通用规则

1. n 个元素的静态查找表，判定树有 n 个内部结点 + $n+1$ 个外部结点（n 个关键字把整个值域切成 $n+1$ 段）。
2. 成功结点的查找长度 = 自身层数（根为第 1 层）。
3. 失败结点的查找长度 = 其父结点所在层数，外部结点自身不产生比较。

（三）一个最小的算例

设查找表为有序表 $\{7, 12, 20\}$，判定树为：根 12，左孩子 7，右孩子 20。

- $ASL_{\text{成功}} = \dfrac{1+2+2}{3} = \dfrac{5}{3}$。
- 四个失败区间分别在 7 左、7–12 之间、12–20 之间、20 右，父结点层数依次为 2、1……严格来说，左侧失败结点挂在 7 上（层数 2），7–12 之间的失败结点挂在 12 上（层数 1），12–20 之间

挂在 12 上（层数 1），右侧挂在 20 上（层数 2）。故 $ASL_{失败}=\dfrac{2+1+1+2}{4}=\dfrac{3}{2}$。

二、顺序查找法

顺序查找（**sequential search**）：从表的一端开始，逐个把元素的关键字与给定值比较。

（一）一般线性表的顺序查找

```
typedef struct {
    int *elem;      // 动态数组基址
    int tablelen;   // 表长
} SSTable;

// 朴素写法：每次循环要做两次判断
int seq_search(SSTable ST, int target) {
    for (int i = 0; i < ST.tablelen; ++i) {
        if (ST.elem[i] == target) return i;   // 判断是否命中
        // for 的条件里还要判断 i 是否越界
    }
    return -1;   // 查找失败
}
```

朴素写法每一轮要做两次判断：一次判断「当前元素是不是目标」，一次判断「表是否已经走完」。把这两个判断合并成一个，就是哨兵的作用。

（二）哨兵（监视哨）的作用

哨兵：在查找表的一端放置一个与待查关键字相等的元素。这样循环条件里只剩「当前元素是否等于目标」一个判断，越界检查被哨兵自然「兜住」。

```
// 哨兵版：elem 至少分配 tablelen+1 个单元
int seq_search(SSTable ST, int target) {
    ST.elem[ST.tablelen] = target;   // 表尾放置哨兵
    int i = 0;
    while (ST.elem[i] != target) ++i; // 少一次越界判断
    return i;   // 返回 tablelen 说明是哨兵命中，即查找失败
}
```

哨兵的位置不止一种约定

- 放在表尾（下标 n），从下标 0 向表尾扫：找不到时必然停在 n，靠「返回值是否等于 n」判失败。
- 放在表头（下标 **1**），数组 `elem[1..n]` 存数据，从下标 n 向表头扫：找不到时必然停在 0，靠「返回值是否为 0」判失败。这是王道教材的写法。

两种约定都对，关键是自己算 **ASL** 时不要和约定混用。无论哪种，哨兵都只减少判断次数，不改变 **ASL** 的值（哨兵本身也要参与一次比较，而这次比较正好等价于原来那次越界判断）。

（三）无序表的 ASL 推导

设表中有 n 个元素，等概率查找，逐个从前向后比较。

查找成功：找第 1 个元素比 1 次，第 2 个比 2 次，……，第 n 个比 n 次。

$$ASL_{\text{成功}} = \frac{1}{n}\sum_{i=1}^{n} i = \frac{1}{n}\cdot\frac{n(n+1)}{2} = \frac{n+1}{2}$$

查找失败：无序表只有真正走完整张表（比较了全部 n 个元素）才能确定失败。所以每一种失败情况都是 n 次比较：

$$ASL_{\text{失败}} = \frac{1}{n}\cdot\underbrace{(n+n+\cdots+n)}_{n\text{个}} = n$$

无序表顺序查找的必背结论

$$ASL_{\text{成功}} = \frac{n+1}{2}, \qquad ASL_{\text{失败}} = n$$

时间复杂度 $O(n)$；顺序查找既适用于顺序存储，也适用于链式存储（这是它相对折半查找最大的优势）。

（四）有序表顺序查找的 **ASL** 推导

若表已按关键字递增有序，顺序查找可以提前终止：一旦发现 $ST.elem[i] > target$，就能确定表中不存在 $target$，不必再往后找。

```
int seq_search(SSTable ST, int target) {
    for (int i = 0; i < ST.tablelen; ++i) {
        if (ST.elem[i] == target) return i;   // 命中
        if (ST.elem[i] > target)  break;      // 已经不可能再找到
    }
    return -1;
}
```

查找成功的 ASL 不变，仍是 $\frac{n+1}{2}$：成功命中仍发生在第 i 个位置，比较 i 次。

查找失败变得复杂。失败情况共有 $n+1$ 种，画成判定树，失败结点分布在 $n+1$ 个区间里。用 T_i 表示落在第 i 个内部结点左侧或中间的失败情况，其需要比较的次数就是它撞上的那个内部结点的层数：

失败区间	撞上的内部结点	比较次数
第 1 个元素之前	$elem[1]$	1
$elem[1]$ 与 $elem[2]$ 之间	$elem[2]$	2
⋮	⋮	⋮
$elem[n-1]$ 与 $elem[n]$ 之间	$elem[n]$	n
第 n 个元素之后	$elem[n]$	n

前 n 个失败区间的比较次数是 $1, 2, \cdots, n$，最后一个失败区间比较 n 次（要走完整张表才知道没有比它大的）。等概率时：

$$ASL_{\text{失败}} = \frac{(1+2+\cdots+n)+n}{n+1} = \frac{\frac{n(n+1)}{2}+n}{n+1} = \frac{n}{2}+\frac{n}{n+1}$$

有序表顺序查找的两个坑

1. 成功 **ASL** 不变，只有失败 **ASL** 变小：n 对比 $\frac{n}{2}+\frac{n}{n+1}$。$n$ 趋于无穷时两者都趋于 $\frac{n}{2}$，但有限 n 下后者略小——不要记成「有序就都减半」。
2. 题目若问「查找失败最多需要比较多少次」，有序表答案是 n，不是 $n+1$。因为最早撞上「大于目标」的元素时就已经能判定失败，最坏情况是目标比所有元素都大，此时走完 n 个元素、比较 n 次。

（五）概率不等时：把常查的元素放前面

若各元素被查概率 $p_1, p_2, \cdots, p_n$ 不等，$ASL = \sum p_i c_i$ 就是按概率加权。要让它最小，应把概率大的元素放在靠前的位置（查找代价小的位置）。这就是「按频度排序」的优化。

统考考法

顺序查找在统考中多以应用题形式出现，要求写出查找过程、计算 $ASL_{成功}$ 与 $ASL_{失败}$，或者比较不同查找策略的比较次数。选择题则常考「哪种存储结构适用顺序查找」（答案是顺序存储和链式存储都适用）。

三、分块查找法

分块查找（**block search**），又叫索引顺序查找。它把顺序查找和折半查找的优点结合起来：查找效率高于顺序查找，而插入删除又不像有序顺序表那样需要移动大量元素。

1. 索引顺序表的结构

分块查找把长度为 n 的表分成 b 块，每块 s 个元素，并要求：

- 块内无序：块内的记录不要求有序；
- 块间有序：第 $i+1$ 块中所有记录的关键字都大于第 i 块中所有记录的关键字。

再为每一块建立一个索引项，保存这一块的最大关键字和存储区间：

```
typedef struct {
    int maxvalue;       // 本块的最大关键字
    int low, high;      // 本块在表中占据的下标区间
} IndexItem;
IndexItem indexTable[100];
```

因为索引表本身是块间有序的，所以索引表上可以用顺序查找，也可以用折半查找。

（一）查找过程：两步

1. 在索引表中确定待查关键字可能所在的块。索引表有序 ⇒ 可顺序查、也可折半查。
2. 在所确定块的块内顺序查找。块内通常无序 ⇒ 只能顺序查。

索引表命中不等于查找成功

如果块内是无序的，即便在索引表里已经查到了「某个块的最大关键字恰好等于 $target$」，也必须进入该块做块内查找才算真的找到——因为块内无序，那个最大关键字可能并不真实存在于该块中（比如索引表写的是块的上界，而块内元素都小于它）。

算 ASL 时忘记这一点，会漏掉块内的那几次比较。这是分块查找最容易出错的地方。

3. 用折半查找查索引表时的落点

如果索引表用折半查找，要注意终止时的指针位置：

- 若某次比较命中了某个索引项，直接去该索引项所指的块内找即可；
- 若索引表中不存在目标关键字，折半查找终止于 $low > high$，此时 low 指向第一个关键字大于目标的索引项。由于索引项保存的是各块的最大关键字，目标只可能落在 low 所指的那一块里，因此应该去 low 所指的块内继续找；
- 若终止时 low 已经超出索引表范围，说明目标比所有块的最大关键字都大，查找失败。

（二）ASL 推导与最优块长

设表长 n，分成 b 块，每块 s 个元素，于是

$$n = b \cdot s$$

设索引查找的平均长度为 L_I，块内查找的平均长度为 L_S，则

$$ASL = L_I + L_S$$

（1）索引表用顺序查找

索引表有 b 项，顺序查找的平均长度为 $\dfrac{b+1}{2}$；块内有 s 个元素，顺序查找平均长度为 $\dfrac{s+1}{2}$。所以

$$ASL_{\text{成功}} = L_I + L_S = \frac{b+1}{2} + \frac{s+1}{2}$$

把 $b = \dfrac{n}{s}$ 代入：

$$ASL_{\text{成功}} = \frac{1}{2}\left(\frac{n}{s} + 1\right) + \frac{s+1}{2} = \frac{n}{2s} + \frac{s}{2} + 1 = \frac{s^2 + 2s + n}{2s}$$

（2）求最优块长 s

把 s 看成连续变量，对 s 求导：

$$\frac{\mathrm{d}}{\mathrm{d}s}\left(\frac{n}{2s} + \frac{s}{2} + 1\right) = -\frac{n}{2s^2} + \frac{1}{2}$$

令导数为 0，得 $s^2 = n$，即

$$\boxed{s = \sqrt{n}}$$

此时块数 $b = \dfrac{n}{s} = \sqrt{n}$，最小平均查找长度为

$$ASL_{\min} = \frac{n}{2\sqrt{n}} + \frac{\sqrt{n}}{2} + 1 = \frac{\sqrt{n}}{2} + \frac{\sqrt{n}}{2} + 1 = \sqrt{n} + 1$$

分块查找必背

$$ASL_{\text{成功}} = \frac{b+1}{2} + \frac{s+1}{2} = \frac{s^2 + 2s + n}{2s}$$

最优块长 $s = \sqrt{n}$，此时 $ASL_{\min} = \sqrt{n} + 1$。

自验证：$n=10000$，$s=100$，$b=100$，$ASL=\frac{101}{2}+\frac{101}{2}=101=\sqrt{10000}+1$，与公式一致。

（3）索引表用折半查找

索引表有 b 项，折半查找的 ASL 约为 $\lceil\log_2(b+1)\rceil$，块内仍为 $\frac{s+1}{2}$，故

$$ASL_{\text{成功}}=\lceil\log_2(b+1)\rceil+\frac{s+1}{2}$$

此时 ASL 关于 s 单调递减，趋于 $\lceil\log_2 b\rceil+\frac{s}{2}$，因此块长应取得更小——块数多、每块小，让索引表承担更多查找工作。极限情形 $s=1$ 就退化成「索引 + 逐个」的二层结构，接近折半查找的水平。

（4）查找失败的 ASL

分块查找的失败 ASL 计算繁琐（要枚举落进每个块后块内一次不中的情况），统考一般不要求整表求 $ASL_{\text{失败}}$，但可能问某个具体关键字的查找失败比较次数，此时按「索引表比较次数 + 块内比较次数」细致数一遍即可。

（三）优缺点与动态版本

- 优点：查找效率高于顺序查找；插入删除只需在块内移动，不必整体搬移。
- 缺点：需额外存储索引表，且要维护「块间有序」这一前提。
- 动态版本：块内改用链表存储，插入删除进一步简化，适合「动态查找表」。

统考考法

分块查找是考纲内长期未被单独考过的条目，属于「冷门但可能突袭」的考点。一旦出现，几乎必然考 $ASL_{\min}=\sqrt{n}+1$ 这个结论（注意有 $+1$）以及最优块长的推导。选择题也可能问「分块查找为什么块内可以无序」。

四、折半查找法

折半查找（**binary search**），又称二分查找。它每一步都把待查区间缩小一半。

（一）适用条件

折半查找的两个必要条件

1. 存储结构必须是顺序存储（数组）。因为折半查找需要 $O(1)$ 时间跳到区间中点，也就是要随机存取能力。
2. 元素必须按关键字有序（通常要求递增）。

因此：有序链表不适合；无序数组不适合；有序静态链表不适合（逻辑顺序由链接字段决定，不能按下标直接取中点）；无序静态链表更不适合。

只有「有序 + 顺序存储（可随机存取）」才适合。

（二）基本实现（左闭右闭）

```
int binary_search(int nums[], int n, int target) {
    int low = 0, high = n - 1, mid;
    while (low <= high) {
        mid = low + (high - low) / 2;   // 等价于 (low+high)/2，下取整
        if (nums[mid] == target) return mid;      // 命中
        else if (nums[mid] > target) high = mid - 1;  // 前半区间
        else low = mid + 1;                        // 后半区间
    }
```

```
    return -1;   // 失败
}
```

循环终止条件是 $low > high$。终止时 low 恰好指向第一个大于 $target$ 的元素的下标——这个性质在「查找插入位置」类问题里很有用。

（三）判定树的构造

折半查找判定树：把每次比较的 mid 作为结点、把递归的左右半个区间作为左右子树，得到的二叉树。

设 low, h 为当前区间的两端，取 $mid = \left\lfloor \dfrac{low + high}{2} \right\rfloor$（下取整），则：

- 区间元素个数为奇数时，mid 分割后左右两部分元素个数相等；
- 区间元素个数为偶数时，mid 分割后左半部分比右半部分少一个元素。

由此得到判定树最重要的形态特征：

$$\text{对判定树中任一结点：右子树结点数} - \text{左子树结点数} \in \{0, 1\}$$

反过来，若采用上取整 $mid = \left\lceil \dfrac{low + high}{2} \right\rceil$，则对任一结点：

$$\text{左子树结点数} - \text{右子树结点数} \in \{0, 1\}$$

mid 取整方式必须全程一致

上取整和下取整产生的判定树是互为镜像的两棵树，但同一棵树里不能混用。统考常给若干棵二叉树，问「哪一棵可能是折半查找判定树」，判定方法就是：在每个结点上都检查左右子树规模的关系是否符合同一个取整规则。

只要有一处不符（比如某个偶数长度区间把多出来的一半放到了错误的一侧），该树就不可能是判定树。这是 2017 年那道判定树选择题的核心判据。

（四）判定树是平衡二叉树

由上面的规模关系可以推出，判定树中任一结点的左右子树高度差不超过 1，因此：

折半查找判定树一定是平衡二叉树

判定树的形态与关键字的具体取值无关，只与元素个数 n 有关。也就是说，对给定 n，折半查找判定树是唯一的（同一取整约定下）。

这一点与二叉搜索树形成鲜明对照：n 个相同关键字、不同插入顺序会得到不同的二叉搜索树，形态不唯一。

（五）树高、查找成功与失败的比较次数上限

树高：判定树只有最后一层可能不满，前 $h-1$ 层是满的（最多容纳 $2^{h-1}-1$ 个内部结点）。内部结点总数 n 满足

$$2^{h-1} - 1 < n \le 2^h - 1$$

两边加 1 取以 2 为底的对数：

$$2^{h-1} < n+1 \le 2^h \implies h-1 < \log_2(n+1) \le h$$

所以（不含失败结点的层数）

$$\boxed{h = \lceil \log_2(n+1) \rceil}$$

由「成功结点的查找长度 = 自身层数」和「失败结点的查找长度 = 父结点层数」，两种情况的比较次数都不会超过树高：

$$ASL_{\text{成功}} \le h, \qquad ASL_{\text{失败}} \le h$$

折半查找比较次数的上限

- 查找成功最多比较 $h = \lceil \log_2(n+1) \rceil$ 次。
- 查找失败最多也比较 $h = \lceil \log_2(n+1) \rceil$ 次。

自验证：$n = 16$，$\lceil \log_2 17 \rceil = \lceil 4.09 \rceil = 5$。前 4 层最多放 $2^4 - 1 = 15$ 个内部结点，第 16 个必须落在第 5 层，所以某些失败查找要沿路径比较 5 次才进入空区间——答案 5 次。

$n = 600$，$\lceil \log_2 601 \rceil = \lceil 9.23 \rceil = 10$，最多比较 10 次。

统考最爱的两个数字：「长度为 n 的有序顺序表折半查找，最多比较多少次」是高频选择题，标准答案就是 $\lceil \log_2(n+1) \rceil$。注意不要把 n 是 2^k 的特殊情况当成通式：$n = 16$ 时不是 4 次而是 5 次。

（六）$ASL_{\text{成功}}$ 的推导

设判定树高为 h，前 $h-1$ 层是满的，第 h 层剩下 k 个结点：

$$k = n - (2^{h-1} - 1)$$

第 i 层有 2^{i-1} 个结点（$i < h$），第 h 层有 k 个结点。按层数加权求和：

$$ASL_{\text{成功}} = \frac{1}{n}\left[\sum_{i=1}^{h-1} i \cdot 2^{i-1} + h \cdot k\right]$$

利用恒等式 $\sum_{i=1}^{h-1} i \cdot 2^{i-1} = (h-2)2^{h-1} + 1$，得

$$ASL_{\text{成功}} = \frac{1}{n}\Big[(h-2)2^{h-1} + 1 + h \cdot \big(n - 2^{h-1} + 1\big)\Big] = h - 1 + \frac{3 - 2^h + 2^{h-1}}{n}$$

在 n 较大、h 较小时，后面那一项相对 n 可忽略，于是得到统考常用的结论：

$$\boxed{ASL_{\text{成功}} \approx \log_2(n+1) - 1}$$

更严格地说：当 $n = 2^h - 1$（判定树是满二叉树）时，$ASL_{\text{成功}} = h - 1 = \log_2(n+1) - 1$，这是下界；当 n 只是刚超过 $2^{h-1} - 1$ 时，$ASL_{\text{成功}}$ 略大于 $h-1$，但始终满足

$$\log_2(n+1) - 1 \le ASL_{\text{成功}} < h = \lceil \log_2(n+1) \rceil$$

算例：$n = 11$，判定树为满二叉树前 4 层去掉第 4 层的 4 个结点（第 4 层剩 4 个）。各层结点数：第 1 层 1 个，第 2 层 2 个，第 3 层 4 个，第 4 层 4 个。

$$ASL_{\text{成功}}=\frac{1\times1+2\times2+3\times4+4\times4}{11}=\frac{1+4+12+16}{11}=\frac{33}{11}=3$$

用近似公式检验：$\log_2 12-1\approx3.585-1=2.585$，实际 3，落在 $[2.585,\,4)$ 内，符合上面的不等式。

（七）$ASL_{\text{失败}}$ 的推导

失败结点（外部结点）的总数恒为 $n+1$。设判定树高为 h，第 h 层上有

$$k=n-\left(2^{h-1}-1\right)$$

个内部结点。外部结点按父结点层数分两类：

- 父结点在第 h 层的：第 h 层的每个内部结点各带 2 个外部孩子，共 $2k$ 个，查找长度为 h；
- 父结点在第 $h-1$ 层的：剩下 $A=(n+1)-2k$ 个，查找长度为 $h-1$。

于是通用公式为

$$ASL_{\text{失败}}=\frac{(h-1)\cdot A+h\cdot 2k}{n+1},\qquad A=n+1-2k$$

算例 **1**：$n=11$，$h=4$，$k=11-7=4$，$2k=8$，$A=12-8=4$：

$$ASL_{\text{失败}}=\frac{3\times4+4\times8}{12}=\frac{12+32}{12}=\frac{44}{12}=\frac{11}{3}\approx3.67$$

算例 **2**：$n=16$，$h=5$，$k=16-15=1$，$2k=2$，$A=17-2=15$：

$$ASL_{\text{失败}}=\frac{4\times15+5\times2}{17}=\frac{60+10}{17}=\frac{70}{17}\approx4.12$$

算例 **3**：$n=15=2^4-1$（满二叉树），$h=4$，$k=15-7=8$，$2k=16$，$A=16-16=0$，全部外部结点的父层数都是 4：

$$ASL_{\text{失败}}=\frac{4\times0+4\times16}{16}=4=h$$

自验证：A 必为非负整数。若算出 $A<0$，说明 k 或 h 取错了。

做题时的可靠做法

画出判定树逐个数外部结点，比套公式更稳。两条规律：

- 只有最下方两层的内部结点可能「孩子不全」；
- 父层数越浅，外部结点越多。

特殊情形：若 $n=2^h-1$（判定树是满二叉树），则 $k=2^{h-1}$，$A=0$，全部外部结点的父层数都是 h：

$$ASL_{\text{失败}}=h=\lceil\log_2(n+1)\rceil=\log_2(n+1)$$

$ASL_{\text{失败}}$ 与失败查找次数上限

最大失败查找次数 = 树高 $h=\lceil\log_2(n+1)\rceil$。

判定树中最深的外部结点，其父结点恰在第 h 层，所以失败查找最多比较 h 次——与成功查找的上界相同。$n=16$ 时 $h=5$，失败查找最多比较 5 次。

$ASL_{失败}$ 的三个易错点

1. 分母是 $n+1$（外部结点数），不是 n。
2. 外部结点自身不计比较，用的是父结点层数。把外部结点当成第 $h+1$ 层来算，结果会整体偏大。
3. 只有 $n=2^h-1$ 时才有干净的 $ASL_{失败}=h$；一般情况下必须画出判定树逐个数，不能直接套 h。

$n=5$ 的例子：$h=3$，$k=5-3=2$，$2k=4$，$A=6-4=2$：

$$ASL_{失败}=\frac{2\times 2+3\times 4}{6}=\frac{4+12}{6}=\frac{8}{3}\approx 2.67$$

（八）时间复杂度与优缺点

折半查找的时间复杂度为 $O(\log_2 n)$。

优点：时间效率高（$n=2^{20}$ 时顺序查找需约 10^6 次比较，折半查找只需 20 次）；不需要额外辅助空间。

局限：

- 只适用于有序数据。为用折半而先排序是 $O(n\log n)$，得不偿失；频繁插入时保持有序也要 $O(n)$ 的搬移成本。
- 只适用于数组，链表上「跳跃访问」代价太高。
- 小数据量下顺序查找反而更快（折半每轮约 4–6 个基本操作，顺序查找只有 1 次判断）。

统考考法

折半查找是选择题重灾区，常见问法四种：

1. 求最多比较次数——答 $\lceil\log_2(n+1)\rceil$（$n=16$ 答 5，$n=600$ 答 10）。
2. 给一串关键字比较序列，判断「能否是某次折半查找的比较序列」。判据：每一步都在前面比较所确定的上下界之间。如 $500,200,450,180$：比较 500 后知目标 <500；比较 200 后知目标 >200，范围 $(200,500)$；比较 450 后知目标 <450，范围 $(200,450)$——180 已在下界 200 之外，不可能出现。
3. 判断某二叉树能否是折半查找判定树——检查每层左右子树规模是否始终符合同一取整规则。
4. 判断哪些存储结构适合折半查找——只有「有序 + 顺序存储」。

BST 查找路径的判定题用的是完全相同的上下界收缩法，只是一次比较后可能只更新一侧边界。

五、树形查找

（一）二叉搜索树（BST）

二叉搜索树（**Binary Search Tree**），又称二叉排序树、二叉查找树，满足：

1. 对任意结点，左子树中所有结点的值 < 该结点的值 < 右子树中所有结点的值；
2. 左、右子树本身也都是二叉搜索树。

BST 的两条基本性质

- 中序遍历 BST 得到的序列是严格递增序列。这条性质是判断一棵树「是不是 BST」的标准方法，也是解「第 k 小元素」「判断合法性」类问题的关键。
- BST 中不允许存在重复关键字（若允许，需在定义中明确放在哪一侧）。

1.（1）查找

从根出发，逐层比较：

- `cur->val < target`：目标在右子树，`cur = cur->right;`
- `cur->val > target`：目标在左子树，`cur = cur->left;`
- 相等：查找成功；
- `cur == NULL`：查找失败。

```
TreeNode *BST_Search(BSTree root, int num) {
    TreeNode *cur = root;
    while (cur != NULL) {
        if (cur->val < num)       cur = cur->right;
        else if (cur->val > num)  cur = cur->left;
        else                      break;      // 命中
    }
    return cur;   // NULL 表示查找失败
}
```

非递归版本空间复杂度 $O(1)$，递归版本空间复杂度 $O(h)$（递归工作栈）。时间复杂度为 $O(h)$，h 为树高：最好 $O(\log n)$，最坏 $O(n)$。

2.（2）插入

插入分两步：

1. 查找插入位置：从根出发，按大小关系向下，直到走到 `NULL`；
2. 在该位置挂上新结点（新结点一定是叶结点）。

```
void insert(BSTree &root, int num) {
    if (root == NULL) { root = new TreeNode{num, NULL, NULL}; return; }
    TreeNode *cur = root, *pre = NULL;
    while (cur != NULL) {
        if (cur->val == num) return;      // 已存在，不插入
        pre = cur;                        // pre 记录父结点
        cur = (cur->val > num) ? cur->left : cur->right;
    }
    TreeNode *node = new TreeNode{num, NULL, NULL};
    if (pre->val < num) pre->right = node;
    else                pre->left  = node;
}
```

新插入的结点一定是叶结点

这一点在判断题里反复出现。推论：**AVL** 树插入后可能旋转，旋转会把新结点抬升成非叶结点，所以「最后插入的元素一定是叶结点」对 BST 成立、对 AVL 树不成立。

3.（3）构造

从空树开始，把关键字序列逐个插入即可：

```
void Create_BST(BSTree &root, const int str[], int n) {
    root = NULL;
```

```
    for (int i = 0; i < n; ++i) insert(root, str[i]);
}
```

由此可得一条重要区别：

BST 的形态不唯一

对同一组关键字，插入顺序不同会得到不同形态的 **BST**（但中序序列相同）。

例：$\{1,2,3\}$ 按 $2,1,3$ 插入得到平衡的树；按 $1,2,3$ 插入退化成右斜链。因此 BST 的 ASL 与形态强相关，只有折半查找判定树是唯一确定的。

4.（4）删除：三种情况

先在树中找到待删结点，然后按度分三种情况：

情况	待删结点的度	处理方式
情况 **1**	度为 0（叶子）	直接删除，其父结点的相应指针置空
情况 **2**	度为 1（只有左子树或只有右子树）	用其唯一的子结点顶替它的位置（即让父结点直接指向该子结点）
情况 **3**	度为 2（左右子树都有）	用中序前驱（左子树最右下结点）或中序后继（右子树最左下结点）替换其值，再在相应子树中递归删除那个前驱/后继

情况 3 为什么可以用前驱或后继替换？因为 BST 的中序序列是递增序列，待删结点在序列中的位置，恰好可以由它紧邻的前一个元素或后一个元素顶上，替换后整棵树仍满足 BST 性质。前驱一定是左子树的最大值（左子树最右下），后继一定是右子树的最小值（右子树最左下），二者都是叶子或至多带一个子树，删除它们就退化成情况 1 或 2。

```
TreeNode *deleteNode(TreeNode *root, int key) {
    if (root == NULL) return NULL;
    if (root->val < key) { root->right = deleteNode(root->right, key); return root; }
    if (root->val > key) { root->left  = deleteNode(root->left,  key); return root; }
    // 找到待删结点
    if (!root->left && !root->right) { delete root; return NULL; }      // 情况 1
    if (!root->left)  { TreeNode *c = root->right; delete root; return c; }  // 情况 2
    if (!root->right) { TreeNode *c = root->left;  delete root; return c; }  // 情况 2
    // 情况 3：用中序后继替换
    TreeNode *succ = root->right;
    while (succ->left) succ = succ->left;      // 右子树最左下
    root->val = succ->val;                     // 只复制值，不搬动结点
    root->right = deleteNode(root->right, succ->val);
    return root;
}
```

删除再插入：形态是否复原？

设对 BST T_1 删除结点 v 得到 T_2，再把 v 插入 T_2 得到 T_3。

- 若 v 是叶结点：删除不改变其他结点的相对结构；重新插入时按下大小比较仍会走到原来那个空位置，故 T_3 与 T_1 相同。
- 若 v 不是叶结点：删除时用前驱/后继调整了结构；而普通 BST 插入总是把新结点放到空指针位置，所以 v 重新插入后一定会变成叶结点。原来它是非叶结点，故 T_3 与 T_1 一定不同。

对 **AVL** 树，结论变弱：删除叶结点可能引发旋转，重新插入时的旋转未必把树还原，所以 T_3 与 T_1 可能不同（不是「一定相同」也不是「一定不同」）；删除非叶结点后重插也不一定不同（存在能还原的例子）。

还有一种「偷懒」的删除写法：把待删结点的左子树整体挂到右子树最左下结点的左指针上，然后让右子树顶替待删结点的位置。这种做法不改变结点之间的父子关系数量，但多次删除后树会退化成链表，不推荐使用，考试也一般不考。

5.（5）查找效率与 ASL

BST 的查找长度等于「从根走到目标结点经过的结点数」，即目标结点的层数。因此

$$ASL = \frac{1}{n}\sum_{i=1}^{n}(\text{第}i\text{ 个结点的层数})$$

- 最好情况：树尽量平衡，高度 $h = \lfloor \log_2 n \rfloor + 1$，$ASL = O(\log n)$；
- 最坏情况：每个结点只有一个分支，退化成单链表，$h = n$，$ASL = O(n)$。

算例：关键字集合 $\{1, 2, \cdots, 9\}$。若按 $1, 2, \cdots, 9$ 顺序插入，得到右斜链，各结点层数为 $1, 2, \cdots, 9$：

$$ASL_{\text{成功}} = \frac{1 + 2 + \cdots + 9}{9} = \frac{45}{9} = 5$$

若构造得较平衡，$ASL_{\text{成功}}$ 可降到 3 左右——**BST** 的效率完全取决于树形。

6.（6）BST 与折半查找的对比

对比项	折半查找	二叉搜索树
存储要求	有序顺序表	任意（链式也可）
判定树/树形	唯一确定，只与 n 有关	不唯一，与插入顺序有关
平均查找时间	$O(\log n)$	平均 $O(\log n)$，最坏 $O(n)$
插入删除	需移动大量元素，$O(n)$	只改指针，$O(\log n)$（平均）
是否适合动态查找表	不适合	适合

统考考法

BST 是高频选择题来源，典型问法：

- 给一串关键字序列，问「不可能是某棵 **BST** 的一条查找路径」（2011 年）。判据是上下界收缩：沿路径每比较一个关键字就收紧一次区间，后续关键字必须始终落在当前区间内。例如路径 $95, 22, 91, 24, 94, \ldots$：比较 95 后目标 < 95；比较 22 后目标 > 22，区间 $(22, 95)$；比较 91 后目标 < 91，区间 $(22, 91)$；比较 24 后目标 > 24，区间 $(24, 91)$——此时出现 94 已超过上界 91，故该序列不可能是查找路径。
- 给一棵 **BST** 图，问某个子树中所有关键字满足的不等式（2018、2024 年）。只要沿「属于谁的左/右子树」把不等式全部列出来取交集即可。
- 给关键字序列，问「不能生成某棵给定 **BST**」的是哪个（2020 年）。逐个插入模拟即可。
- 删除再插入的形态判断（2013 年）。
- 中序遍历相邻结点的关系判断（2022 年）。

（二）平衡二叉树（AVL）

BST 的效率取决于树形，而一连串有序插入或反复「删右子树最小值」的操作会让树偏向一侧。平衡二叉树就是为解决这个问题而引入的自平衡结构。

平衡二叉树（**AVL 树**）：一棵 BST，且其中任意结点的左、右子树高度之差的绝对值不超过 **1**。AVL 树是最早提出的自平衡二叉查找树。

1.（*1*）结点高度与平衡因子

结点高度：从该结点到最远叶结点的边数（空结点高度为 0，叶结点高度为 1）。

平衡因子（**Balance Factor, BF**）：

$$BF(\text{node}) = h(\text{node} \to \text{left}) - h(\text{node} \to \text{right})$$

AVL 的平衡条件

AVL 树中任一结点的平衡因子 $BF \in \{-1, 0, 1\}$；空结点的平衡因子规定为 0。

$|BF| > 1$ 的结点称为失衡结点。插入或删除后只要出现失衡结点，就必须调整。

为了 $O(1)$ 地取得高度和平衡因子，AVL 的结点要额外维护 `height` 域：

```
typedef struct AVLNode {
    int val;
    int height;
    struct AVLNode *left, *right;
} AVLNode, *AVLTree;

int height(AVLNode *node) { return node == NULL ? 0 : node->height; }

void updateHeight(AVLNode *node) {
    node->height = (height(node->left) > height(node->right)
                    ? height(node->left) : height(node->right)) + 1;
}

int balanceFactor(AVLNode *node) {
    return node == NULL ? 0 : height(node->left) - height(node->right);
}
```

2.（*2*）最小失衡子树与四种旋转

最小失衡子树（最小不平衡子树）：以离插入（或删除）位置最近的那个失衡结点为根的子树。AVL 的调整永远是自底向上找第一个失衡结点，然后只调整这一棵子树。

关键结论

在插入操作中，只要把最小不平衡子树调整平衡，它上面的所有祖先结点都会自动恢复平衡，不必再往上调。

（删除操作不一定：子树高度降低可能继续向上传导失衡，需要反复回溯调整。）

记失衡结点为 `node`，其高度较高的孩子为 `child`。按「插入位置相对 `node` 的方向」分四种情况：

① **LL** 型（左孩子的左子树插入）——右旋

`node` 的 $BF > 1$（左偏），且 `child` 的 $BF \geq 0$。以 `child` 为轴把 `node` 向右旋转：`child` 上升为子树新根，`node` 下降为 `child` 的右孩子；若 `child` 原本有右子树 β（记为 `grandChild`），则把 β 改

挂为 node 的左子树。

结构变化（α, β, γ 表示子树）：

旋转前	旋转后
node 的左孩子是 child；child 的左子树 α、右子树 β；node 的右子树 γ	child 为根；左子树 α，右孩子 node；node 的左子树 β、右子树 γ

② **RR** 型（右孩子的右子树插入）——左旋

node 的 $BF < -1$（右偏），且 child 的 $BF \leq 0$。与 LL 完全镜像：以 child 为轴把 node 向左旋转，child 上升为新根，node 成为 child 的左孩子，child 原来的左子树 β 改挂为 node 的右子树 γ 位置。

旋转前	旋转后
node 的右孩子是 child；node 的左子树 α；child 的左子树 β、右子树 γ	child 为根；左孩子 node，右子树 γ；node 的左子树 α、右子树 β

③ **LR** 型（左孩子的右子树插入）——先左旋再右旋

node 的 $BF > 1$（左偏），但 child 的 $BF < 0$（child 自己右偏），单旋解决不了，要两次旋转：

1. 先对 child 执行左旋，把 child 的右孩子（记为 grandChild，其左子树 β_1、右子树 β_2）抬到 child 的位置；
2. 再对 node 执行右旋，把 grandChild 抬到 node 的位置。

最终 grandChild 成为该子树的新根：左孩子 child（左子树 α、右子树 β_1），右孩子 node（左子树 β_2、右子树 δ）。

步骤	变化
初始	node 的左孩子 child，node 的右子树 δ；child 的左子树 α、右孩子 gc；gc 的左子树 β_1、右子树 β_2
对 child 左旋	gc 顶替 child 的位置，child 成为 gc 的左孩子（child 的右子树变为 β_1）
对 node 右旋	gc 顶替 node 的位置，node 成为 gc 的右孩子（node 的左子树变为 β_2）

LR/RL 的本质

双旋后升上去的是原来的「孙子」，而不是「儿子」。这正是红黑树插入中 LR/RL 型要「儿换爷」染色、而 LL/RR 型要「父换爷」染色的原因。

④ **RL** 型（右孩子的左子树插入）——先右旋再左旋

node 的 $BF < -1$（右偏），但 child 的 $BF > 0$，与 LR 完全镜像：

1. 先对 child 执行右旋：child 的左孩子 gc 上升，child 成为 gc 的右孩子，而 gc 原来的右子树 β_2 改挂为 child 的左子树；
2. 再对 node 执行左旋：gc 上升为子树新根，node 成为 gc 的左孩子，而 gc 原来的左子树 β_1 改挂为 node 的右子树。

最终 grandChild 成为该子树的新根：左孩子 node（左子树 α、右子树 β_2），右孩子 child（左子树 β_1、右子树 δ）。

步骤	变化
初始	node 的左子树 α，右孩子 child；child 的左孩子 gc、右子树 δ；gc 的左子树 β_1、右子树 β_2
对 child 右旋	gc 顶替 child 的位置；child 成为 gc 的右孩子（child 的左子树变为 β_2）
对 node 左旋	gc 顶替 node 的位置；node 成为 gc 的左孩子（node 的右子树变为 β_1）

四种旋转的判定表（背下来）

失衡结点 BF	较高孩子 BF	类型	旋转方法
>1（左偏）	≥ 0	LL	右旋
>1（左偏）	<0	LR	先左旋后右旋
<-1（右偏）	≤ 0	RR	左旋
<-1（右偏）	>0	RL	先右旋后左旋

一句话记忆：「偏向哪边，就先把它正过来」。LL/RR 是「偏心在外侧」，单旋即可；LR/RL 是「先偏外边再偏里边」，要双旋。

旋转的代码只改指针。右旋：

```
AVLNode *rightRotate(AVLNode *node) {
    AVLNode *child = node->left;
    AVLNode *grand = child->right;
    child->right = node;       // child 上升
    node->left   = grand;      // 原 child 的右子树改挂
    updateHeight(node);        // 先更新下层 node
    updateHeight(child);       // 再更新上层 child
    return child;              // 返回新子树根
}
```

左旋把上述代码中所有 left/right 互换即可：

```
AVLNode *leftRotate(AVLNode *node) {
    AVLNode *child = node->right;
    AVLNode *grand = child->left;
    child->left  = node;
    node->right  = grand;
    updateHeight(node);
    updateHeight(child);
    return child;
}
```

旋转的三条注意事项

1. 旋转不改变中序遍历序列——这是 AVL 旋转既能恢复平衡、又保持 BST 性质的根本原因。

2. 必须自底向上找第一个失衡结点（最小不平衡子树）来调整，而不是直接调根；插入情形下调整它后祖先自动恢复。
3. 更新高度必须先 `node` 后 `child`（先下后上），顺序反了高度会算错。

3.（*3*）插入后的调整过程

步骤：

1. 按 BST 规则找到插入位置，插入新结点（新结点一定是叶结点），并回溯更新沿途结点的高度；
2. 自底向上找到第一个失衡结点 `node`；
3. 记 `node` 中高度较高的孩子为 `child`，若 `child` 也有较高的孙子 `grandChild`，则由 `node` → `child` → `grandChild` 的形状判定是 LL / RR / LR / RL；
4. 在最小不平衡子树上执行相应旋转；
5. 因插入只会让子树高度增加，调整完第一个失衡结点后整棵树就平衡了，结束。

完整算例：依次插入 $1,2,3,4,5,6,7$ 到空 AVL 树。

插入后	出现的失衡	类型	调整结果（根）
1,2	无（根 1，右孩子 2，$BF=-1$）	—	1
3	结点 1 的 $BF=-2$，孩子 2 的 $BF=-1$	RR	左旋 1 → 根变 2
4	结点 3 的 $BF=-1$，无失衡	—	2
5	结点 3 的 $BF=-2$，孩子 4 的 $BF=-1$	RR	左旋 3 → 4 顶替
6	结点 2 的 $BF=-2$，孩子 4 的 $BF=-1$	RR	左旋 2 → 4 成为总根
7	结点 5 的 $BF=-2$，孩子 6 的 $BF=-1$	RR	左旋 5 → 6 顶替

最终整棵树是以 4 为根、左右子树各 3 个结点的平衡形态。这个例子说明：连续有序插入不会退化成链，因为每两步就会触发一次旋转。

这是 **2013** 年那道题的原型

「将 $1,2,3,4,5,6,7$ 依次插入初始为空的 AVL 树，求平衡因子为 0 的分支结点个数」，就是照上表模拟到底，再数 $BF=0$ 的非叶结点。

4.（*4*）删除后的调整过程

删除比插入麻烦，因为它不保证一次旋转就结束：

1. 先用 **BST** 的删除方法删掉结点（三种情况同上）；
2. 从删除位置向上回溯，找第一个失衡结点；找不到说明树仍平衡，结束；
3. 记失衡结点为 `node`，取高度最高的儿子为 `child`、高度最高的孙子为 `grandChild`（必须调整长得更高的那一侧才可能恢复平衡），按 LL/RR/LR/RL 执行相应旋转；
4. 若调整后子树高度减 1 导致上层继续失衡，回到第 2 步继续调整，直到整棵树平衡。

删除与插入的两条差别

- 插入：只调一个最小不平衡子树即结束。
- 删除：可能需要多次调整，一直回溯到根。

5.（5）n 个结点的 AVL 树的高度范围

这块是统考计算题的常客，核心是递推。

最大高度（最少结点数）

设 $N(h)$ 表示高度为 h 的 **AVL** 树所含的最少结点数。

要让高度为 h 的 AVL 树结点数最少，根的两棵子树必须高度差取到最大的 **1**，并且两棵子树各自也取最少结点。于是根的一棵子树高度为 $h-1$（取最少结点 $N(h-1)$），另一棵高度为 $h-2$（取最少结点 $N(h-2)$），再加上根本身：

$$\boxed{N(h)=N(h-1)+N(h-2)+1}$$

初值：

$$N(0)=0,\quad N(1)=1,\quad N(2)=2$$

（$h=0$ 是空树，$h=1$ 只有根，$h=2$ 必须左右各一个孩子，共 2 个结点。）

依次递推：

h	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
$N(h)$	1	2	4	7	12	20	33	54	88	143	232	376

与斐波那契数列的关系：记斐波那契数列 $F_1=1,F_2=1,F_3=2,F_4=3,\ldots$。把 $N(h)+1$ 列出来：$2,3,5,8,13,21,34,\ldots$，正好是 F_{h+2}：

$$N(h)=F_{h+2}-1$$

验证：$h=4$ 时 $F_6=8$，$N(4)=7$，成立；$h=6$ 时 $F_8=21$，$N(6)=20$，成立。

由斐波那契的通项 $F_k\approx\dfrac{\varphi^k}{\sqrt{5}}$（$\varphi=\dfrac{1+\sqrt{5}}{2}\approx1.618$），可推出高度与结点数的关系：

$$h\approx\log_{\varphi}\left((n+1)\sqrt{5}\right)-2$$

换成以 2 为底：

$$h\le1.44\log_2(n+1)$$

AVL 树的高度范围（必背）

- 最少结点数递推：$N(h)=N(h-1)+N(h-2)+1$，$N(0)=0,N(1)=1,N(2)=2$；与斐波那契的关系 $N(h)=F_{h+2}-1$。
- 含 n 个结点的 AVL 树，最坏高度

  $$h\approx1.44\log_2 n$$

  所以查找、插入、删除最坏都是 $O(\log n)$，远好于普通 BST 的 $O(n)$。
- 最大结点数：高度为 h 的 AVL 树最多 2^h-1 个结点（满二叉树，此时所有结点 $BF=0$）。

常见题型一：给定高度求最少结点数

设 $h=6$，则 $N(6)=20$，所以高度为 6 的 AVL 树至少 20 个结点。

常见题型二：给定结点数求最大高度

$n=20$ 时，$N(6)=20$，故最大高度恰好是 6。

常见题型三：所有非叶结点的平衡因子都相同

2012 年那道题：「高度为 6、所有非叶结点的平衡因子均为 1 的 AVL 树，结点总数是多少」。

分析：$BF=1$ 表示每个非叶结点的左子树比右子树高 1。设 $M(h)$ 为满足该条件的树的最少结点数（也是唯一确定的结点数——因为每个结点都必须左高 1，两侧高度被唯一确定：左子树高度 $h-1$、右子树高度 $h-2$）：

$$M(h)=1+M(h-1)+M(h-2)$$

这与 $N(h)$ 的递推完全相同，但初值不同：$BF=1$ 意味着叶结点的 $BF=0$。逐层推：

$$M(1)=1,\quad M(2)=2,\quad M(3)=4,\quad M(4)=7,\quad M(5)=12,\quad M(6)=20$$

所以答案为 **20** 个结点（选项 B）。注意这道题的树形是被 BF 条件唯一钉死的，不是「最少结点」的优化问题。

AVL 三类数字别混

问题	公式/答案
高度 h 的 AVL 树最少结点数	$N(h)=N(h-1)+N(h-2)+1$，$N(6)=20$
高度 h 的 AVL 树最多结点数	2^h-1，$h=6$ 时 63
n 个结点的 AVL 树最大高度	解 $N(h)\le n$，$n=20$ 时 $h=6$
n 个结点的 AVL 树最小高度	$\lceil\log_2(n+1)\rceil$，$n=20$ 时 5

「最少结点」和「最大高度」其实是同一个问题换了个问法——不要分别死记。

6.（6）AVL 的典型应用

- 组织与存储大数据集合，适用于高频查找、低频增删的场景；
- 数据库索引系统；
- 当散列表的某个链表过长时，可以把该链表转换成 **AVL** 树，把链表查找从 $O(n)$ 优化到 $O(\log n)$。

（三）红黑树（RBT）

AVL 树的平衡条件非常严格（$|BF|\le 1$），因此插入删除很容易破坏平衡，必须频繁调整：每次都要算平衡因子、找最小不平衡子树（时间开销大），再做 LL/RR/LR/RL 旋转。

红黑树（**Red-Black Tree, RBT**）放松了平衡条件，换来更少的调整次数，因此在「插入删除频繁」的场景中比 AVL 更实用。

1.（1）五条性质

红黑树首先是一棵二叉搜索树，额外满足：

1. 每个结点或是红色，或是黑色；
2. 根结点是黑色的；
3. 叶结点（外部结点 / **NULL** 结点 / 失败结点）都是黑色的；
4. 不存在两个相邻的红结点（即红结点的父结点和孩子结点都必须是黑的）；
5. 对每个结点，从该结点到任一叶结点的所有简单路径上，所含黑结点的数目相同。

五条性质的口诀

「左根右，根叶黑，不红红，黑路同」

- 左根右：它首先是 BST，满足左 < 根 < 右；
- 根叶黑：根黑，外部叶结点黑；
- 不红红：没有连续两个红结点；
- 黑路同：任一点到任一叶结点的黑结点数相同（这叫黑高一致）。

> 注意红黑树中的「叶结点」指空结点（外部结点），不是带关键字的最下层结点——这一点和 B 树一致，考试极爱在这里设陷阱。

结点结构需要多一个父指针和颜色域：

```
typedef struct RBNode {
    int val;
    struct RBNode *parent;
    struct RBNode *left, *right;
    int color;      // 0 表示黑，1 表示红
} RBNode;
```

*2.（2）*两条重要推论

推论 **1**：从根到叶结点的最长路径不大于最短路径的两倍。

证明思路：由「黑路同」，最长路径与最短路径上的黑结点数目相同，设为 bh。由「不红红」，最长路径上只能「黑、红、黑、红……」交替出现，所以最长路径的结点数至多为 $2bh$；而最短路径可以全是黑结点，结点数为 bh。故最长不超过最短的两倍。

这条性质也是判断一棵树是否为红黑树的核心依据（配合「不红红」和「黑路同」一起检查）。

推论 **2**：含 n 个内部结点的红黑树，高度 $h \le 2\log_2(n+1)$，因此查找的时间复杂度为 $O(\log n)$。

推导：定义结点的黑高（**bh, black height**）为「从该结点出发（不含该结点）到达任一叶结点的简单路径上的黑结点总数」。

若根结点的黑高为 bh，则内部结点数最少的情形是「共 bh 层、全是黑结点」的满二叉树，此时内部结点数至少为 $2^{bh}-1$，即

$$n \ge 2^{bh} - 1$$

另一方面，由「不红红」，总高度 h 最多是黑高的两倍（红黑交替），所以

$$bh \ge \left\lfloor \frac{h}{2} \right\rfloor$$

两式联立：

$$n \ge 2^{\lfloor h/2 \rfloor} - 1 \implies \frac{h}{2} \le \log_2(n+1) \implies \boxed{h \le 2\log_2(n+1)}$$

3.（*3*）与 *AVL* 的对比

对比项	**AVL** 树	红黑树
平衡条件	任一结点左右子树高度差 ≤ 1	最长路径 $\leq$ 最短路径的 2 倍
树高上界	$\approx 1.44\log_2 n$	$\leq 2\log_2(n+1)$
查找效率	略高（树更矮）	略低（树更高，但仍是 $O(\log n)$）
插入/删除的调整	频繁：要算平衡因子、找最小不平衡子树，可能多次旋转	较少：很多时候变色即可，调整可在常数级时间内完成
适用场景	以查为主、很少插入删除	频繁插入、删除，实用性更强

为什么红黑树在插入删除频繁时更优

AVL 的平衡条件严格，插入删除后绝大多数情况都要调整，而且每次调整前都要计算平衡因子、定位最小不平衡子树，定位本身就要 $O(\log n)$ 的代价（虽然可以靠维护 height 域降到 $O(1)$，但旋转次数仍多）。

红黑树的条件宽松，插入删除后很多时候不破坏红黑特性，直接结束；即使需要调整，也常在常数级时间内通过变色完成，旋转次数显著更少。因此在增删频繁的工程场景（如标准库的 `map`/`set`、Linux 内核的调度器与虚拟内存管理）中，红黑树比 AVL 更受欢迎。

4.（*4*）插入操作：变色 + 旋转

插入分三步：

第一步：按 BST 规则查找插入位置（新结点一定是叶结点）。

第二步：给新结点染色——

- 新结点是根：染为黑色；
- 新结点非根：染为红色（为什么染红？因为染红只可能违反「不红红」，而不改变任何路径的黑结点数；染黑必然违反「黑路同」，更难修）。

第三步：若插入后仍满足红黑性质，结束；否则按「新结点的叔叔的颜色」分两类调整：

① 黑叔：旋转 + 染色（交换旋转中涉及的两个结点的颜色）

类型	情形	处理
LL 型	新结点是祖父的左孩子的左孩子	右单旋，然后「父换爷」染色（父与爷互换颜色）
RR 型	新结点是祖父的右孩子的右孩子	左单旋，然后「父换爷」染色
LR 型	新结点是祖父的左孩子的右孩子	先左旋再右旋（左右双旋），然后「儿换爷」染色（儿与爷互换颜色）
RL 型	新结点是祖父的右孩子的左孩子	先右旋再左旋（右左双旋），然后「儿换爷」染色

记忆要点：「黑叔 $\rightarrow$ 旋转 + 染色」。LL/RR 单旋时是「父」和「爷」换色；LR/RL 双旋时最终升上去的是原来的「儿」，所以是「儿」和「爷」换色。

② 红叔：染色 + 变新（不旋转）

做法是「叔父爷染色，爷变新」：

1. 把叔叔、父亲、祖父三个结点的颜色全部取反（红变黑、黑变红）；
2. 把祖父当成「新插入的结点」，回到第二步重新处理（可能继续向上传递，最坏传到根，此时把根染黑即可）。

红黑树插入的两个易错点

1. 红叔时不需要旋转，只染色并向上传递。很多考生一见失衡就条件反射地旋转，这是错的。
2. 根结点被染红后必须立刻染回黑。红叔情形向上传递到根时，若把根染成了红，不满足性质 2，必须强制染黑。此时所有路径的黑结点数 +1，仍然满足「黑路同」。

完整算例：从空红黑树开始，依次插入 $20, 10, 5, 30, 40, 57, \ldots$。

- 插入 20：作为根，染黑。
- 插入 10：非根染红，挂 20 左边，父 20 为黑，满足性质，结束。
- 插入 5：挂 10 左边，父 10 为红——违反「不红红」。叔叔为空（黑）⇒ 黑叔 + **LL** 型：对祖父 20 右旋并「父换爷」染色（10 黑、20 红），根变 10。
- 插入 30：挂 20 右边，父 20 红、叔叔为空 ⇒ 黑叔 + **RR** 型：对祖父 10 左旋并换色，根变 20。
- 插入 40：挂 30 右边，父 30 红、叔叔 10 为红 ⇒ 红叔：叔、父、爷三色全取反，爷爷 20 作为「新结点」向上处理；它此时是根且为红，强制染黑。
- 后续每次插入按同样的两步判定处理。

判断练习：问「某树是不是红黑树」，只需查三条——① 根是否为黑；② 有无连续两个红结点；③ 从根到每个外部叶结点的黑结点数是否相等。三条全过即为红黑树。

5.（5）删除操作用不用掌握?

红黑树删除的考试要求

- 红黑树的删除操作过于复杂，统考不作要求。
- 只需知道三条事实：
  1. 删除操作的时间复杂度是 $O(\log_2 n)$；
  2. 删除结点的处理方式与二叉搜索树的删除相同（三种情况）；
  3. 删除后可能破坏红黑特性，需要调整结点颜色和位置使其重新满足红黑特性。

从 2022 年起「红黑树」进入考纲，考查方式很可能是判断某棵树是否为红黑树（用五条性质逐个排除），或与 AVL 对比优劣。插入调整考的可能性低于性质判定，但也要会做。

六、B 树及其基本操作、B+ 树的基本概念

B 树是本章分值最高、难度最大的考点。理解它的关键是把三个字拆开：「平衡」「多路」「查找」。

- 平衡：B 树中每个结点的任意两棵子树高度之差为 0，因此所有叶结点都在同一层（和 AVL 的「差不超过 1」、红黑树的「不超过 2 倍」都不同，B 树要求绝对平衡）；
- 多路：路数 = 结点分叉数，也叫阶数。m 阶 B 树就是 m 叉树，$m \geq 2$；$m = 2$ 时 B 树退化成二叉树的样子；
- 查找：结点内关键字有序，整体「中序遍历」（用环线法）得到单调递增序列。

一句话理解 B 树

B 树 = 树 + 顺序表：整体是一棵树，每个结点是一个有序顺序表。因此它兼具两者的优点——既有树的对数级查找路径，又有顺序表的局部性与有序性，非常适合做磁盘索引（一次结点访问 = 一次磁盘 I/O）。

（一）m 阶 B 树的定义

一棵 m 阶 B 树，或为空树，或为满足下列特性的 m 叉树：

1. 每个结点至多有 m 棵子树（即最多 $m-1$ 个关键字）；
2. 若根结点不是叶结点，则至少有 **2** 棵子树（即根至少有 1 个关键字）；
3. 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树（即至少有 $\lceil m/2 \rceil - 1$ 个关键字）；
4. 所有非叶结点包含信息 $(n, A_0, K_1, A_1, K_2, A_2, \cdots, K_n, A_n)$，其中：
   - K_i 为关键字，满足 $K_1 < K_2 < \cdots < K_n$；
   - A_i 为指向子树根结点的指针；
   - A_{i-1} 所指子树中所有关键字均小于 K_i，A_i 所指子树中所有关键字均大于 K_i；
   - n 为该结点的关键字个数，除根结点外的非空结点满足

$$\left\lceil \frac{m}{2} \right\rceil - 1 \le n \le m - 1$$

根结点满足 $1 \le n \le m-1$；

1. 所有叶结点（空结点）都出现在同一层上，且不带信息（可视为外部结点 / 失败结点，指向它们的指针为空）。

「叶结点」这个词在 B 树里有两种用法

- 经典定义（教材、**2023** 年真题第 **7** 题的语境）：B 树的叶结点是指空结点（外部结点 / 失败结点），与红黑树一致。带关键字的结点叫内部结点。
- 口语化用法：把最下面一层带关键字的结点也叫「叶结点」，上面带关键字的叫「非叶结点」。

两种说法都有人用，必须按上下文判断。一般结论：

- 考试里说「所有叶结点都在同一层」，通常指空结点层；
- 考试里说「含关键字的结点个数」「插入的新关键字最终位于叶结点」，用的是口语化用法。

遇到判断对错的选项时，先确认题目用的是哪一套。

把性质 1–5 用「口语化」重述一遍（这是考试最常用的形式）：

1. 树中每个结点至多有 m 棵子树；
2. 若根结点不是叶结点，则至少有两棵子树；
3. 除根结点外的所有非叶结点至少有 $\lceil m/2 \rceil$ 棵子树，即至少含有 $\lceil m/2 \rceil - 1$ 个关键字；
4. 除根结点外的非空结点满足 $\lceil m/2 \rceil - 1 \le n \le m-1$，根结点满足 $1 \le n \le m-1$；
5. 所有叶结点都出现在同一层上。

*1.*关键性质：*5* 阶 *B* 树（$m = 5$）

项	值
每结点最多子树数	5
每结点最多关键字数	$m - 1 = 4$
非根非叶结点最少子树数	$\lceil 5/2 \rceil = 3$
非根非叶结点最少关键字数	$\lceil 5/2 \rceil - 1 = 2$
根最少关键字数	1

*2.*一棵 m 阶 *B* 树的关键字数量范围

高度为 h 的 m 阶 **B** 树，关键字数最少为 $2\lceil m/2 \rceil^{h-1} - 1$。

推导：设含关键字的结点所在层为第 1 到第 h 层。要让关键字数最少，就让每层结点数最少、每个结点关键字数最少：

- 第 1 层（根）：1 个结点，最少 1 个关键字；
- 第 2 层：根至少 2 棵子树 $\Rightarrow$ 2 个结点，每个最少 $\lceil m/2 \rceil - 1$ 个关键字；
- 第 3 层：每个第 2 层结点至少 $\lceil m/2 \rceil$ 棵子树 $\Rightarrow$ 最少 $2\lceil m/2 \rceil$ 个结点；
- $\vdots$
- 第 h 层：最少 $2\lceil m/2 \rceil^{h-2}$ 个结点。

高度为 h 的 m 阶 **B** 树，关键字数最多为 $m^h - 1$。

推导：让每个结点都装满，即每个结点 $m-1$ 个关键字、m 棵子树：

$$n \le (m-1)\left(1 + m + m^2 + \cdots + m^{h-1}\right) = (m-1) \cdot \frac{m^h - 1}{m-1} = m^h - 1$$

*3.*高度范围的推导（两个方向都要会）

方向一：已知 n 个关键字，求最大高度 h

要让高度最大，就要让各层的分叉尽可能少：根只有 2 个分叉，其余每个非叶结点只有 $\lceil m/2 \rceil$ 个分叉。

含关键字的结点层数分布：第 1 层 1 个，第 2 层 2 个，第 3 层 $2\lceil m/2 \rceil$ 个，……，第 h 层 $2\lceil m/2 \rceil^{h-2}$ 个。

若约定第 h 层的下一层（第 $h+1$ 层）是叶结点（失败结点）层，则第 $h+1$ 层共有

$$2\left\lceil \frac{m}{2} \right\rceil^{h-1}$$

个叶结点。而 n 个关键字恰好把值域切成 $n+1$ 个区间，所以叶结点数必为 $n+1$：

$$n + 1 \ge 2\left\lceil \frac{m}{2} \right\rceil^{h-1}$$

两边取对数：

$$\boxed{h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1}$$

方向二：已知 n 个关键字，求最小高度 h

要让高度最小，就要让每个结点尽可能满（$m-1$ 个关键字、m 个分叉）：

$$n \le (m-1)\left(1 + m + m^2 + \cdots + m^{h-1}\right) = m^h - 1$$

$$\boxed{h \ge \log_m (n+1)}$$

合并结论：含 n 个关键字的 m 阶 B 树，高度满足

$$\log_m (n+1) \le h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$$

用哪个公式要看清题目对「高度」的定义

- 题目说「高度为 2」并配图，一般表示根在第 **1** 层、最下面一层含关键字的结点在第 **2** 层（不含空结点层）。
- 题目说「不包含空结点层」「根为第 1 层」时，用上面的公式。
- 若题目把空结点层也算进高度，所有高度要再 +1，公式也要相应平移。

做 B 树的高度题，第一件事是数清楚题目给的「高度」是否包含空结点层。

算例 **1**：5 阶 B 树有 53 个关键字，高度最高是多少（不含空结点层）？

$h \le \log_3 \dfrac{53+1}{2} + 1 = \log_3 27 + 1 = 4$。答案 **4**。

算例 **2**：高度为 2 的 5 阶 B 树最少含多少关键字？根 1 个关键字 +2 棵子树，每棵子树最少 $\lceil 5/2 \rceil - 1 = 2$ 个关键字，故 $1 + 2 \times 2 = 5$。答案 **5**。

算例 **3**：高度为 5 的 3 阶 B 树最少含多少关键字？3 阶树的非根结点最少 1 个关键字、2 棵子树，各层最少结点数 $1, 2, 4, 8, 16$，合计 31，每结点 1 个关键字。答案 **31**。

算例 **4**：一棵有 20 个关键字的 3 阶 B 树，含关键字的结点个数最多是几个？

3 阶树每结点最多 2 个关键字。要让结点数最多，就让每个结点只放 1 个关键字：此时含关键字的结点构成满二叉树形，$2^5 - 1 = 31 \ge 20 \ge 2^4 - 1 = 15$，$h = 5$ 时 31 个位置足以放 20 个关键字。

关键是不能只放 **10** 个结点（$20 \div 2$）——若某层放满 2 个关键字、另一层只放 1 个，叶结点就不在同一层了，违反 B 树性质。所以正确答案是 **20**（2014 年题的陷阱正是「叶结点必须在同一层」）。

算例 **5**：具有 15 个关键字的 4 阶 B 树，结点个数最多是多少？每个结点放 1 个关键字，构成满二叉形（$2^4 - 1 = 15$），满足同层要求。答案 **15**。

算例 **6**：高度为 3 的 3 阶 B 树，根为第 1 层，第 2 层有 4 个关键字，则结点个数最多是多少？

根最多 3 棵子树 $\Rightarrow$ 第 2 层最多 3 个结点。设这 3 个结点的关键字数为 $n_1, n_2, n_3 \ge 1$，$n_1+n_2+n_3 = 4$。第 3 层结点数 $= \sum(n_i + 1) = 4 + 3 = 7$。总结点数 $= 1 + 3 + 7 = 11$。

答案 **11**。注意第 3 层结点数只由「子树数 = 关键字数 + 1」和第 2 层关键字总数决定，与关键字在各结点间的分布无关。

（二）B 树的查找

B 树的查找与 BST 类似，只是每个结点内部要在有序表中定位。

设查找关键字为 x，从根开始：

- 基本情况：
  - 搜索到的结点为空（到达最下层空结点）$\Rightarrow$ 查找失败，结束；
  - 结点非空且包含 $x \Rightarrow$ 查找成功，结束。
- 迭代情况：结点非空且不含 x，该结点为 $(n, A_0, K_1, A_1, \cdots, K_n, A_n)$：
  - 若 $x < K_1$，进入孩子 A_0；
  - 若 $K_i < x < K_{i+1}$（$i = 1, \ldots, n-1$），进入孩子 A_i；
  - 若 $x > K_n$，进入孩子 A_n。

B 树查找与 B+ 树查找的根本区别

B 树的每个结点里存的都是真实存在的关键字，所以在内部结点命中就立刻成功，不必走到最下层。

B+ 树只有叶结点存记录信息，所以无论成功还是失败，查找都必须走到最下面一层。

查找效率：结点内部关键字有序，可用线性查找（$O(n)$）或折半查找（$O(\log n)$）。

设树高为 h，则查找路径长度至多为 h。若结点内部采用折半查找，查找的代价为

$$O(\log m) \cdot O(\log_m n) = O(\log n)$$

（三）B 树的插入

要领：先查找确定插入位置，一定从终端结点（最下层含关键字的结点）插入；若插入后关键字数超过上限 $m-1$，则把当前结点的中间关键字（第 $\lceil m/2 \rceil$ 个）上移到父结点，当前结点分裂成左右两部分；父结点关键字数 +1 后若也超限，则继续向上分裂。

第一步：查找插入位置并插入

- 若 B 树为空：新建结点作为根，放入关键字；
- 否则：从根开始单程向下查找，一直到最下层含关键字的结点为止，把关键字按序插入该结点。

第二步：调整以维护 **B** 树性质

- 不分裂（仅插入）：被插入结点关键字个数 $\leq m-1$，无需调整，结束。若插入前树为空、新建了根，树高 +1。
- 分裂（插入后分裂，需要迭代）：被插入结点关键字个数达到 m（超上限），需要分裂：
  1. 取该结点的第 $\lceil m/2 \rceil$ 个关键字上移插入父结点；
  2. 第 1 到 $\lceil m/2 \rceil - 1$ 个关键字（及对应的孩子指针）留在左结点；
  3. 第 $\lceil m/2 \rceil + 1$ 到 m 个关键字（及对应的孩子指针）移到新建的右结点；
  4. 把「上移的关键字」和「父结点」分别作为新的「待插入关键字」和「待插入结点」，返回第一步迭代，直到出现「不分裂」情形；
  5. 若分裂一直传导到根，导致根也分裂，则新建一个根，树高 +1。这是 B 树长高的唯一方式。

中间关键字是「下中位数」

上移的是关键字数组的下中位数位置，即第 $\lceil m/2 \rceil$ 个：

- $m=4$（4 阶）：结点最多 3 个关键字，插入后变 4 个，上移第 $\lceil 4/2 \rceil = 2$ 个关键字；左 1 个、右 2 个。
- $m=5$（5 阶）：结点最多 4 个关键字，插入后变 5 个，上移第 $\lceil 5/2 \rceil = 3$ 个关键字；左 2 个、右 2 个。

分裂后的左孩子通常在原结点上修改，右孩子需要新建结点。

算例：往 4 阶 B 树中插入 66。4 阶树每结点最多 3 个关键字，若终端结点原本是 $\{60, 64, 70\}$，插入后变 $\{60, 64, 66, 70\}$ 超限：

1. 上移第 $\lceil 4/2 \rceil = 2$ 个关键字 **64** 到父结点；
2. 左结点保留 $\{60\}$，右结点保留 $\{66, 70\}$；
3. 父结点若因此超限，继续向上分裂。

*1.*插入操作的推论（*2023* 年真题考点）

- 插入操作可能增加树的高度（对：根分裂时新建根，树高 +1）；
- 插入的新关键字最终不一定留在叶结点（对：它可能被同级分裂推到父结点，甚至更高层）。「插入总是发生在最下层」是对的，但「新关键字最终留在最下层」是错的。

（四）B 树的删除

删除是 B 树所有基本操作中最难的，分两步：先删除（可能先做替换），后调整。

第一步：查找并删除

- 若关键字位于终端结点：直接删除；
- 若关键字位于非终端结点：用它的直接前驱或直接后继替代该关键字，从而把对非终端结点的删除转化为对终端结点的删除：
  - 直接前驱 = 该关键字左侧指针所指子树中「最右下」的元素；
  - 直接后继 = 该关键字右侧指针所指子树中「最左下」的元素。

第二步：调整以维护 **B** 树性质

设删除后该结点的关键字数为 n。

- 情况 **1**：直接删除（$n \geq \lceil m/2 \rceil - 1$，仍满足下限）：在该结点内把后面的关键字前移即可，结束。
- 情况 **2**：需要调整（$n < \lceil m/2 \rceil - 1$，下溢）：
  - 情况 **2.1**「兄弟够借」（旋转）：若右兄弟关键字个数 $\geq \lceil m/2 \rceil$（借走一个仍合法），则做旋转——把父结点中分隔这两个兄弟的关键字下移到被删结点，把右兄弟中的最小关键字上移到父结点。若右兄弟不够借而左兄弟够，则镜像处理（把左兄弟中的最大关键字上移父结点，把父结点中的分隔关键字下移到被删结点）。本质就是「用当前结点的后继（或前驱）来填补空缺」。
  - 情况 **2.2**「兄弟不够借」（合并）：若左右兄弟都恰含 $\lceil m/2 \rceil - 1$ 个关键字，则把被删结点、其一个兄弟结点、以及父结点中分隔它们的关键字三者合并成一个结点。
    - 合并后父结点关键字数减 1。若父结点是根且关键字数减到 0（原根只有 1 个关键字、2 棵子树），则直接删除根，合并后的新结点成为新的根，树高减 **1**。
    - 若父结点不是根且关键字数减到 $\lceil m/2 \rceil - 2$（下溢），则对父结点重复上述「借 / 合并」过程，继续向上传导，直到满足 B 树要求。

B 树删除的三个易错点

1. 删除非终端结点关键字不是直接删，必须先用前驱或后继替换，再删终端结点里的那个值。
2. 合并是「三方合并」：左（或右）兄弟 + 父结点中的分隔关键字 + 当前结点，三者合成一个结点，不要漏掉父结点中的那个关键字。
3. 根的特殊处理：根的关键字数下限是 1（不是 $\lceil m/2 \rceil - 1$），所以根下溢到 0 时要删根、降低树高，而不是去和兄弟借。

关于「删一个关键字的结果是否唯一」：删除非终端结点的关键字时，用前驱还是用后继替换会造成不同的中间形态。2022 年那道 5 阶 B 树删 260 的题，四个选项分别对应「用后继 + 与右兄弟合并」「用后继 + 与左兄弟合并」「用前驱 + 左兄弟够借」「用前驱 + 右兄弟合并」四种路径。但从最终 **B** 树的合法性与算法确定性角度看，只需按 B 树性质逐一验证选项是否合法即可——把所有可能路径都画出来，看哪个选项构造不出合法的 5 阶 B 树。

（五）B 树的判定

判断一棵树是不是合法的 m 阶 B 树，三步：

1. 查平衡：所有叶结点（含关键字的结点层）必须都在同一层；
2. 查有序：用环线法做「中序遍历」，得到的关键字序列必须单调递增；

3. 查关键字个数：对除根外的每个结点，关键字个数 n 满足 $\lceil m/2 \rceil - 1 \le n \le m-1$；根结点满足 $1 \le n \le m-1$。

2022 年那道题的最好解法就是「判定法」

「在 5 阶 B 树中删除 260 得到 T_1，下列不可能的是 T_1 根结点的关键字序列」。从删除算法入手很繁琐，正确做法是：按每个选项反推出整棵 **B** 树，再检查它是否满足 **5** 阶 **B** 树的性质（$m=5$ 时非根结点关键字数须满足 $2 \le n \le 4$）。选项 D 中含 100 的那个结点只有 1 个关键字，违反 $2 \le n \le 4$，所以不可能。

（六）B+ 树的基本概念

B+ 树的产生源于操作系统的文件索引和数据库索引。可以把它理解成「分块查找的进化——多级分块查找」。

一棵 m 阶 B+ 树满足：

1. 每个分支结点最多有 m 棵子树（孩子结点）；
2. 非叶根结点至少有两棵子树，其他每个分支结点至少有 $\lceil m/2 \rceil$ 棵子树；
3. 结点的子树个数与关键字个数相等（对比 B 树的「子树数 = 关键字数 + 1」）；
4. 所有叶结点包含全部关键字及指向相应记录的指针；叶结点中关键字按大小顺序排列，并且相邻叶结点按大小顺序相互链接（支持顺序查找）；
5. 所有分支结点中仅包含它的各个子结点中关键字的最大值及指向其子结点的指针（只起索引作用，不含记录信息）。

B+ 树的查找：B+ 树有两个头指针，一个指向根结点，一个指向关键字最小的叶结点，因此支持两种查找：

- 从最小关键字开始的顺序查找（沿叶结点链）——这正是 B+ 树支持范围查询的原因；
- 从根结点开始的多路查找。

B+ 树查找必然走到最下层

与 B 树「查找成功可能停在任意一层」不同，**B+** 树中无论查找成功与否，最终一定要走到最下面一层结点，因为只有叶结点包含信息（记录指针）。因此 B+ 树的查找速度稳定，而 B 树不稳定。

（七）B 树与 B+ 树的对比

对比项	m 阶 **B** 树	m 阶 **B+** 树
类比	二叉查找树的进化——m 叉查找树	分块查找的进化——多级分块查找
关键字与分叉	n 个关键字对应 $n+1$ 棵子树	n 个关键字对应 n 棵子树
结点存的信息	所有结点都包含记录的信息（含记录的存储地址）	只有最下层叶结点包含记录信息；非叶结点仅起索引作用，只存「对应子树的最大关键字 + 指针」
根结点关键字数（非叶根）	最少 1 个，最多 $m-1$ 个	最少 **2** 个，最多 m 个（根若同时是叶结点，可以只有 1 个关键字）
非根结点	最少 $\lceil m/2 \rceil - 1$ 个关键字	至少 $\lceil m/2 \rceil$ 棵子树
查找路径	查找成功时可能停在任意一层，速度不稳定	成功或失败都必到最后一层，速度稳定

对比项	m 阶 **B** 树	m 阶 **B+** 树
顺序查找	不支持	支持（叶结点之间有顺序链）
关键字重复	各结点中的关键字互不重复	叶结点包含全部关键字；非叶结点出现过的关键字也会再次出现在叶结点中
应用场景	文件系统、需要随机访问内部结点数据的场景	关系数据库索引、需要范围查询与顺序扫描的场景
相同点	除根外最少 $\lceil m/2 \rceil$ 个分叉（避免结点太「空」）；所有结点的子树一样高（绝对平衡，所有叶结点同层）	同左

B+ 树为什么适合数据库索引

非叶结点不存记录信息，只存最大关键字和指针，故一个磁盘块能装下更多关键字，B+ 树阶更大、树更矮，读磁盘次数更少；叶结点有序链接又天然支持范围查询（`BETWEEN`、`ORDER BY`）与顺序扫描。

B 树的结点要携带记录地址，同样大小的磁盘块装下的关键字少，阶数小、树高更大，且不支持高效范围扫描。这是数据库普遍选 B+ 树的原因。

七、散列（**hash**）表

（一）基本概念

散列表（**Hash Table**，哈希表）：一种根据数据元素的关键字直接计算出它在表中的存储地址的数据结构，是典型的以空间换时间。

散列函数（哈希函数）：$Addr = H(key)$，建立「关键字 $\to$ 存储地址」的映射。

冲突（碰撞）：插入元素时，若根据关键字算出的地址已经被其他元素占用，称为冲突。

同义词：若不同的关键字通过同一个散列函数映射到同一个地址，则称这些关键字互为同义词。

装填因子 α：

$$\alpha = \frac{\text{表中记录数} n}{\text{散列表长度} m}$$

装填因子的地位

散列表的平均查找长度依赖于装填因子 α，而不直接依赖于 n 或 m。

α 越大，表越「满」，冲突可能性越大，ASL 越大。为什么题目问「影响散列表 ASL 的因素」时答案包含装填因子、散列函数、冲突解决策略三项——因为它们分别决定了表的拥挤程度、初始分布的均匀性、以及冲突后的探测代价。

反过来说：不能通过「增大装填因子」来提高查找效率（那是 2011 年那道题的干扰项）。

（二）散列函数的构造方法

设计散列函数要注意四点，四个反面例子很值得记住：

要求	反面例子	问题
定义域必须涵盖所有可能出现的关键字	$H(key) = \sqrt{key} \bmod 13$	不支持负值

要求	反面例子	问题
值域不能超出散列表地址范围	$H(key) = key \bmod 15$（表长 13）	可能映射到非法地址 13、14
尽量减少冲突，地址尽量均匀分布	$H(key) = (key \times 13) \bmod 13$	任何关键字都被映射到 0
散列函数应尽量简单、能快速计算	$H(key) = (key!) \bmod 13$	关键字大时阶乘计算耗时极高

（法一）除留余数法：$H(key) = key \bmod p$

表长为 m 时，取不大于 m 但最接近或等于 m 的质数 p。适用场景最广，只要关键字是整数即可。

为什么对质数取余：取余会被公因子影响。若 p 有多个因子，关键字与 p 的公共因子会导致分布不均；取质数能最大程度均匀分布、减少冲突。

除留余数法算 $ASL_{失败}$ 时分母取 p 还是 m?

取 $\min(m, p) = p$。因为初次散列只可能落在 $0 \sim p-1$ 这些地址上，p 及以后的地址永远不会是「初次探测点」，不属于任何一种失败情况。

例：表长 $m = 11$，$H(key) = key \bmod 7$，则 $ASL_{失败}$ 的分母是 **7**，不是 11。这是 2019 年那道题的得分点。

而 $ASL_{成功}$ 的分母始终是关键字的个数 n。

（法二）直接定址法：$H(key) = key$ 或 $H(key) = a \cdot key + b$。计算最简单且不会冲突；缺点是关键字分布不连续、空位多时浪费空间。适用场景：关键字分布基本连续。

（法三）数字分析法：选取数码分布较为均匀的若干位作为散列地址。关键字是 r 进制数时，某些数位上各种数码出现机会均等，某些数位上只有少数几种数码常出现，应选取分布均匀的那些位。

典型例子：以「手机号码」为关键字把用户信息存入长度 10000 的散列表——手机号前几位（运营商、地区号段）重复度高，应取分布较均匀的后四位。

（法四）平方取中法：取关键字平方值的中间几位作为散列地址。结果的每一位都受原关键字各位影响，故分布较均匀。适用场景：关键字每一位取值都不够均匀。典型例子：以「底盘号 + 发动机号 + 外观号」为车型标识，把 100 种车型存入长度 100 的散列表，取拼接后平方值的中间若干位。

（三）处理冲突的方法一：拉链法

拉链法（又称链接法、链地址法）：把所有同义词存储在同一个链表中。

插入两步：

1. 计算新元素的散列地址；
2. 把新元素插入该地址对应的链表（头插法或尾插法皆可）。

例：散列表长度为 13，$H(key) = key \bmod 13$，用拉链法依次插入

$$\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$$

各地址上的链表为（下表按头插法的最终状态给出，即新元素插在链头）：

地址	0	1	2	3	4	5	6	7	8	9	10	11	12
链表	空	79 → 27 → 1 → 14	空	55 → 68	空	84 → 19	空	20	空	空	23 → 10	11	空

映射关系（key mod 13）：$14, 1, 27, 79$ 都映射到 **1**；$68, 55$ 都映射到 **3**；$19, 84$ 都映射到 **5**；20 映射到 **7**；$23, 10$ 都映射到 **10**；11 映射到 **11**。

若改用尾插法，链中顺序与插入顺序一致，但链长不变。链中元素的先后顺序只影响「第几次比较命中」，不影响链长，因此不影响 $ASL_{失败}$。

下面第 7 节的算例会给出完整的 ASL 计算。

拉链法算查找长度的关键规则

若该地址为空，则查找长度为 **0**——空指针的比较不计入查找长度。

这是拉链法与开放定址法最本质的区别：

- 拉链法：探测到空链表时，那个「空指针」的比较不算一次关键字比较，所以查找长度为 0。
- 开放定址法：探测到空单元时，那个空单元的比较要计入查找长度（因为要先读一次该单元、发现它为空，才能判定失败）。

对比两种方法算 $ASL_{失败}$ 时，这一点必须分清，否则数字会差 1。

拉链法的其他要点：

- 删除：算出地址，顺序查找该链表后按链表删除即可——可直接删除，不需要「删除标记」（相对开放定址法的优势）。
- 优化：插入时保持链表有序可提高效率；链表很长时可转换成 **AVL** 树或红黑树，把查找从 $O(n)$ 降到 $O(\log n)$（Java `HashMap` 即如此）。
- 经验公式：$ASL_{成功} \approx 1 + \dfrac{\alpha}{2}$，$ASL_{失败} \approx \alpha$。统考一般不要求套公式，而是按实际链表逐个数比较次数。

（四）处理冲突的方法二：开放定址法

主要思想：发生冲突时，给新元素另外找一个空闲位置。

为什么叫「开放定址」：一个散列地址，既对同义词开放，也对非同义词开放。

思路是确定一个探测顺序，从初始散列地址出发寻找下一个空闲位置：

$$H_i = \big(H(key) + d_i\big) \bmod m$$

其中 d_i 是增量序列，表示第 i 次冲突时下一个探测地址与初始散列地址的相对偏移量；m 是表长。

按 d_i 的取法分四种：

① 线性探测法：$d_i = 1, 2, 3, \ldots, m-1$

- 插入：算地址后若冲突，顺序查看下一单元（到表尾后回绕表头），直到找到空闲单元或查遍全表。
- 查找：按探测序列比较，命中则成功；探测到空单元则失败。
- 删除：不能简单置空，必须打「已删除」标记。

② 平方探测法（二次探测再散列）：$d_i = 1^2, -1^2, 2^2, -2^2, 3^2, -3^2, \ldots, \pm k^2$

③ 双散列法(再散列法)：$d_i = i \cdot hash_2(key)$，需两个散列函数，$H_i = (H(key) + i \cdot hash_2(key)) \bmod m$。常用套路是令表长 m 为质数、$hash_2(key) = m - (key \bmod m)$，这样 $hash_2(key)$ 与 m 一定互质，保证能探测到所有单元。

④ 伪随机序列法：d_i 取一个伪随机序列

四种探测方法的覆盖率对比

方法	能否保证探测到所有位置
线性探测法	一定可以。只要表中有空闲位置，就一定能插入成功（最坏冲突 $m-1$ 次即可走遍全表）
平方探测法	至少能探测到表中一半的位置；即便表中有空闲位置，也未必能插入成功
双散列法	取决于 $hash_2(key)$ 的设计；若 $hash_2(key)$ 与 m 互质，则能探测所有单元
伪随机序列法	取决于伪随机序列设计是否合理

开放定址法删除元素：算出散列地址，按探测序列找到匹配的关键字后删除。

开放定址法必须用「删除标记」

删除元素不能简单地把该位置置为空，否则会截断它之后的探测路径——之后查找其他元素时，探测到这个空位就会误判为「不存在」，而实际目标还在更后面。

正确做法是设一个「已删除」标志位做逻辑删除。被标记为已删除的空间可以再次使用：插入新元素时若探测到「已删除」位置，可直接占用，既不破坏探测序列连续性，也不浪费空间。

2023 年真题正考这一点：表长 5，$H(k) = (k+4) \bmod 5$，依次插入 2022、12、25 后删除 25，求 $ASL_{失败}$。删除后地址 4 是「删除标记」而不是空位，故从地址 4 出发的失败查找要越过标记再探测地址 **0**，比「真为空」多一次比较。

（五）线性探测的堆积（聚集）现象

堆积（聚集、**clustering**）：线性探测时，冲突后的元素一定被放在某个连续的位置上。数组中连续被占用的区域越长，这块区域发生哈希冲突的可能性就越大，进一步促使聚堆生长，形成恶性循环，最终导致增删查改效率劣化。

堆积现象的特点

- 线性探测法最容易产生堆积，因为它只向相邻的下一格探测；
- 堆积是「同义词 + 非同义词」共同造成的；纯同义词（如拉链法的一条链）不叫堆积；
- 堆积直接影响的是「平均查找长度」，不影响存储效率、散列函数、装填因子；
- 平方探测法、双散列法可以缓解堆积，但双散列法只是缓解，不能完全避免。

（六）平方探测法与表长 $4k+3$

平方探测法的增量序列是

$$d_i = 1^2,\ -1^2,\ 2^2,\ -2^2,\ 3^2,\ -3^2,\ \ldots,\ \pm k^2$$

为什么要求表长是 $4k+3$ 形式的素数

平方探测只能保证探测到表中至少一半的位置。也就是说，即使表里还有空位，也可能插不进去。

若表长 m 是一个可以表示成

$$m = 4k + 3$$

的素数（例如 $m = 7, 11, 19, 23$），则 $m \equiv 3 \pmod 4$。此时：

- 平方序列 $\{i^2 \bmod m\}$ 的取值集合恰好是 $\{0, 1, 2, \ldots, m-1\}$，即覆盖全部 m 个位置；
- 也就是说，只要表中有空位，平方探测一定能找到它。

反之：

- $m \equiv 1 \pmod 4$ 时，平方序列只能覆盖 $\{0, 1, \ldots, m-1\}$ 中的一部分位置（例如 $m = 13$ 时，$\{i^2 \bmod 13\} = \{0, 1, 3, 4, 9, 10, 12\}$，只覆盖 7 个位置）；
- m 不是素数时，情况更复杂，也可能只覆盖一半。

结论：考试中记「表长为 $4k+3$ 的素数时，平方探测可探测到全部位置」即可。

（七）散列表 ASL 的计算（三个完整算例）

*1.*算例 *1*：平方探测法，$ASL_{\text{成功}}$

设关键字 $(9, 1, 23, 14, 55, 20, 84, 27)$，散列函数 $H(key) = key \bmod 7$，表长 10，用平方探测再散列 $H_i = (H(key) + d_i) \bmod 10$，$d_i = 1^2, 2^2, 3^2, \ldots$ 构造散列表并求 $ASL_{\text{成功}}$。

key	**9**	**1**	**23**	**14**	**55**	**20**	**84**	**27**
$H(key)$	2	1	2	0	6	6	0	6
第一次冲突 $+1$			3			7	1	7
第二次冲突 $+4$							4	0
第三次冲突 $+9$								5
实际位置	2	1	3	0	6	7	4	5
成功比较次数	1	1	2	1	1	2	3	4

散列表状态（地址 $0 \sim 9$）：

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**
14	1	9	23	84	27	55	20		

$$ASL_{\text{成功}} = \frac{1+1+2+1+1+2+3+4}{8} = \frac{15}{8}$$

*2.*算例 *2*：线性探测法，$ASL_{\text{成功}}$ 与 $ASL_{\text{失败}}$

设表长 $m = 15$，散列函数 $H(key) = key \bmod 13$，关键字集合 $(53, 17, 12, 61, 89, 70, 87, 25, 64, 46)$，用线性探测法处理冲突，求 $ASL_{\text{成功}}$ 与 $ASL_{\text{失败}}$。

key	**53**	**17**	**12**	**61**	**89**	**70**	**87**	**25**	**64**	**46**
$H(key)$	1	4	12	9	11	5	9	12	12	7
第一次冲突 $+1$							10	13	13	
第二次冲突 $+2$									14	
实际位置	1	4	12	9	11	5	10	13	14	7
成功比较次数	1	1	1	1	1	1	2	2	3	1

散列表状态：

0	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**
	53			17	70		46		61	87	89	12	25	64

$$ASL_{\text{成功}} = \frac{1+1+1+1+1+1+2+2+3+1}{10} = \frac{14}{10} = \frac{7}{5} = 1.4$$

$ASL_{\text{失败}}$：散列函数值域是 $0 \sim 12$（$p = 13$），所以分母是 **13**，不是 15。对每个可能的初始地址 i，从 i 出发一直线性探测到第一个空位，数比较次数（空位的比较要计入）：

初始地址	探测路径	比较次数
0	0（空）	1
1	1, 2（空）	2
2	2（空）	1
3	3（空）	1
4	4, 5, 6（空）	3
5	5, 6（空）	2
6	6（空）	1
7	7, 8（空）	2
8	8（空）	1
9	9, 10, 11, 12, 13, 14, 0（空）	7
10	10, 11, 12, 13, 14, 0（空）	6
11	11, 12, 13, 14, 0（空）	5
12	12, 13, 14, 0（空）	4

$$ASL_{\text{失败}} = \frac{1+2+1+1+3+2+1+2+1+7+6+5+4}{13} = \frac{36}{13} \approx 2.77$$

注意 $i = 2$、3、8 这些位置的探测次数为什么是 1

因为它们本身就是空位。「探测到空位」这一次访问要算一次比较，所以是 1 而不是 0。如果按「访问到元素才计数」来算就会得到 0，这是最常见的错误。

对比：拉链法在同样情形下的查找长度是 **0**（空指针不计比较）。两套规则不要混。

*3.*算例 *3*：拉链法

表长 13，$H(key) = key \bmod 13$，依次插入 $\{19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79\}$，用拉链法解决冲突，求 $ASL_{成功}$ 与 $ASL_{失败}$。

各地址的链表（按头插法，新元素插在链头，查找长度从表头算）：

地址	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
链	空	$79 \to 27 \to 1 \to 14$	空	$55 \to 68$	空	$84 \to 19$	空	20	空	空	$23 \to 10$	11	空

$ASL_{成功}$：逐个关键字数它在自己链中的位置（即比较次数）：

关键字	**14**	**1**	**23**	**68**	**20**	**84**	**27**	**55**	**11**	**10**	**79**	**19**
比较次数	4	3	2	2	1	2	3	2	1	2	1	2

即比较次数为 1 的有 $20, 11, 79$（3 个），为 2 的有 $23, 68, 84, 55, 10, 19$（6 个），为 3 的有 $1, 27$（2 个），为 4 的有 14（1 个）：

$$ASL_{成功} = \frac{1 \times 3 + 2 \times 6 + 3 \times 2 + 4 \times 1}{12} = \frac{3 + 12 + 6 + 4}{12} = \frac{25}{12} \approx 2.08$$

$ASL_{失败}$：分母为 13（散列函数值域 $0 \sim 12$ 共 13 个地址）。对每个地址，沿着链一直比较到 NULL 都查不到，比较次数即该地址的链长：

初始地址	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
链长	0	4	0	2	0	2	0	1	0	0	2	1	0

$$ASL_{失败} = \frac{0 + 4 + 0 + 2 + 0 + 2 + 0 + 1 + 0 + 0 + 2 + 1 + 0}{13} = \frac{12}{13} \approx 0.92$$

头插法 / 尾插法：影响的是「谁第几个被比到」，不影响 ASL

同一条链 $\{14, 1, 27, 79\}$：

- 头插法链序 $79 \to 27 \to 1 \to 14$：命中次数依次 $1, 2, 3, 4$（79 最前面）；
- 尾插法链序 $14 \to 1 \to 27 \to 79$：命中次数依次 $1, 2, 3, 4$（14 最前面）。

两种情况下这 4 个关键字的比较次数集合都是 $\{1, 2, 3, 4\}$，总和都是 10，ASL 完全相同——只是「哪个关键字最省事」不同。

做题时仍要按题目给定的插入方式（头插 / 尾插）逐个确定比较次数，否则若题目给的链序与你的假设不一致，个别关键字的次数会数错。

而 $ASL_{失败}$ 只与各地址的链长有关，与链内顺序完全无关。

开放定址法 **vs** 拉链法：$ASL_{失败}$ 的两套规则

	探测到空位	空位比较是否计数
开放定址法	查找失败	计数（要读一次才知道是空的）
拉链法	走到 NULL 指针，查找失败	不计数（空指针不是关键字）

同一次失败查找，开放定址法的比较次数通常比拉链法多 **1**。这是统考反复设置的对比陷阱。

（八）一个经典小题

问：有 k 个关键字互为同义词，用线性探测法把它们全部存入散列表，至少需要多少次探测?

解：第 1 个元素探测 1 次即可存入；第 2 个元素先探测到第 1 个（冲突），再探测 1 次存入，共 2 次；……第 k 个元素共探测 k 次。总计

$$1+2+\cdots+k=\frac{k(k+1)}{2}$$

答案 $\frac{k(k+1)}{2}$ 次。

散列表统考考法汇总

- 提高散列表查找效率的措施（2011 年）：设计冲突少的散列函数、处理冲突时避免堆积。增大装填因子是错的。
- 堆积直接影响什么（2014 年）：平均查找长度。不影响存储效率、散列函数、装填因子。
- 影响 ASL 的因素（2022 年）：装填因子、散列函数、冲突解决策略——三项都影响。
- 给插入序列求 $ASL_{成功}$（2018 年）：线性探测法，逐个数比较次数。
- 给插入序列求 $ASL_{失败}$（2019 年）：注意分母是 $p=7$ 而不是表长 11。
- 插入后删除再求 $ASL_{失败}$（2023 年）：关键是「删除标记」不能当空位，要越过它继续探测。
- 拉链法的 ASL：注意空链的查找长度为 0。
- 装填因子：$\alpha=n/m$，是唯一能描述「表满程度」的量。

八、字符串模式匹配

模式匹配：在主串（目标串）S 中查找模式串 T 第一次出现的位置。

（一）朴素算法（**BF** 算法）

思路：从主串的第 1 个字符起与模式串逐个比较；若某个位置失配，则主串指针回退到本次起始位置的下一位，模式串指针回到开头，重新比较。

```
int NaiveMatch(const char *s, const char *t) {
    int i = 0, j = 0;
    while (s[i] != '\0' && t[j] != '\0') {
        if (s[i] == t[j]) { ++i; ++j; }        // 继续比较后继字符
        else { i = i - j + 1; j = 0; }         // 指针回退，重新开始
    }
    if (t[j] == '\0') return i - j;            // 匹配成功
    return -1;                                 // 匹配失败
}
```

时间复杂度：设主串长 n、模式串长 m。最坏情况下，每个起始位置都要比较 m 次，起始位置有 $n-m+1$ 个，所以

$$O\big((n-m+1)\cdot m\big)=O(nm)$$

最坏情形举例：主串 `aaaaaaaaab`，模式串 `aaab`。前 3 个字符每次都匹配，第 4 个失配，然后回退重来，比较次数接近 nm。

（二）KMP 算法的思想

核心观察：当在模式串的第 j 位失配时，主串指针不回退，模式串指针回退到某个位置。

为什么可以不回退？因为失配前已经匹配的那一段 $T[0..j-1]$ 与主串是对应的。如果要让模式串继续与主串匹配，模式串必须向右滑动尽量少的距离——等价于在 $T[0..j-1]$ 中找最长的、既是真前缀又是真后缀的子串，长度记为 k，则模式串指针只需回到位置 k，主串指针原地不动。

用「甲（主串）乙（模式串）」的比喻：不匹配发生时，前面匹配的那一小段对二者是相同的。用这一小段的头部去匹配它的尾部，最长的那段就是答案。

例：主串 `abbaabbaaba`，模式串 `abbaaba`，在第 7 个字符失配。已经匹配的前缀是 `abbaab`：

- 真前缀：`a`，`ab`，`abb`，`abba`，`abbaa`（不含最后一个字符）
- 真后缀：`b`，`ab`，`aab`，`baab`，`bbaab`（不含第一个字符）

最长的相等真前缀与真后缀是 `ab`，长度 2。所以模式串指针回退到第 3 个字符（下标 2）继续比较，主串不回退。

（三）部分匹配表 PMT 与 next 数组

部分匹配表（**PMT, Partial Match Table**）：对模式串的每个位置 i，记录子串 $T[0..i]$ 的「最长相等真前缀与真后缀」的长度。

高效计算 PMT 的关键洞察

次大匹配必定落在最大匹配之中。

若 $T[0..i]$ 的最大匹配长度是 L，则次大匹配一定是前缀 $T[0..L-1]$ 的最大匹配长度，可以直接查已经算好的表。这就是 KMP 求 `next` 数组的递推来源。

算例：模式串 `abababzabababa`

子串位置	**a**	**b**	**a**	**b**	**a**	**b**	**z**	**a**	**b**	**a**	**b**	**a**	**b**	**a**
PMT	0	0	1	2	3	4	0	1	2	3	4	5	6	5

最后一个位置：前面 `abababz` 的 PMT 是 0，但整体 `abababzababab` 的最大匹配是 6（`ababab`）。末尾再来一个 `a` 时，不能直接 $6+1=7$（因为 `abababz` 的第 7 个字符是 `z`，不等于 `a`）。于是回退到次大匹配 4（`abab`），发现 `abab` 之后的字符是 `a`，与末尾的 `a` 相等，所以 PMT 为 $4+1=5$。

从 **PMT** 到 **next**：为了编程方便，不使用 `PMT` 数组本身，而是把 `PMT` 整体向右偏移一位，并把第 0 位置设为 -1，得到的新数组称为 `next` 数组。

next 数组的两种教材约定（统考必须掌握）

约定 **A**（**0** 基下标，`next[0] = -1`）——多数国内教材与代码实现的写法：

- `next[j]` 表示「$T[0..j-1]$ 的最长相等真前后缀长度」，再整体后移一位、首项置 -1；
- 失配于 j 时执行 `j = next[j]`；
- `next[0] = -1` 只是一个哨兵值，用来表示「模式串第 1 个字符就失配，主串指针后移一位」。

约定 **B**（王道教材，**1** 基下标，`next[1] = 0`）：

- `next[j]` 表示「$T[1..j-1]$ 的最长相等真前后缀长度再加 **1**」；
- 失配于 j 时令 `j = next[j]`；
- `next[1] = 0` 是哨兵，`next[j] = 0` 表示模式串已退到开头，需要主串指针后移。

两者的换算：把 0 基的 next 数组每一项加 **1**，就得到 1 基的 next 数组；反之减 1。

0 基 next	**-1**	**0**	**0**	**1**	**2**	**3**	**1**	**1**	**2**	**3**	**4**	**5**
1 基 next	0	1	1	2	3	4	2	2	3	4	5	6

考试时先看题目给的 `next[1]` 是 **0** 还是 -1，再决定用哪套。

求 **next** 数组的代码（**0** 基约定）：

```
void getnext(const char *s, int next[]) {
    next[0] = -1;
    int i = 0, j = -1;
    while (s[i + 1] != '\0') {
        if (j == -1 || s[i] == s[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];      // 回退到次大匹配
        }
    }
}
```

KMP 匹配主程序（**0** 基约定）：

```
int KMP(const char *a, const char *b, const int next[]) {
    int i = 0, j = 0;
    while (a[i] != '\0' && b[j] != '\0') {
        if (j == -1 || a[i] == b[j]) { ++i; ++j; }
        else { j = next[j]; }     // 主串指针 i 不回退
    }
    if (b[j] == '\0') return i - j;   // 匹配成功
    return -1;
}
```

（四）手算 **next** 数组的流程

以模式串 ababaaababaa 为例（0 基）：

第 **1** 步：求 **PMT**

对每个位置 i，看子串 $T[0..i]$ 的最长相等真前后缀长度。

char	a	b	a	b	a	a	a	b	a	b	a	a
index	0	1	2	3	4	5	6	7	8	9	10	11
PMT	0	0	1	2	3	1	1	2	3	4	5	6

验证几个：

- $i=4$：子串 ababa，真前缀 aba 与真后缀 aba 相等，长度 3 ✓
- $i=5$：子串 ababaa，真前缀最长匹配是 a（首尾都是 a，ab 不行因为末尾是 aa），长度 1 ✓
- $i=11$：子串 abababaababaa，真前缀 ababaa 与真后缀 ababaa 相等，长度 6 ✓

第 **2** 步：**PMT** 右移一位，首项置 -1，得到 **next**

index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**
next	-1	0	0	1	2	3	1	1	2	3	4	5

第 **3** 步：若要 **1** 基约定，每项 **+1**

index	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
next	0	1	1	2	3	4	2	2	3	4	5	6

next 数组的速算法

不需要每次都画 PMT 表。直接用定义：

$next[j]$（**1** 基约定）= $T[1..j-1]$ 的最长相等真前后缀长度 $+1$。

- $next[1]=0$（哨兵）；
- $next[2]=1$（前 1 个字符没有真前后缀，长度 0，$0+1=1$）；
- 从 $j=3$ 起：看 $T[1..j-1]$ 的最长相等真前后缀长度 k，则 $next[j]=k+1$。

（五）nextval 数组（优化）

问题：如果 $T[j]$ 失配，按 next 回退到 $j'=next[j]$，而恰好 $T[j']=T[j]$，那么这次比较必然还是失配，白比一次。

优化思路：若 $T[j]=T[next[j]]$，则把 nextval[j] 直接设为 nextval[next[j]]（递归跳过所有「回去也必然失配」的位置）。

```
void getnextval(const char *s, const int next[], int nextval[]) {
    nextval[0] = -1;                    // next[0] 为 -1，不能访问 s[-1]
    for (int i = 1; s[i] != '\0'; i++) {
        if (next[i] == -1 || s[i] != s[next[i]])
            nextval[i] = next[i];       // 字符不同，保留原 next
        else
            nextval[i] = nextval[next[i]]; // 字符相同，继续往前跳
    }
}
```

算例：$s=$ abababaababaa

index	0	1	2	3	4	5	6	7	8	9	10	11
char	a	b	a	b	a	a	a	b	a	b	a	a
next	-1	0	0	1	2	3	1	1	2	3	4	5
nextval	-1	0	-1	0	-1	3	1	0	-1	0	-1	3

逐项验证：

- $i=1$：`s[next[1]]=s[0]='a'`，`s[1]='b'`，不同 ⇒ `nextval[1] = next[1] = 0` ✓
- $i=2$：`s[next[2]]=s[0]='a'`，`s[2]='a'`，相同 ⇒ `nextval[2] = nextval[0] = -1` ✓
- $i=4$：`s[next[4]]=s[2]='a'`，`s[4]='a'`，相同 ⇒ `nextval[4] = nextval[2] = -1` ✓
- $i=5$：`s[next[5]]=s[3]='b'`，`s[5]='a'`，不同 ⇒ `nextval[5] = 3` ✓

另外几个常用练习：

模式串	**next（0 基）**	**nextval（0 基）**
ababaabab	$-1,0,0,1,2,3,1,2,3$	$-1,0,-1,0,-1,3,0,-1,0$
pqppqpqp	$-1,0,0,1,1,2,3,2$	$-1,0,-1,1,0,-1,3,-1$
abaabcac	$-1,0,0,1,1,2,0,1$	$-1,0,-1,1,0,2,-1,1$

nextval 的三个细节

1. `nextval[0]` 一定等于 `next[0]`（$=-1$ 或 **1** 基约定下的 **0**），因为不存在 `s[-1]` 可比。
2. 判断的是「字符是否相同」，不是「下标是否相同」：`s[i] != s[next[i]]` 时保留原值。
3. `nextval` 只影响比较次数，不影响匹配结果的正确性，也不改变 $O(n+m)$ 的复杂度量级——它减少的是常数因子。

（六）匹配过程与复杂度

匹配过程要点：

- 主串指针 i 只前进不回退；
- 模式串指针 j 失配时跳到 `next[j]`（或 `nextval[j]`），可能多次跳跃，但总跳跃距离不超过模式串长度；
- 匹配成功时返回 $i-j$（模式串在主串中的起始下标）。

为什么 KMP 是 $O(n+m)$

- 求 `next`（或 `nextval`）数组只扫描一遍模式串，$O(m)$；
- 匹配过程中，主串指针 i 单调递增、最多移动 n 次；模式串指针 j 每失配一次就减小，而 j 的增大量总和不超过 i 的增量，所以 j 的减小总量也是 $O(n)$。均摊下来匹配是 $O(n)$。

总计 $O(n+m)$，比朴素算法的 $O(nm)$ 好一个量级。

KMP 统考考法汇总

- 给主串、模式串，问第一次失配后 i 和 j 分别是多少（2015 年）。答：i 不变（仍是失配位置），j 变成该位置对应的 `next` 值。例：$S=$ `abaabaabacacaabaabcc`，$t=$ `abaabc`，第一次失配时 $i=j=5$。失配前已匹配的模式串前缀是 `abaab`，其最长相等真前后缀是 `ab`（长度 2）。所以 $i=5$ 不动，$j=2$。

- 问到匹配成功为止一共比较了多少次（2019 年）。做法：逐段累加「成功比较的次数 + 1 次失配 + 后续成功比较的次数」。模式串 abaabc 若前 5 个匹配、第 6 个失配，则这一段比较 6 次；回退到 $j=2$ 后续匹配若干次。总比较次数需按题目给定的主串逐字符数，不能凭公式猜。
- 问 **nextval**（修正 **next**）下模式串最大滑动距离（2024 年）。失配于 j 时，模式串向右滑动的距离为 $j-nextval[j]$（0 基约定）。对 $S=$ aabaab，0 基 nextval 为 $[-1,0,1,0,1,2]$，各位置滑动距离 $j-nextval[j]$ 依次为 $1,1,1,3,3,3$，最大为 **3**。（若按 1 基约定计算，注意换算。）

九、查找算法的分析及应用

（一）各查找方法的横向对比

查找方法	存储结构要求	是否支持动态（插入删除）	时间复杂度	$ASL_{成功}$	$ASL_{失败}$
顺序查找（无序）	顺序表或链表	插入简单，删除 $O(1)$（链表）	$O(n)$	$\frac{n+1}{2}$	n
顺序查找（有序）	顺序表或链表	同上	$O(n)$	$\frac{n+1}{2}$	$\frac{n}{2}+\frac{n}{n+1}$
分块查找	索引表（顺序）+ 块（顺序或链表）	较方便	$O(\sqrt{n})$（$s=\sqrt{n}$ 时）	$\frac{b+1}{2}+\frac{s+1}{2}$，最优 $\sqrt{n}+1$	需逐个分析
折半查找	有序顺序表	不支持（插入删除 $O(n)$）	$O(\log n)$	$\approx \log_2(n+1)-1$	$\frac{(h-1)A+h\cdot 2k}{n+1}$（逐个数更稳）
二叉搜索树	任意（链式亦可）	支持，$O(\log n)$ 平均、$O(n)$ 最坏	平均 $O(\log n)$，最坏 $O(n)$	与树形相关	与树形相关
AVL 树	任意	支持，但要旋转	$O(\log n)$ 最坏	与树形相关，$O(\log n)$	$O(\log n)$
红黑树	任意	支持，调整次数少	$O(\log n)$ 最坏	$O(\log n)$	$O(\log n)$
B 树 / **B+** 树	任意（面向磁盘）	支持（分裂/合并）	$O(\log_m n)$ 层磁盘访问	与结点内查找方式有关	与结点内查找方式有关
散列表	散列表	支持	期望 $O(1)$	与 α 和冲突处理有关	与 α 和冲突处理有关

（二）选用原则

查找方法的选择

1. 表是静态的（只查不改）+ 顺序存储 + 有序 → 折半查找。效率最高（$O(\log n)$）且不需要额外空间。
2. 表是静态的 + 链式存储 / 无序 → 顺序查找。顺序查找是唯一在链表上也能直接用的方法。
3. 表是动态的（频繁插入删除） → 树形结构。查多改少用 **AVL**；增删频繁用红黑树；数据量大、面向磁盘用 **B/B+** 树。
4. 要求 $O(1)$ 级别的等值查找，且不需要范围查询和有序遍历 → 散列表。散列表的缺点是：不支持范围查询（无序）、最坏情况下会退化（所有关键字同义词）、需要额外空间、装填因子高时 ASL 迅速上升。
5. 既要有序又要快速，且数据量大到内存放不下 → **B+** 树（数据库索引的标准选择）。
6. 数据量小 → 顺序查找反而更快（常数因子小）。
7. 需要范围查询 / 顺序遍历 → B+ 树、有序表；散列表不适合。

（三）三个必须能自己推的结论

1. 折半查找判定树的树高 $h=\lceil\log_2(n+1)\rceil$，它同时是成功与失败查找的比较次数上限。
2. 折半查找的平均查找长度：$n=2^h-1$ 时为 $h-1=\log_2(n+1)-1$（最优）；一般情形下介于 $\log_2(n+1)-1$ 与 $\lceil\log_2(n+1)\rceil$ 之间。
3. 散列表的 ASL 只依赖装填因子 α，不直接依赖 n 或 m——所以「装填因子相同，ASL 就大致相同」是散列表的核心工程性质。

本章小结表

考点	核心结论
查找长度	比较关键字的次数，不是访问结点数
ASL	$ASL=\sum P_iC_i$；成功用 n 做分母，失败用失败位置数做分母
判定树	n 个内部结点 + $n+1$ 个外部结点；失败结点查找长度 = 父结点层数
顺序查找	无序：$ASL_{成功}=\frac{n+1}{2}$，$ASL_{失败}=n$；有序：$ASL_{成功}$ 不变，$ASL_{失败}=\frac{n}{2}+\frac{n}{n+1}$
哨兵	减少一次循环判断，不改变 ASL；可放表头（王道）或表尾
分块查找	$ASL=\frac{b+1}{2}+\frac{s+1}{2}$；最优 $s=\sqrt{n}$，$ASL_{\min}=\sqrt{n}+1$；索引表命中后仍要进块内查（块内无序时）
折半查找适用条件	有序 + 顺序存储；有序链表、无序数组、静态链表都不适合
折半判定树	一定是平衡二叉树；形态唯一，只与 n 有关
mid 取整	下取整 ⇒ 右子树结点数 − 左子树结点数 $\in\{0,1\}$；上取整则相反
折半比较次数上限	$\lceil\log_2(n+1)\rceil$（成功与失败都是）
折半 $ASL_{成功}$	$\log_2(n+1)-1$（满树时取等，一般略大但小于树高）
折半 $ASL_{失败}$	分母 $n+1$；公式 $\frac{(h-1)A+h\cdot 2k}{n+1}$，其中 $k=n-(2^{h-1}-1)$、$A=n+1-2k$；满树（$n=2^h-1$）时 $A=0$，$ASL_{失败}=h$
折半失败比较次数上限	$\lceil\log_2(n+1)\rceil$（与成功查找上界相同）
BST 定义	左 < 根 < 右；中序遍历递增；无重复关键字
BST 插入	新结点一定是叶结点
BST 删除	三种情况：叶子直接删；单子树用子结点顶替；双子树用中序前驱（左子树最右下）或中序后继（右子树最左下）替换
BST 删除再插入	v 是叶结点 ⇒ 形态相同；v 非叶 ⇒ 形态一定不同
BST 与折半	折半判定树唯一；BST 树形随插入顺序变化；BST 支持动态插入删除
AVL 定义	任一结点 $\lvert BF\rvert\le 1$；空结点 $BF=0$
AVL 四种旋转	LL 右旋；RR 左旋；LR 先左后右；RL 先右后左
AVL 插入	调最小不平衡子树即可，祖先自动恢复
AVL 删除	可能多次向上调整
AVL 最少结点	$N(h)=N(h-1)+N(h-2)+1$，$N(0)=0,N(1)=1,N(2)=2$；$N(h)=F_{h+2}-1$
AVL 高度范围	最多 2^h-1 个结点；n 个结点的最大高度 $\approx 1.44\log_2 n$
红黑树五性质	口诀「左根右、根叶黑、不红红、黑路同」；注意「叶结点」指空结点
红黑树结论	最长路径 ≤ 最短路径的 2 倍；$h\le 2\log_2(n+1)$；查找 $O(\log n)$

考点	核心结论
红黑树插入	新结点染红；黑叔 → 旋转 + 染色；红叔 → 染色 + 变新（爷爷变新结点）
红黑树 vs AVL	红黑树平衡条件松、调整少，适合增删频繁；AVL 树更矮，适合以查为主
m 阶 B 树定义	每结点 $\le m$ 棵子树；非根非叶 $\ge \lceil m/2 \rceil$ 棵子树（$\ge \lceil m/2 \rceil - 1$ 个关键字）；根 ≥ 1 个关键字；所有叶结点同层
B 树关键字范围	高度 h 时最少 $2\lceil m/2 \rceil^{h-1} - 1$，最多 $m^h - 1$
B 树高度范围	$\log_m(n+1) \le h \le \log_{\lceil m/2 \rceil} \frac{n+1}{2} + 1$
B 树查找	内部结点命中即成功；失败到达空结点
B 树插入	一定从终端结点插入；超限则上移第 $\lceil m/2 \rceil$ 个关键字并分裂；可能长高（根分裂）
B 树删除	非终端结点先用前驱/后继替换；下溢则借（旋转）或合并（三方合并）；根下溢到 0 则删根降高
B 树判定	三步：叶结点同层、中序递增、关键字个数在 $[\lceil m/2 \rceil - 1, m-1]$ 内（根为 $[1, m-1]$）
B+ 树特点	子树数 = 关键字数；只有叶结点存记录信息；叶结点有序链接；查找必到最后一层
B+ 树应用	关系数据库索引（阶大、树矮、读盘少、支持范围查询）
散列函数	除留余数法 $H(key) = key \bmod p$（p 为不大于 m 的最大质数）；直接定址法；数字分析法；平方取中法
装填因子	$\alpha = n/m$；ASL 只依赖 α，不直接依赖 n 或 m
拉链法空位	空链表查找长度为 **0**，空指针不计比较；可直接删除
开放定址空位	空单元的比较计入查找长度；删除必须留标记
开放定址 $ASL_{失败}$ 分母	散列函数值域大小（通常是 p），不是表长 m
线性探测堆积	同义词 + 非同义词共同造成；直接影响 ASL
平方探测	至少能探测一半位置；表长为 $4k+3$ 的素数时可探测全部位置
双散列	只是缓解堆积，不能完全避免；$hash_2$ 与 m 互质时可探测全部
KMP next（0 基）	`next[0] = -1`；`next[j]` = $T[0..j-1]$ 最长相等真前后缀长度后移一位
KMP next（1 基）	`next[1] = 0`；`next[j]` = $T[1..j-1]$ 最长相等真前后缀长度 +1
nextval	若 $s[i] = s[next[i]]$ 则 `nextval[i]=nextval[next[i]]`，否则等于 `next[i]`
KMP 复杂度	$O(n+m)$；主串指针不回退
朴素匹配复杂度	$O(nm)$

易错清单

查找章 20 个高频陷阱

1. $ASL_{失败}$ 的分母不是 n。顺序查找是 $n+1$；折半查找是 $n+1$ 个外部结点；散列表开放定址是散列函数值域大小（常为 p）。
2. 失败结点的查找长度 = 父结点层数，外部结点自身不产生比较。把它当成第 $h+1$ 层会整体偏大。
3. 折半查找的失败比较次数上限也是 $\lceil \log_2(n+1) \rceil$，不是 $\lfloor \log_2 n \rfloor$。$n = 16$ 时答案是 5 不是 4。
4. $ASL_{成功} = \log_2(n+1) - 1$ 只在 $n = 2^h - 1$ 时取等，一般情况下要略大，且始终小于树高。

5. **mid** 的取整方式必须全程一致。判断某树是否为判定树时，逐层检查左右子树规模关系是否符合同一规则。
6. 折半查找判定树唯一，**BST** 不唯一。别把两者的性质互换。
7. 有序表用顺序查找时，只有 $ASL_{\text{失败}}$ 变小（$\frac{n}{2}+\frac{n}{n+1}$），$ASL_{\text{成功}}$ 仍是 $\frac{n+1}{2}$。
8. 哨兵不改变 ASL，只减少一次循环判断。两种哨兵位置的返回值判失败方式不同（表尾看是否等于 n，表头看是否为 0）。
9. 分块查找：索引表命中不等于查找成功，块内无序时必须进块内再查一次。
10. 分块查找最优块长 $s=\sqrt{n}$，$ASL_{\text{min}}=\sqrt{n}+1$，别漏掉 $+1$。
11. **BST** 新插入的结点一定是叶结点；对 **AVL** 树不成立（旋转会把新结点抬上去）。
12. **BST/AVL** 删除再插入：BST 中 v 是叶结点则形态相同、非叶则一定不同；**AVL** 中只能说「可能不同」，不能用「一定」。
13. **AVL** 插入只需调整最小不平衡子树（祖先自动恢复）；**AVL** 删除可能需要多次调整直到根。
14. **AVL** 的「最少结点数」和「最大高度」是同一个问题的两种问法，不要分别背两套数字。
15. 红黑树的「叶结点」是空结点（外部结点），不是带关键字的最下层结点。性质 3 说的是所有外部结点都是黑的。
16. 红黑树插入遇红叔时只染色不旋转，并把祖父当成新结点继续向上处理；传到根时把根强制染黑。
17. m 阶 **B** 树的关键字数下限是 $\lceil m/2\rceil-1$（非根）和 **1**（根）。给定结点数求「结点数最多」时，别忘了「所有叶结点必须在同一层」这个约束，不能简单用 $n/(m-1)$。
18. **B** 树删除非终端结点关键字必须先替换（前驱或后继），替换后再删终端结点里的值；合并时别漏掉父结点中的分隔关键字（三方合并）。
19. **B+** 树查找无论成功失败都要走到最下一层；B 树可能在内部结点就成功。B+ 树的「子树数 = 关键字数」，B 树是「子树数 = 关键字数 + 1」。
20. 散列表的三组对比：① 开放定址法空位计入比较（拉链法空指针不计）；② 开放定址法删除要留标记（拉链法可直接删）；③ $ASL_{\text{失败}}$ 的分母是 p 不是 m。堆积影响的是 ASL，不是装填因子。

第七章排序

本章考纲要求

> 七、排序（一）排序的基本概念（二）直接插入排序（三）折半插入排序（四）起泡排序（bubble sort）（五）简单选择排序（六）希尔排序（shell sort）（七）快速排序（八）堆排序（九）二路归并排序（merge sort）（十）基数排序（十一）外部排序（十二）排序算法的分析与应用

排序是「算法题 + 选择题」双线考察的章节。选择题集中在前十二个条目的过程模拟（给一趟结果反推算法、给一趟结果反推增量、问比较/移动次数）与算法选择；应用题则考察堆的手工建堆与调整、快排划分、基数排序手工模拟、外部排序的置换-选择与最佳归并树。因此本章的写法是：每个算法都给出可直接照做的分步流程，并用带具体数字的序列把每一趟的状态写出来。

一、排序的基本概念

（一）什么是排序

排序（**sorting**）：重新排列表中的元素，使表中元素按关键字递增（或递减）有序。设含 n 个记录的序列为 $\{R_1, R_2, \ldots, R_n\}$，其对应关键字为 $\{K_1, K_2, \ldots, K_n\}$，则排序即确定一种排列 $p_1, p_2, \ldots, p_n$，使 $K_{p_1} \le K_{p_2} \le \cdots \le K_{p_n}$（升序）。

若待排序列中存在两个及以上关键字相同的记录，排序后它们之间的相对次序可能改变，这就是稳定性问题的来源。

（二）稳定性（**stability**）

稳定性的定义

对序列中任意两个关键字相等的记录 R_i 与 R_j（$K_i = K_j$ 且排序前 $i < j$），若排序后 R_i 仍排在 R_j 之前，则该排序算法稳定；否则不稳定。

- 只有「相等关键字元素在原始序列中的相对次序保持不变」才叫稳定。
- 稳定与否只取决于算法本身，与关键字的具体取值无关。讲稳定性时必须给出存在两个相等关键字的例子，否则无从谈起。

判断一个算法是否稳定，最可靠的方法不是在脑子里试一个例子，而是分析算法的核心操作：

1. 该操作是否会把某个元素跨越另一个与它关键字相等的元素？
2. 比较时用 $<$ 还是 $\le$？若在两元素相等时选择「不动」，通常有利于保持稳定。

稳定性的意义在于多关键字排序。例如先按总分排序、再按语文成绩排序，若第二次排序是稳定的，则语文成绩相同的记录仍保持总分从高到低的次序，等价于一次性完成了「按语文为主关键字、总分为次关键字」的排序。

稳定性易错点

- 折半插入排序是稳定的，容易误判为不稳定。它在折半查找时用的是「小于等于则向右」的策略，使得相等元素被插到已有序的相等元素右边，顺序不变。
- 希尔排序、简单选择排序、快速排序、堆排序不稳定；直接插入、折半插入、起泡、归并、基数排序稳定。

- 「某算法存在稳定版本与不稳定版本」时，统考按稳定理解（例如交换时判 $\geq$ 则不交换）。
- 排序算法的稳定性与「某一趟结果是否稳定」无关，别把「一趟排序后序列没变」当成稳定的证据。

（三）内部排序与外部排序

- 内部排序（**internal sort**）：待排序记录全部存放在内存中，排序全过程在内存完成。前面十种算法都是内部排序。
- 外部排序（**external sort**）：待排序文件很大，内存无法一次容纳全部记录，排序过程中需要多次在内存与外存之间交换数据。外部排序的时间主要由读写外存次数决定，而非比较次数。

内部排序关心「比较次数 + 移动次数」，外部排序关心「磁盘 I/O 次数」，这是两者分析指标的根本区别。

（四）比较次数与移动次数

评价一个内部排序算法，除了渐进时间复杂度，统考经常要求精确计数：

- 比较次数：关键字之间进行大小比较的次数。
- 移动次数：记录整体搬移的次数。一次关键字赋值算一次移动；一次 `swap` 需要 3 次移动（$temp \leftarrow a$；$a \leftarrow b$；$b \leftarrow temp$）。

三种计数模式要分清

1. 与初始状态无关：简单选择排序的比较次数恒为 $\frac{n(n-1)}{2}$；折半插入排序的比较次数只与 n 有关；基数排序的移动次数只与 n 和位数 d 有关。
2. 与初始状态有关：直接插入、起泡、希尔、快速、堆排序的比较/移动次数都随初始序列状态变化。
3. 最好情况：直接插入与优化后的起泡在完全有序时只需 $n-1$ 次比较、0 次移动，达到 $O(n)$。

还有一个常用工具——逆序数。若 $i<j$ 而 $a_i>a_j$，称 (a_i,a_j) 构成一个逆序对，逆序对的总数即逆序数。排序过程就是不断消除逆序对直到逆序数为 0 的过程。

- 只做相邻交换的算法（起泡、直接插入），一次交换/右移最多消除一个逆序，所以平均交换次数为 $O(n^2)$，算法不可能优于 $O(n^2)$。
- 能做长距离移动的算法（希尔、快速、堆、归并），一次操作可能同时消除多个逆序，因此能够突破 $O(n^2)$。

（五）基于比较的排序的下界

起泡、插入、选择、希尔、快速、堆、归并都属于基于比较的排序（comparison sort）：只能通过关键字之间的比较来确定相对次序。这类算法的时间下界是 $\Omega(n\log n)$。

决策树证明

把算法对 n 个元素的所有比较过程画成一棵二叉决策树：每个内部结点是一次比较，两条分支对应两种比较结果；每个叶结点对应一个可能的输出排列。n 个元素共有 $n!$ 种排列，因此叶结点数 l 满足 $n!\leq l$。设树高为 h，二叉树的叶结点数最多 2^h，故

$$n!\leq l\leq 2^h \implies h\geq\lceil\log_2(n!)\rceil=\Omega(n\log n)$$

即最坏情况下至少需要 $\Omega(n\log n)$ 次比较。用斯特林公式 $n!\approx\sqrt{2\pi n}\,(n/e)^n$ 可得 $\log_2(n!)\approx n\log_2 n-1.44n+O(\log n)$。

结论：任何基于比较的排序算法，最坏时间都不能低于 $O(n\log n)$。想达到线性时间，必须放弃比较，改用「按关键字的值直接确定位置」的办法——这就是基数排序。

真题考法

2019 年第 7 题直接考察「选择排序算法时还需考虑哪些因素」，正确答案是数据规模、存储方式、稳定性、初始状态四项都要考虑。这类题要求你对每个算法的「适用条件」有清晰记忆，而不是只记复杂度。

2012 年第 10 题、2023 年第 10 题、2015 年第 9 题、2017 年第 11 题分别考察「每趟至少确定一个元素最终位置的算法有哪些」「移动次数与初始状态无关的算法」「更换为链式存储后效率降低的算法」「不稳定的算法有哪些」，本质都是本章（十二）那张对比表的内容。

二、直接插入排序

（一）算法思想

直接插入排序（**Insertion Sort**）：每次将一个待排序的记录按其关键字大小插入到前面已经排好序的子序列中，直到全部记录插入完成。

整个序列在逻辑上被划为两部分：左侧的有序区（初始只有第 1 个元素）和右侧的无序区。每趟取无序区的第一个元素，在有序区中从右往左找到插入位置，把它插进去。n 个元素共需 $n-1$ 趟。

（二）手工过程（逐步模拟）

以序列 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例（49^* 表示与前面的 49 关键字相同、用来观察稳定性的第二个记录）：

趟次	待插入元素	插入过程	本趟结束后序列
初始	—	49 视为已排序	`[49] 38 65 97 76 13 27 49*`
1	38	38<49，49 后移，38 放最前	`[38 49] 65 97 76 13 27 49*`
2	65	65≥49，不动	`[38 49 65] 97 76 13 27 49*`
3	97	97≥65，不动	`[38 49 65 97] 76 13 27 49*`
4	76	76<97、76<65？不成立（76>65），97 后移，76 插在 65 后	`[38 49 65 76 97] 13 27 49*`
5	13	13 比 97、76、65、49、38 都小，全体后移，13 放最前	`[13 38 49 65 76 97] 27 49*`
6	27	27<97、76、65、49、38，但 27>13，38 及其后元素后移	`[13 27 38 49 65 76 97] 49*`
7	49*	49\<*97*、*76*、*65*，但 *49*\≥49，故 97、76、65 后移，49* 插在 49 之后	`13 27 38 49 49* 65 76 97`

注意第 7 趟：判断条件是「$a_j > base$ 就继续往前找」，遇到相等的 49 时停止，于是 49^* 落在原 49 的右边，相对次序不变——这就是直接插入排序稳定的原因。

（三）参考代码

```
void InsertSort(int a[], int n) {
    int i, j, base;
    for (i = 1; i < n; ++i) {      // 共 n-1 趟
```

```
        base = a[i];                // 暂存待插入元素，a[i] 的位置成为“空位”
        j = i - 1;
        while (j >= 0 && a[j] > base) {  // 严格大于才后移，保证稳定
            a[j + 1] = a[j];
            --j;
        }
        a[j + 1] = base;            // 插入到正确位置
    }
}
```

（四）复杂度与特性

- 最好情况：序列本身有序，每趟只比较 1 次、不移动，比较次数 $n-1$，移动次数 0，时间 $O(n)$。
- 最坏情况：序列逆序，第 i 趟比较 i 次、移动 $i+1$ 次，总比较次数 $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$，时间 $O(n^2)$。
- 平均情况：$O(n^2)$；记录平均后移 $n/2$ 个位置，约需 $n^2/4$ 次移动。
- 空间：只需一个暂存变量，$O(1)$，原地排序。
- 稳定性：稳定。
- 适用存储：顺序存储和链式存储都适用。链表上无需移动元素，只需从头查找插入位置后改指针（但查找位置仍需 $O(n)$，总时间仍 $O(n^2)$）。

直接插入排序的三句话

1. 时间复杂度：最好 $O(n)$，最坏与平均 $O(n^2)$，空间 $O(1)$。
2. 稳定、原地、自适应（效率强烈依赖初始有序程度）。
3. 特别适合「记录基本有序」或「待排记录数 n 很小」的场合。

直接插入排序易错点

- 第 i 趟结束后，前 $i+1$ 个元素有序，但这部分元素不一定在最终位置上。这与起泡、选择、堆排序「每趟至少确定一个元素的最终位置」不同，是反推算法类型的关键判别点。
- 排序趟数与初始状态无关，恒为 $n-1$ 趟。
- 判断条件必须是 `a[j] > base`（严格大于），写成 `>=` 会使算法变成不稳定。

真题考法

2022 年第 11 题：问「采用直接插入排序而不采用快速排序」的可能原因，正确答案为 I（大部分元素已有序）、II（待排序元素数量很少）、III（要求空间复杂度 $O(1)$）、IV（要求排序稳定）全部正确。

2020 年第 11 题：对大部分元素已有序的数组，直接插入比简单选择效率高的原因是「直接插入的比较次数更少」。注意辅助空间两者都是 $O(1)$，移动次数也不能笼统断言谁更少。

2009 年第 10 题、2012 年第 11 题也从不同角度考过该算法。

三、折半插入排序

（一）与直接插入排序的唯一区别

折半插入排序（**binary insertion sort**）：把直接插入排序中「在有序区从后往前顺序查找插入位置」这一步，换成折半查找（二分查找）。确定位置后，元素的搬移方式与直接插入排序完全相同。

因此两者唯一的差别只在「比较次数」：

比较项	直接插入排序	折半插入排序
排序总趟数	$n-1$	$n-1$（相同）
元素的移动次数	相同（同一初始序列下完全一致）	相同
辅助空间	$O(1)$	$O(1)$（相同）
查找插入位置的比较次数	与初始状态有关；基本有序时极少，最坏 $\frac{n(n-1)}{2}$	只与 n 有关，约 $O(n\log n)$
稳定性	稳定	稳定

（二）比较次数与移动次数的分别分析

比较次数：对第 i 个元素（它前面有 i 个已排序元素），折半查找最多比较 $\lceil \log_2(i+1) \rceil$ 次。全部 $n-1$ 趟加起来

$$\sum_{i=1}^{n-1} \lceil \log_2(i+1) \rceil = O(n\log n)$$

平均约为 $n\log_2 n$ 次。这个次数与待排序序列的初始状态无关，只取决于元素个数 n。

移动次数：插入位置确定后，插入位置之后的元素仍要逐个后移一位，平均每次移动约一半元素，因此总移动次数仍为

$$\sum_{i=1}^{n-1} \frac{i+1}{2} \approx \frac{n^2}{4} = O(n^2)$$

折半插入排序的核心结论

- 比较次数 $O(n\log n)$，移动次数 $O(n^2)$，因此总时间复杂度仍为 $O(n^2)$。
- 比较次数与初始序列状态无关，这是它与直接插入排序最大的区别。
- 只适用于顺序存储（顺序表），不能用于链表。链式存储无法在 $O(1)$ 时间内随机访问中间元素，折半查找无法进行。

（三）折半查找的边界（决定稳定性）

为了保持稳定，折半查找要找的是最后一个不大于 $base$ 的元素的后一个位置。比较时若 `a[mid] <= base`，则插入位置在右半区（`low = mid + 1`），这样相等元素总被插到已有相等元素的右侧。

```
// 在有序区 a[0..i-1] 中找 base 的插入位置，返回应插入的下标
int binarySearch(int a[], int low, int high, int base) {
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] <= base) low = mid + 1;   // 相等时向右，保证稳定
        else high = mid - 1;
    }
    return low;   // low 即为插入位置
}

void BinaryInsertSort(int a[], int n) {
    for (int i = 1; i < n; ++i) {
        int base = a[i];
        int pos = binarySearch(a, 0, i - 1, base);
```

```
        for (int j = i; j > pos; --j)   // 后移，为 base 腾位置
            a[j] = a[j - 1];
        a[pos] = base;
    }
}
```

折半插入排序易错点

- 说「折半插入排序时间比直接插入排序快」是错的。它只减少了比较次数，移动次数没变，而移动比比较更耗时，所以总时间没有实质改善，反而在序列基本有序时比直接插入排序更慢（直接插入此时只需 $O(n)$ 次比较，而折半插入固定要 $O(n\log n)$ 次）。
- 「折半插入排序的比较次数与初始状态无关」是正确的。
- 折半插入排序不能用于链表，这条几乎年年出现在算法选择的判断题里。

真题考法

2012 年第 11 题：对同一待排序序列分别进行折半插入排序和直接插入排序，问两者可能的不同之处。四个选项分别是排序总趟数、元素移动次数、辅助空间数量、元素之间的比较次数。正确答案是 **D**（元素之间的比较次数）。总趟数都是 $n-1$，移动次数相同，辅助空间都是 $O(1)$。

四、起泡排序（bubble sort）

（一）算法思想

起泡排序（**Bubble Sort**）：从前往后依次比较相邻两个元素，若「左 > 右」则交换，一趟扫描结束后，当前无序区中关键字最大的元素被交换到无序区末尾。重复进行，直到某一趟没有发生任何交换（或已完成 $n-1$ 趟）。

逻辑分区与插入排序相反：「无序区在前、有序区在后」，有序区是从后往前逐步扩大的。

（二）手工过程（逐步模拟）

仍以 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例：

第 **1** 趟（无序区为整个序列，从左到右两两比较）：

- 49 与 38：49>38，交换 → `38 49 65 97 76 13 27 49*`
- 49 与 65：不换 → `38 49 65 97 76 13 27 49*`
- 65 与 97：不换 → `38 49 65 97 76 13 27 49*`
- 97 与 76：交换 → `38 49 65 76 97 13 27 49*`
- 97 与 13：交换 → `38 49 65 76 13 97 27 49*`
- 97 与 27：交换 → `38 49 65 76 13 27 97 49*`
- 97 与 49*：交换 → *'38 49 65 76 13 27 49* 97'*

第 1 趟结束，**97** 到达最终位置（下标 7）。

第 **2** 趟（只扫描前 7 个元素，97 不参与）：49 与 65 不换；65 与 76 不换；76 与 13 交换；76 与 27 交换；76 与 49* 交换。末尾再与 97 比较时不交换。

结果：`38 49 65 13 27 49* 76 97`，**76** 到达最终位置。

第 **3** 趟（只扫描前 6 个元素）：38 与 49 不换；49 与 65 不换；65 与 13 交换 → 49 与 13 交换 → 38 与 13 交换，之后 65 与 27 交换、65 与 49* 交换。

结果：13 38 49 27 49* 65 76 97，**65** 到达最终位置。

第 **4** 趟：13 与 38、38 与 49 不换；49 与 27 交换；49 与 49* 不换（相等不交换）。

结果：13 38 27 49 49* 65 76 97，**49*** 到达最终位置。

第 **5** 趟：38 与 27 交换，其余不再发生交换。

结果：13 27 38 49 49* 65 76 97，**38** 到达最终位置。

第 **6** 趟：从头到尾没有任何交换，`flag` 保持 `false`，算法提前结束。剩余 3 个元素 13、27、38 自然有序。

可以看到：每一趟至少把一个元素（当前无序区的最大值）送到它的最终位置，且已排好的有序后缀一定是全局有序的（这与插入排序的「局部有序前缀」截然不同，是判别算法类型的重要特征）。

（三）参考代码（带提前结束优化）

```
void BubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; ++i) {   // 最多 n-1 趟
        int flag = 0;                    // 本趟是否发生交换
        for (int j = 0; j < n - 1 - i; ++j) {
            if (a[j] > a[j + 1]) {       // 相等不交换，保证稳定
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                flag = 1;
            }
        }
        if (!flag) break;                // 本趟无交换，说明已全部有序
    }
}
```

（四）复杂度与特性

- 最好情况：序列完全有序，加 `flag` 优化后只做 1 趟，比较 $n-1$ 次，交换 0 次，时间 $O(n)$。
- 最坏情况：序列逆序，比较次数 $\sum_{i=1}^{n-1}(n-i)=\frac{n(n-1)}{2}$，交换次数也是 $\frac{n(n-1)}{2}$，每次交换 3 次移动，时间 $O(n^2)$。
- 平均情况：$O(n^2)$。
- 空间：$O(1)$，原地。
- 稳定性：稳定（相等时不交换）。
- 适用存储：顺序存储与链式存储都可用（链表上相邻结点仍可顺序比较交换）。

起泡排序的两个关键性质

1. 每一趟至少确定一个元素的最终位置，且产生的有序区在末尾、是全局有序的。
2. 加 `flag` 优化后，比较次数与初始序列状态有关：完全有序时 $n-1$ 次；逆序时 $\frac{n(n-1)}{2}$ 次。不加优化时，比较次数恒为 $\frac{n(n-1)}{2}$，与初始状态无关。

起泡排序易错点

- 起泡排序的趟数与初始状态有关：有序时 1 趟即可结束，逆序时需要 $n-1$ 趟。这是它与插入、选择排序的重要差别（插入、选择的趟数与初始状态无关）。
- 加不加 `flag`，直接影响「比较次数是否与初始状态有关」这个结论。答题时先确认题目中的代码是否带优化。
- 起泡排序的交换操作涉及 3 次移动，插入排序的移动只涉及 1 次赋值。因此虽然同为 $O(n^2)$，起泡排序的常数开销通常大于直接插入排序。

- 某些教材把带 flag 的优化版本与「冒泡界优化（记录上一趟最后一次交换的位置）」混为一谈。冒泡界优化包含了提前结束优化，但提前结束优化并不带界的收缩，答题时要按题面代码判断。

真题考法

2010 年第 11 题：给出某排序的前三趟结果为 <2,12,16,5,10,88> → <2,12,5,10,16,88> → <2,5,10,12,16,88>，问采用的排序方法。观察可知每趟都把当前最大值推到末尾（88 → 16 → 12），符合起泡排序特征，答案 **A**。

2009 年第 10 题与 2012 年第 10 题都涉及「每趟至少确定一个最终位置」这一判别特征。

五、简单选择排序

（一）算法思想

简单选择排序（**Selection Sort**）：每一趟在待排序元素（无序区）中选出关键字最小（或最大）的元素，与无序区的第一个元素交换，从而把有序区从前往后扩大一个位置。

n 个元素需要 $n-1$ 趟：第 i 趟（i 从 0 计）在区间 $[i,\ n-1]$ 中找最小值下标 `minIdx`，若 `minIdx != i` 则与 `a[i]` 交换。

（二）手工过程（逐步模拟）

以 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*\}$ 为例：

趟次	无序区	选出的最小元素	交换操作	本趟结束后序列
1	`[49 38 65 97 76 13 27 49*]`	13（下标 5）	49 ↔ 13	`[13] 38 65 97 76 49 27 49*`
2	`[38 65 97 76 49 27 49*]`	27（下标 6）	38 ↔ 27	`[13 27] 65 97 76 49 38 49*`
3	`[65 97 76 49 38 49*]`	38（下标 6）	65 ↔ 38	`[13 27 38] 97 76 49 65 49*`
4	`[97 76 49 65 49*]`	49（下标 4）	97 ↔ 49	`[13 27 38 49] 76 49* 65 97`
5	`[76 49* 65 97]`	49*（下标 4）	76 ↔ 49*	`13 27 38 49 49* 65 97`
6	`[65 97]`	65（下标 5）	不交换	`13 27 38 49 49* 65 97`
7	`[97]`	只剩一个，无需处理	—	完成

观察第 4、5 趟：第 4 趟找到的 49 恰好在原 49\ 的左边，交换后保持不变；第 *5* 趟在 *'[76, 49*, 65, 97]' 中选最小值 49\（下标 *4*），与 *76* 交换，*49*\ 落到下标 4，此时 49\ 位于原 *49* 的右边，位置正确。本例中两个 *49* 的相对次序恰好没有被打乱，说明「举一个例子没乱」不能证明算法稳定 *，必须分析操作机制。

（三）参考代码

```
void SelectSort(int a[], int n) {
    for (int i = 0; i < n - 1; ++i) {
        int minIdx = i;
        for (int j = i + 1; j < n; ++j)
            if (a[j] < a[minIdx]) minIdx = j;    // 找无序区最小值下标
```

```
        if (minIdx != i) {                          // 位置不变时不做无谓交换
            int t = a[i]; a[i] = a[minIdx]; a[minIdx] = t;
        }
    }
}
```

（四）比较次数与移动次数

- 比较次数：第 1 趟比较 $n-1$ 次，第 2 趟 $n-2$ 次……最后一次 1 次，总数

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

与序列的初始状态完全无关，恒为 $\frac{n(n-1)}{2}$。这是所有排序算法中比较次数最”死”的一个。

- 移动次数：每趟最多交换一次，每次交换 3 次移动，因此最多 $3(n-1)$ 次移动（若代码写成「位置相同时不交换」，则更少）。这是所有内部排序算法中移动次数最少的之一（仅次于基数排序这类不靠移动的算法）。
- 空间：$O(1)$。
- 适用存储：顺序存储与链式存储都可用（链表上「找最小值」只需顺序扫描）。

简单选择排序的核心结论

- 比较次数固定为 $\frac{n(n-1)}{2} = O(n^2)$，与初始状态无关。
- 移动次数最多 $n-1$ 次交换，即 $O(n)$ 级别，是「比较得多、交换得少」的典型。
- 时间复杂度最好/最坏/平均都是 $O(n^2)$，非自适应。
- 不稳定。

（五）不稳定的原因与反例

简单选择排序为什么不稳定

不稳定的根源是：每趟把「无序区最小值」与「无序区第一个元素」直接交换，这是一次跨越多个元素的长距离交换。若在「无序区第一个元素」与「最小值所在位置」之间还存在与前者关键字相等（甚至与最小值相等）的元素，这次交换就会把它们之间的相对次序颠倒。

反例（最小规模）：序列 $\{8,\ 5,\ 5^*,\ 1\}$，要求升序。

- 第 1 趟：无序区 `[8，5，5*，1]`，最小值是 1（下标 3），与下标 0 的 8 交换 $\rightarrow \{1,\ 5,\ 5^*,\ 8\}$

排序结果中，原本在前的 5（下标 1）仍在 5*（下标 2）之前，这个例子恰好稳定。换一个例子：

反例：序列 $\{2,\ 2^*,\ 1\}$，要求升序。

- 第 1 趟：无序区 `[2，2*，1]`，最小值是 1（下标 2），与下标 0 的 2 交换 $\rightarrow \{1,\ 2^*,\ 2\}$

排序前 2 在 2^* 之前，排序后 2^* 在 2 之前，相对次序被改变，故简单选择排序不稳定。

另一个常用反例：$\{7,\ 7^*,\ 2\}$ 同理，第 1 趟交换第一个 7 与 2，得到 $\{2,\ 7^*,\ 7\}$。

简单选择排序易错点

- 「比较次数与初始状态无关」是正确的；「移动次数与初始状态无关」是错误的（移动次数与初始状态有关，最多 $n-1$ 次交换，可能一次都不交换）。
- 「每趟至少确定一个元素的最终位置」是正确的（每趟结束，最小值落到下标 i 的最终位置）。

- 排序趟数与初始状态无关，恒为 $n-1$ 趟。
- 判断不稳定时必须给出含两个相等关键字的具体反例，不能只说「因为它做跨越交换」。

真题考法

2020 年第 11 题：对大部分元素已有序的数组，直接插入比简单选择效率更高，原因是前者比较次数更少——因为简单选择排序的比较次数恒为 $\frac{n(n-1)}{2}$，不因数据接近有序而减少。

六、希尔排序（shell sort）

（一）算法思想

希尔排序（**Shell Sort**），又称缩小增量排序（**diminishing increment sort**）：先取一个小于 n 的增量 gap，把序列中所有下标相差 gap 的记录分为同一组，对每组分别进行直接插入排序；然后缩小 gap 重复上述过程；当 $gap=1$ 时，整个序列作为一组再做一次直接插入排序，排序完成。

每一次组内排序用的都是直接插入排序（这是 2015 年真题的直接考点）。

（二）为什么能改进——两条理由

希尔排序能突破 $O(n^2)$ 的两个依据

1. 直接插入排序对「基本有序」的序列效率极高。当 gap 很小时，序列在前几轮已经被整理得大致有序，此时插入排序的效率接近线性 $O(n)$。
2. 直接插入排序每次比较最多让元素移动一位，效率太低。希尔排序在 gap 较大时允许元素一次跨越 gap 个位置移动，一次操作可能消除多个逆序对，从而以低于 n^2 阶的次数把逆序数降到 0。

从逆序数角度看：起泡和直接插入只做相邻交换，一次最多消除一个逆序，所以交换次数为 $O(n^2)$。而希尔排序在 $gap>1$ 时做的是长距离交换，能一次消除多个逆序，因此总操作次数可以低于 $O(n^2)$。

（三）手工过程（逐步模拟）

以序列 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49^*,\ 55,\ 4\}$ 为例，共 $n=10$，采用 Shell 增量 $gap=5,\ 2,\ 1$。

第 **1** 趟，$gap=5$：下标相差 5 的元素归为一组，共 5 组（下标 0–5、1–6、2–7、3–8、4–9）。

- 组 1：（49，13）→ 13 插到 49 前 → 13 49
- 组 2：（38，27）→ 27 插到 38 前 → 27 38
- 组 3：（65，49*）→ 49\ 插到 *65* 前 → *49*\ 65
- 组 4：（97，55）→ 55 插到 97 前 → 55 97
- 组 5：（76，4）→ 4 插到 76 前 → 4 76

写回原数组：$\{13,\ 27,\ 49^*,\ 55,\ 4,\ 49,\ 38,\ 65,\ 97,\ 76\}$

第 **2** 趟，$gap=2$：下标相差 2 的元素归为一组，共 2 组（偶数组 0,2,4,6,8 和奇数组 1,3,5,7,9）。

- 偶数组：下标 0,2,4,6,8 上的元素为（13，49*，4，38，97），直接插入排序后为（4，13，38，49*，97）
- 奇数组：下标 1,3,5,7,9 上的元素为（27，55，49，65，76），直接插入排序后为（27，49，55，65，76）
- 交错写回：$\{4,\ 27,\ 13,\ 49,\ 38,\ 55,\ 49^*,\ 65,\ 97,\ 76\}$

第 **3** 趟，$gap=1$：全序列做一次直接插入排序（此时序列已基本有序，只需较少比较与移动），得到最终结果

$$\{4,\ 13,\ 27,\ 38,\ 49,\ 49^{*},\ 55,\ 65,\ 76,\ 97\}$$

高频题型：给一趟结果反推增量

方法：若某一趟采用的增量为 gap，则排序后所有下标相差 gap 的元素构成的子序列都必须各自有序。逐个候选 gap 取出这些子序列检查是否有序即可。

例题（**2014** 年第 **10** 题）：某序列第 1 趟希尔排序结果为 9，1，4，13，7，8，20，23，15，则该趟采用的增量可能是多少?

- 检验 $gap=2$：下标 0,2,4,6,8 上的元素为 (9，4，7，20，15)——9>4，不有序，排除；奇数组 (1，13，8，23)——13>8，也不有序。
- 检验 $gap=3$：下标 0,3,6 为 (9，13，20) 有序；下标 1,4,7 为 (1，7，23) 有序；下标 2,5,8 为 (4，8，15) 有序。三组全部有序，故 $gap=3$ 可行。
- 检验 $gap=4$、$gap=5$ 均能找到组内逆序，排除。答案 **B**（**3**）。

例题（**2018** 年第 **10** 题）：初始序列 (8,3,9,11,2,1,4,7,5,10,6)，第 1 趟结果为 (1,3,7,5,2,6,4,9,11,10,8)，第 2 趟结果为 (1,2,6,4,3,7,5,8,11,10,9)。求两趟的增量。

第 1 趟按 $gap=5$ 分组：(8,1,6)→(1,6,8)、(3,4)→(3,4)、(9,7)→(7,9)、(11,5,10)→(5,10,11)、(2,11...) 逐组检查后交错写回恰好得到第 1 趟结果，故 $gap=5$。第 2 趟按 $gap=3$ 检查同样吻合。答案 **D**（**5, 3**）。

（四）参考代码

```
void ShellSort(int a[], int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {      // 增量序列: n/2, n/4, ..., 1
        for (int s = 0; s < gap; ++s) {             // 当前增量下有 gap 个子序列
            for (int i = s + gap; i < n; i += gap) {// 对每个子序列做直接插入排序
                int base = a[i], j = i - gap;
                while (j >= 0 && a[j] > base) {
                    a[j + gap] = a[j];
                    j -= gap;
                }
                a[j + gap] = base;
            }
        }
    }
}
```

Shell 增量无论 n 多大都从 $\lfloor n/2 \rfloor$ 开始不断除 2，直到 $gap=1$。（Donald Shell 1959 年原论文中 $n=11$ 的示例即取 $gap=5,2,1$。）

（五）复杂度分析

时间复杂度与增量序列的选择直接相关，最优增量序列至今没有明确结论。

增量序列	最坏时间复杂度
Shell 增量 $n/2^k$（$n/2,\ n/4,\ \ldots,\ 1$）	$\Theta(n^2)$；在 n 处于某些特定范围时可达到 $O(n^{1.3})$
Hibbard 增量	优于 Shell 增量

增量序列	最坏时间复杂度
Knuth 增量、Sedgewick 增量	进一步改进，最坏可达 $O(n^{1.5})$ 甚至更好

统考层面需要记住的表述是：希尔排序的时间复杂度约为 $O(n^{1.3})$，与增量序列有关，最坏可达 $O(n^2)$；当输入数组已有序时达到最好时间复杂度 $O(n)$。空间复杂度 $O(1)$。

（六）不稳定的原因与反例

希尔排序为什么不稳定

不稳定根源：当 $gap > 1$ 时，关键字相同的记录可能被划分到不同的子序列中。它们各自在自己的子序列里做插入排序，然后交错写回原数组，相对次序因此被打乱。也就是说，跳步移动让相等元素可以跨越彼此。

反例：序列 $\{0,\ 1,\ 4,\ 3_1,\ 3_2,\ 5,\ 6\}$，取 $gap = 2$。（下标 3 上的 3 记为 3_1，下标 4 上的 3 记为 3_2，3_1 原本在 3_2 之前。）

按下标相差 2 分组：

- 偶数组（下标 0,2,4,6）：`(0,\ 4,\ 3_2,\ 6)` → 直接插入排序后 `(0,\ 3_2,\ 4,\ 6)`
- 奇数组（下标 1,3,5）：`(1,\ 3_1,\ 5)` → 已有序，不变

交错写回原数组，得到

$$\{0,\ 1,\ 3_2,\ 3_1,\ 4,\ 5,\ 6\}$$

排序前 3_1 在 3_2 之前，排序后 3_2 在 3_1 之前，相对次序被改变。故希尔排序不稳定。

希尔排序易错点

- 希尔排序的组内排序是直接插入排序，不是折半插入、快排或归并（2015 年第 11 题）。
- 希尔排序的最后一趟增量一定是 **1**，且 $gap = 1$ 那一趟等价于对整个序列做一次直接插入排序。
- 希尔排序是原地排序（$O(1)$ 空间），这一点与快排、归并不同。
- 希尔排序不能简单地照搬到链表，因为它需要频繁按下标跳转访问下标相差 gap 的元素。
- 「希尔排序每趟至少能确定一个元素的最终位置」是错的。一趟只是让各个增量子序列有序，通常不能保证某个元素已经落在它整个序列的最终位置上（2012 年第 10 题）。
- 希尔排序的复杂度不能写成一个确定值，必须写成「与增量序列有关」。

真题考法

希尔排序是高频考点，2014、2015、2018 年连续考察。三种典型问法：

1. 给一趟结果反推增量（2014-10、2018-10）：检验「下标相差 gap 的子序列是否有序」。
2. 问组内排序用什么算法（2015-11）：答直接插入排序。
3. 与其他算法对比（2012-10、2017-11）：希尔排序不能保证每趟确定一个最终位置；换成链式存储后效率会降低。

七、快速排序

（一）算法思想与分治结构

快速排序（**Quick Sort**）基于分治思想，核心操作是一趟划分（**partition**）：选定一个基准元素（枢轴，**pivot**），通过一次扫描把整个序列分成三部分——左子序列（所有元素 $\leq$ 枢轴）、枢轴、右子序列（所有元素 $\geq$ 枢轴），并且枢轴被放到它的最终位置上。然后对左右两个子序列递归地重复划分，直到子序列长度不超过 1。

（二）一趟划分（**Partition**）的完整手工过程

统考（王道教材）采用挖坑法：取最左端元素为枢轴，暂存到变量 `key`，该位置成为「坑」；先从右往左找比 `key` 小的元素填到左边的坑，再从左往右找比 `key` 大的元素填到右边的坑；如此交替，直到左右指针相遇，最后把 `key` 放入相遇位置。

挖坑法的四条操作规则

设区间为 `[begin, end]`，`key = a[begin]`，`begin` 处为第一个坑。

1. `end` 从右往左移动，跳过所有 $\geq$ `key` 的元素；遇到 `< key` 的元素时，执行 `a[begin] = a[end]`（填左坑），此后 `end` 处成为新坑。
2. `begin` 从左往右移动，跳过所有 $\leq$ `key` 的元素；遇到 `> key` 的元素时，执行 `a[end] = a[begin]`（填右坑），此后 `begin` 处成为新坑。
3. 重复 1、2，直到 `begin == end`。
4. 令 `a[begin] = key`，返回 `begin`，它即枢轴的最终下标。

若在最左端挖坑，必须先移动 `end`（即「先右后左」）。反之若选最右端元素为枢轴，则必须先移动 `begin`。

样例——对序列 $\{25, 32, 12, 45, 8, 60, 17, 5\}$ 做第一趟划分：

初始 `a = [25, 32, 12, 45, 8, 60, 17, 5]`，`key = 25`，`begin = 0`，`end = 7`。下表逐步记录，`_` 表示当前的「坑」。

步骤	操作	序列状态	说明
初始	取出枢轴	`_ 32 12 45 8 60 17 5`	`key = 25`，`begin = 0`，`end = 7`，下标 0 为坑
1	`end`（7）左移找 `< 25`	`_ 32 12 45 8 60 17 5`	`a[7] = 5 < 25`，`end` 停在下标 7
2	填左坑	`5 32 12 45 8 60 17 _`	`a[0] = 5`，坑移到下标 7
3	`begin`（0）右移找 `> 25`	`5 32 12 45 8 60 17 _`	`a[0]=5` 跳过；`a[1] = 32 > 25`，停
4	填右坑	`5 _ 12 45 8 60 17 32`	`a[7] = 32`，坑移到下标 1
5	`end`（6）左移找 `< 25`	`5 _ 12 45 8 60 17 32`	`a[6] = 17 < 25`，停
6	填左坑	`5 17 12 45 8 60 _ 32`	`a[1] = 17`，坑移到下标 6
7	`begin`（1）右移找 `> 25`	`5 17 12 45 8 60 _ 32`	`a[1]=17` 跳过、`a[2]=12` 跳过；`a[3] = 45 > 25`，停
8	填右坑	`5 17 12 _ 8 60 45 32`	`a[6] = 45`，坑移到下标 3

步骤	操作	序列状态	说明
9	end（5）左移找 < 25	5 17 12 _ 8 60 45 32	a[5]=60 跳过；a[4] = 8 < 25，停
10	填左坑	5 17 12 8 _ 60 45 32	a[3] = 8，坑移到下标 4
11	begin（3）右移	5 17 12 8 _ 60 45 32	a[3]=8 跳过，begin 增到 4，与 end 相遇（begin == end == 4）
12	枢轴归位	5 17 12 8 25 60 45 32	a[4] = key = 25

最终结果：{5, 17, 12, 8, 25, 60, 45, 32}。

检验：25 左侧 {5，17，12，8} 全部小于 25；右侧 {60，45，32} 全部大于 25。枢轴 **25** 已到达它的最终位置（下标 **4**）。✓

注意第 7、9 两步：begin 从左往右扫时跳过的是小于等于枢轴的元素（12 被跳过），end 从右往左扫时跳过的是大于等于枢轴的元素（60 被跳过）。这一步的「跳过条件」写反，是手工模拟出错的最常见原因。

继续递归（下述完整过程已逐步核对）：

- 子区间 [0,3] 即 {5,17,12,8}，枢轴 5，划分后为 {5，8，12，17}（5 归位下标 0）
- 再对 {17,12,8}，枢轴 17，划分后为 {8，12，17}（17 归位下标 3）
- 再对 {8,12}，枢轴 8，划分后 {8,12}（8 归位下标 1）
- 子区间 [5,7] 即 {60,45,32}，枢轴 60，划分后为 {32，45，60}（60 归位下标 7）
- 再对 {32,45}，枢轴 32，划分后 {32,45}（32 归位下标 5）

最终结果：{5, 8, 12, 17, 25, 32, 45, 60}，排序正确。

从递归结构可以看到，每次划分都恰好确定一个元素（枢轴）的最终位置，这正是「快速排序每趟至少确定一个元素最终位置」的来源。

（三）参考代码

```
// 挖坑法划分：以 a[low] 为枢轴，返回枢轴最终下标
int Partition(int a[], int low, int high) {
    int key = a[low];                  // 暂存枢轴，low 处成为第一个坑
    while (low < high) {
        while (low < high && a[high] >= key) --high;  // 从右往左找比 key 小的
        a[low] = a[high];                             // 填左坑
        while (low < high && a[low] <= key) ++low;    // 从左往右找比 key 大的
        a[high] = a[low];                             // 填右坑
    }
    a[low] = key;                      // 枢轴归位
    return low;
}

void QuickSort(int a[], int low, int high) {
    if (low < high) {
        int pivotPos = Partition(a, low, high);
        QuickSort(a, low, pivotPos - 1);
        QuickSort(a, pivotPos + 1, high);
    }
}
```

划分顺序不能随意调换

以最左端元素为枢轴时，必须先「从右往左」再「从左往右」。

原因：`Partition` 的最后一步是 `a[low] = key`，要求相遇位置上的元素 $\leq$ `key`。若先从左往右找，在找不到比枢轴更大的元素时会以 `low == high` 跳出，此时相遇位置的元素可能 `> key`，把它填到枢轴位置就会导致划分失败。

反例：`[0, 0, 0, 0, 1]`。若先从左往右找，会得到错误结果 `[1, 0, 0, 0, 0]`。

读者可自行验证：以最右端元素为枢轴时，顺序正好反过来，必须先从左往右。

（四）时间复杂度的三种情况

设序列长度为 n，每次划分的扫描代价为 $O(n)$。

- 最好情况：每次划分都把序列均分为两个长度约 $n/2$ 的子序列。递归深度为 $\lceil \log_2 n \rceil + 1$，每层的总扫描代价为 $O(n)$，因此

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

- 最坏情况：序列已经基本有序（正序或逆序）且固定取第一个元素为枢轴。此时每次划分只能分出一个长度为 0 的子序列和一个长度为 $n-1$ 的子序列，分治退化为「每次只减少一个元素」，递归深度达到 $n-1$，

$$T(n) = T(n-1) + O(n) = O(n^2)$$

- 平均情况：枢轴位置的期望接近中点，$T(n) = O(n \log n)$。

快速排序最坏为什么是 $O(n^2)$

关键在于划分的均衡程度取决于「枢轴在序列中的相对大小排名」：

- 枢轴恰是中位数 $\rightarrow$ 均分 $\rightarrow$ 递归树高度 $\log n \rightarrow O(n \log n)$
- 枢轴是最大或最小值 $\rightarrow$ 划分出 0 与 $n-1$ $\rightarrow$ 递归树退化成链 $\rightarrow$ 高度 $n \rightarrow O(n^2)$

而取第一个（或最后一个）元素为枢轴的策略，在序列正序或逆序时必然选出极值，所以「有序」恰恰是这种实现的最坏输入，这时快速排序退化成起泡排序级别的效率。

（五）递归深度与栈空间

快速排序是递归算法，空间开销来自递归工作栈，栈的深度等于递归树的深度：

- 平均 / 最好情况：递归树高 $\log n$，栈空间 $O(\log n)$。
- 最坏情况（正序或逆序输入 + 取首元素为枢轴）：递归深度为 $n-1$，栈空间 $O(n)$。

因此空间复杂度的标准写法是 $O(\log n)$ 到 $O(n)$，平均 $O(\log n)$、最坏 $O(n)$。排序操作本身在原数组上进行，不需要额外数组，所以是原地排序。

基准选取的三种优化：

1. 随机化选取：随机选一个元素作枢轴，与首元素交换后再划分，使最坏情况不容易被人为构造，但伪随机数序列固定的语言里仍有被针对的可能。
2. 三数取中：取首、尾、中点三个元素，把中位数换到最左端作枢轴。这样枢轴「既不太小也不太大」的概率大幅提高。
3. 尾递归优化：划分完成后，先递归较短的那个子区间，将较长的子区间用循环处理。由于较短子序列长度不超过 $\frac{n}{2}$，递归深度被限制在 $\lceil \log_2 n \rceil$ 以内，最坏空间降为 $O(\log n)$。

递归次数相关的两个易混结论

- 递归次数（递归调用的总次数）与初始序列状态有关，与「先处理左分区还是先处理右分区」无关（2010 年第 10 题答案）。分区处理顺序只改变递归调用的先后，不改变需要划分的分区集合。
- 尾递归优化后，「先处理较短的子数组」这一策略确实能降低栈深度，但这是空间优化，不是减少递归次数。答题时要看清题目问的是「递归次数」还是「栈深度」。

（六）稳定性

快速排序为什么不稳定

不稳定根源：划分过程中的交换是远距离的。特别是在挖坑法/左右指针法的最后一次操作把枢轴换到分界线上时，枢轴可能被交换到与它关键字相等的元素右侧；此外左右指针的交换也会让相等的元素跨越彼此。

反例：序列 $\{7,\ 2,\ 4_1,\ 4_2,\ 8,\ 9\}$（$4_1$ 原本在 4_2 之前），以最左端元素 7 为枢轴做一趟划分。

- `end` 从右往左找 `< 7` 的元素：9 跳过、8 跳过，`a[3] = 4_2 < 7`，停。`a[0] = 4_2` $\rightarrow \{4_2,\ 2,\ 4_1,\ 4_2,\ 8,\ 9\}$（坑移到下标 3）
- `begin` 从左往右找 `> 7` 的元素：`a[0]=4_2` 跳过、`a[1]=2` 跳过、`a[2]=4_1` 跳过，`begin` 增到 3，与 `end` 相遇
- `a[3] = key = 7` $\rightarrow \{4_2,\ 2,\ 4_1,\ 7,\ 8,\ 9\}$

排序前 4_1 在 4_2 之前，一趟划分后 4_2 落到下标 **0**、4_1 落到下标 **2**，相对次序颠倒。故快速排序不稳定。

这个例子还有一个附带结论：4_2 跨到了枢轴 7 的左侧——注意它是小于 7 的，所以划分结果本身是正确的，不稳定的原因仅仅在于两个 4 的次序被换掉了。

（七）快速排序为什么”快”

平均时间复杂度与归并、堆排序同为 $O(n\log n)$，但实际运行中快速排序通常更快：

1. 出现最差情况的概率很低（随机输入下枢轴排名极端的情况罕见）。
2. 缓存效率高：划分操作在连续的一段子数组上扫描，符合程序访问的局部性原理；而堆排序需要跳跃式访问元素，缓存命中率低。
3. 常数系数低：比较、赋值、交换的总操作数在三者中最少。

（八）不适合的存储结构

快速排序需要从两端向中间扫描、频繁按下标随机访问与交换元素，因此只适合顺序存储（顺序表）。链式存储无法 $O(1)$ 定位到表尾并向前移动，散列存储不保持线性次序，都不合适（2011 年第 10 题，答案：顺序存储）。

快速排序易错点

- 空间复杂度不能只写 $O(\log n)$ 或只写 $O(n)$，完整表述是「平均 $O(\log n)$，最坏 $O(n)$」，来自递归栈。
- 「初始序列基本有序时快排效率高」是错的，那恰恰是取首元素为枢轴时最坏的情形。正确说法是「初始序列越随机、划分越均匀，快排效率越高」。
- 快排的排序趟数与初始状态有关（递归深度随初始状态变化）。
- 快排的比较次数与初始状态有关。
- 快排不稳定，且不要求空间 $O(1)$——这两条使得「要求空间 $O(1)$ 且要求稳定」时不能选快排（2022 年

第 11 题的 III、IV 正是在说这一点）。

真题考法

快速排序是排序章考频最高的考点，至少有三种典型问法。

考法一：判断某个序列「是否可能是快速排序第 k 趟的结果」。思路：每趟划分至少让一个元素（枢轴）落到最终位置。所以第 k 趟结束后，处于最终位置的元素个数至少为 k。

- 2014 年第 11 题：序列 `2,3,5,4,6,7,9` 等四个选项中，先写出全部元素最终升序 `2,3,4,5,6,7,9`，再逐项对照有几个元素已在最终位置。C 项 `3,2,5,4,7,6,9` 只有 9 在最终位置，不足 2 个，故不可能是第 2 趟结果，答案 **C**。
- 另外常用的一条经验：若第一趟确定的枢轴在最左或最右，第二趟只能再确定 1 个位置；若第一趟枢轴不在两端，第二趟能确定 2 个位置。

考法二：给出一次划分后的结果，反推枢轴是哪个元素。思路：枢轴必须满足「左侧全部 $\le$ 它，右侧全部 $\ge$ 它」，即它已在其最终位置上。

- 2023 年第 11 题：一次划分后为 `68，11，70，23，80，77，48，81，93，88`。逐个检验：11 左侧有 68 $>$ 11，不行；70 右侧有 23、48 $<$ 70，不行；80 右侧有 77、48，不行；81 左侧的 `68,11,70,23,80,77,48` 全部小于 81，右侧只有 93、88 全部大于 81，**81** 可行，答案 **D**。

考法三：考察划分结果的块间/块内性质。 2024 年第 8 题：第一趟排序把除枢轴外的 $n-1$ 个元素划分为均非空的 P、Q 两块，则正确的是「P 和 Q 块间有序」。因为一次升序划分只保证 $\forall x \in P, \forall y \in Q$ 都有 $x \le y$，但 P、Q 各自内部仍可能无序，两块大小取决于枢轴排名不一定接近，存在重复关键字时块内仍可能有相等元素。

八、堆排序

（一）堆的定义与存储

堆（**Heap**）是满足特定条件的完全二叉树：

- 大顶堆（大根堆，**Max Heap**）：任意结点的值 $\ge$ 其左右孩子的值，堆顶是最大值。
- 小顶堆（小根堆，**Min Heap**）：任意结点的值 $\le$ 其左右孩子的值，堆顶是最小值。

由于堆是完全二叉树，用数组顺序存储最合适。

下标映射（两种约定都要会）

从下标 **1** 开始存储（王道教材惯用）：结点 i 的左孩子为 $2i$，右孩子为 $2i+1$，父结点为 $\lfloor i/2 \rfloor$；$i > \lfloor n/2 \rfloor$ 的结点都是叶子。

从下标 **0** 开始存储（C/C++ 数组惯用）：结点 i 的左孩子为 $2i+1$，右孩子为 $2i+2$，父结点为 $\lfloor (i-1)/2 \rfloor$；$i \ge \lfloor n/2 \rfloor$ 的结点都是叶子。

答题时先看清题目的下标约定，这是最容易丢分的地方。

（二）筛选（下滤）操作

筛选（**sift down** / 下滤）：把某个结点与其两个孩子中较大的那个交换，然后继续对交换后的位置做同样操作，直到该结点不小于两个孩子或成为叶子。

```
// 下标从 1 开始；对以 i 为根的子树做下滤，堆的有效长度为 n
void SiftDown(int a[], int i, int n) {
    a[0] = a[i];                    // a[0] 作暂存单元
    for (int j = 2 * i; j <= n; j = 2 * i) {
```

```
        if (j < n && a[j] < a[j + 1]) ++j;   // 取两个孩子中较大的
        if (a[0] >= a[j]) break;             // 已满足大顶堆性质
        a[i] = a[j];
        i = j;
    }
    a[i] = a[0];
}
```

（三）建堆过程（自下而上筛选，$O(n)$）

建堆：把任意一个完全二叉树调整为堆。方法是从最后一个非叶子结点开始，依次向前对每个结点做一次筛选，直到根结点。

用下标 1 存储时，最后一个非叶子结点是 $\lfloor n/2 \rfloor$。一句话概括：对最后一个非叶子结点到根结点，依次执行下滤（**SiftDown**）。

样例——把 $\{49,\ 38,\ 65,\ 97,\ 76,\ 13,\ 27,\ 49\}$ 建成大顶堆（数组下标 $1-8$，双重循环记 $a_1 = 49, \ldots, a_8 = 49$）：

步骤	对哪个结点筛选	比较与交换	数组状态（下标 **1–8**）
初始	—	未调整	49 38 65 97 76 13 27 49
1	$i=4$（$\lfloor 8/2 \rfloor$）	$97 \geq 49$，无需交换	49 38 65 97 76 13 27 49
2	$i=3$	$65 \geq 13$ 且 $65 \geq 27$，无需交换	49 38 65 97 76 13 27 49
3	$i=2$	孩子 97、76 中较大者 $97 > 38$，交换 38 与 97；换到下标 4 后 $38 \geq 49$？不成立（38<49），再交换 38 与 49	49 97 65 49 76 13 27 38
4	$i=1$	孩子 97、65 中较大者 $97 > 49$，交换；49 换到下标 2 后，孩子 49、76 中较大者 $76 > 49$，再交换	97 76 65 49 49 13 27 38

建堆完成，得到大顶堆 $\{97,\ 76,\ 65,\ 49,\ 49,\ 13,\ 27,\ 38\}$。

建堆为什么是 $O(n)$ 而不是 $O(n \log n)$

- 需要下滤的结点（非叶结点）约有 $n/2$ 个，每个结点下滤最多走 $O(\log n)$ 层。相乘得到 $O(n \log n)$ 是错误估计。
- 正确算法：按层求和，把「每层结点数 × 该层结点到叶子的距离」累加。设完美二叉树高为 h，则

$$T(h) = 2^0 h + 2^1 (h-1) + 2^2 (h-2) + \cdots + 2^{h-1} \cdot 1$$

用错位相减法：$2T(h) - T(h) = -h + 2^1 + 2^2 + \cdots + 2^h = 2^{h+1} - h - 2 = O(2^h)$。

而高度为 h 的完美二叉树结点数 $n = 2^{h+1} - 1$，所以 $T(h) = O(2^h) = O(n)$。

- 直觉：越靠近底层的结点数量越多，但它们需要下滤的距离越短；越靠近根结点的结点下滤距离长，但数量极少。加权求和后总和是 $O(n)$。

两种建堆方式对比（**2021** 年真题考点）

1. 逐个插入建堆：从空堆开始，依次把每个元素插入堆底并向上堆化。每个元素插入代价 $O(\log n)$，总代价 $O(n\log n)$。**2021 年第 11 题考察的就是这种方式**，注意它得到的堆与「对整个序列自下而上建堆」的结果不一定相同。
2. 自下而上筛选建堆：把全部元素先原样放入数组，再从最后一个非叶子结点到根依次下滤。总代价 $O(n)$，效率更高。

答题时务必先确认题目说的是「依次插入建堆」还是「对给定序列建堆」。

（四）堆排序的每一趟

堆排序（**Heap Sort**）（升序排序用大顶堆）：

1. 把待排序序列建成大顶堆，此时堆顶是最大值。
2. 把堆顶元素与当前堆的最后一个元素交换，最大值到达其最终位置，堆的有效长度减 **1**。
3. 对新的堆顶做一次下滤，恢复大顶堆性质。
4. 重复 2、3 共 $n-1$ 轮。

样例——接上建堆结果 $\{97, 76, 65, 49, 49, 13, 27, 38\}$。表中用 | 分隔「当前堆区」与「已经排好的有序区」：

趟次	交换	交换后（堆区	已排好的有序区）	下滤后恢复的堆
1	$97 \leftrightarrow 38$	`38 76 65 49 49 13 27 | 97`	`76 49 65 38 49 13 27 | 97`
2	$76 \leftrightarrow 27$	`27 49 65 38 49 13 | 76 97`	`65 49 27 38 49 13 | 76 97`
3	$65 \leftrightarrow 13$	`13 49 27 38 49 | 65 76 97`	`49 49 27 38 13 | 65 76 97`
4	$49 \leftrightarrow 13$	`13 49 27 38 | 49 65 76 97`	`49 38 27 13 | 49 65 76 97`
5	$49 \leftrightarrow 13$	`13 38 27 | 49 49 65 76 97`	`38 13 27 | 49 49 65 76 97`
6	$38 \leftrightarrow 27$	`27 13 | 38 49 49 65 76 97`	`27 13 | 38 49 49 65 76 97`
7	$27 \leftrightarrow 13$	`13 | 27 38 49 49 65 76 97`	完成

最终结果：$\{13, 27, 38, 49, 49, 65, 76, 97\}$（升序）。

注意这里两个 49 的相对次序已经无法与初始状态对应（初始时 $a_4 = 97$ 之后的 $a_8 = 49$ 与 $a_1 = 49$ 是两个不同记录），这正是不稳定性的表现之一。

（五）复杂度与特性

- 时间复杂度：建堆 $O(n)$；之后 $n-1$ 轮，每轮交换一次并做一次下滤，每次下滤 $O(\log n)$，共 $O(n\log n)$。所以

$$T(n) = O(n) + O(n\log n) = O(n\log n)$$

最好、最坏、平均都是 $O(n\log n)$，与初始状态无关（非自适应）。

- 空间复杂度：$O(1)$，原地排序。这是它相对于快速排序的最大优势之一——最坏时间仍是 $O(n\log n)$，且空间为 $O(1)$。
- 稳定性：不稳定。

（六）不适用的存储结构与 Top-K 应用

- 堆排序依赖完全二叉树的数组下标关系（$2i$、$2i+1$、$\lfloor i/2 \rfloor$）快速定位父子结点，改用链式存储后无法 $O(1)$ 定位，效率降低（2017 年第 11 题）。
- **Top-K** 问题（2022 年真题：从 n 个数中找最小的 10 个数）：
  - 要在 n 个数中找最大的 k 个 $\rightarrow$ 用小顶堆（堆顶是当前候选中最小的，新元素比堆顶大就替换）。
  - 要在 n 个数中找最小的 k 个 $\rightarrow$ 用大顶堆（堆顶是当前候选中最小的，新元素比堆顶小就替换）。
  - 时间复杂度 $O(n \log k)$，空间 $O(k)$。另外还可以用快排的划分思想做到平均 $O(n)$。

堆排序易错点

- 升序排序要用大顶堆，降序排序要用小顶堆。用「小顶堆 + 依次弹出堆顶」也能得到升序，但那样需要额外的辅助数组，不是原地堆排序。
- 建堆的时间复杂度是 $O(n)$，不是 $O(n \log n)$。但逐个插入建堆是 $O(n \log n)$。
- 堆不是二叉排序树。大顶堆只要求「父 $\geq$ 子」，不要求「左子树 < 根 < 右子树」，因此中序遍历堆得不到有序序列（2020 年第 9 题：III 错）。
- 大根堆中「次大值一定在根的下一层」是正确的：根的某个孩子一定是次大值，其它任意更深结点都受其祖先约束而不可能超过它。
- 堆的插入（堆底加入后向上调整）和删除（堆顶与堆底交换、删尾、向下调整）都是 $O(\log n)$。
- 堆排序趟数与初始状态无关，恒为 $n-1$ 趟。

真题考法

堆是 2009、2011、2015、2018、2020、2021、2022、2024 年反复考察的考点，主要三种问法。

1. 给定序列建堆，问建堆后的样子（2018 年第 11 题）：严格按「从最后一个非叶子结点到根」的顺序逐个下滤，每一步都要写出中间状态。
2. 插入/删除后重建堆，问关键字比较次数（2011 年第 11 题：大根堆 `25,13,10,12,9` 插入 18；2015 年第 10 题：小根堆删除堆顶；2024 年第 9 题：连续两次删除）：必须按下标依次比较，逐一计数，注意「比较两个孩子」也算一次比较。
3. 堆的性质判断（2020 年第 9 题）：是否完全二叉树（是）、能否顺序存储（能）、是否是二叉排序树（不是）、次大值位置（在根下一层）。

九、二路归并排序（merge sort）

（一）算法思想

二路归并排序（**Merge Sort**）基于分治思想，分两个阶段：

- 划分阶段：递归地把数组从中点处分成左右两个子数组，直到每个子数组长度为 1（长度为 1 的数组天然有序）。
- 合并阶段：把两个已经有序的相邻子数组合并成一个有序数组，逐层向上，直到整个数组有序。

核心操作 **Merge**（二路归并）：设两个有序表 `L` 与 `R`，用指针 `i`、`j` 分别指向各自的首元素，每次比较 `L[i]` 与 `R[j]`，把较小者取出放入结果表，被取出的那一侧指针后移；当某一侧取空后，把另一侧剩余元素直接复制到结果表。

2022 年第 10 题的考点

「二路归并排序操作的功能」问的是 **Merge** 这一步本身，答案是「将两个有序表合并为一个新的有序表」。

- 「将 M 划分为元素个数大致相等的两部分」描述的是分解过程，不是归并操作。
- 「将 M 分成 n 个部分，每部分仅一个元素」描述的也是分解阶段。
- 「将 M 划分为两部分，一部分的值均小于另一部分」描述的是快速排序的划分结果。

归并的递归顺序与二叉树的后序遍历相同：先递归左子数组，再递归右子数组，最后处理合并。

（二）手工过程（逐步模拟）

以 $\{49, 38, 65, 97, 76, 13, 27, 49^*\}$ 为例（$n = 8$，向上归并时取等号放左侧以保证稳定）：

第 **1** 趟（相邻两个元素两两归并，归并段长度变为 2）：

- `[49]` 与 `[38]` 归并 → `[38 49]`
- `[65]` 与 `[97]` 归并 → `[65 97]`
- `[76]` 与 `[13]` 归并 → `[13 76]`
- `[27]` 与 `[49*]` 归并 → `[27 49*]`

结果：`38 49 | 65 97 | 13 76 | 27 49*`

第 **2** 趟（每两个长度为 2 的归并段合成一个长度为 4 的归并段）：

- `[38 49]` 与 `[65 97]`：38<65 取 38；49<65 取 49；左空，接上 65、97 → `[38 49 65 97]`
- `[13 76]` 与 `[27 49*]`：13<27 取 13；76>27 取 27；76>49\ 取 *49*\；右空，接上 76 → `[13 27 49* 76]`

结果：`38 49 65 97 | 13 27 49* 76`

第 **3** 趟（两个长度为 4 的归并段合成整个序列）：

- 38 与 13 比 → 取 13
- 38 与 27 比 → 取 27
- 38 与 49* 比 → 取 38
- 49 与 49\ 比 → 相等取左侧的 *49**（保证稳定）
- 65 与 49\ 比 → 取 *49*\
- 65 与 76 比 → 取 65
- 97 与 76 比 → 取 76
- 右侧空，接上 97

结果：`13 27 38 49 49* 65 76 97`（排序完成）

共 **3** 趟，恰好等于 $\lceil \log_2 8 \rceil = 3$。

（三）参考代码

```
// 合并 a[left..mid] 与 a[mid+1..right] 为一个有序段
void Merge(int a[], int left, int mid, int right) {
    int *tmp = (int *)malloc(sizeof(int) * (right - left + 1));
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right)
        if (a[i] <= a[j]) tmp[k++] = a[i++];   // 相等取左侧，保证稳定
        else              tmp[k++] = a[j++];
    while (i <= mid)   tmp[k++] = a[i++];      // 左段剩余
    while (j <= right) tmp[k++] = a[j++];      // 右段剩余
```

```
    for (k = 0, i = left; i <= right; ++i, ++k) a[i] = tmp[k];
    free(tmp);
}

void MergeSort(int a[], int left, int right) {
    if (left >= right) return;                  // 子数组长度 <= 1，终止
    int mid = left + (right - left) / 2;
    MergeSort(a, left, mid);                    // 递归左子数组
    MergeSort(a, mid + 1, right);               // 递归右子数组
    Merge(a, left, mid, right);                 // 合并
}
```

（四）归并趟数与复杂度

二路归并排序的趟数

对 n 个元素做二路归并，从长度为 1 的子序列开始，每趟把归并段长度翻倍，因此

$$\text{趟数} = \lceil \log_2 n \rceil$$

更一般地，对 r 个初始归并段做 k 路平衡归并，趟数 $S = \lceil \log_k r \rceil$。

- 时间复杂度：划分产生高度为 $\lceil \log_2 n \rceil$ 的递归树，每层合并的总操作量为 n（每个元素恰好被搬运两次：一次进 `tmp`、一次回原数组），因此

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

最好、最坏、平均都是 $O(n \log n)$，与初始序列状态无关（非自适应）。这也意味着归并排序不因数据接近有序而变快。

- 空间复杂度：归并需要长度 n 的辅助数组，为 $O(n)$；递归栈深度 $O(\log n)$ 是低阶项，通常忽略。所以空间 $O(n)$，是非原地排序。这直接导致它不适合内存紧张的场景（2017 年第 10 题）。
- 稳定性：稳定。合并时遇到相等元素取左侧（`a[i] <= a[j]` 时取 `a[i]`），相对次序不变。

（五）链表上的归并排序

归并排序特别适合链表，且空间可以优化到 $O(1)$：

1. 链表结点的增删只需改指针，合并两个有序链表无需辅助数组。
2. 用迭代划分替代递归划分，可以省去递归栈空间。

代价是：链表上无法 $O(1)$ 随机访问中点，找中点需要快慢指针，但总时间仍是 $O(n \log n)$。

哪些排序适合链表

- 适合：直接插入排序、简单选择排序、起泡排序（顺序扫描即可）、二路归并排序（最优选择）。
- 不适合：折半插入排序（需随机访问）、希尔排序（需按下标跳转）、快速排序（需从两端向中间扫描）、堆排序（需按下标定位父子）。

归并排序易错点

- 归并排序每趟不一定能确定任何一个元素的最终位置。中途几趟只形成越来越长的有序段，段内元素在后续归并时仍可能整体后移（2012 年第 10 题）。
- 归并排序的元素移动次数与初始状态无关（非原地排序，每趟都要把 n 个元素搬进搬出辅助数组），这是很多考生的知识盲区（对比表里「移动次数是否与初始状态有关」这一列，归并填「无关」）。

- 归并排序不稳定的常见误解：只要合并时写成 `if (a[i] < a[j])`（严格小于）就会变不稳定，必须用 `<=` 取左侧。
- 一般教材给出的 `MergeSort` 是自顶向下递归版；考试里也可能出现自底向上迭代版，两者的趟数与结果相同，但迭代版第一趟是相邻两两归并（写结果时要说清是哪一种）。

真题考法

- 2017 年第 10 题：选归并而不选插入的理由，答案是「归并排序的运行效率更高」——归并的代码更长、占用空间更多，所以 I、II 都错。
- 2022 年第 10 题：二路归并操作的功能 = 将两个有序表合并为一个新的有序表。
- 2024 年第 10 题：三个有序序列 (3,5)、(7,9)、(6) 从左至右依次做二路归并。第一次归并 (3,5) 与 (7,9)：3<7 取 3，5<7 取 5，左空接上 7、9，比较 **2** 次；再与 (6) 归并：3<6 取 3，5<6 取 5，6<7 取 6，右空接上 7、9，比较 **3** 次。总比较 **5** 次，答案 **C**。

十、基数排序

（一）算法思想（非比较排序）

基数排序（**Radix Sort**）是一种非比较排序：「基」指数的位。把关键字看成由若干位（十进制即个位、十位、百位……）组成，从最低位开始，逐位对全部元素做一次「分配 + 收集」，直到最高位处理完毕。

- 分配（**distribute** / 装桶）：按当前位的值，把每个元素放进对应的桶（十进制需要 10 个桶，编号 0–9）。
- 收集（**collect**）：按桶的编号从小到大，把各桶中的元素依次连接起来，写回原序列。

每一趟结束后，序列在「已考察的位」范围内是有序的。

（二）MSD 与 LSD

方式	全称	处理顺序	特点
LSD	最低位优先（Least Significant Digit first）	从个位 → 十位 → 百位……逐位排序	实现简单、无需递归、是统考与主流实现。要求每一位的排序必须稳定。
MSD	最高位优先（Most Significant Digit first）	从百位 → 十位 → 个位逐层划分成更小的子序列，最后依次连接	可用于关键字取值范围很大（位数多）的情形，通常需要递归，实现复杂

为什么普遍采用 LSD

后一轮排序会覆盖前一轮排序的结果：若第一轮给出 $a < b$，第二轮却给出 $a > b$，则第二轮的结果取代第一轮。

由于高位的优先级高于低位，所以应该先排低位、再排高位——这样高位的排序结果覆盖低位结果时，正是我们想要的效果。反过来（MSD）必须借助「逐层划分子序列」才能避免低位被覆盖。

（三）手工过程（逐步模拟）

以序列 $\{278, 109, 63, 930, 589, 184, 505, 269, 8, 83\}$ 为例，$d = 3$（最大数是三位数），$r = 10$。

第 **1** 趟，按个位分配（把 8 看作 008、63 看作 063、83 看作 083）：

桶号（个位）	0	1	2	3	4	5	6	7	8	9
装入元素	930	—	—	63, 83	184	505	—	—	278, 8	109, 589, 269

按桶号从小到大收集：930，63，83，184，505，278，8，109，589，269

第 **2** 趟，按十位分配：

桶号（十位）	0	1	2	3	4	5	6	7	8	9
装入元素	505, 8	—	—	930	—	—	63, 269	278	83, 184	589

（8 的十位为 0；63 与 269 的十位都是 6，按第 1 趟收集后的先后顺序，63 在 269 之前——这就是稳定的价值。）

按桶号收集：505，8，109，930，63，269，278，83，184，589

第 **3** 趟，按百位分配：

桶号（百位）	0	1	2	3	4	5	6	7	8	9
装入元素	8, 63, 83	109, 184	269, 278	—	—	505, 589	—	—	—	930

按桶号收集：8，63，83，109，184，269，278，505，589，930

排序完成，结果升序且唯一。

（四）复杂度与稳定性

设最大元素的位数为 d（即排序趟数），待排序数据为 r 进制（r 为桶的个数）。

- 时间复杂度：每趟分配要扫描 n 个元素（$O(n)$）、收集要扫描 r 个桶（$O(r)$），共 d 趟，因此

$$T(n) = O(d(n + r))$$

这是线性级别的复杂度（当 d、r 为常数时即 $O(n)$），因为它不做关键字比较。

- 空间复杂度：需要 r 个桶的辅助空间，若用链队列实现则为 $O(r)$（桶的头尾指针数组）。若同时用计数数组实现分配，则为 $O(n + r)$。
- 稳定性：稳定。前提是每一位用的分配/收集（或该位上的排序）必须是稳定的。
- 移动次数：与关键字的初始排列次序无关（2015 年第 9 题）。

基数排序的稳定性是「正确性的前提」

基数排序要求对每一位的排序必须是稳定的，否则无法得到正确结果。

反例：对 {25, 13, 24} 做 LSD 基数排序。第 1 趟按个位稳定排序得 13，24，25。若第 2 趟按十位排序时不稳定，可能得到 13，25，24——结果错误。

只有当每一位的排序都稳定时，LSD 基数排序才既正确又稳定。反之，若底层排序算法不稳定，采用基数排序后未必正确也未必稳定。

基数排序易错点

- 基数排序不基于比较，所以不受 $\Omega(n \log n)$ 下界约束。
- **”** 从最低位开始**”** 是 **LSD** 的核心。若题目给的序列已经按高位分好组，那是 MSD 的做法。
- 基数排序一趟不保证任何元素到达最终位置（它只是让某一位有序），因此不满足「每趟至少确定一个元素最终位置」。
- 基数排序的适用前提是关键字可以分解为若干位（如整数、字符串、定长编码）。对任意可比类型（如浮点数区间、复杂结构）不能直接用。
- 当关键字的取值范围很大（位数很多）时，趟数 d 增大，效率下降；此时应考虑 MSD 或改用其他算法。
- 基数排序每一趟都要把所有元素” 装桶再倒出”，所以虽然不比较关键字，但元素移动次数是 $n \cdot d$ 级别，常数并不小。

真题考法

- 2013 年第 11 题：对 `110`，`119`，`007`，`911`，`114`，`120`，`122` 做基数排序，问第 2 趟分配收集后的序列。第 **1** 趟按个位：个位 0 的 `110`，`120`；个位 9 的 `119`；个位 7 的 `007`；个位 1 的 `911`；个位 4 的 `114`；个位 2 的 `122`。收集得 `110,120,911,122,114,007,119`。第 **2** 趟按十位：十位 1 的 `110,911,114,119`；十位 2 的 `120,122`；十位 0 的 `007`。收集得 `007,110,911,114,119,120,122`，答案 **A**。
- 2021 年第 10 题：对 `{93,946,372,9,146,151,301,485,236,327,43,892}` 做 LSD 基数排序，问第 1 趟后 372 前后紧邻的元素。第 1 趟只看个位，个位为 2 的有 `372`，`892`（按原序），个位为 3 的有 `93`，`43`，个位为 1 的有 `151`，`301`。收集后个位 1 的桶最靠前：`151`，`301`，`372`，`892`，`93`，`43`，`...`。所以 372 之前紧邻 301、之后紧邻 892，答案 **C**。

十一、外部排序

（一）为什么需要外部排序

当待排序文件比内存可用容量还大时，无法一次性放入内存排序，必须借助外部存储器，这就是外部排序（**external sort**）。

（二）外部排序的两个阶段

外部排序的完整步骤

阶段一：生成初始归并段（顺段，**run**）

把大文件按内存可用容量分成若干长度为 L 的子文件（L 应小于内存可用容量）。依次把每个子文件读入内存，用适当的内部排序算法排序，把排好序的子文件（称为初始归并段或顺段）写回外存，为下一个子文件腾出内存。

若文件共 N 个记录，内存工作区可容纳 L 个记录，则等分法生成的初始归并段数量为

$$r = \left\lceil \frac{N}{L} \right\rceil$$

阶段二：多趟归并

对这些初始归并段反复做 k 路归并，直到得到整个有序文件。归并一趟后归并段数量变为原来的约 $1/k$。

例：一个含 10000 个记录的文件，内存可用容量为 1000 个记录。

- 阶段一：等分为 10 个临时文件，各含 1000 个记录，依次读入内存排序，得到 **10** 个初始归并段写回外存。
- 阶段二：对这 10 个段做两两归并（**2** 路平衡归并），共进行 4 趟，直到得到完整的有序文件。

「k 路平衡归并」的准确含义

常见误区：认为 k 路平衡归并的归并树必须是严格的 k 叉树（只有度为 0 和度为 k 的结点）。

正确理解：k 路平衡归并指的是

- 最多只能有 k 个段归并为一个；
- 每一趟归并中，若有 m 个归并段参与归并，则经过这一趟处理得到 $\lceil m/k \rceil$ 个新的归并段。

所以「8 个段做 4 路归并，第一趟得到 3 个段」是错误的（不平衡）；正确做法下第一趟应得到 $\lceil 8/4 \rceil = 2$ 个段。

（三）外部排序的时间开销与优化方向

外部排序时间开销的构成

$$\text{外部排序总时间} = \text{读写外存的时间} + \text{内部排序所需时间} + \text{内部归并所需时间}$$

其中读写外存的时间占绝对主导（外存与内存速度差几个数量级）。因此，外部排序的优化目标只有一个：减少读写外存的次数。

归并趟数公式：对 r 个初始归并段做 k 路平衡归并，归并趟数为

$$S = \lceil \log_k r \rceil$$

由该式可以看出，减少 I/O 次数有两条路：

减少外存 I/O 的两条路

路一：增加 k（增大归并路数）

代价 1：k 路归并需要 k 个输入缓冲区，内存开销增大。代价 2：每次从 k 个归并段中选一个最小元素需要 $k-1$ 次关键字比较，内部归并时间增加。→ 用「败者树」可以把每次选最小值的比较次数降到 $\lceil \log_2 k \rceil$，从而抵消这一代价。

路二：减少初始归并段数量 r（增大每个归并段的长度）

等分法下 $r = \lceil N/L \rceil$，而 L 受内存容量限制无法增大。→ 用「置换-选择排序」可以在内存容量不变的前提下生成更少、更长的初始归并段。

例题：某文件经内部排序得到 100 个初始归并段。

1. 要使多路归并三趟完成，k 至少为多少？解 $\lceil \log_k 100 \rceil \le 3$，即 $k^3 \ge 100$，$k \ge 5$（$4^3 = 64 < 100$，$5^3 = 125 \ge 100$）。答案 **5**。
2. 若操作系统要求一个程序同时可用的输入/输出文件总数不超过 13，则按多路归并至少需几趟？又若要 2 趟完成，可取的最低路数是多少？每个归并段需要一个输入文件，输出需要一个输出文件，故 $k+1 \le 13$，$k \le 12$。$\lceil \log_{12} 100 \rceil = 2$ 趟。若要 2 趟完成，$k^2 \ge 100$，$k \ge 10$。答案 **2** 趟，**10** 路。

例题（读写次数计算）：一次 I/O 的物理块大小为 150 个记录，每次可对 750 个记录进行内部排序。对含 150000 个记录的磁盘文件做 **4** 路平衡归并，需要多少次 I/O？

- 文件共占 $150000/150 = 1000$ 个物理块。
- 每次内部排序可处理 $750/150 = 5$ 个物理块，故初始归并段数 $r = 1000/5 = 200$。
- 4 路平衡归并趟数 $S = \lceil \log_4 200 \rceil = 4$。
- 生成初始归并段：读 1000 块、写 1000 块。
- 4 趟归并：每趟读 1000 块、写 1000 块。

- 总 **I/O** 次数 $= 1000 \times 2 + 1000 \times 2 \times 4 = 10000$ 次。

（四）败者树（**loser tree**）

败者树（**loser tree**）是树形选择排序的一种变形，用于 k 路归并中快速选出最小关键字。

败者树的结构与规则

- 败者树是一棵完全二叉树，k 个叶子结点分别对应 k 个归并段当前参与比较的元素；另有「冠军结点」保存最终胜者。
- 双亲结点中存放的是左右孩子比较后的失败者（败者所在归并段的段号），胜者继续向上与其它胜者比较。
- 最顶层再设一个结点（`ls[0]`）存放最终的胜者。
- 由于败者树是完全二叉树，叶子数为 k，则度为 2 的结点数为 $k-1$，不计冠军结点时总结点数为 $2k-1$；计冠军结点则为 $2k$。

5 路归并败者树的工作过程：设 5 个归并段当前的关键字依次为 $b_0 = 9,\ b_1 = 10,\ b_2 = 12,\ b_3 = 15,\ b_4 = 8$。

- 初始构造需比较 $k-1 = 4$ 次，最终胜者是段 4（关键字 8）。
- 输出 8 之后，从段 4 读入下一个关键字，设为 17，此时只需要沿从该叶子到根的路径重新比较：17 与兄弟结点（段 3 的 15）比较，15 胜；15 再与上一层胜者比较……每输出一个记录最多比较 $\lceil \log_2 k \rceil$ 次。

败者树的三个关键结论

1. 败者树的作用：进行 k 路归并时，减少选出最小关键字所需的关键字比较次数，使得增大 k 时代价 2 消失。
2. 构造 k 路归并的初始败者树，需要比较关键字 $k-1$ 次。
3. 基于构造好的败者树，每次从 k 个归并段中找出最小关键字，最多比较 $\lceil \log_2 k \rceil$ 次（最少比较次数取决于新元素来自哪个叶子，即路径长度，可能少于 $\lceil \log_2 k \rceil$）。

处理某个归并段变空的技巧：在每个归并段末尾附加一个关键字为最大值（∞）的记录。当某次选出的冠军是 ∞ 时，说明所有段都已归并完毕。

2024 年第 **11** 题

在外部排序中，利用败者树对初始为升序的归并段进行多路归并，败者树中记录「冠军」的结点保存的是什么？

答：升序归并段多路归并时，每次应输出各段当前首元素中的最小者，比较中的「冠军」是当前最小关键字。但败者树的冠军结点保存的不是关键字本身，而是最小关键字所在的归并段号——因为输出冠军后，需要据此到对应归并段读取下一个记录。答案 **D**（最小关键字所在归并段号）。

（五）置换-选择排序（生成初始归并段）

置换-选择排序（**replacement selection**）用于在内存容量不变的前提下生成更少、更长的初始归并段，从而减小 r。

为什么不能用内部排序：所有内部排序算法的前提是全部记录都在内存中，而内存容量 L 是固定的，无法通过增大 L 来加长归并段。所以要另想办法。

置换-选择排序的操作步骤

设内存工作区可容纳 L 个记录。

1. 从初始文件中输入 L 个记录到内存工作区。
2. 从工作区中选出关键字最小的记录，记为 $MINIMAX$。
3. 把 $MINIMAX$ 记录输出到归并段文件。
4. 若初始文件不为空，从初始文件中输入下一个记录到工作区（补足 L 个）。
5. 从工作区中所有关键字大于当前 $MINIMAX$ 的记录中，选出关键字最小的记录，作为新的 $MINIMAX$。
6. 重复 3–5，直到工作区中选不出新的 $MINIMAX$（即剩余记录的关键字都不大于当前 $MINIMAX$）为止，此时得到一个初始归并段。
7. 重复 2–6（对工作区中剩下的记录重新开始），直到内存工作区为空，得到全部初始归并段。

这个「不断选出不小于旧 $MINIMAX$ 的最小值」的操作，需要借助败者树高效实现。

样例：设内存工作区可容纳 **3** 个记录，文件的关键字序列为

`51, 94, 37, 92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`

逐步过程（**#** 表示该归并段结束）：

输出文件 **FO**	工作区 **WA**	输入文件 **FI**（待读）
—	`51, 94, 37`	`92, 14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37	`51, 94, 92`	`14, 63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51	`14, 94, 92`	`63, 15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92	`14, 94, 63`	`15, 99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92, 94	`14, 15, 63`	`99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
37, 51, 92, 94, #	`14, 15, 63`	`99, 48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14	`99, 15, 63`	`48, 56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15	`99, 48, 63`	`56, 23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48	`99, 56, 63`	`23, 60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56	`99, 23, 63`	`60, 31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63	`99, 23, 60`	`31, 17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63, 99	`31, 23, 60`	`17, 43, 8, 90, 166, 100`
14, 15, 48, 56, 63, 99, #	`31, 23, 60`	`17, 43, 8, 90, 166, 100`
23	`31, 17, 60`	`43, 8, 90, 166, 100`
23, 31	`43, 17, 60`	`8, 90, 166, 100`
23, 31, 43	`8, 17, 60`	`90, 166, 100`
23, 31, 43, 60	`8, 17, 90`	`166, 100`
23, 31, 43, 60, 90	`8, 17, 166`	`100`
23, 31, 43, 60, 90, 166	`8, 17, 100`	—（已读完）
23, 31, 43, 60, 90, 166, #	`8, 17, 100`	—
8	`17, 100`	—

输出文件 **FO**	工作区 **WA**	输入文件 **FI**（待读）
8, 17	`100`	—
8, 17, 100	—（工作区空）	—
8, 17, 100, #	—	—

共得到 **4** 个初始归并段：

- RUN1 = `37，51，92，94`（长度 4）
- RUN2 = `14，15，48，56，63，99`（长度 6）
- RUN3 = `23，31，43，60，90，166`（长度 6）
- RUN4 = `8，17，100`（长度 3）

对比：若用普通内部排序等分，19/3 只能得到 **7** 个长度为 3 的归并段。置换-选择排序只用 **4** 个段（且长度不均），大幅减少了 r。

读写外存次数：每条记录初始时占一个磁盘块，全部 19 条记录要依次读入内存、再逐一写回外存，因此读磁盘 **19** 次、写磁盘 **19** 次。

置换-选择排序的两个极值结论

- 输入文件关键字呈非递减（已有序）时，得到的初始归并段最长，长度为全部记录总数 N。
- 输入文件关键字呈递减（逆序）时，得到的初始归并段最短，最短长度为缓冲区长度 L。
- 若对置换-选择排序的输出文件再做一次置换-选择排序，结果保持原样（因为已经有序）。

2023 年第 42 题

2023 年应用题深入考察了外部排序的置换-选择排序。改编练习（19 个记录、工作区 3 个记录、3 路归并）要求回答：能生成几个初始归并段、各是什么、置换-选择过程读/写磁盘次数。读 **19** 次、写 **19** 次是标准答案的要点。

需注意的还有 2016 年第 11 题（考察外部排序思想，10 TB 数据文件应用归并类外部排序）与 2019 年第 11 题（最佳归并树虚段数）。

（六）最佳归并树与哈夫曼树

经过置换-选择排序后，各初始归并段的长度不等。此时一个自然的问题：按什么顺序归并，才能使读写外存的次数最少？

归并树与 WPL 的对应关系

把归并过程画成一棵树：叶子结点表示初始归并段，权重为该段的记录数；非叶结点表示归并得到的临时文件。

若每次归并都涉及「读 + 写」，则读写外存的总次数恰好等于该归并树带权路径长度 WPL 的 **2** 倍：

$$\text{磁盘 I/O 总次数} = 2 \times WPL = 2 \times \sum_{i=1}^{m} w_i \, l_i$$

其中 w_i 是第 i 个归并段的长度，l_i 是该叶子到根的路径长度（即它参与归并的趟数）。

因此在 k 路归并中，使 **I/O** 次数最少 $\Leftrightarrow$ 使 k 叉归并树的 **WPL** 最短 $\Leftrightarrow$ 构造哈夫曼树。这样得到的归并树称为最佳归并树（**optimal merge tree**）。

哈夫曼树从二叉树推广到 k 叉树同样成立：带权路径长度最短的 k 叉树即 k 叉哈夫曼树。

样例：9 个初始归并段的长度分别为 9，30，12，18，3，17，2，6，24，采用 **3** 路平衡归并。

- 按「任意顺序、凑够 3 个就归并」的普通 3 路归并：总长 $9+30+12+18+3+17+2+6+24=121$，两次归并，读写次数 $=121\times2\times2=484$。
- 按最佳归并树（**3** 叉哈夫曼树）归并：$\mathrm{WPL}=2\times3+3\times3+6\times3+9\times2+12\times2+17\times2+18\times2+24\times2+30\times1=223$，读写次数 $=223\times2=446$。

最佳归并树把 **I/O** 次数从 **484** 降到了 **446**。

（七）虚段（补空归并段）的数目计算

为什么需要虚段

k 叉的最佳归并树一定是一棵严格的 k 叉树（树中只包含度为 k 和度为 0 的结点）。

设度为 k 的结点有 n_k 个，度为 0 的结点（叶子）有 n_0 个，总结点为 n。

- 结点数：$n=n_0+n_k$
- 边数：$k\,n_k=n-1$（树中边数 = 结点数 -1）

联立可得

$$n_0=(k-1)n_k+1 \quad\Longrightarrow\quad n_k=\frac{n_0-1}{k-1}$$

n_k 必须是整数，所以初始归并段数量 m 必须满足 $(m-1)\bmod(k-1)=0$。若不然，就需要补充若干个权值为 **0** 的「虚段」来凑成严格 k 叉树。

虚段数目的计算公式（直接套用）

对 k 路平衡归并，若初始归并段数量为 m：

- 若 $(m-1)\bmod(k-1)=0$：不需要增加虚段。
- 否则，计算 $u=(m-1)\bmod(k-1)$，需要附加

$$\text{虚段个数}=(k-1)-u$$

虚段的权值为 **0**，设置它只是为了方便构造哈夫曼树，构树完成后自动去掉即可（不参与归并、不影响 WPL）。

验算：

- $k=3$，$m=9$：$(9-1)\bmod(3-1)=8\bmod2=0$，不需虚段。（与第 6 节样例一致，可构造真正的 3 叉树。）
- $k=3$，$m=8$：$(8-1)\bmod2=1\neq0$，虚段 $=(3-1)-1=1$ 个。
- $k=12$，$m=120$：$(120-1)\bmod(12-1)=119\bmod11=9\neq0$，虚段 $=(12-1)-9=2$ 个。（2019 年第 11 题，答案 **B**（**2**）。）

2019 年第 11 题与改编应用题的完整解法

题目：外存上有 120 个初始归并段，进行 12 路归并，为实现最佳归并，需要补充的虚段个数是多少?

解答：$k=12$，$m=120$。$(m-1)\bmod(k-1)=119\bmod11=9=u\neq0$，故虚段个数 $=(k-1)-u=11-9=2$。答案 **B**。

改编应用题（19 个记录、工作区 3 个记录，3 路归并）的完整解法：

1. 置换-选择排序得到 **4** 个初始归并段，长度分别为 **4**、**6**、**6**、**3**。
2. 3 路归并：$(4-1)\bmod(3-1)=3\bmod2=1=u\neq0$，虚段个数 $=(3-1)-1=$ **1** 个。

3. 补 1 个权值为 0 的虚段后，5 个叶子的权为 4，6，6，3，0。构造 3 叉哈夫曼树：取最小的 3 个 0，3，4 合成 1 个权为 7 的结点；剩下 6，6，7 合成 1 个权为 19 的根。
4. $WPL=(0+3+4)\times 2+(6+6)\times 1=26$。最佳归并树形态不唯一，但 **WPL** 一定等于 **26**。
5. 读磁盘次数 = **WPL** = **26** 次；写磁盘次数 = **WPL** = **26** 次；磁盘 **I/O** 总次数 $=26\times 2=52$ 次。

逐趟验算：第一趟把 4、6、6 三段归并（或补虚段后的等价组合），带来 $0+3+4=7$ 次读、7 次写；第二趟把得到的 7 与另两段 6、6 归并，带来 $7+6+6=19$ 次读、19 次写。合计读 $7+19=26$ 次、写 $7+19=26$ 次，与 WPL 一致。✓

最佳归并树易错点

- 读写外存次数 = **WPL**（读）与 **WPL**（写），**I/O** 总次数 = $2\times WPL$。不要漏乘 **2**。
- 哈夫曼树只保证 **WPL** 最小，树形不唯一，但 WPL 值唯一。
- 虚段的权值必须取 **0**，且要放在哈夫曼树的最深层（因为权 0 的叶子路径最长也不增加 WPL）。
- 只有当 $(m-1)\bmod(k-1)\neq 0$ 时才需要补虚段；已满足整除条件时补虚段是错误做法。
- 计算 WPL 时不要把非叶结点（临时文件）的权漏掉或重算，只累加「叶子权 × 叶子到根的路径长度」。

十二、排序算法的分析与应用

（一）完整对比表

下表是本章最需要背诵的内容。约定：「比较次数与初始状态有关」指该算法在最好与最坏情况下比较次数不同。

算法	最好时间	平均时间	最坏时间	空间	稳定性	适合链表	适合基本有序	比较次数与初态有关
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	适合	非常适合	有关
折半插入排序	$O(n\log n)$（仅比较）	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	不适合	不适合（比较次数反而固定为 $O(n\log n)$）	无关
起泡排序	$O(n)$（带 flag）	$O(n^2)$	$O(n^2)$	$O(1)$	稳定	适合	较适合	带 flag 有关；不带则无关
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定	适合	不适合（比较次数固定）	无关
希尔排序	$O(n)$	与增量序列有关，约 $O(n^{1.3})$	$O(n^2)$（Shell 增量）	$O(1)$	不稳定	不适合	适合	有关
快速排序	$O(n\log n)$	$O(n\log n)$	$O(n^2)$	$O(\log n)\sim O(n)$	不稳定	不适合	不适合（有序时性能最差）	有关
堆排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(1)$	不稳定	不适合	无关（不敏感）	有关（堆化次数随状态变化）
二路归并排序	$O(n\log n)$	$O(n\log n)$	$O(n\log n)$	$O(n)$	稳定	非常适合	无关	有关（两两归并次数随状态变化）
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(r)$（或 $O(n+r)$）	稳定	适合	无关	无关（非比较排序）
桶排序	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	取决于桶内排序	不适合	无关	无关（非比较排序）

对比表里最容易考的七条

1. 稳定的五个：直接插入、折半插入、起泡、归并、基数。
2. 不稳定的四个：希尔、简单选择、快速、堆。（2023 年第 10 题正是问这组。）
3. 最好时间是 $O(n)$ 的三个：直接插入、起泡（带 flag）、希尔（已有序时）。
4. 最好/平均/最坏完全相同的三个：简单选择、堆排序、二路归并（都是 $O(n^2)/O(n\log n)/O(n\log n)$，且与初始状态无关）。
5. 空间不是 $O(1)$ 的三个：快速 $O(\log n)\sim O(n)$、归并 $O(n)$、基数 $O(r)$。
6. 比较次数恒定的两个：简单选择 $\frac{n(n-1)}{2}$、折半插入（只与 n 有关）。
7. 移动次数最少的是简单选择排序（最多 $n-1$ 次交换）。

「每趟至少确定一个元素最终位置」的算法

满足的：简单选择排序、起泡排序、快速排序、堆排序。

不满足的：直接插入排序（前缀有序但不在最终位置）、折半插入排序、希尔排序、二路归并排序、基数排序。

记忆口诀：「选择、起泡、快排、堆排」每趟钉死一个元素（2012 年第 10 题答案：仅 I、III、IV）。

（二）什么情况下选哪个算法

算法选择的判定流程

按下面顺序逐条判断，第一个满足条件的就是答案。

1. 数据在外部存储器、内存放不下 → 多路归并的外部排序（先置换-选择生成初始归并段，再建最佳归并树归并）。
2. 要求算法稳定（多关键字排序、保留原始相对次序）→ 只有直接插入、折半插入、起泡、归并、基数可选；数据量大时选归并或基数。
3. 要求空间复杂度 $O(1)$ → 排除快排（递归栈）、归并（$O(n)$ 辅助数组）、基数（$O(r)$ 桶）。
4. 要求最坏情况下仍是 $O(n\log n)$ → 选堆排序或二路归并；优先堆排序（空间 $O(1)$）。
5. 序列基本有序 → 选直接插入排序或起泡排序（都能接近 $O(n)$）。此时绝不能选快速排序。
6. 序列完全随机、数据量大、只要求平均快、不要求稳定 → 选快速排序（常数系数最小、缓存友好）。
7. 数据元素的关键字可以分解为若干位（整数、定长编码），且取值范围不大 → 选基数排序，可达到线性时间。
8. 要求移动次数最少 → 选简单选择排序或基数排序。
9. 存储结构是链表 → 排除折半插入、希尔、快排、堆排序；优先直接插入或二路归并。
10. 数据量很小（如 $n\le 20$） → 选直接插入排序或简单选择排序，常数开销小。

真题考法

2019 年第 7 题：「选择一个排序算法时，除算法的时空效率，还需要考虑的是?」四个选项为数据规模、数据存储方式、算法稳定性、数据初始状态，四项都需要考虑，答案 D。这道题的四个因素正是上面选择流程的四个维度。

（三）算法性质反推（考试最常见的形式）

给出「某排序方法的第 k 趟结果」，反推用的是哪个算法，按下表特征定位：

观察到的现象	对应算法
前 i 个元素局部有序，后面未处理	直接插入排序
后 i 个元素是全局最大的几个、有序	起泡排序

观察到的现象	对应算法
前 i 个元素是全局最小的几个、有序	简单选择排序
有 i 个元素（或以上）已在各自最终位置上，其余无序	快速排序
序列由若干长度为 2^k 的有序段拼接而成	二路归并排序
序列由若干长度不等的有序段拼接而成，总段数减少	希尔排序（增量为 gap 时）
所有下标相差 gap 的子序列各自有序	希尔排序，增量为 gap
各元素只按某一位稳定分组，整体尚未有序	基数排序

2009 年第 10 题完整解析

序列 11，12，13，7，8，9，23，4，5 是某排序方法第二趟后的结果，问是哪种方法。

- 检查直接插入排序：第 1 趟后前 2 个元素有序，第 2 趟后前 3 个元素有序。本例前三个 11，12，13 恰好有序，后面未处理部分可以任意，符合。
- 排除起泡排序：两趟后末尾应有两个全局最大元素就位，但末尾是 4，5。
- 排除简单选择排序：两趟后开头应有两个全局最小元素就位，但开头是 11，12（全局最小是 4、5）。
- 排除二路归并排序：自底向上两趟后应形成长度约 4 的有序段，但开头 11，12，13，7 并非有序。

答案 **B**（插入排序）。

本章小结表

条目	一句话要点	必记公式 / 数字
（一）基本概念	稳定性 = 相等元素相对次序不变；内部排序比比较/移动，外部排序比 I/O 次数	基于比较的下界 $\lceil \log_2(n!) \rceil = \Omega(n \log n)$
（二）直接插入	无序区首元素插入有序区合适位置	最好 $O(n)$、最坏/平均 $O(n^2)$、空间 $O(1)$、稳定、$n-1$ 趟
（三）折半插入	只把「找位置」换成折半，其余同直接插入	比较 $O(n \log n)$、移动 $O(n^2)$、$O(1)$、稳定、不能用于链表
（四）起泡	每趟把无序区最大值冒到末尾；有序区在尾部且全局有序	最好 $O(n)$（带 flag）、最坏 $O(n^2)$、$\frac{n(n-1)}{2}$ 次比较、稳定
（五）简单选择	每趟选最小值放到无序区首位	比较恒为 $\frac{n(n-1)}{2}$、交换至多 $n-1$ 次、$O(n^2)$、不稳定
（六）希尔	缩小增量、组内直接插入	与增量序列有关（约 $O(n^{1.3})$，最坏 $O(n^2)$）、$O(1)$、不稳定
（七）快速排序	一趟划分把枢轴放到最终位置，再递归左右	平均/最好 $O(n \log n)$、最坏 $O(n^2)$、空间 $O(\log n) \sim O(n)$、不稳定、不适用链表
（八）堆排序	建大顶堆 → 堆顶与堆尾交换 → 下滤，循环 $n-1$ 轮	建堆 $O(n)$、调整 $O(\log n)$、总 $O(n \log n)$、空间 $O(1)$、不稳定、不适用链表
（九）二路归并	分治 + 合并两个有序表	趟数 $\lceil \log_2 n \rceil$、恒为 $O(n \log n)$、空间 $O(n)$、稳定、适合链表
（十）基数排序	LSD：按位分配 + 收集，逐位稳定排序	时间 $O(d(n+r))$、空间 $O(r)$、稳定、不比较关键字、要求关键字可分解

条目	一句话要点	必记公式 / 数字
（十一）外部排序	生成初始归并段 → 多路归并	趟数 $S=\lceil \log_k r \rceil$；败者树构造 $k-1$ 次、每次选最小 $\lceil \log_2 k \rceil$ 次；虚段数 $(k-1)-u$；I/O 总次数 $=2\times WPL$
（十二）分析应用	稳定选插入/归并/基数，$O(1)$ 空间选堆/插入/选择，基本有序选插入	「每趟确定一个最终位置」的是：选择、起泡、快排、堆排

易错清单

逐条核对，每一条都是统考反复设置的陷阱

1. 稳定性判断必须给出含相等关键字的具体反例。「举一个例子没乱」不等于稳定。反例必须包含两个关键字相等的记录，且排序前后它们的相对次序发生颠倒。
2. 折半插入排序是稳定的，且它的比较次数与初始状态无关。不要因为「折半查找」就认为它不稳定。
3. 折半插入排序不能用于链表，希尔排序同理（需按下标跳转）。插入、选择、起泡、归并可用于链表。
4. 直接插入排序的比较次数与初始状态有关，简单选择排序无关。简单选择的比较次数恒为 $\frac{n(n-1)}{2}$。
5. 直接插入与折半插入的移动次数相同，差别只在比较次数。
6. 起泡排序加不加 `flag` 直接改变「比较次数是否与初始状态有关」的结论。不带优化时恒为 $\frac{n(n-1)}{2}$。
7. 排序「趟数」与初始状态的关系：直接插入、折半插入、简单选择、堆排序的趟数与初始状态无关；起泡、希尔、快速、归并的趟数与初始状态有关。
8. 「每趟至少确定一个元素最终位置」只在选择、起泡、快排、堆排成立。希尔、归并、插入、基数都不成立。
9. 快速排序最坏是 $O(n^2)$，而有序序列恰恰是取首元素为枢轴时的最坏输入。「基本有序时选快排」是经典错误。
10. 快速排序的空间复杂度是 $O(\log n)\sim O(n)$，来自递归栈，不是 $O(1)$。最坏情况下递归深度可达 $n-1$。
11. 快速排序的递归次数与初始序列状态有关，与「先处理长分区还是短分区」无关。只有做了尾递归优化后，先处理短分区才能把栈深度限制在 $O(\log n)$。
12. 挖坑法以最左端元素为枢轴时，必须先「从右往左」扫。以最右端元素为枢轴时顺序相反。
13. 建堆的时间复杂度是 $O(n)$，不是 $O(n\log n)$；但逐个插入建堆是 $O(n\log n)$，两种建堆结果可能不同。
14. 升序排序用大顶堆。堆的插入、删除单个元素是 $O(\log n)$。
15. 堆不是二叉排序树，中序遍历不产生有序序列。大根堆中次大值一定在根的下一层。
16. 堆排序改用链式存储后效率降低（失去下标定位能力）。同理希尔排序。
17. 归并排序每趟不保证任何元素到达最终位置。「每趟确定一个最终位置」的算法里没有归并。
18. 归并排序的移动次数与初始状态无关（非原地，每趟都要搬 n 个元素）。合并时必须用 `<=` 取左侧才能保证稳定。
19. 归并排序的空间是 $O(n)$，不是 $O(\log n)$（$O(\log n)$ 只是递归栈）。所以「要求空间 $O(1)$」时不能选归并。
20. 基数排序是稳定排序，但前提是每一位的分配/收集必须稳定。底层不稳定会导致结果既不正确也不稳定。
21. 基数排序一趟不保证任何元素到达最终位置，它只让「已考察的位」有序。
22. **LSD** 是「从最低位开始」，不能反过来。因为高位的排序结果会覆盖低位。
23. 基数排序的时间 $O(d(n+r))$ 中 r 是「基数」（桶的个数），不是记录数。位数 d 很大时效率会下降。
24. 外部排序的优化目标是减少读写外存次数，不是减少关键字比较次数。增加 k 的代价之一是内部归并比较次数增加，靠败者树解决。
25. k 路平衡归并 $\neq$ 严格 k 叉树。判据是「每趟由 m 个段得到 $\lceil m/k \rceil$ 个段」。
26. 败者树的每个双亲结点存的是「败者」的段号，冠军结点存的是「最小关键字所在段号」，不是关键字本身。

27. 构造 k 路败者树比较 $k-1$ 次；之后每次选最小关键字最多比较 $\lceil \log_2 k \rceil$ 次。（不计冠军结点时败者树有 $2k-1$ 个结点。）
28. 置换-选择排序得到的初始归并段长度不均；输入已有序时最长（等于记录总数 N），输入逆序时最短（等于缓冲区长度 L）。
29. 最佳归并树的 **I/O** 总次数 $= 2 \times WPL$，不要漏乘 **2**。WPL 是「叶子权 × 路径长度」之和，不含非叶结点权。
30. 虚段权值必须为 **0**；仅当 $(m-1) \bmod (k-1) \neq 0$ 时才补，个数为 $(k-1)-u$（$u=(m-1) \bmod (k-1)$）。满足整除条件时补虚段是错的。
31. 简单选择排序的移动次数最多 $n-1$ 次交换（即 $3(n-1)$ 次赋值），是移动次数最少的算法之一；但它的比较次数始终是 $\frac{n(n-1)}{2}$，不会因数据接近有序而减少。
32. 外部排序和内部排序的分析指标不同：前者看磁盘 I/O 次数，后者看比较次数与移动次数。